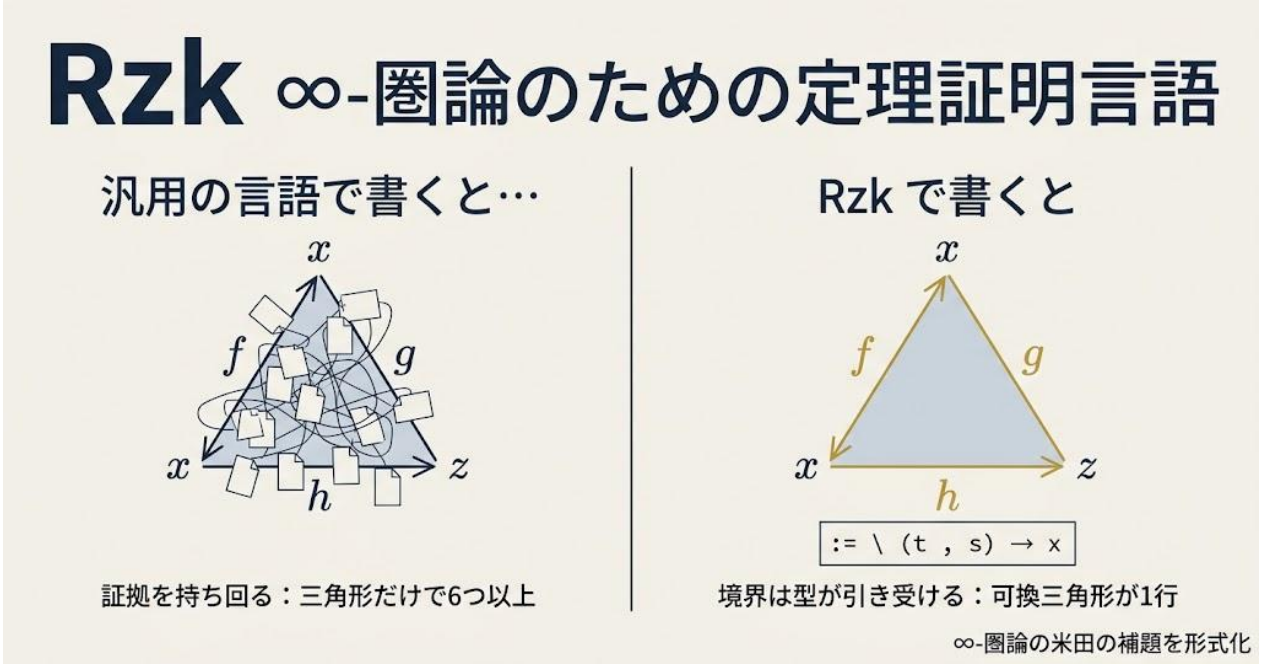
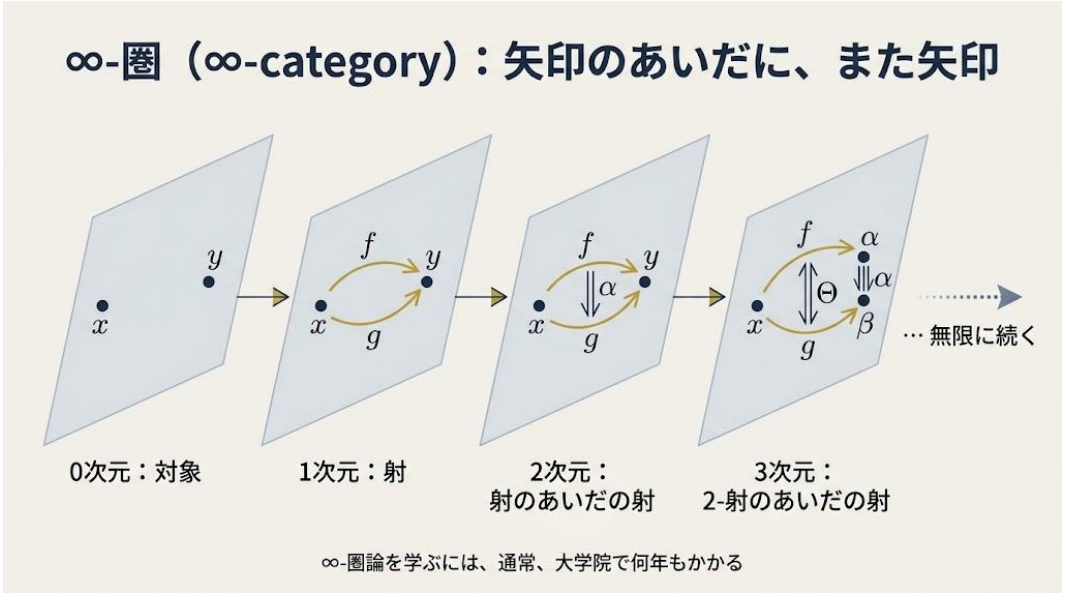
∞-圏論のためだけに作られた定理証明言語 Rzk
—— extension type から、∞-圏を表す Rezk 型、そして米田の補題まで

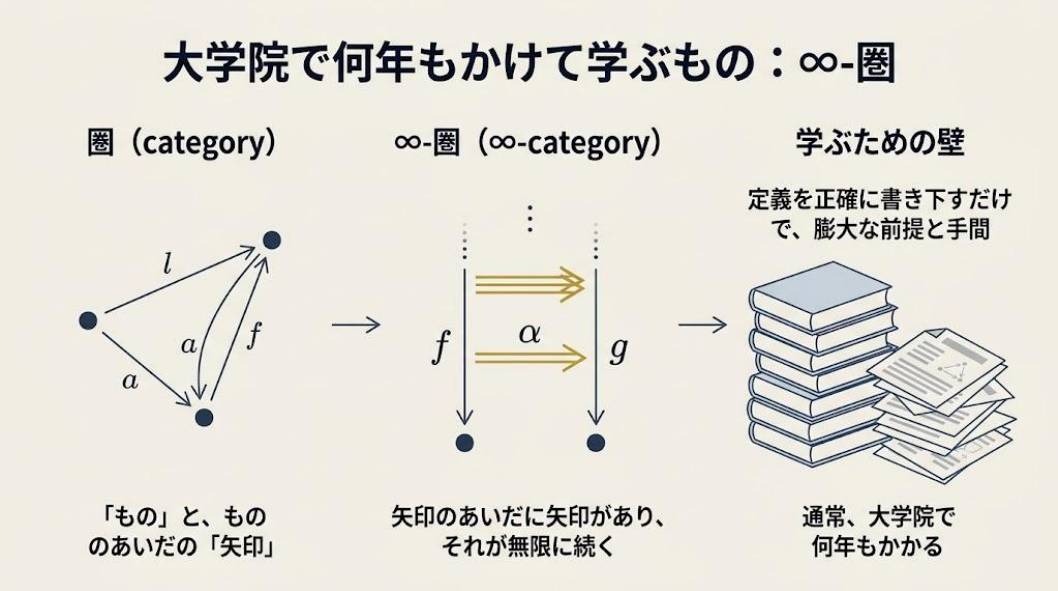


【1】Executive Summary



ある数学の分野のためだけに、言語が一から作られることがあります。本稿は、その一例をたどります。

大学院で何年もかけて学ぶもの

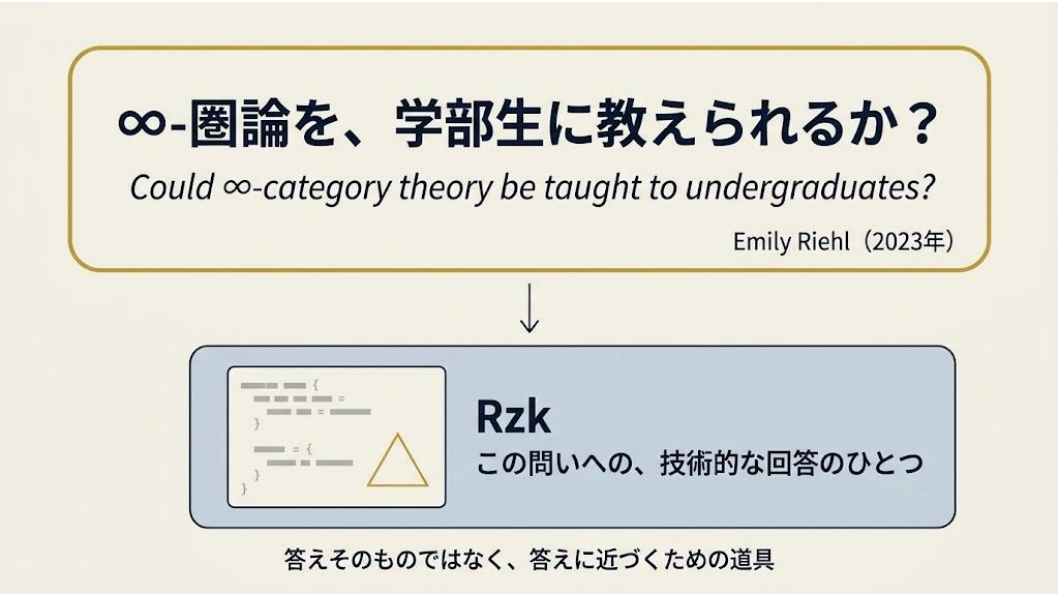


∞-圏（*∞-category*）という数学の概念があります。

圏（*category*）とは、「もの」と、もののあいだの「矢印」からなる構造です。∞-圏では、**矢印のあいだにまた矢印があり、そのあいだにまた矢印があり、それが無限に続きます。**

この概念を学ぶには、通常、大学院で何年もかかります。数学の中身が難しいこともありますが、それ以上に、**定義を正確に書き下すだけで、膨大な前提と手間が要る**からです。

「学部生に教えられるか」

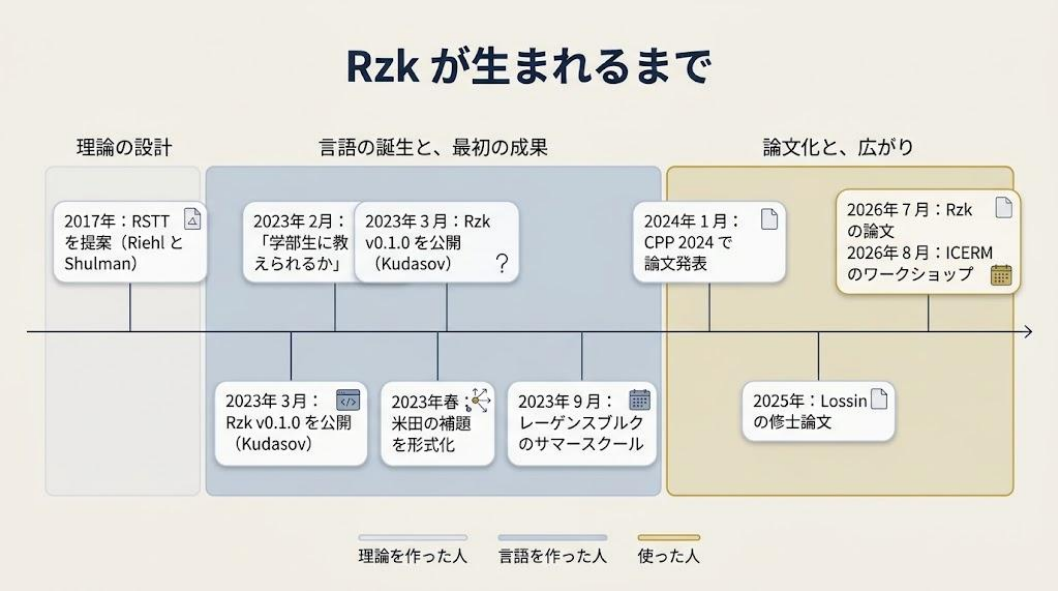


2023 年、**数学者の Emily Riehl** は、こう問いかけました。

『∞-圏論を、学部生に教えられるか？』（***Could ∞-category theory be taught to undergraduates ?***）

本稿が扱う **Rzk** は、**この問いへの技術的な回答のひとつ** なのです。

Rzk が生まれるまで



Rzk は、ある日突然できた言語ではありません。

紙の上の理論が先にあり、それを動かす言語が作られ、その言語で大きな定理が検査されるまで、**いくつかの段階** を経ています。

Rzk が公開されるまでの時系列 を年表にまとめたものが以下です。

時期	出来事	役割
2017 年	Emily Riehl と Michael Shulman が、単体的型理論（RSTT）を論文で提案	理論の設計（紙の上）
2023 年 2 月	Riehl が論考 *Could ∞-category theory be taught to undergraduates?* を arXiv で公開（Notices of the AMS 2023 年 5 月号に掲載）	動機
2023 年 3 月	Nikolai Kudasov が Rzk の最初のリリース（v0.1.0）を公開	言語の設計・実装
2023 年	v0.2.0 で全面的に書き直し	言語の設計・

時期	出来事	役割
4 月		実装
2023 年春	Kudasov、Riehl、Weinberger が、Rzk で ∞-圏論の米田の補題を形式化	最初の大きな成果
2023 年 9 月	レーゲンスブルクのサマースクール（ITP-2023）で、Rzk が教材として使われる。前は利用者 3 人、v0.5.4。期間中に 8 営業日で 8 回のリリース	教育での利用の始まり
2023 年 10 月	Kudasov が HoTTEST セミナーで Rzk を発表	公の場での発表
2024 年 1 月	米田の補題の形式化の論文が CPP 2024 で発表される	成果の論文化
2025 年	Benno Lossin が、修士論文の結果を Rzk で形式化	学生による利用
2026 年 7 月	Rzk そのものを記述した論文が arXiv で公開される（Kudasov、Sim、Ahrens）	理論と実装の対応づけ
2026 年 8 月	ICERM のワークショップで、Rzk が実習の土台になる	教育での本格的な利用

理論を作った人、言語を作った人、それを使った人が、それぞれ違うことに注目してください。

Riehl と Shulman が設計した紙の理論を、Kudasov が動く言語にし、Riehl たちが最初の利用者としてそれを使いました。

Rzk とは何か

年表の内容と一部重複しますが、**Rzk** の主要項目を列挙してみます。

- **読み方**：R-Z-K と一文字ずつ読むか、「Rezk（レツク）」と読む
- **正体**：∞-圏を扱うために、一から設計された定理証明言語
- **実装**：Haskell 製。ブラウザでも動く
- **設計・実装**：Nikolai Kudasov（イノポリス大学、ロシア）
- **理論の土台**：Emily Riehl と Michael Shulman が 2017 年に提案した単体的型理論（RSTT）

なお、**定理証明言語**とは、数学の証明を書き、それが正しいかを機械に検査してもらうための言語です。

Lean 、**Agda** 、**Rocq**（旧称 **Coq** ）といった名前を聞いたことがあるかもしれません。

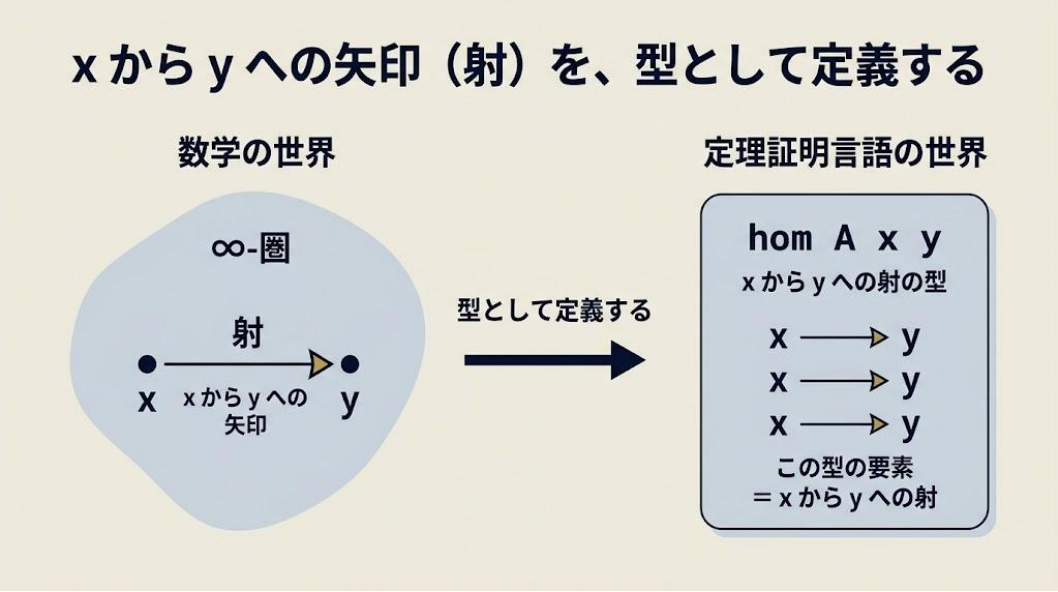
数学のどの分野でも使えるように作られています。

Rzk は違います。∞-圏論のためだけに作られました。

∞-圏論に特化した **Rzk** と比較するために、どの分野の数学にも使える定理証明言語を、本稿では『**汎用の言語**』と呼ぶことにします（**あくまで本稿だけの用語** です）。

Rezk の登場で、何が変わったのか

∞-圏 を扱うには、「x から y への **矢印（射）** 」を、**型として定義する必要** があります。



Lean や **Agda**、**Rocq** のように、数学のどの分野でも使えるように作られた定理証明言語（**本稿では「汎用の言語」と呼びます** ）で、素朴に書くと、次のようになります。

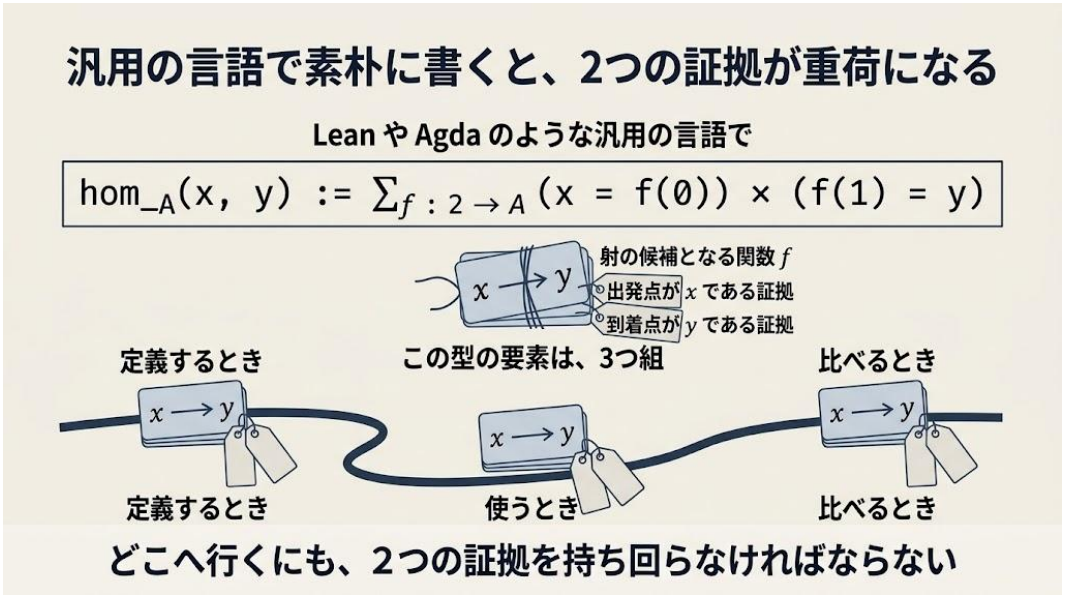
```
hom_A(x, y) := Σ_{f : 2 → A} (x = f(0)) × (f(1) = y)
```

この型の要素は、**3 つ組** です。

```
( 射の候補となる関数 f,  出発点が x である証拠,  到着点が y である証拠 )
```

2 つの「証拠」が重荷になります。

どこへ行くにも、持ち回らなければなりません。



この型理論 を提案した Riehl と Shulman の 2017 年の論文は、こう述べています（筆者訳）。

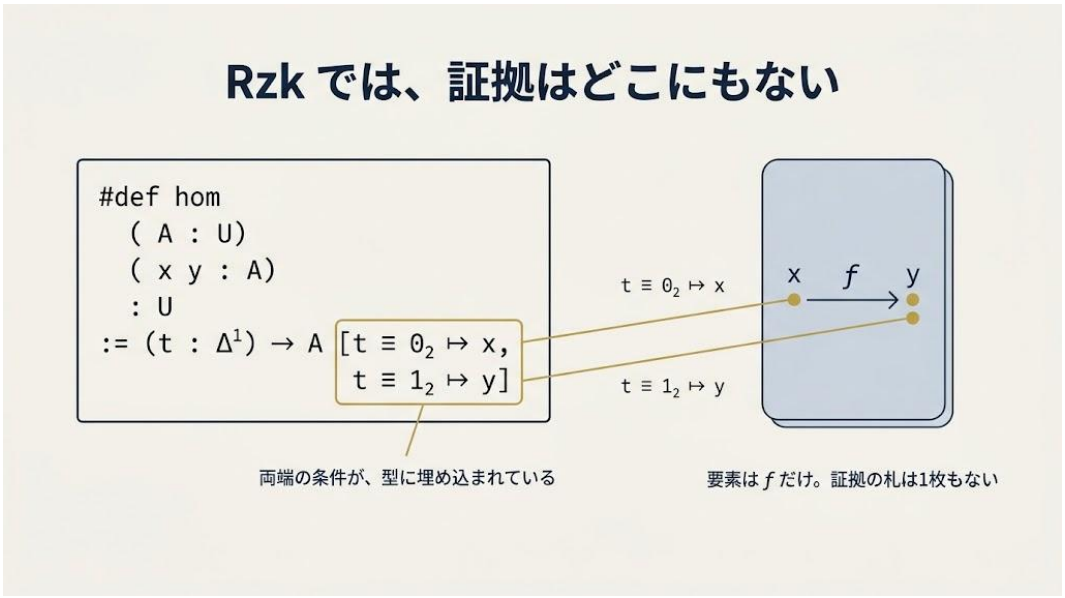
> これはきわめて煩雑であり、可換三角形、まして可換四面体を定義する段になると、技術的な困難がほとんど乗り越えられないものになる。

Rzk では、同じものがこう書けます。

```
#def hom
  ( A : U)
  ( x y : A)
  : U
  := (t : Δ¹) → A [ t ≡ 0₂ ↦ x, t ≡ 1₂ ↦ y ]
```

証拠は、どこにもありません。

「両端で x と y に一致する」という条件が、型そのものに埋め込まれているからです。

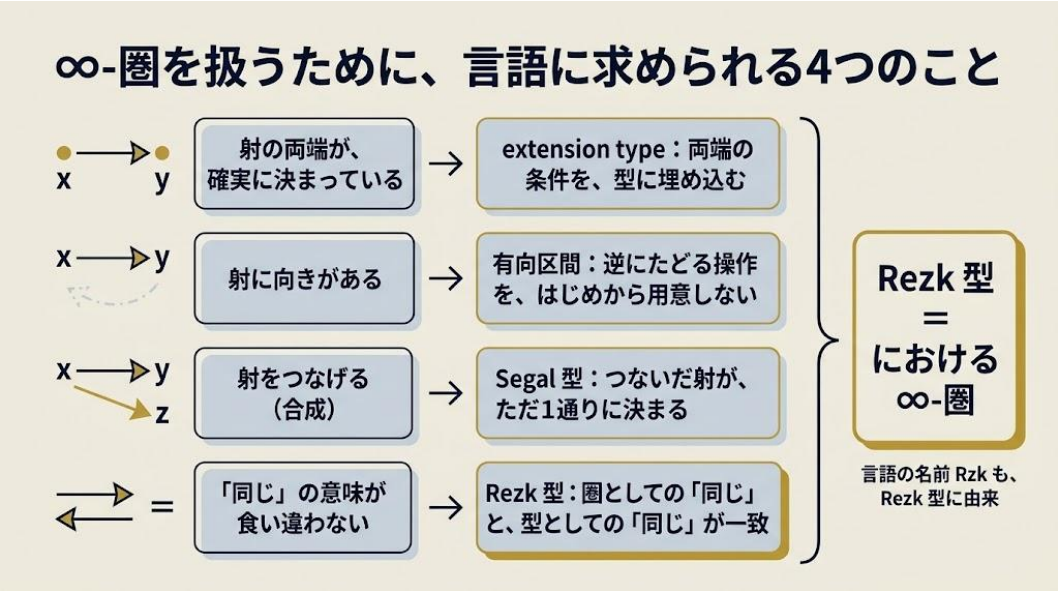


「ほとんど乗り越えられない」とされた可換三角形も、本体は 1 行 です。

```
#def id-triangle (A : U) (x : A)
  : hom2 A x x x (id-hom A x) (id-hom A x) (id-hom A x)
  := \ (t , s) → x
```

筆者は、Rzk 0.11.4 を実際に動かし、これらが型検査を通ることを確認しました。誤った定義が型エラーで拒否されることも確かめています（第 1 章 1-6）。

∞-圏を扱うために、言語に求められること



ここまでで、**射が型として書けること** を見ました。

けれども、**∞-圏を扱うには、射が書けるだけでは足りません。**

∞-圏を扱うために、言語には、次の 4 つのことが求められます。

1. **射の両端が、確実に決まっていること**
   - 「x から y への射」は、両端がきちんと x と y でなければならない
2. **射に向きがあること**
   - x から y への射があっても、y から x へ戻れるとは限らない
3. **射をつなげること（合成）**
   - x から y への射と、y から z への射をつないで、x から z への射が、ただ 1 通りに決まること
4. **「同じ」の意味が食い違わないこと**

- **圏として「同じ」（逆向きの射で行き来できる）ものと、型として「同じ」（等しい）ものが、一致していること**

Rzk は、これらの要請に対して、**次のように応えています。**

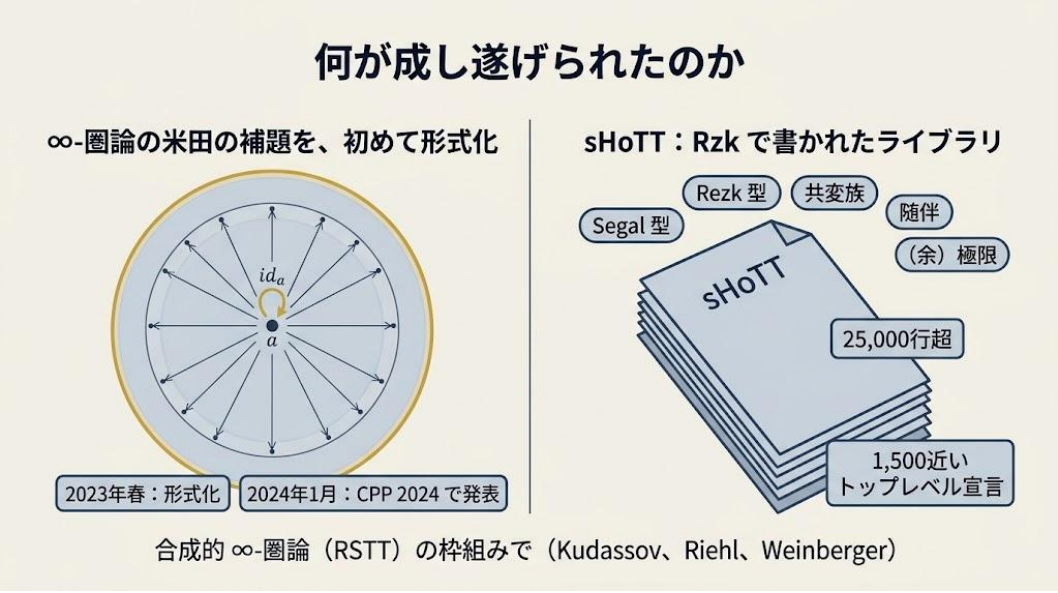
求められること	Rzk の仕組み	何をしてくれるか
射の両端が決まっている	**extension type**	「両端では x と y になる」という条件を、**型そのものに埋め込む**
射に向きがある	**有向区間**	区間に「逆にたどる操作」を **はじめから用意しない**
射をつなげる	**Segal 型**	**2 本の射をつないだ射が、ただ 1 通りに決まる型** を選び出す
「同じ」が食い違わない	**Rezk 型**	Segal 型のうち、**圏としての「同じ」と型としての「同じ」が一致する型** を選び出す

この 4 つがそろった型、つまり **Rezk 型** が、**Rzk** における ∞-圏です。

言語の名前の **Rzk** も、この **Rezk 型** に由来しています。

本稿では、この 4 つの仕組みを、1 つずつ順に見ていきます。

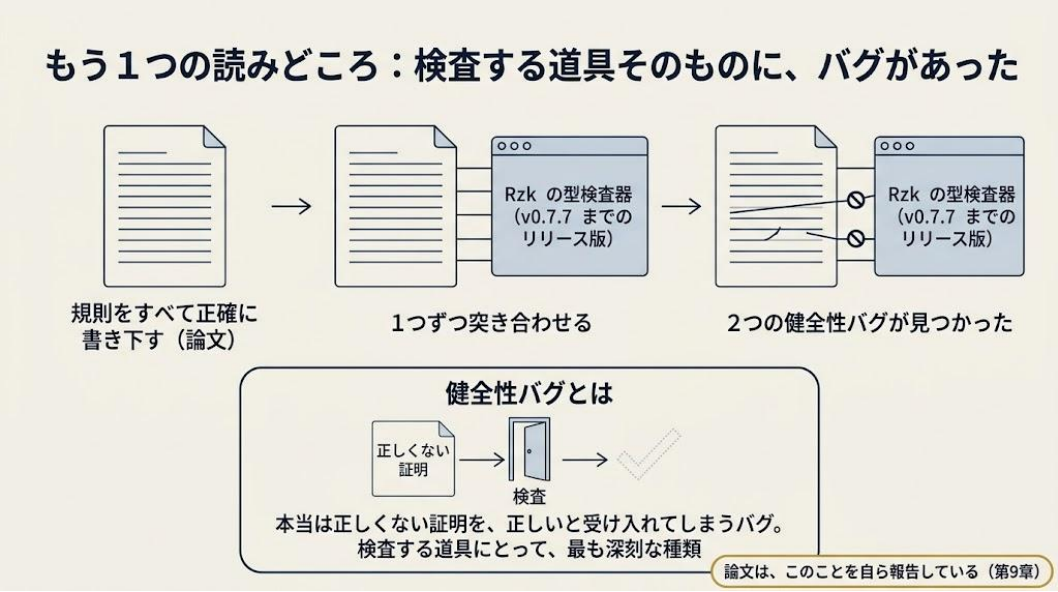
何が成し遂げられたのか



- **2023 年～2024 年**
  - 合成的 ∞-圏論（RSTT）の枠組みで、∞-圏論の米田の補題を初めて形式化した
  - 形式化は 2023 年春、論文の発表は 2024 年 1 月の CPP 2024
  - （Kudasov、Riehl、Weinberger）

- **sHoTT**
  - Rzk で書かれたライブラリ
  - 25,000 行超、1,500 近いトップレベル宣言
  - Segal 型、Rezk 型、共変族、随伴、（余）極限 など

もう 1 つの読みどころ



Rzk を紹介する 2026 年の論文（Kudasov、Sim、Ahrens *Rzk: a Proof Assistant for Synthetic ∞-Categories*）には、もう 1 つ読みどころがあります。

著者たちは、Rzk がどんな規則で型を検査しているのかを、論文の中ですべて正確に書き下しました。

そして、その規則と、実際に動いている Rzk の型検査器のプログラムとを、1 つずつ突き合わせました。

すると、すでに公開されていた Rzk（v0.7.7 までのリリース版）に、**2 つの健全性バグ** が見つかったのです。

健全性バグ とは、本当は正しくない証明を、正しいと受け入れてしまうバグのことです。

証明を検査するための道具にとって、最も深刻な種類のバグです。

- **「証明を検査する道具」そのものに、バグがあった。**
- **理論を精密に書き下す作業が、それを見つけた。**

論文は、このことを隠さずに自ら報告しています。

詳しくは第 9 章で扱います。

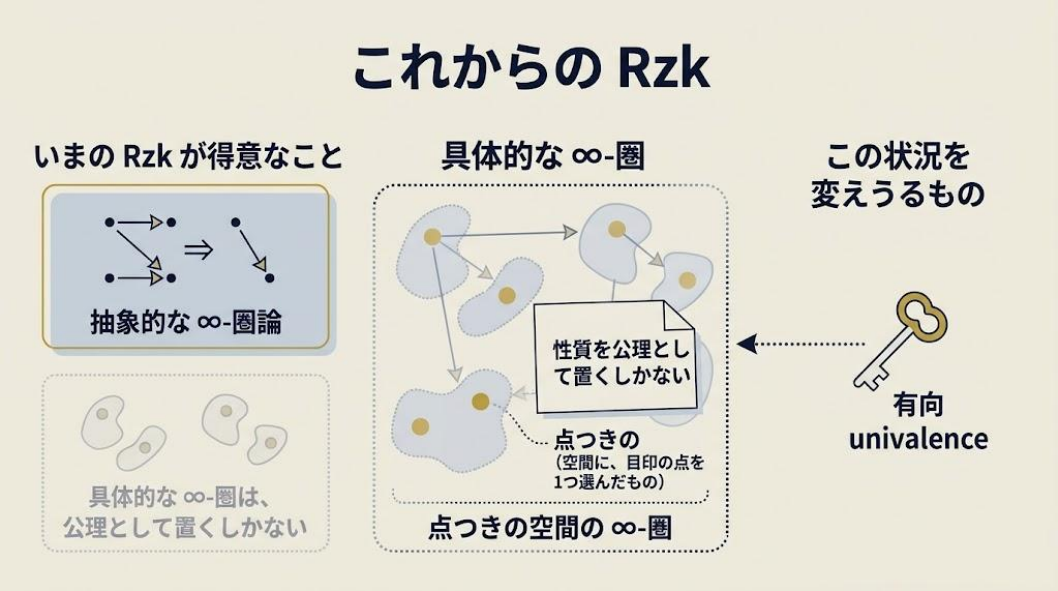
これからの Rzk

Rzk は、∞-圏論の議論を書き下すための言語として生まれました。

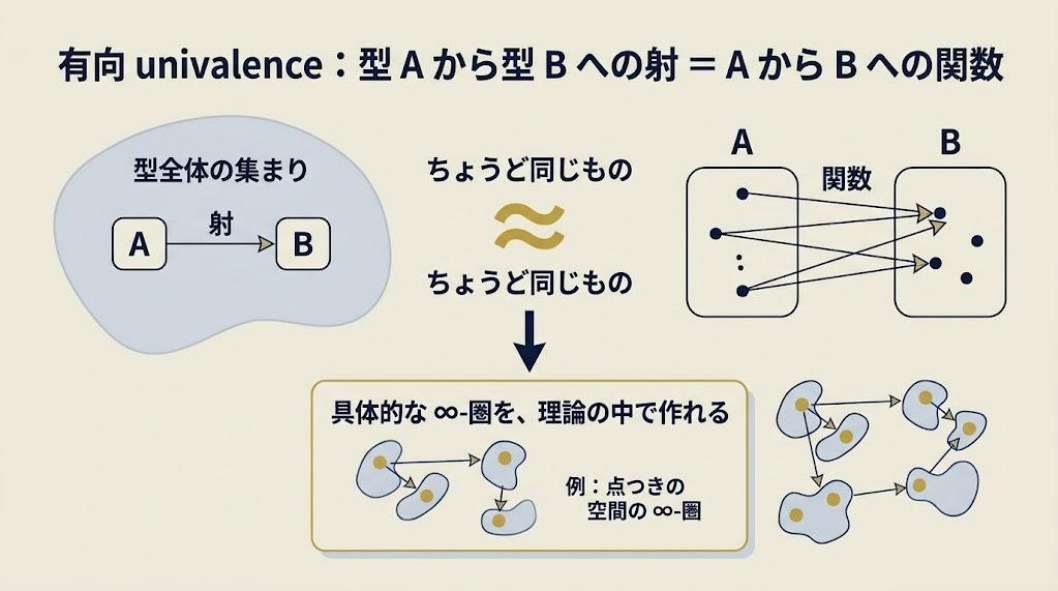
ただし、いまの **Rzk** が得意なのは、**抽象的な ∞-圏論** です。

点つきの空間（空間に、目印の点を 1 つ選んだもの）**の ∞-圏** のような **具体的な ∞-圏** は、いまの **Rzk** では、**性質を公理として置くしかありません。**

この状況を変えうるのが、**有向 univalence** という原理です。

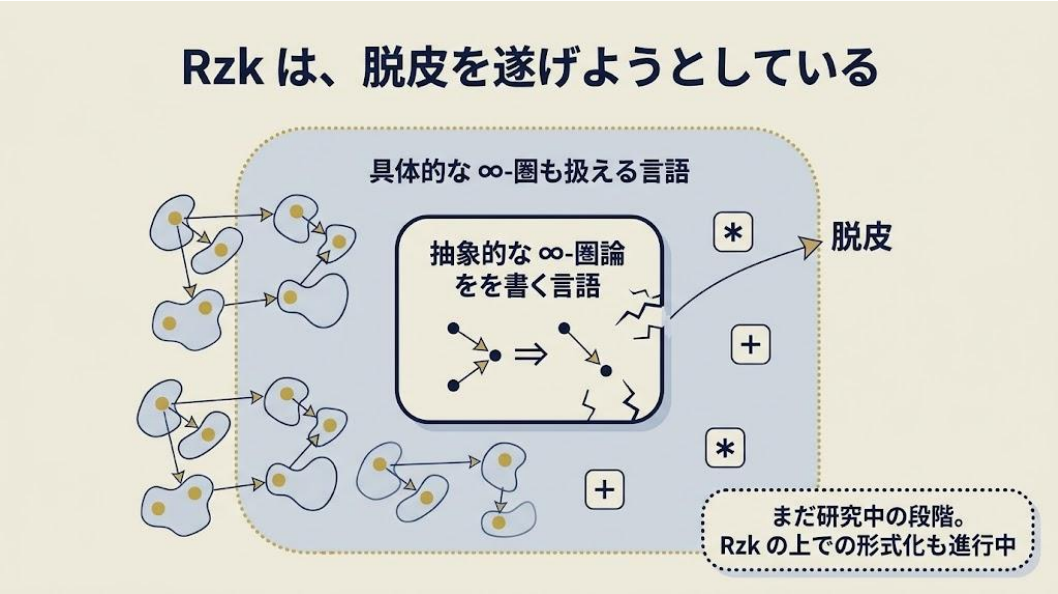


型全体の集まり の中で、「型 A から型 B への **射** 」が「A から B への **関数** 」とちょうど同じものになる、という原理で、**これを使えば、具体的な ∞-圏を理論の中で作れるようになります。**



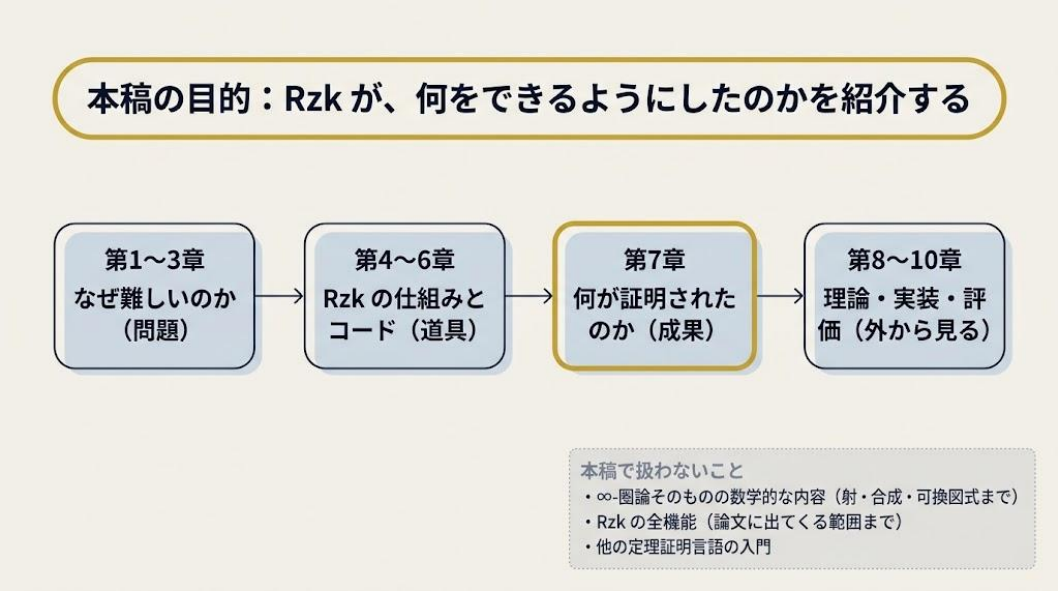
Rzk は、**抽象的な ∞-圏論を書く言語** から、**具体的な ∞-圏も扱える言語** へと、**脱皮を遂げようとしている** のです。

ただし、これはまだ研究中の段階で、**Rzk** の上での形式化も進行中です。



詳しくは第 7 章で扱います。

本稿の目的



> 本稿は、∞-圏論という抽象的な数学の研究のために一から設計された定理証明言語 Rzk が、何をできるようにしたのかを紹介することを、ひとつの目的にしています。

【2】この記事には何が書かれているのか

問い	答える章
定理証明言語とは何か。Lean や Agda と何が違うのか	第 1 章
∞-圏とは何か。なぜ難しいのか	第 2 章
汎用の言語で書くと、何が起きるのか	第 3 章
Rzk は、どういう仕組みでできているのか	第 4 章
実際のコードは、どう書かれるのか	第 5 章・第 6 章
何が証明されたのか。誰が使っているのか	第 7 章
もとの理論（RSTT）に、何を足したのか	第 8 章
理論を精密に書くと、何が起きたのか	第 9 章
強みと弱みは何か。今後どうなるのか	第 10 章

本稿で扱わないこと

∞-圏論そのものの数学的な内容

- 射、合成、可換図式の範囲にとどめます

Rzk の全機能

- 論文に出てくる範囲に絞ります

他の定理証明言語の使い方

- 比較のために言及しますが、入門は扱いません

【3】想定読者と、この記事から得られるもの

想定読者



本稿は、次の 4 つの条件に該当する方々を想定して書いています。

とくに、1 と 2 の方に向けて、前提となる知識をできるだけ本文の中で説明するようにしました。

	どんな方か	本稿のどこが役に立つか
1	**Haskell を学び始めた方**	型が何を保証するか、という話の延長（1-6、第 5 章）
2	**圏論の学習を始めたばかりの方**	圏、射、可換図式が、コードとしてどう書かれるか（第 2 章、第 5 章）
3	定理証明支援系に関心のある方	分野専用の言語という設計（第 1 章、第 10 章）
4	型システムの設計に関心のある方	3 層構造と extension type（第 4 章、第 8 章）

1 つも当てはまらない方でも、お読みいただけます。必要な前提は、すべて本文で説明します。

前提とする知識・しない知識

前提とするもの

- プログラムを書いた経験
- 型があることの意味（コンパイル時に検査される、など）
- 高校理系コースの数学

前提としないもの

- 圏論。圏の定義から説明します
- Haskell の高度な機能。モナドも型クラスも前提としません
- ∞-圏、ホモトピー型理論。本文で説明します
- 筆者の他の記事

この記事から得られるもの



1　圏と射が、コードとしてどう書かれるかが分かります。

教科書で「x から y への射」と書かれていたものが、`hom A x y` という型になります。

その対応が、具体的に見えます。

2　型が保証できることの範囲が広がります。

言語	型	型が保証すること

言語	型	型が保証すること
Haskell	`f :: Int -> Int`	受け取るのは Int、返すのも Int
Rzk	`hom A x y`	左端が x、右端が y であることまで保証される

3　Rzk のコードを、自分で読めるようになります。

4　ブラウザで、実際に動かせます。 インストールは不要です。

https://rzk-lang.github.io/rzk/playground/

5　本稿で扱う 22 語を、英語の原語と記号つきで学習済みにできます。 圏、射、∞-圏、∞-亜群、合成的、単体的型理論、有向型理論、cube、tope、shape、有向区間、単体、inner horn、extension type、cofibration、Segal 型、Rezk 型、忠実性、保存性、双方向型検査、部分型、判定的等しさの 22 語です。一覧は付録 A にあります。

6　「分野専用の言語」という設計の是非を、自分で判断できるようになります。 強みと弱みを 5 つずつ示します。そして、この問いにまだ答えが出ていないことも示します。

【4】この記事の論旨展開

本稿は、1 つの問いから出発します。

∞-圏論を、学部生に教えられるか？

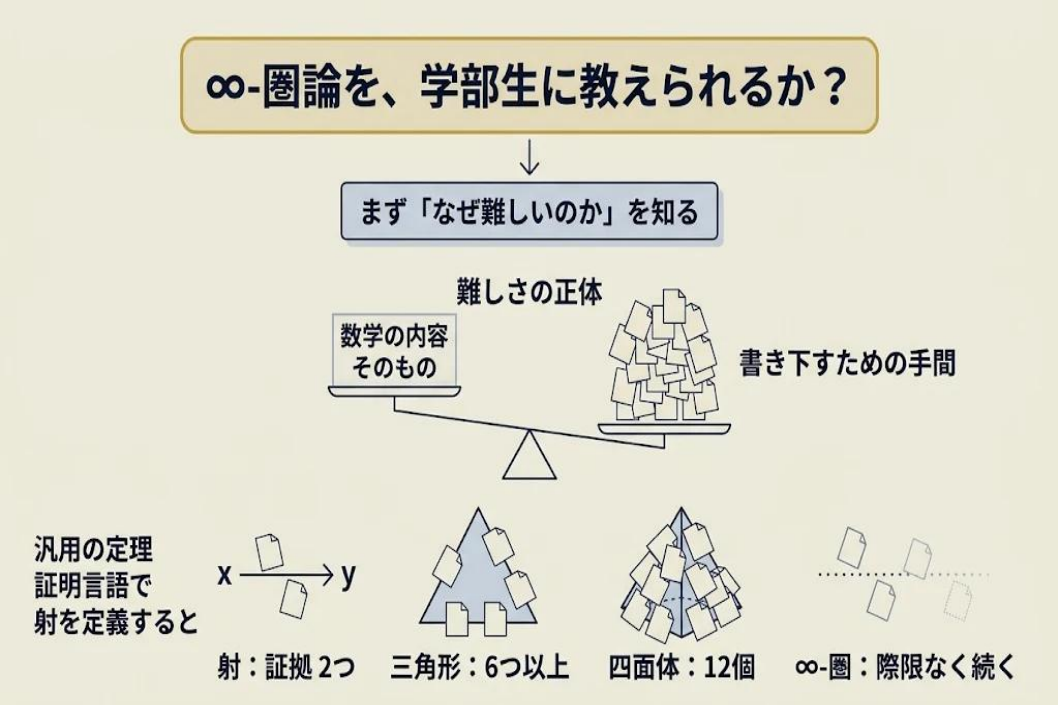
この問いに答えるには、まず **「なぜ難しいのか」を知る必要** があります。

難しさの正体 は、数学の内容そのものより、**それを書き下すための手間**にありました。

汎用の定理証明言語で **射** を定義すると、**2 つの「証拠」を持ち回ることになります。**

三角形なら少なくとも 6 つ、四面体なら 12 個。

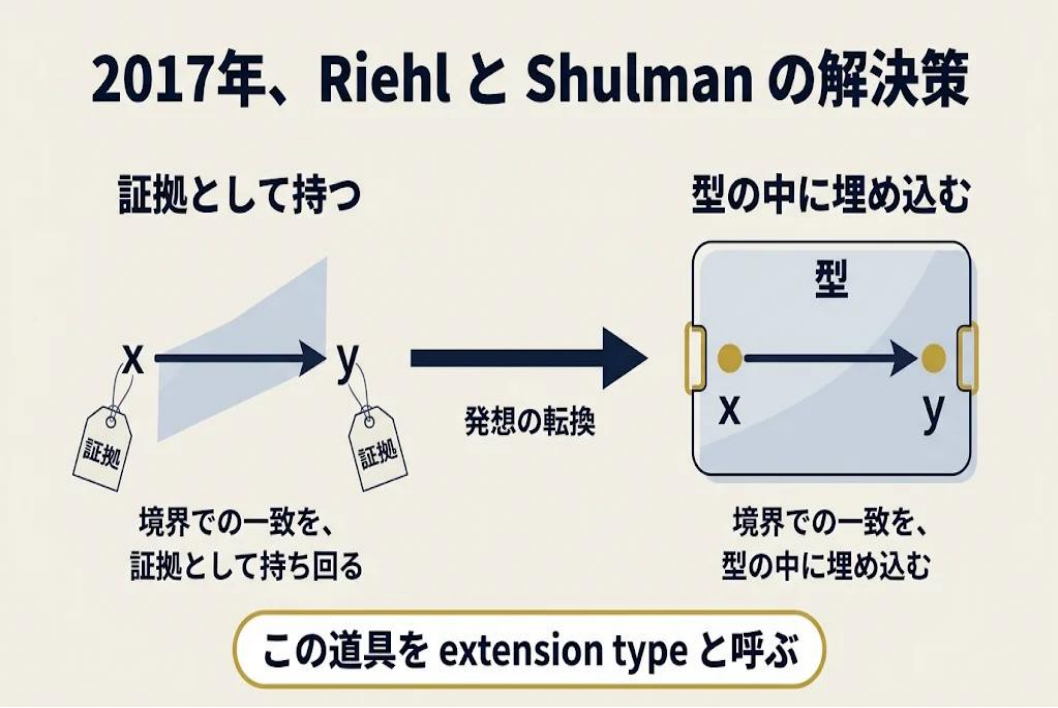
∞-圏では、これが際限なく続きます。



Riehl と Shulman は、**2017 年に解決策を提案しました** 。

境界での一致を、証拠ではなく型の中に埋め込む。

この道具を **extension type** と呼びます。



しかし、**それは紙の上の理論でした。動く処理系はありませんでした。**

Kudasov は「論文を生きたものにする」ことを目指し、Rzk を実装しました。

Rzk では、境界での一致を、証拠として持ち回らずに型に埋め込む extension type を、次のように書きます。

```
(t : Δ¹) → A [ t ≡ 0₂ ↦ x, t ≡ 1₂ ↦ y ]
```

- 前半の `(t : Δ¹) → A`：線分の上の点 t を受け取って、A の値を返す関数
  - `Δ¹` は、始点 0_2 から終点 1_2 までの **線分** を表す記号です（「デルタ・ワン」と読みます）
  - `(t : Δ¹)` は、「t はこの線分の上の点」という意味です
- 後半の `[ … ]`：境界での一致の条件。「t が始点 0_2 なら x、終点 1_2 なら y」

後半の `[ … ]` が、境界での一致を型に埋め込んでいる部分 です。

この型を持つ関数は、両端で必ず x と y になることが、**証拠なしに、型そのものによって保証されます。**

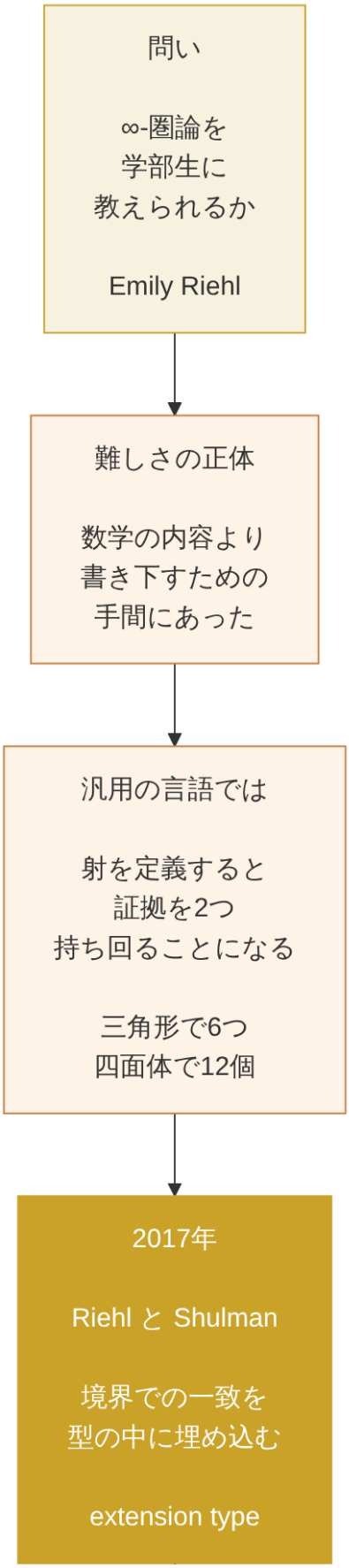
なお、**Kudasov** が RSTT を **Rzk** として実装するにあたっては、**紙の理論にはなかった工夫が 5 つ**必要でした。

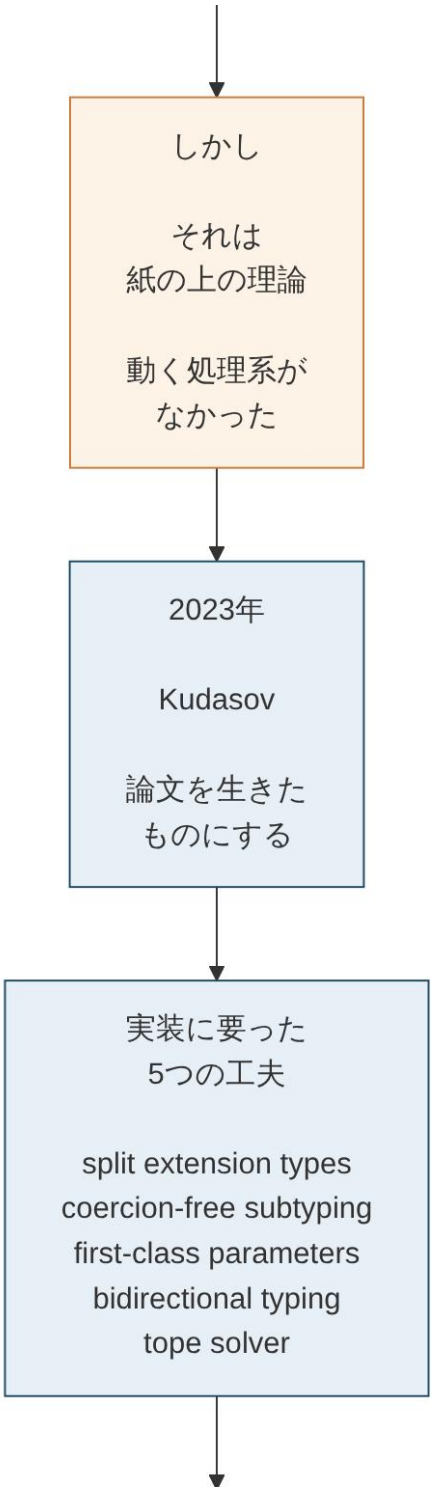
1. **extension type を 2 つの部品に分ける**（*split extension types*）
   - 「形の上の関数」の部分と、「境界の制約」の部分を、別々に扱えるようにする
2. **値に手を加えずに読み替える規則**（*coercion-free subtyping*）
   - ある型の値を、別の型の値として、そのまま使ってよい場合を決めておく
3. **点の空間や形を、定義の引数として書けるようにする**（*first-class parameters*）
   - 紙の理論では文章で補っていた前提を、コードの中に書けるようにする
4. **双方向の型検査**（*bidirectional typing*）
   - 「この型を持つか」を確かめる検査と、「型は何か」を求める検査を、行き来しながら進める
5. **形の包含を自動で確かめる仕組み**（*tope solver*）
   - 紙の証明で「明らか」として省かれていた確認を、型検査器が自動で行う

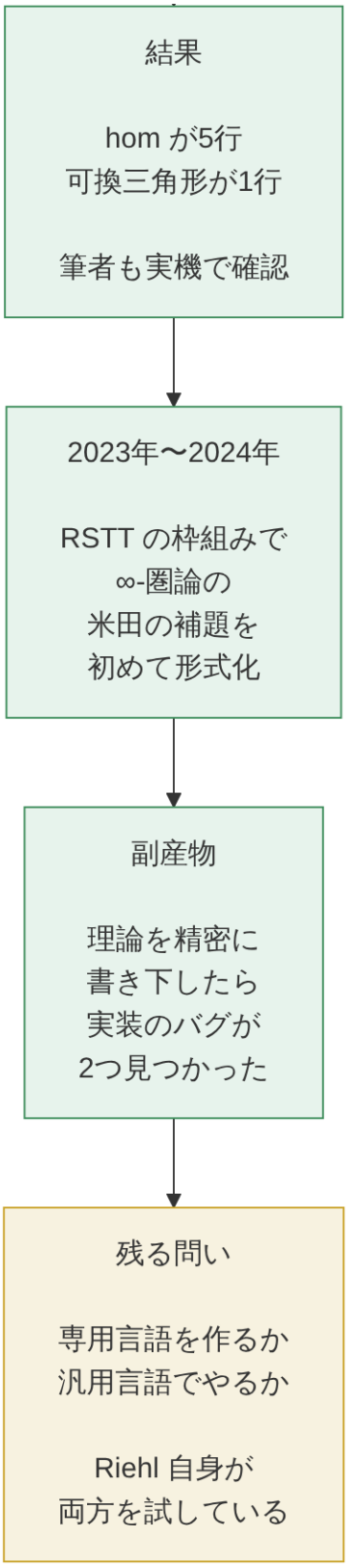
詳しくは第 8 章で扱います。

結果として、合成的 ∞-圏論（RSTT）の枠組みで、∞-圏論の米田の補題が初めて形式化されました。

そして、理論を精密に書き下す作業が、実装のバグを 2 つ見つけました。


問い
∞-圏論を
学部生に
教えられるか
Emily Riehl
難しさの正体
数学の内容より
書き下すための
手間にあった
汎用の言語では
射を定義すると
証拠を2つ
持ち回ることになる
三角形で6つ
四面体で12個
2017年
Riehl と Shulman
境界での一致を
型の中に埋め込む
extension type


しかし
それは
紙の上の理論
動く処理系が
なかった
2023年
Kudasov
論文を生きた
ものにする
実装に要った
5つの工夫
split extension types
coercion-free subtyping
first-class parameters
bidirectional typing
tope solver



橙は難しさ、金は問いと道具、青は実装、緑は到達点を表します。

図の最初と最後が、どちらも「問い」であることにご注意ください。 本稿は、答えを出して終わる記事ではありません。1 つの道が、ここまで来たことを示す記事です。

第 1 章　汎用の言語と、専用の言語

この章の要点：Lean や Agda は「どの数学でも書ける」ように作られた。Rzk は「∞-圏論だけを、自然に書ける」ように作られた。Rzk は、紙の上の型理論を「動くもの」にしたいという動機から生まれた。

1-1　Lean、Agda、Rocq は、何のために作られたか

定理証明言語（*proof assistant*、定理証明支援系とも）は、数学の証明をプログラムのように書き、機械に検査させる言語です。

紙の証明では、読者が行間を補います。「明らかに」「同様にして」と書かれていれば、読者は自分の頭で残りを埋めます。定理証明言語では、行間を補ってくれる読者はいません。**省略された推論も含めて、型検査器が、定められた規則に従って検査します。** この確かめる作業を **型検査**（*type checking*）と呼び、数学を機械が検査できる形に書き直すことを **形式化**（*formalization*）と呼びます。

なぜ「型」検査なのでしょうか。多くの定理証明言語では、命題を型として、証明をその型の値（項）として表します。「命題 P の証明を書く」ことは、「型 P を持つ値を作る」ことと同じになります。だから、証明が正しいかどうかは、値の型が合っているかどうかを確かめる型検査で判定できるのです。この対応は、プログラミング言語の世界でもよく知られています。

代表的な言語を並べます。

言語	実装言語	主なライブラリ	特徴
Lean 4	C++ / Lean	mathlib	現代数学の大規模な形式化で知られる
Agda	Haskell	agda-stdlib など	依存型プログラミング言語としての性格が強い
Rocq（旧称 Coq）	OCaml	Mathematical Components など	長い歴史を持ち、検証済みソフトウェアにも使われる

これらは、**汎用**です。整数論も、解析学も、圏論も書けます。

数学のどの分野でも使えるように、言語の核はできるだけ小さく、一般的に作られています。分野ごとの概念は、その上に**ライブラリとして**積み上げます。

∞-圏論に近い分野で言えば、ホモトピー型理論（第 2 章で説明します）を扱うための仕組みも、汎用言語の上に用意されています。

Rzk の論文によれば、Rocq と Agda には、ホモトピー型理論と両立するモードがあります。

Lean 2 はホモトピー型理論を言語として直接サポートしていましたが、その後の Lean ではサポートされなくなりました。

1-2　それでも、専用の言語が作られることがある

汎用の言語で「書ける」ことと、「書きやすい」ことは別です。

ある分野の概念が、汎用の言語の核とうまく噛み合わないことがあります。そのとき、利用者は概念そのものではなく、**概念を言語に押し込むための手間**に時間を取られます。

身近な例えで言えば、表計算ソフトでも文章は書けますが、長い文章を書くならワープロのほうが楽です。どちらも「文字を入力できる」点では同じです。違うのは、その道具が何を前提に設計されているかです。

その手間が大きすぎるとき、別の選択肢が現れます。**その分野の概念を、はじめから言語の核に組み込んでしまう**という選択です。

型理論の世界には、すでにその先例があります。

ホモトピー型理論の一種である「キュービカル型理論」には、cubicaltt、Cubical Agda、RedPRL、redtt、cooltt といった処理系が作られてきました。これらは、区間という図形を言語の核に持っています。

Rzk は、同じ発想を ∞-圏論に向けたものです。

Rzk の核に組み込まれているのは、∞-圏論の「形」（線分、三角形、四面体）と、その形の境界で値を固定する仕組みです。詳しくは第 4 章で見ます。

論文の著者らによれば、Rzk は、著者らの知る限り、Riehl と Shulman の型理論を実装した最初の、そして現時点で唯一の定理証明言語です。

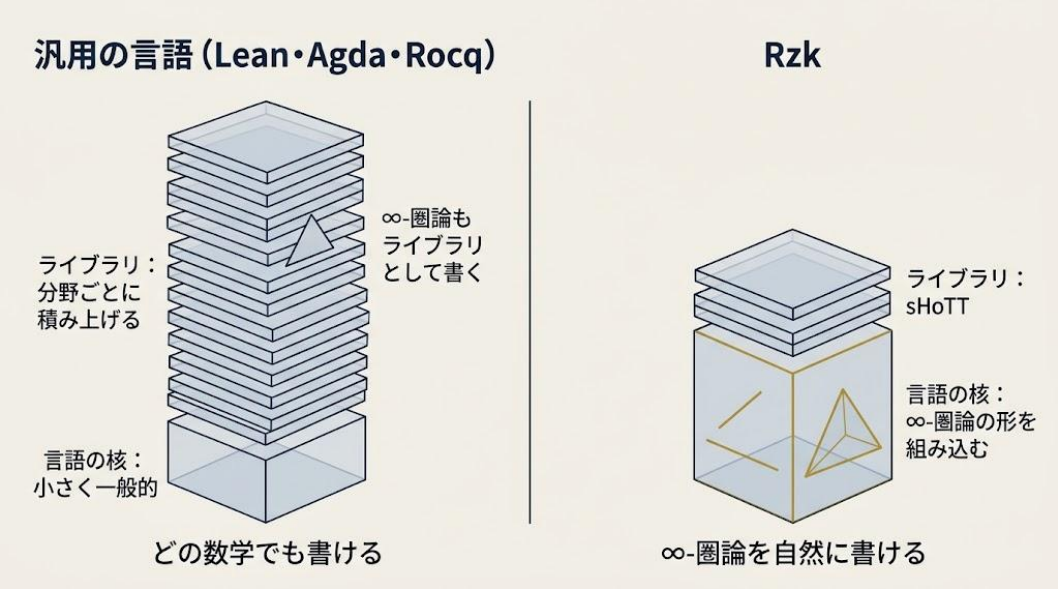
もちろん、専用の言語を作ることには代償があります。

- **専用の言語の利点**
  - その分野の記法が、ほぼそのまま書ける
  - 分野に固有の手間を、処理系が引き受けられる
- **専用の言語の代償**
  - 利用者が少なくなりやすい
  - 他の分野で作られたライブラリを使えない
  - 処理系そのものの正しさを、一から確かめる必要がある

最後の代償は、見落とされがちです。 汎用の言語は、長い年月と多くの利用者によって、処理系の誤りが洗い出されてきました。

新しい専用言語には、その蓄積がありません。

この点は第 9 章で、Rzk 自身の論文が報告したバグの話として、具体的に見ることになります。



1-3　Rzk という名前の由来

名前は、R-Z-K と一文字ずつ読むか、「Rezk」と読みます。

由来は、∞-圏を表す **Rezk 型**（*Rezk type*）という概念です。Rezk 型は第 6 章で扱います。その Rezk 型という名前は、さらに数学者 **Charles Rezk** による「Rezk 完備化」（*Rezk completion*）にちなみます。

名前	何にちなむか
Charles Rezk（数学者）	──
Rezk 完備化	Charles Rezk
Rezk 型（∞-圏を表す型）	Rezk 完備化
Rzk（その型を扱う言語）	Rezk 型

言語の名前が、言語の目的そのものを指しています。

Rzk は「Rezk 型を書くための言語」です。

汎用の言語が「どんな数学でも」を目指すのに対して、Rzk は名前の段階から、扱う対象を 1 つに絞っています。

1-4　「∞-圏論を、学部生に教えられるか」

∞-圏論は、長らく大学院で何年もかけて学ぶものでした。

理由は、内容の深さだけではありません。

∞-圏の標準的な定義（たとえば「準圏」と呼ばれるもの）は、組合せ論的な構成の上に立っています。

そこから「射」「合成」といった基本的な概念を取り出すだけで、多くの補題が要ります。

基本の言葉を話せるようになる前に、文法書を何冊も読まされるような状況です。

Emily Riehl（ジョンズ・ホプキンス大学）は、2023 年の論考で、こう問いかけました。

> ∞-圏論を、学部生に教えられるか。（*Could ∞-category theory be taught to undergraduates?*）

Riehl の答えの方向は、「∞-圏を外から組み立てるのではなく、はじめからそこにある世界で考えればよい」というものです。

この考え方を **合成的**（*synthetic*）と呼びます。

対になるのは、組み立てて定義する **解析的**（*analytic*）な考え方です。

例えで言えば、ユークリッド幾何学は「点」や「直線」を、座標を使って組み立てずに、はじめからあるものとして扱います。

これが合成的な考え方です。座標を入れて、点を数の組として定義するのが解析的な考え方です。

どちらも同じ幾何学を扱えますが、出発点が違います。

Rzk の標準ライブラリ sHoTT の README は、このプロジェクトが「この論考に示された哲学に従う」と明記しています。

Rzk は、この問いへの技術的な回答のひとつです。

本稿では、この問いを背骨にして話を進め、第 10 章で回収します。

1-5　開発の経緯 ──「論文を生きたものにする」

Rzk のリポジトリの README には、開発の動機が書かれています（筆者訳）。

> このプロジェクトは、Riehl と Shulman の 2017 年の論文を「生きたもの」にするという着想から始まった。

彼らの shapes を持つ型理論にもとづく定理証明支援系を実装することによって。

経緯を時系列で並べます。

年	出来事
2017 年	Riehl と Shulman が、∞-圏を合成的に扱う型理論を論文で提案（単体的型理論、RSTT。紙の上の理論）
↓	「これを実際に動かしてみよう」
2023 年	Nikolai Kudasov が Rzk を実装。v0.2.0 で全面的に書き直し
2023 年春	∞-圏論の米田の補題を、Rzk で形式化（Kudasov、Riehl、Weinberger）
2023 年	レーゲンスブルクのサマースクールで教材として使われる
2023 年 10 月	HoTTEST セミナーで発表
2024 年 1 月	米田の補題の形式化の論文が CPP 2024 で発表される
2026 年	Rzk そのものを記述した論文が公開される
2026 年 8 月	ICERM のワークショップで、実習の土台になる

実装には、Haskell と、その周辺の技術が使われています。

論文によれば、構文解析と整形表示は、文法の記述から自動生成する BNF Converter というツールで作られています。

型検査器の内部では、項（プログラムの式）を **free scoped monad** という、Kudasov 自身が研究してきた表現で扱います。

この表現では、変数の束縛と、変数の取り違えを起こさない置き換えが、データ型の側で一度だけ実装されます。

型検査器を書くときに起きやすい「変数の捕獲」の誤りを、構造的に防ぐ工夫です。

さらに、Rzk は GHCJS というツールで JavaScript に変換され、ブラウザの中だけで動きます。これが、本稿で何度も紹介する playground です。

本稿の主な情報源は、2026 年の論文です。

> N. Kudasov, V. Sim, B. Ahrens "Rzk: a Proof Assistant for Synthetic ∞-Categories" Proceedings of the ACM on Programming Languages, Vol. 11, POPL, 2027 arXiv:2607.12207 https://arxiv.org/abs/2607.12207

著者	所属
Nikolai Kudasov	イノポリス大学（ロシア、タタールスタン共和国）
Violetta Sim	イノポリス大学
Benedikt Ahrens	デルフト工科大学（オランダ）

論文は、Rzk を 2 つの視点から説明しています。実践の視点からは、Rzk の使い方の短い入門と、大規模なライブラリでの性能の報告。理論の視点からは、Rzk が実装している型理論を正確に書き下し、もとの理論 RSTT との関係を証明しています。本稿も、この 2 つの視点を行き来します。

1-6　型が何を保証するか ── Haskell と Rzk

Haskell を学び始めた人が、最初に感動するのは「型がバグを防いでくれる」ことでしょう。

```
f :: Int -> Int
```

この宣言は、「f は Int を受け取って Int を返す」と保証します。

f に文字列を渡すコードは、コンパイルが通りません。 **間違いが、実行する前に見つかります。**

Rzk の型は、その先を行きます。

```
hom A x y
```

この型の要素は、「A の中の、x から y への射」です。

おおまかに言えば、**A の点を時間 0 から 1 まで並べた道筋で、しかも始点が x、終点が y であるもの**です。

両端が何であるかまで、型が保証しています。

	型が保証すること
Haskell の `Int -> Int`	受け取るのは Int、返すのも Int
Rzk の `hom A x y`	A の中の道筋であり、しかも左端が x、右端が y

筆者は、わざと誤った定義を書いて、Rzk 0.11.4 で検査してみました。

```
-- 両端が x と y であるべきなのに、常に x を返す
#def bad (A : U) (x y : A) : hom A x y := \ t -> x
```

結果は型エラーです。

```
An error occurred when typechecking!
Type Error:
...
cannot unify term
  y
```

「いつでも x を返す」関数は、終点で y になっていません。

だから、x から y への射ではありません。**型検査器が、それを拒否しました。**

Haskell で `Int` の場所に文字列を渡したときと、同じことが起きています。

違うのは、検査される内容が「型の種類」から「境界での値」にまで広がっていることです。

Rzk には、Haskell と似た書き方もあります。

論文の付録には、型を受け取って何もせずに返す恒等関数を、3 通りに書いた例があります。

ここでは、そのうち 2 つを示します。

```
#define identity1
  : ( A : U) → (x : A) → A
  := \ A x → x

#define identity2
  ( A : U)
  : A → A
  := \ x → x
```

`U` は「型全体の集まり」（宇宙）です。

`(A : U)` は「型 A を引数に取る」という意味で、Haskell の型変数 `a` を、明示的な引数にしたものと考えると近いでしょう。

もう 1 つ、Haskell の読者に馴染みのある例を挙げます。 2 引数の関数の引数の順番を入れ替える `swap` です（論文の付録 B.2）。

```
#define swap
  ( A B C : U)
  : ( A → B → C) → (B → A → C)
  := \ f → \ b a → f a b
```

Haskell の標準ライブラリにある `flip` と同じものです。 ここまでは、ふつうの関数型プログラミングです。

Rzk が定理証明言語であることが見えるのは、次の例です。

「swap を 2 回かけると、もとの関数に戻る」ことを証明します（論文の付録 B.4）。

```
#define swap-swap
  ( A B C : U)
  ( f : A → B → C)
  : swap B A C (swap A B C f) = f
  := refl
```

型の部分 `swap B A C (swap A B C f) = f` が、証明したい命題です。「左辺と右辺は等しい」と読みます。

値の部分 `refl` が、その証明です。

`refl` は「両辺は計算すれば同じになる」という意味の、いちばん簡単な証明です。

型検査器は、左辺を計算して `\ a b → f a b` の形にし、これが `f` そのものと同じであることを確かめます。

論文の説明によれば、Rzk は必要に応じて関数を η 展開（`f` を `\ a b → f a b` と書き直すこと）して比べるので、この証明が通ります。

Haskell では「2 回 flip すると元に戻る」ことを型で保証することはできません。Rzk では、それが型として書け、機械が確かめます。

ただし、ここまでの例は、Rzk の「ふつうの型理論」の部分しか使っていません。

論文も、これらの例には Rzk 独自の機能（第 4 章で見る cube や tope）は一切関わらない、と注記しています。Rzk の本領は、第 5 章からです。

> 【Haskell との関係について】Rzk は Haskell で**実装**されていますが、Haskell とは別の言語です。
>
> Rzk のプログラムを Haskell として動かすことはできません。

> また、Rzk はプログラムを書いて実行するための言語ではなく、証明を書いて検査するための言語です。
>
> Haskell の知識は、Rzk の型の読み方を理解する助けになりますが、Rzk を使うための前提ではありません。

1-7　本稿がたどる道筋

章	内容	位置づけ
第 2 章	∞-圏とは何か	（数学の側）
第 3 章	汎用の言語で素朴に書くと、何が起きるか	（問題）
第 4 章	Rzk の 3 つの層	（道具）
第 5 章	射と可換三角形を書く	（道具を使う）
第 6 章	Segal 型と Rezk 型	（∞-圏そのもの）
第 7 章	何が証明されたのか	（成果）
第 8 章	紙の理論に、何を足したのか	（実装の工夫）
第 9 章	理論と実装の往復	（バグの話）
第 10 章	分野専用の言語という設計	（評価と、残る問い）

第 2 章と第 3 章は、問題を理解するための章です。

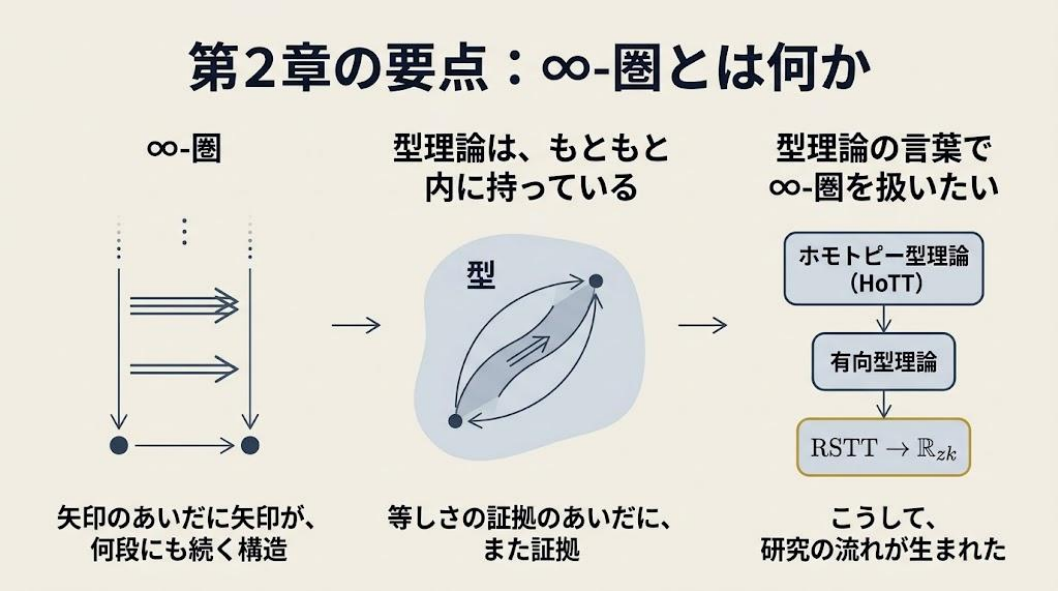
第 4 章から第 6 章は、Rzk のコードを実際に読む章です。

第 7 章以降は、Rzk を外から評価する章です。

急ぐ方は、第 3 章の最後と第 5 章を読めば、Rzk の核心はつかめます。

まずは数学の側から始めます。

第 2 章　∞-圏とは何か



この章の要点：

∞-圏 とは、矢印のあいだに矢印があり、**それが何段にも続く構造** である。

型理論 は、その構造を **もともと内に** 持っている。

だから、**型理論の言葉で ∞-圏を扱いたい**、という研究の流れが生まれた。

2-1　まず、圏とは何か

具体例から入ります。

集合 X から集合 Y への関数 f と、Y から Z への関数 g があれば、つないで X から Z への関数 $g \circ f$ が作れます。

「まず f を当て、その結果に g を当てる」関数です。

どの集合にも、何もしない関数 id（恒等関数）があります。

この「つなぐ」という構造は、関数以外にもあちこちに現れます。

もの（対象）	矢印（射）	つなぐ（合成）
集合	関数	関数の合成
位相空間	連続写像	連続写像の合成
ベクトル空間	線形写像	線形写像の合成

もの（対象）	矢印（射）	つなぐ（合成）
Haskell の型	関数 `a -> b`	`(.)` 演算子

最後の行は、Haskell を学んでいる方には馴染みがあるでしょう。

`(.) :: (b -> c) -> (a -> b) -> (a -> c)` は、まさに関数をつなぐ演算子です。

`id :: a -> a` が、何もしない関数です。

この共通の形に、名前がついています。

圏（*category*）です。

圏は、次のものからなる

構成要素	内容
対象（object）	x, y, z, …
射（morphism）	x から y への矢印 f : x → y
合成（composition）	f : x → y と g : y → z から、g∘f : x → z
恒等射（identity）	各 x に、何もしない射 id_x : x → x

ただし、次の規則を満たす

- 結合律：h∘(g∘f) = (h∘g)∘f
- 単位律：f∘id = f、id∘f = f

射（*morphism*）は、対象から対象への矢印です。

合成（*composition*）は、矢印をつなぐ操作です。

恒等射（*identity morphism*）は、何もしない矢印です。

可換図式（*commutative diagram*）は、圏論でよく使う絵です。

たとえば三角形の図式で、$x \to y \to z$ とたどった合成 $g \circ f$ と、近道 $h: x \to z$ が等しいとき、「この三角形は可換である」と言います。

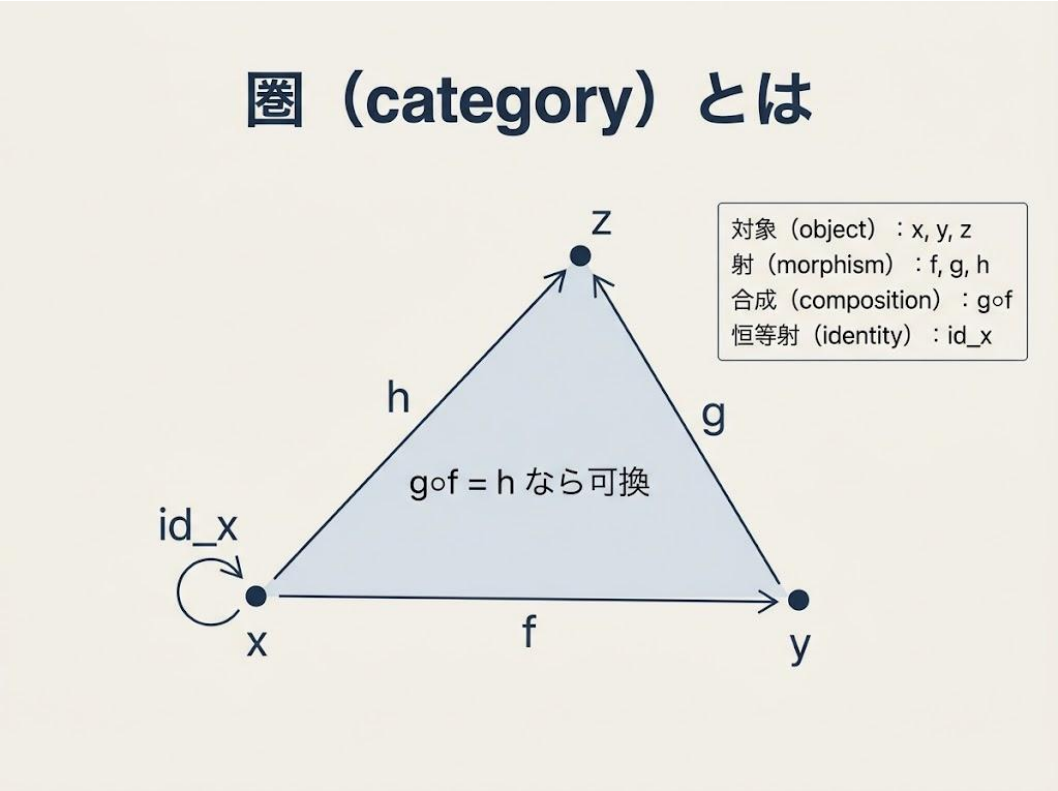
```
        y
   f ↗    ↘ g
  x ――――→ z
        h

可換である　⇔　g∘f = h
```

「どの道を通っても結果が同じ」ことを、絵で表したものが可換図式です。圏論の議論は、多くの場合、可換図式をつなぎ合わせることで進みます。

> 【折り畳みコラム：圏の正確な定義】圏 C は、対象の集まり ob(C) と、対象の各組 (x, y) に対する射の集まり C(x, y)、合成写像 C(y, z) × C(x, y) → C(x, z)、恒等射 id_x ∈ C(x, x) からなり、結合律と単位律を満たすもの。

ここで「集まり」は集合とは限らない（集合全体の圏の対象は集合にならない）など、細かい注意がある。



2-2　射が、何段にも重なる

可換三角形の「等しい」に、注目してください。

集合と関数の世界では、$g \circ f = h$ は「すべての入力で値が一致する」という、はっきりした意味を持ちます。

等しいか、等しくないか、どちらか です。

しかし数学には、「ぴったり等しい」ではなく「ある意味で同じ」としか言えない場面がたくさんあります。

たとえば **位相空間** では、2 つの連続写像が「連続的に変形して移り合える」とき（ホモトピックであるとき）、それらを同じとみなしたいことがよくあります。

例えで言えば、輪ゴムで作った輪と、少し引っ張って楕円にした輪は、「連続的に変形できる」という意味で同じです。

けれども、どう引っ張ったかという「変形のしかた」は、1 通りではありません。

このとき、**「同じである」こと** 自体が、**変形という具体的なデータ**になります。

変形 は、**射と射のあいだの矢印** です。

これを **2-射**（*2-morphism*）と呼びます。

次元	名前	記号	意味
0 次元	対象	x, y	
1 次元	射	f, g : x → y	
2 次元	射のあいだの射	α : f ⇒ g	（f を g に変形する）
3 次元	2-射のあいだの射	Θ : α ⇛ β	（変形を変形する）
…	以下、無限に続く		

変形のしかたが 2 通りあれば、その 2 つの変形が「同じか」がまた問題になり、3-射が必要とされます。

これが際限なく続きます。

すべての次元に射を持つ構造を、**∞-圏**（*∞-category*）と呼びます。

ここで大事なのは、ふつうの圏の規則（結合律、単位律）も、**∞-圏** では **「ぴったり成り立つ」のではなく、「1 つ上の次元の射によって成り立つ」こと** です。

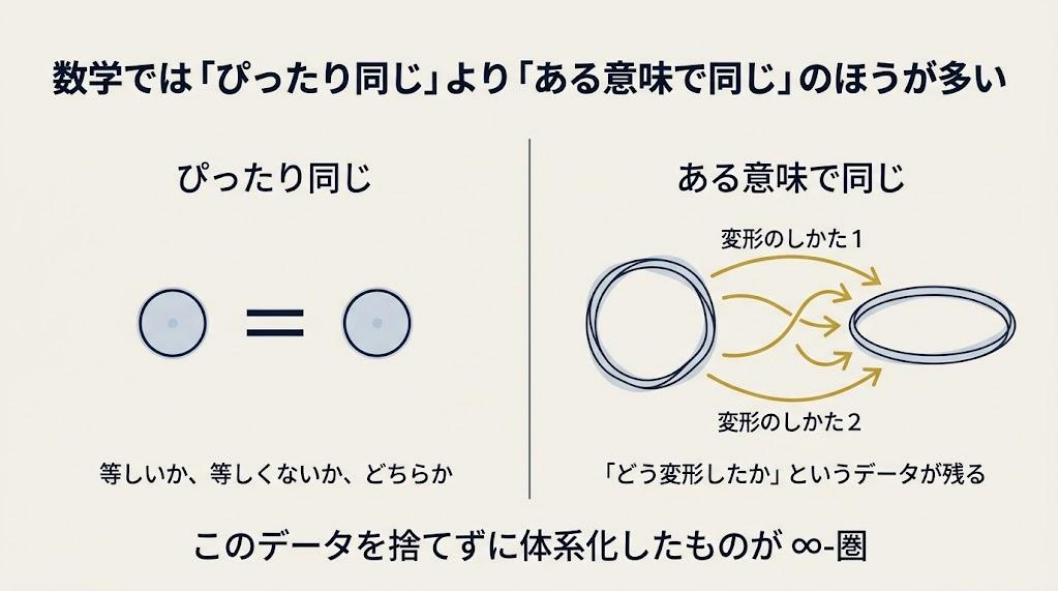
$h \circ (g \circ f)$ と $(h \circ g) \circ f$ は、**等しいのではなく、2-射でつながっています。**

そして、**そのつながり方どうしの整合性が、さらに上の次元の射で保証されます。**

この **「整合性の整合性の…」を正確に扱うことが、∞-圏論の難しさの源** です。

2-3　なぜ、そんな構造を考えるのか

理由を一言で言えば、**数学では「ぴったり同じ」より「ある意味で同じ」のほうが多い**からです。

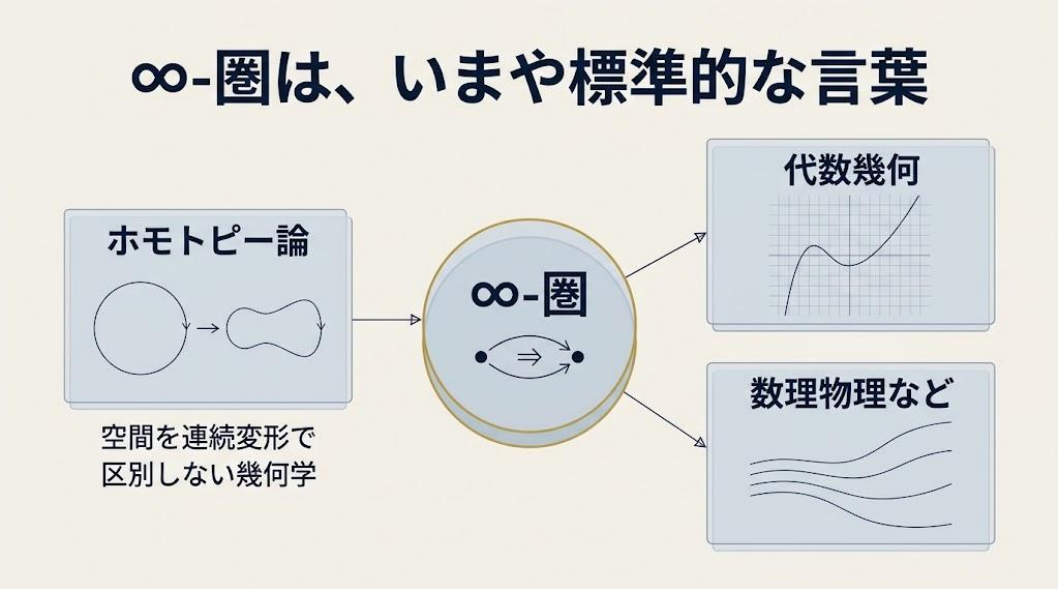


「ある意味で同じ」を正確に扱おうとすると、「どういう意味で同じか」というデータを捨てずに持っておく必要があります。

捨てると、情報が失われて、計算が合わなくなることがある からです。

そのデータを捨てずに体系化したものが、∞-圏 です。

∞-圏 は、**ホモトピー論** （空間を連続変形で区別しない幾何学）、**代数幾何** 、**数理物理など** で、いまや **標準的な言葉** になっています。



「データを捨てるとどうなるか」を、もう少し具体的に見ておきます。

空間と連続写像の圏で、「ホモトピックな写像を同じとみなす」という操作をすると、ふつうの圏が 1 つ得られます。

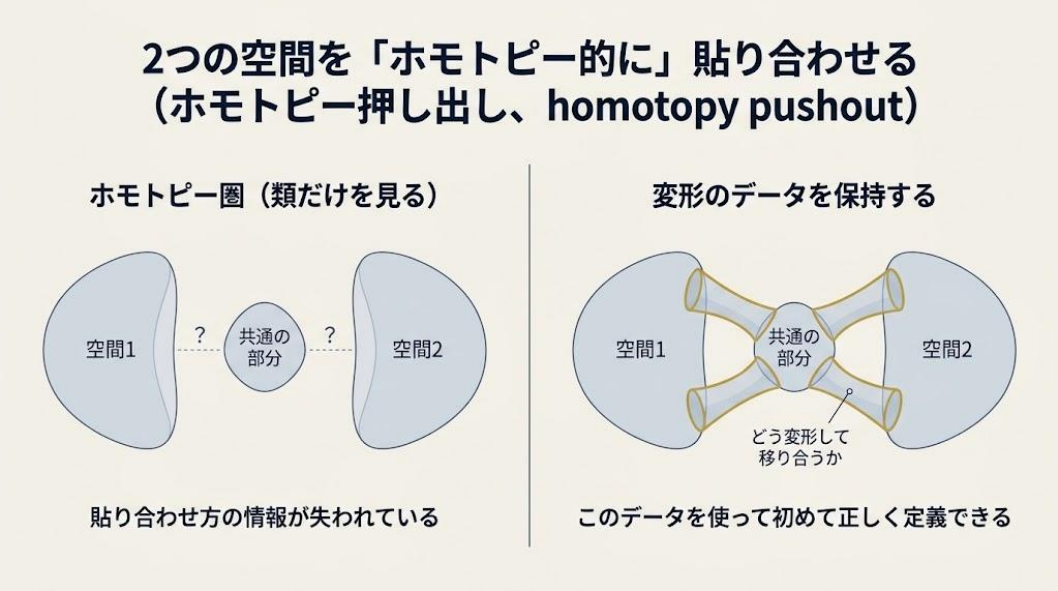
これを　「**ホモトピー圏**」と呼びます。**射** は「**変形で移り合う写像の類**」です。

一見、これで十分に見えます。

ところが、**ホモトピー圏** では、いくつかの基本的な構成がうまくいきません。

たとえば、2 つの空間を共通の部分で「ホモトピー的に」貼り合わせる構成（ホモトピー押し出し、*homotopy pushout*）は、「**どう** 変形して移り合うか」というデータを使って初めて正しく定義できます。

類だけを見ていては、貼り合わせ方の情報が失われているのです。



扱い方	変形のデータ	結果
ふつうの圏として扱う	変形のデータを捨てる	→ 一部の構成が正しくできない
∞-圏として扱う	変形のデータを保持する	→ 構成が正しくできるただし、扱いが格段に難しくなる

∞-圏論は、**この「難しくなる」部分を体系的に引き受ける理論** です。

2-4　なぜ、∞-圏論は難しいのか

難しさの正体を、ここで整理しておきます。

∞-圏を、集合や関数から組み立てて定義しようとすると、「すべての次元の射」と「それらの整合性」を一度に指定する必要があります。

無限に続くデータを、有限の言葉で書き下すのは容易ではありません。

そこで数学者は、いくつかの工夫を考えました。

2-5 のコラムで紹介する **「準圏」** は、その代表です。

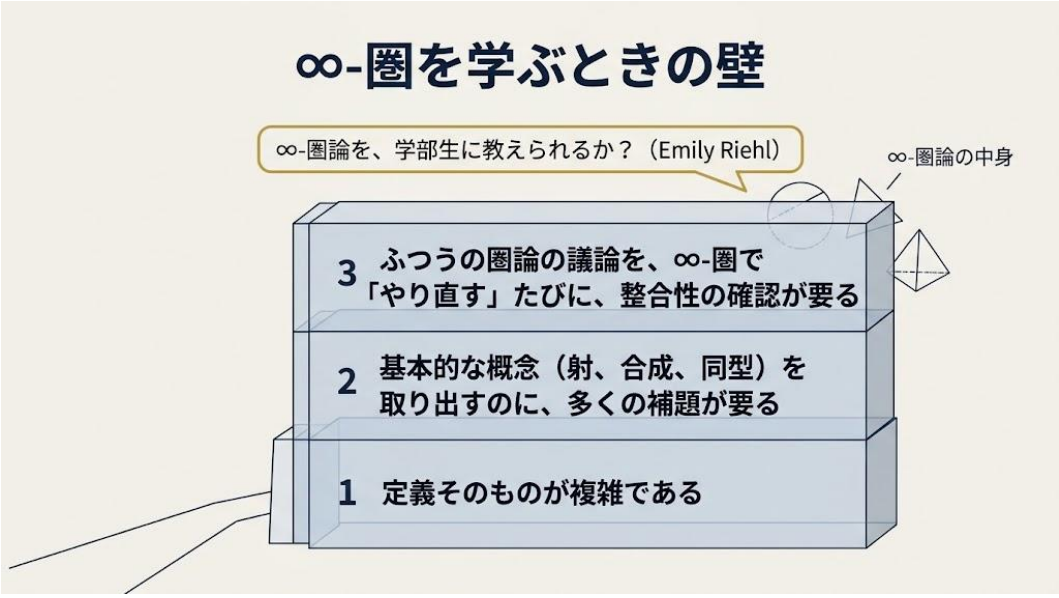
単体 という **図形の組合せ** で、**無限のデータをまとめて** 表します。

しかし、その代わりに、「射とは何か」「合成とは何か」を取り出すだけで、組合せ論的な議論が必要になります。

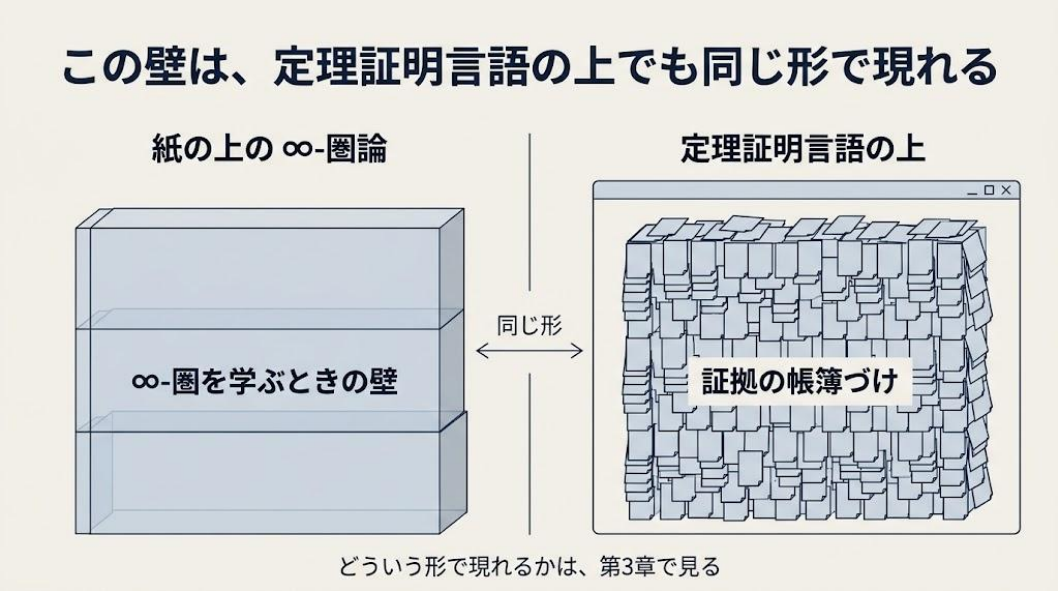
∞-圏を学ぶときの壁

1. 定義そのものが複雑である
2. 基本的な概念（射、合成、同型）を取り出すのに、多くの補題が要る
3. ふつうの圏論の議論を、∞-圏で「やり直す」たびに、整合性の確認が要る

第 1 章で紹介した Riehl の問い「学部生に教えられるか」は、この壁を前にしたものです。



そして、次章で見るように、**この壁は定理証明言語の上でも同じ形で現れます。**



2-5　(∞,1)-圏と (∞,0)-圏

∞-圏には、もう 1 つ大事な区別があります。

射が逆向きにたどれるかどうかです。

- **(∞,0)-圏**
  - すべての射が可逆（逆向きの射を持つ）
  - ＝ ∞-亜群（∞-groupoid）
  - 空間の点、道、道の変形、… がこれにあたる
- **(∞,1)-圏**
  - 1 次元の射は、可逆とは限らない
  - 2 次元以上の射は、すべて可逆
  - 単に「∞-圏」と言えば、ふつうこれを指す

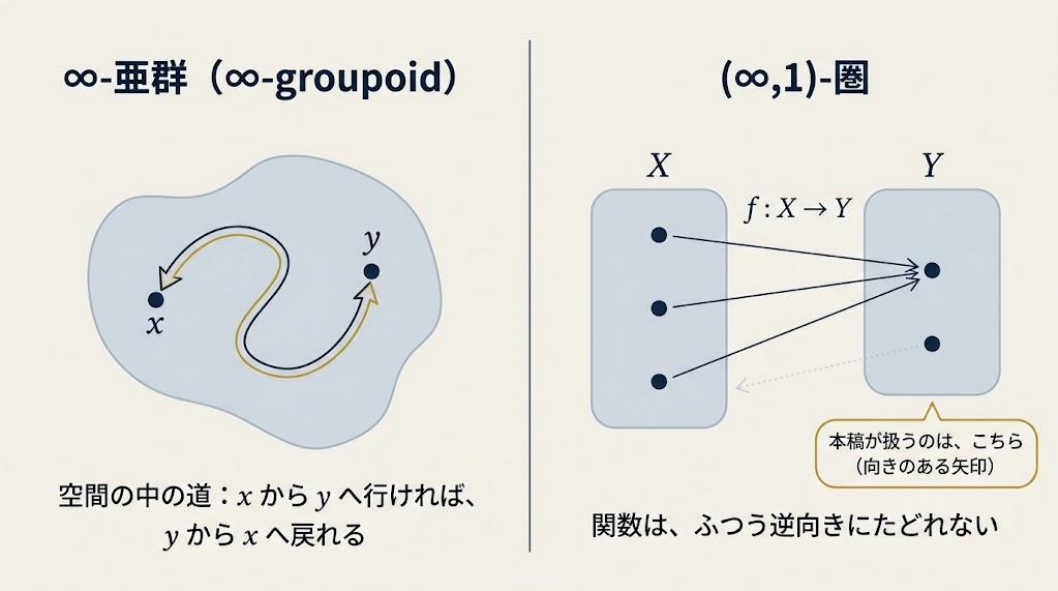
∞-亜群（*∞-groupoid*）では、**x から y へ行ければ、必ず y から x へ戻れます。**

空間の中の道が、その典型です。道は逆向きにたどれます。

一方、**関数 $f: X \to Y$** は、**ふつう逆向きにたどれません。**

向きのある矢印を扱いたいなら、**(∞,1)-圏が必要** となります。

本稿が扱うのは、こちら です。



記号 (∞,1) の読み方も押さえておきます。

左の ∞ は「射が無限の次元まである」こと、右の 1 は「1 次元より上の射はすべて可逆」であることを表します。

(∞,0) なら、**0 次元より上、つまりすべての射が可逆** です。

> 【折り畳みコラム：∞-圏の標準的な定義の一例】
>
> (∞,1)-圏の代表的な定義の 1 つは「準圏」（*quasi-category*）である。
>
> 単体的集合（点・線分・三角形・四面体…を貼り合わせた組合せ論的な対象）のうち、すべての「内側の角」（inner horn）が単体に埋められるもの、と定義される。
>
> 第 4 章に出てくる inner horn Λ^2_1 は、この定義の最も小さな場合にあたる。
>
> この定義から「射」「合成」「同型」を取り出すには、相当の準備が要る。

2-6　なぜ、型理論で扱いたいのか

ここで、話は数学からプログラミング言語に移ります。

型理論（*type theory*）とは、**型と項（値）の規則を定めた形式体系** です。

Lean、Agda、Rocq の土台 になっているのは、Martin-Löf や Coquand と Huet らが発展させた **依存型理論**（*dependent type theory*）です。

依存型とは、型が値に依存できる型のことです。

たとえば「長さ n のリスト」の型 `Vec A n` は、値 n に依存します。

依存型理論 には、**等式型**（*identity type*）と呼ばれる型があります。

a と b が型 A の要素のとき、a = b という型があり、その要素は「a と b が **等しいことの証拠** 」です。

等式型もまた型 なので、**「等しさの証拠どうしの等しさ」を表す型** も作れます。これが、後で重要になります。

論文は、この等式型の歴史を次のようにまとめています。

- **1994 年**
  - Hofmann と Streicher
  - 型を「亜群」として解釈するモデルを作り、「等しさの証拠はつねに 1 つしかない」という原理（UIP）が証明できないことを示した
  - → 2 つの等しさの証拠 p, q が、等しいとは限らない
- **その後**
  - ホモトピー型理論（HoTT）
  - 型を ∞-亜群として解釈する
  - Voevodsky の univalence 公理によって、型の等しさと型どうしの同値を結びつける原理が加わる
- **さらに**
  - キュービカル型理論
  - univalence を公理として足すと失われる
  - 「計算が必ず値まで進む」性質（標準性）を取り戻す

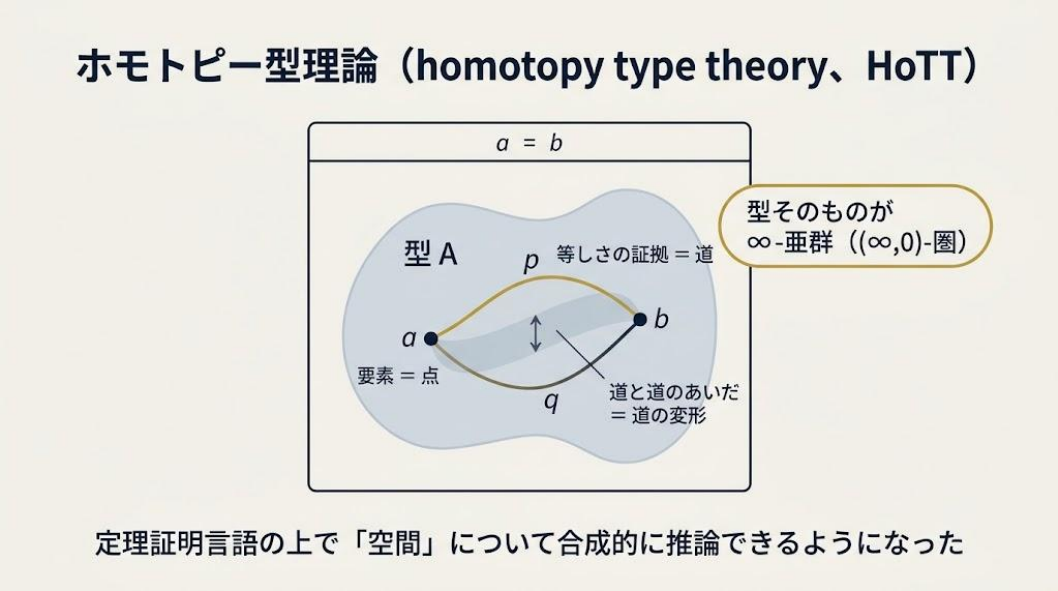
つまり、**等式型** は **「空間の中の道」のように振る舞う** のです。

型理論の側	空間の側
型 A	空間
要素 a : A	空間の点
a = b の要素	点 a から点 b への道
道の等式	道の変形
…	…

この見方を体系にしたものが、**ホモトピー型理論**（*homotopy type theory*、HoTT）です。

HoTT では、型そのものが ∞-亜群、つまり (∞,0)-圏として扱えます。

定理証明言語の上で「空間」について合成的に推論できるようになった のです。



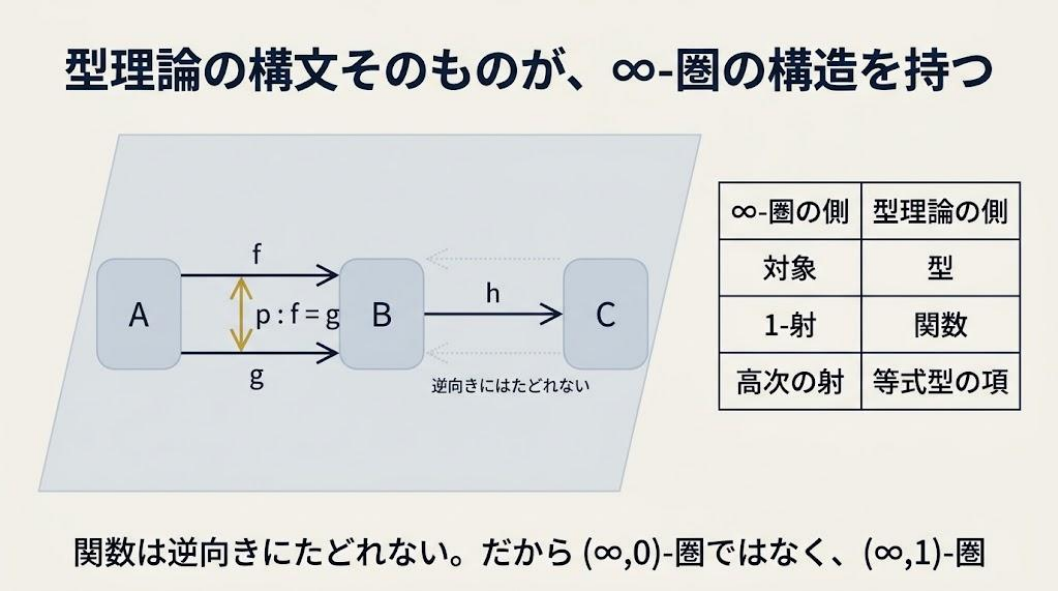
Rzk の論文 は、さらに次の点を指摘しています。

型理論の構文そのものが、∞-圏の構造を持つというのです。

∞-圏の側	型理論の側
対象	型
1-射	関数
高次の射	等式型の項

関数は逆向きにたどれません。だから、ここに現れるのは (∞,0)-圏ではなく、(∞,1)-圏です。

型理論が自分自身を語ろうとすれば、(∞,1)-圏の言葉が要る、ということです。

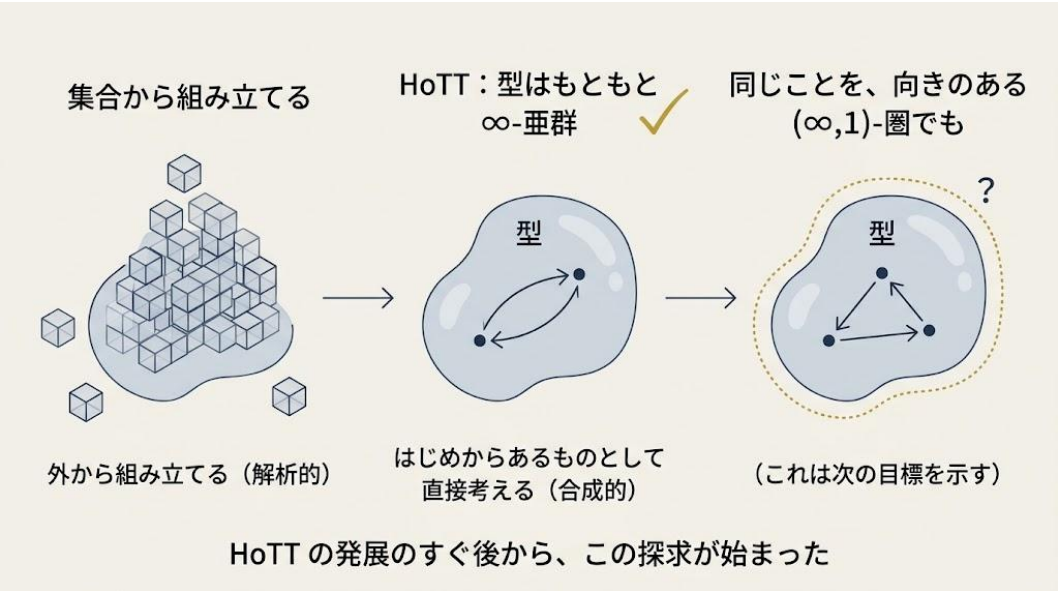


HoTT は ∞-亜群を **合成的に**扱えました。

つまり、∞-亜群を集合から組み立てるのではなく、「型は **もともと ∞-亜群である** 」という世界で **直接** 考えられました。

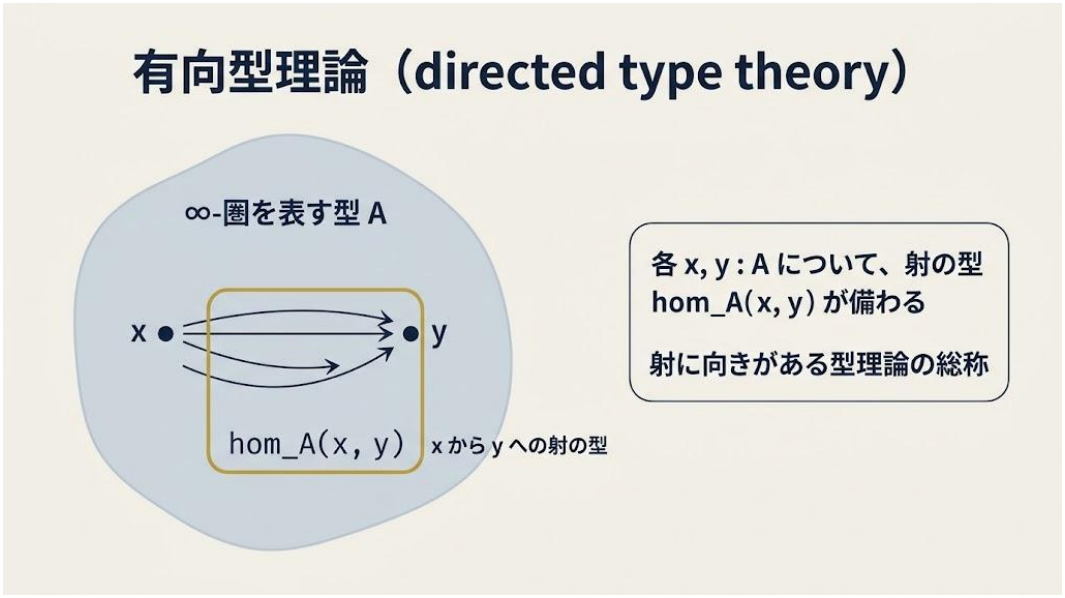
同じことを、向きのある (∞,1)-圏でもやりたい。

論文によれば、**HoTT の発展のすぐ後から、この探求が始まりました。**



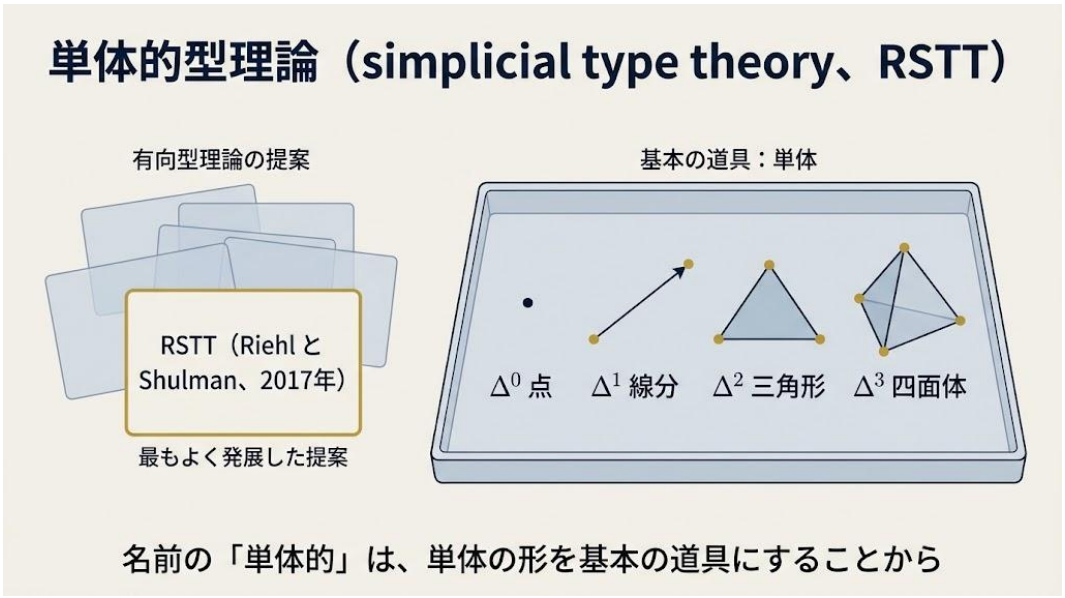
そのような型理論 では、**∞-圏を表す型** A には、各 $x, y : A$ について、**x から y への射の型 hom_A(x, y) が備わっているはずで** す。

射に向きがある型理論 を、総称して **有向型理論**（*directed type theory*）と呼びます。



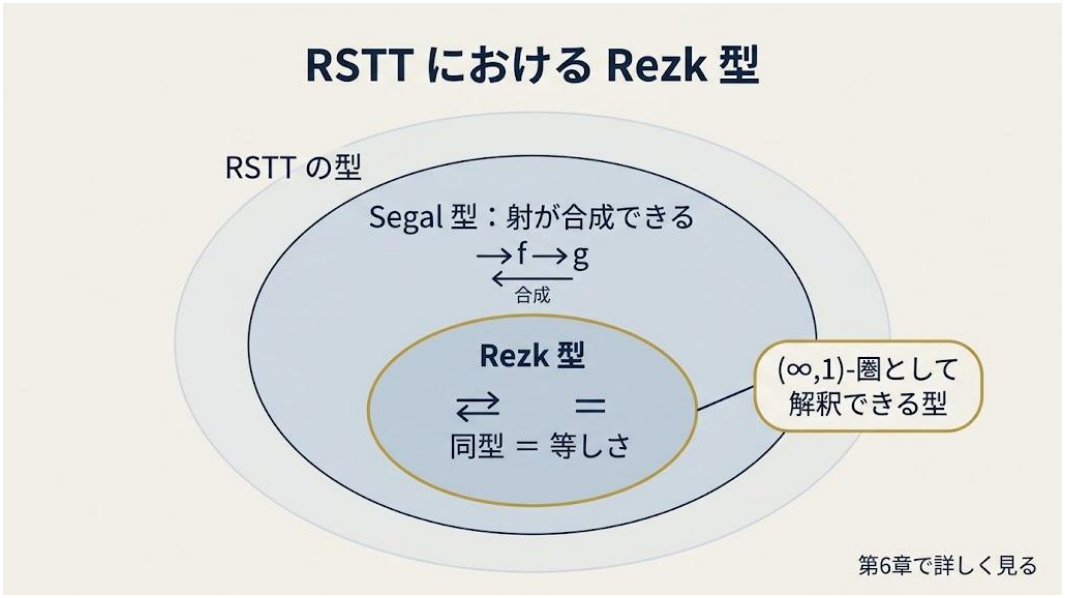
論文によれば、その提案のうち最もよく発展したものが、Riehl と Shulman の **単体的型理論**（*simplicial type theory*、論文では **RSTT** と略記）です。

名前の 「**単体的**」 は、**点・線分・三角形・四面体といった「単体」** の形を **基本の道具にすること** から来ています。



RSTT では、**(∞,1)-圏として解釈できる型** が **Rezk 型** です。

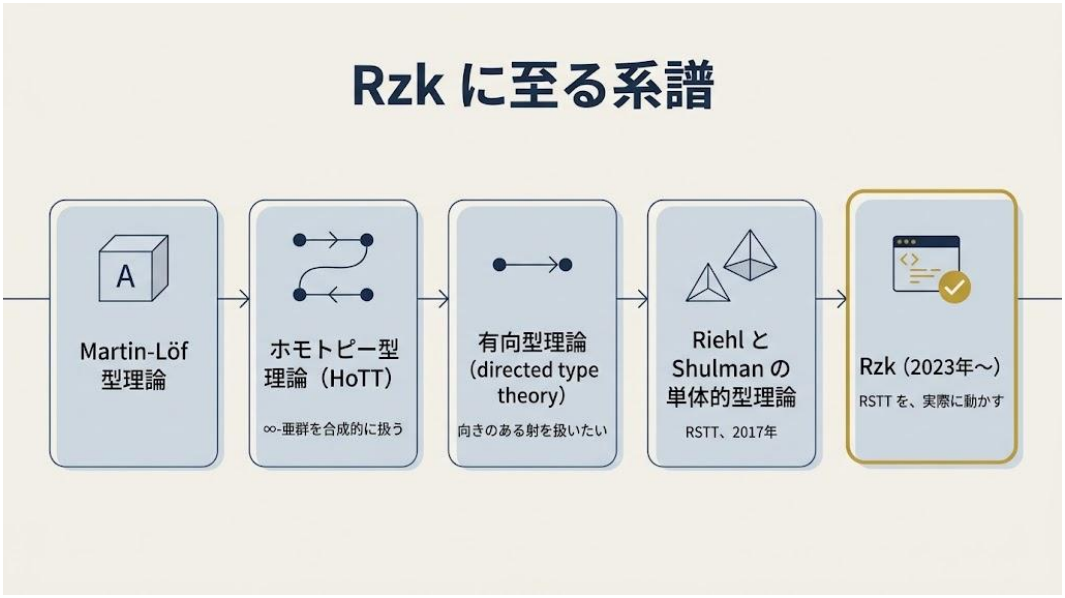
Rezk 型 は、**射が合成できる型（Segal 型）のうち、さらに同型と等しさが一致するもの** です。第 6 章で詳しく見ます。



Rzk は、この RSTT を実装した言語 です。



1. Martin-Löf 型理論
2. ホモトピー型理論（HoTT）：∞-亜群を合成的に扱う
3. 有向型理論（directed type theory）：向きのある射を扱いたい
4. Riehl と Shulman の単体的型理論（RSTT、2017 年）
5. Rzk（2023 年〜）：RSTT を、実際に動かす



有向型理論 には、**ほかにも提案** があります。

論文の **関連研究の節** では、**Weaver と Licata による有向 univalence のモデル（2020 年）** や、**Cisinski、Nguyen、Walde による意味論的な立場からの有向型理論の試み** が挙げられています。

RSTT は、その中で最も多くの数学が展開された理論です。

次章では、**RSTT** が **何を解決したのか** を見るために、まず、「汎用の言語で素朴に書くと何が起きるか」を確かめます。

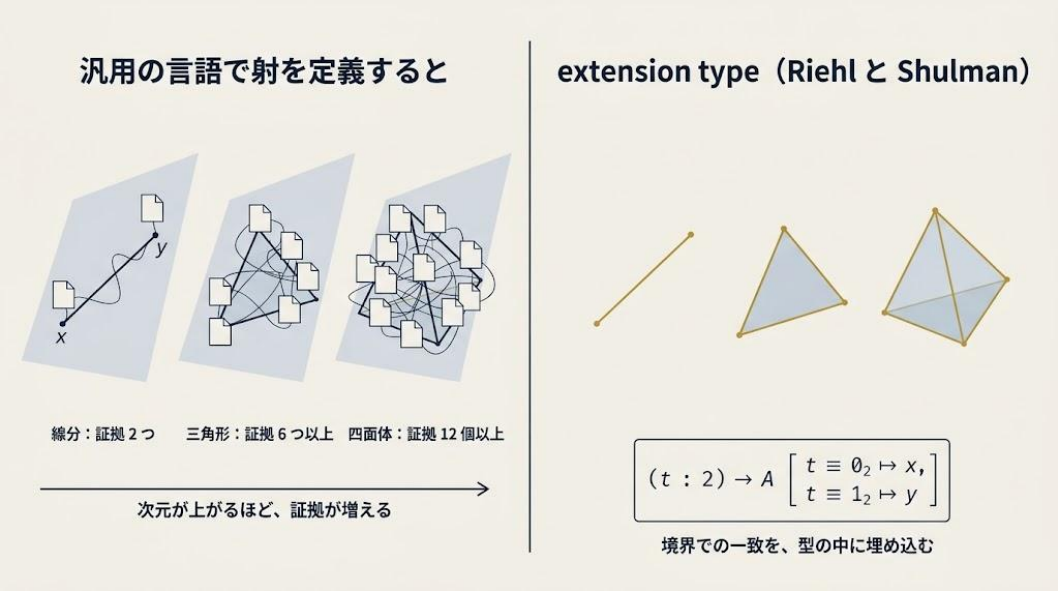
第 3 章　素朴に書くと、何が起きるか

この章の要点：

汎用の言語で **射** を定義すると、**端点についての証拠を持ち回ることになる。**

図形の次元が上がるほど、証拠は増えていく。

Riehl と Shulman は、**境界での一致を型の中に埋め込む extension type** で、**この問題を解いた。**



3-1　射の型を、素朴に定義する

HoTT の世界 で、型 A の中の「x から y への射」を定義したいとします。

射は、時間 0 から 1 までの道筋だと考えます。

そこで、0 から 1 までの **有向区間**（*directed interval*）を表す型 `2` を用意し、関数 `f : 2 → A` を射の候補とします。

$f(0)$ が始点、$f(1)$ が終点です。

ここでの型 `2` は、説明のために仮に置いた「有向区間の型」です。

ふつうの HoTT に、このような型が用意されているわけではありません。（第 4 章で見るとおり、Rzk では 2 は型ではなく、型とは別の層にある cube として扱われます。）

ただし、どんな f でもよいわけではありません。 始点が x、終点が y でなければなりません。

汎用の言語 でこれを表す、いちばん素直な方法は、**等しさの証拠を組にして持つ** ことです。

$$hom_A(x,y) :\equiv \Sigma_{f:2\to A}\big(x = f(0)\big) \times \big(f(1) = y\big)$$

記号の読み方を説明します。

- `Σ_{f : 2 → A} …`：「関数 f と、… の組」（ **依存ペア型**、Σ**型** ）
- `x = f(0)`：「x と $f(0)$ が等しいことの証拠」の型（ **等式型** ）
- `×`：「両方の組」

Haskell に慣れた方なら、次のような **レコード型** を想像すると近いでしょう（あくまで雰囲気を伝えるための擬似コードで、コンパイルはできません）。

ここでは、始点 x と終点 y も、型の引数として固定されているものとします。

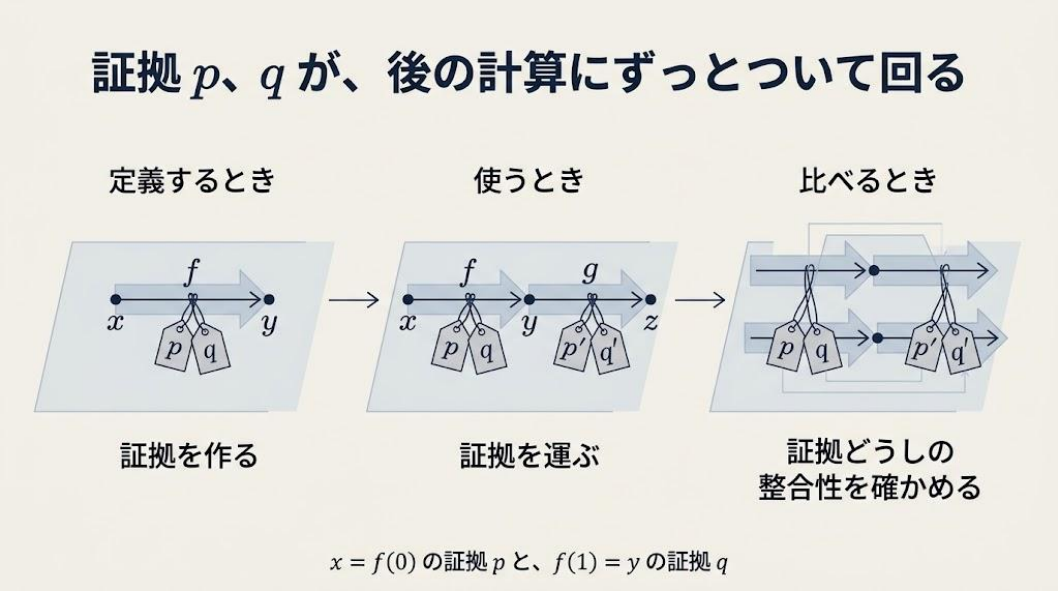
```
data Hom a x y = Hom
  { path      :: Interval -> a      -- 道筋そのもの
  , startProof :: Proof (x == path 0) -- 始点が x である証拠
  , endProof   :: Proof (path 1 == y) -- 終点が y である証拠
  }
```

この型の要素は、**3 つ組** です。

(射の候補となる関数 f, 出発点が x である証拠 p, 到着点が y である証拠 q)

3-2　証拠を持ち回ることになる

問題 は、この **証拠 p 、q が、後の計算にずっとついて回ること** です。

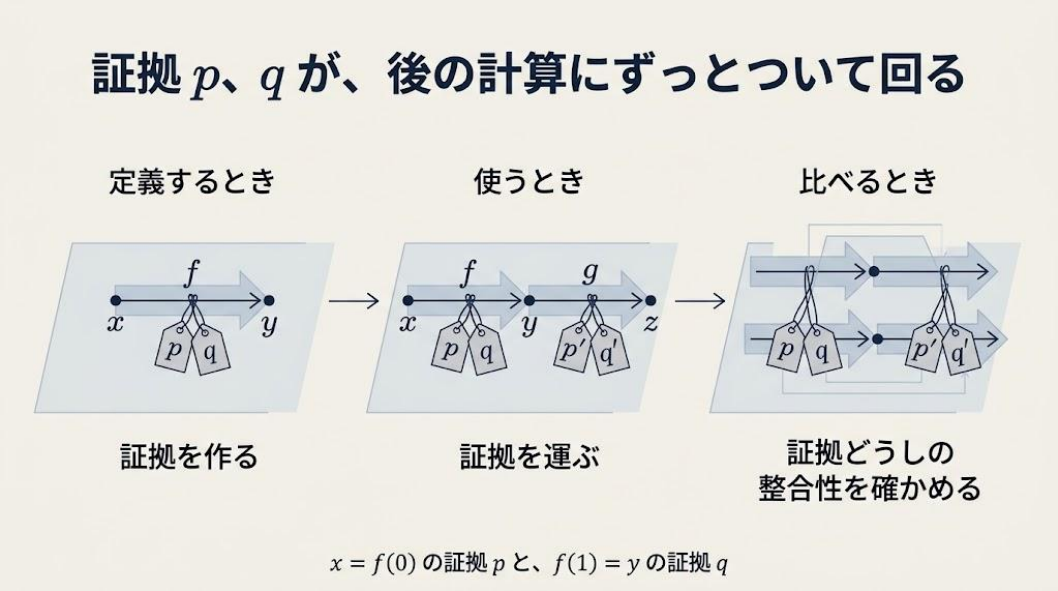


HoTT では、等式型 `x = f(0)` の要素は 「**道**」 です。

道には中身があります。

どの道を通って等しいのかによって、結果が変わりうる からです。

第 2 章で見た Hofmann と Streicher の結果が示すとおり、 **2 つの等しさの証拠は、同じとは限りません。だから、証拠は捨てられません。**



2 つの射 $(f, p, q) : hom(x, y)$ と $(g, p', q') : hom(y, z)$ をつなぐ場面を考えます。

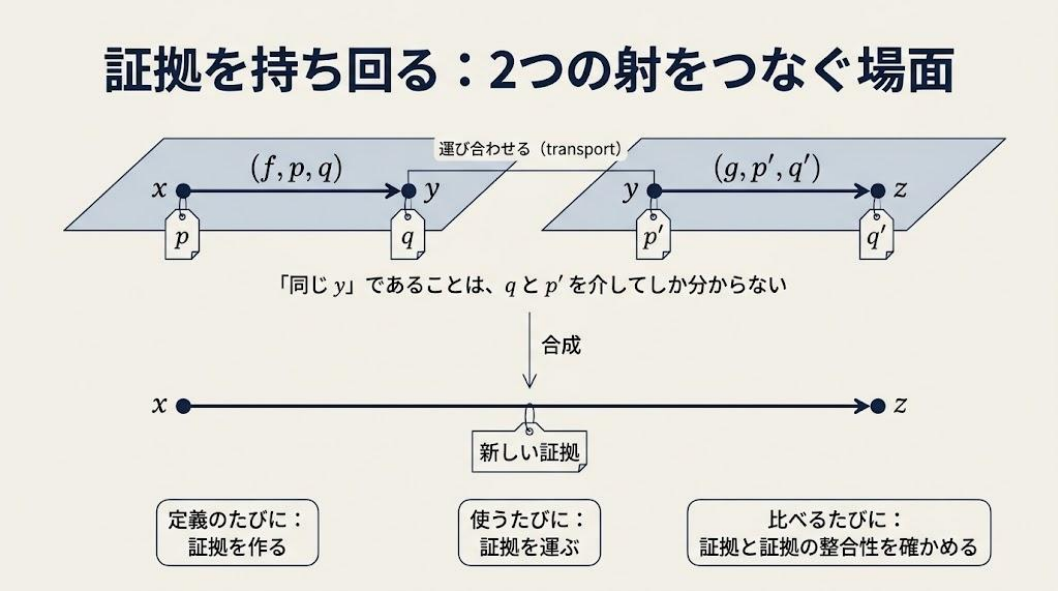
f の終点と g の始点が「同じ y」であることは、q と p' という 2 つの証拠を介してしか分かりません。

合成を定義する には、**その証拠を使って**、f の終点と g の始点を **運び合わせる**（*transport*） **操作が必要** です。

そして、合成した結果にも、**また新しい証拠が付きます。**

- **定義のたびに**：証拠を作る
- **使うたびに**：証拠を運ぶ
- **比べるたびに**：証拠と証拠の整合性を確かめる

これが、本稿で言う **「証拠を持ち回る」ということ** です。



Haskell にも、この **「運び合わせる」操作の小さな版** があります。

`Data.Type.Equality` モジュール には、「型 a と型 b が等しい」という証拠の型 `a :~: b` と、**その証拠を使って値を運ぶ関数** があります。

```
castWith :: (a :~: b) -> a -> b
```

「a と b **が等しい証拠があれば** 、a の値を b の値として使える」という **関数** です。

証拠がなければ、型検査器は a の値を b の値として受け入れてくれません。

だから、**証拠を持っている限り、使うたびに** `castWith` を挟むことになります。

素朴な hom の定義では、これと同じことが、**射を使うたびに** 起きます。

しかも Haskell の `:~:` と違い、HoTT の等しさの証拠は「中身のある道」なので、**どの証拠で** 運んだかによって結果が変わりうるのです。

> 【用語】**判定的等しさ**（*judgmental equality*）／**定義的等しさ**（*definitional equality*）とは、型検査器が **計算によって** 「同じだ」と自動で認める等しさです。
>
> たとえば `(\x -> x + 1) 2` と `3` は、計算すれば同じになるので、証拠なしに同じものとして扱われます。
>
> これに対して、等式型 `a = b` の要素として与える等しさを **命題的等しさ**（*propositional equality*）と呼びます。
>
> こちらは **証拠が必要** です。
>
> 素朴な定義の弱点は、端点の条件を命題的等しさでしか書けないことにあります。

3-3　可換三角形で、証拠が増える

次に、**可換三角形** を考えます。

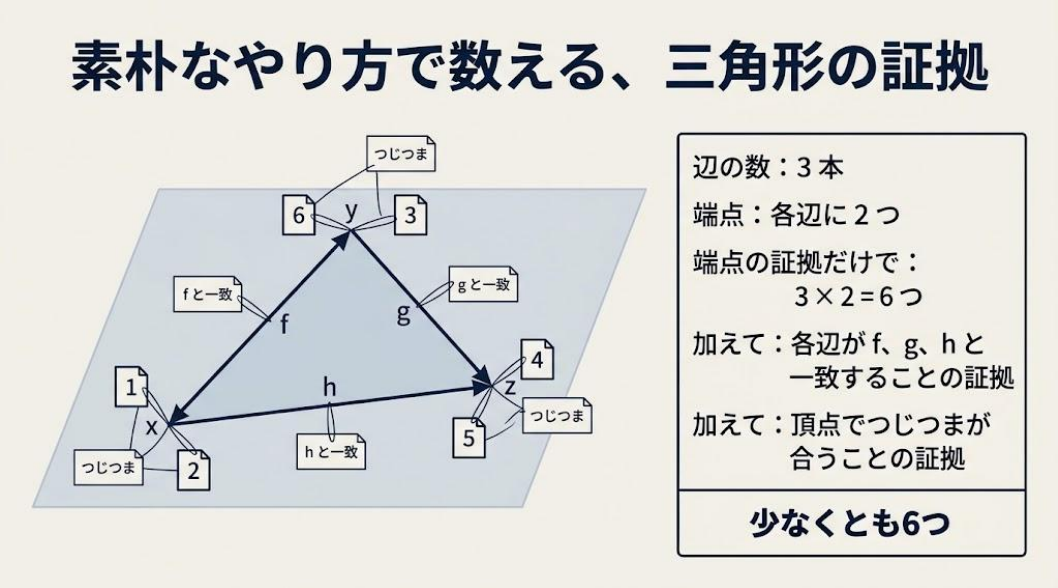
「h が f と g の合成である」ことを、三角形の形をした写像で表したいとします。

三角形には、辺が 3 本あり、それぞれの辺は 2 つの端点を持ちます。

素朴なやり方では、辺ごとに端点の証拠が要り、さらに三角形の各辺が f、g、h に一致する証拠も要ります。

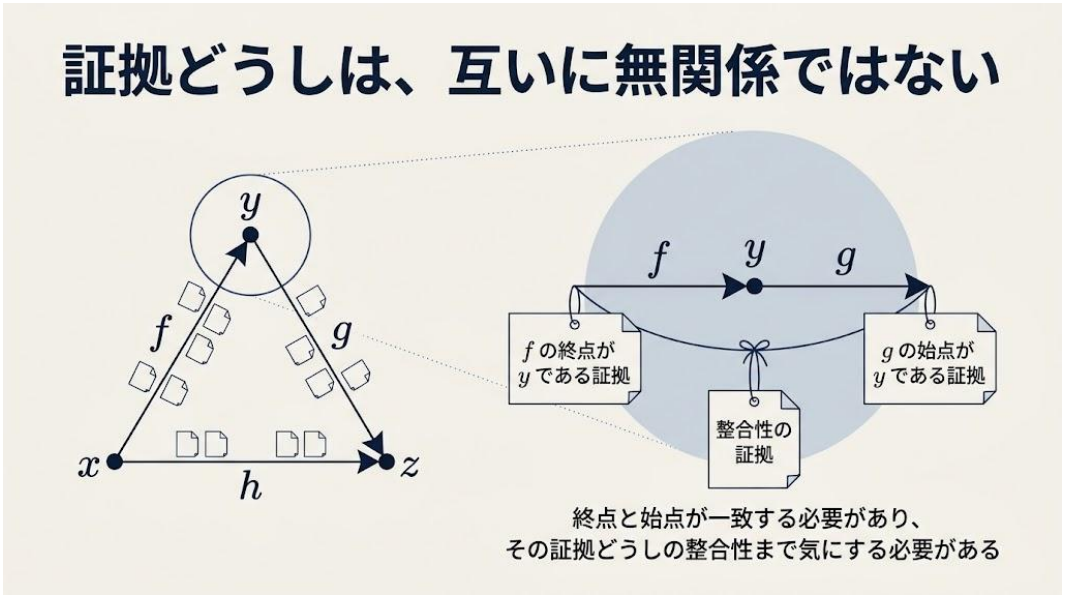
- **辺の数**：3 本
- **端点**：各辺に 2 つ
- **端点の証拠だけで**：3 × 2 = 6 つ
- **加えて**：各辺が f、g、h と一致することの証拠
- **加えて**：それらの証拠どうしが、頂点でつじつまが合うことの証拠

数え方にもよりますが、**端点の証拠だけで少なくとも 6 つ**になります。

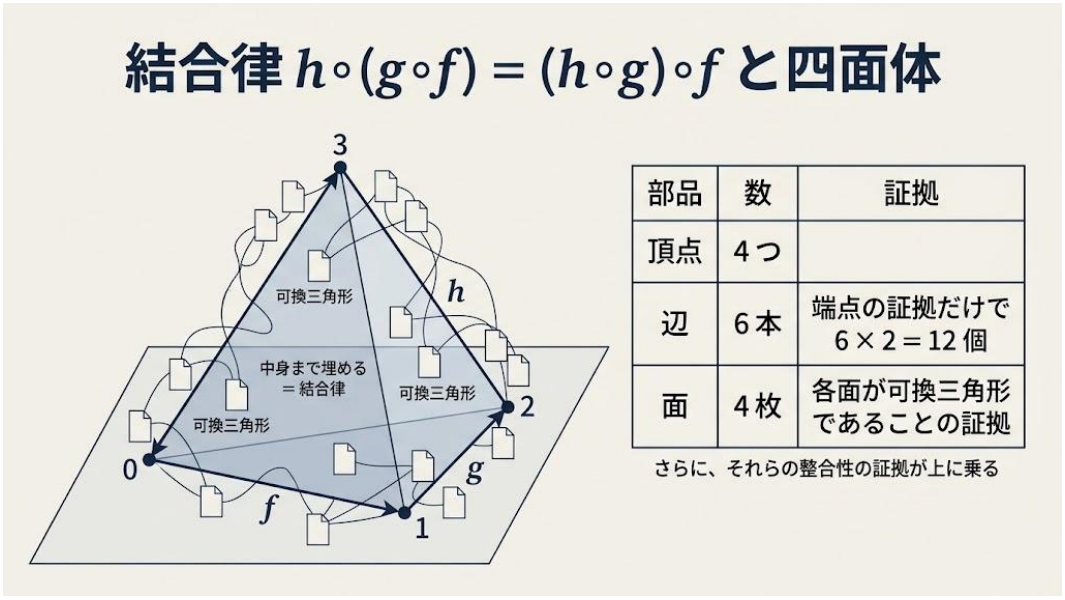


しかも、**これらは互いに無関係ではありません。**

頂点 y では、f の **終点** と g の **始点** が **一致する必要** があり、**その一致を示す証拠どうしの整合性** まで気にする必要があります。



3-4　可換四面体で、さらに増える



合成の結合律、$h \circ (g \circ f) = (h \circ g) \circ f$ を扱うには、4 つの頂点を持つ **四面体**（3 次元の単体）が要ります。

四面体の 4 枚の面は、**それぞれ可換三角形** です。

結合律 は、「4 枚の面がそろった四面体を、中身まで埋められる」こととして表されます。

部品	数	証拠
四面体の頂点	4 つ	
四面体の辺	6 本	端点の証拠だけで 6 × 2 = 12 個

部品	数	証拠
四面体の面	4 枚	各面が可換三角形であることの証拠

端点の証拠だけで 12 個です。

面の証拠と、**それらの整合性の証拠** が、さらに上に乗ります。

3-5　∞-圏では、これが際限なく続く

∞-圏では、四面体で終わりません。

点、線分、三角形、四面体、その先の高次元の図形が、すべての次元で現れます。

こうした図形の列を **単体**（*simplex*）と呼び、n 次元の単体を Δ^n と書きます。

記号	形
Δ^0	点
Δ^1	線分
Δ^2	三角形
Δ^3	四面体
Δ^4	…（4 次元の単体）

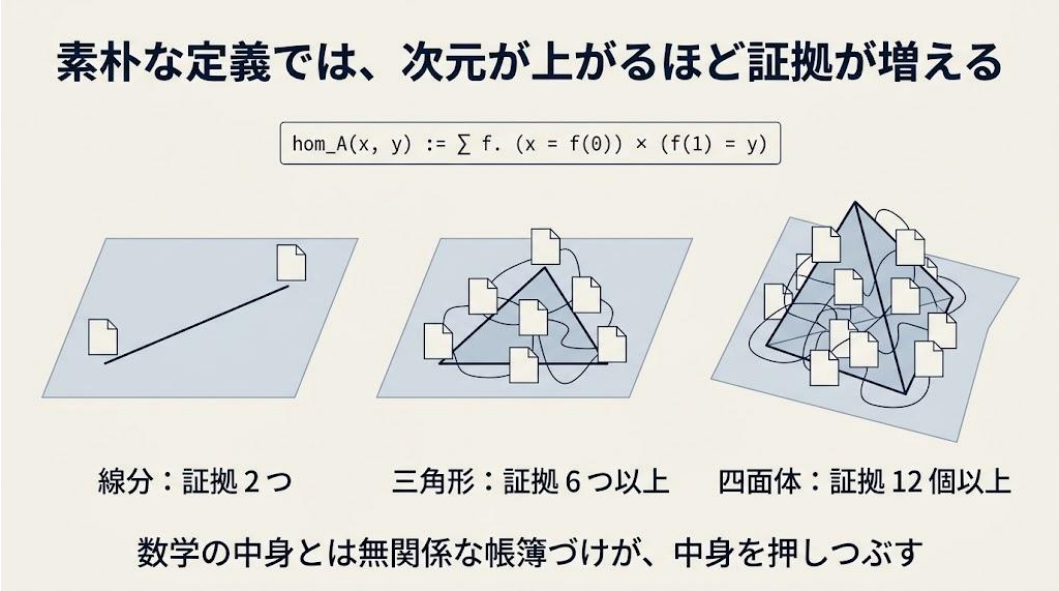
次元が上がるたびに、境界の部品が増え、証拠が増え、証拠どうしの整合性の証拠が増えます。

Riehl と Shulman は、2017 年の論文で、この状況をこう書いています（筆者訳）。

> これはきわめて煩雑であり、可換三角形、まして可換四面体を定義する段になると、技術的な困難がほとんど乗り越えられないものになる。

問題は、∞-圏の数学が難しいことではありません。

数学の中身とは無関係な「証拠の帳簿づけ」が、中身を押しつぶしてしまうことです。



3-6　Riehl と Shulman の解決

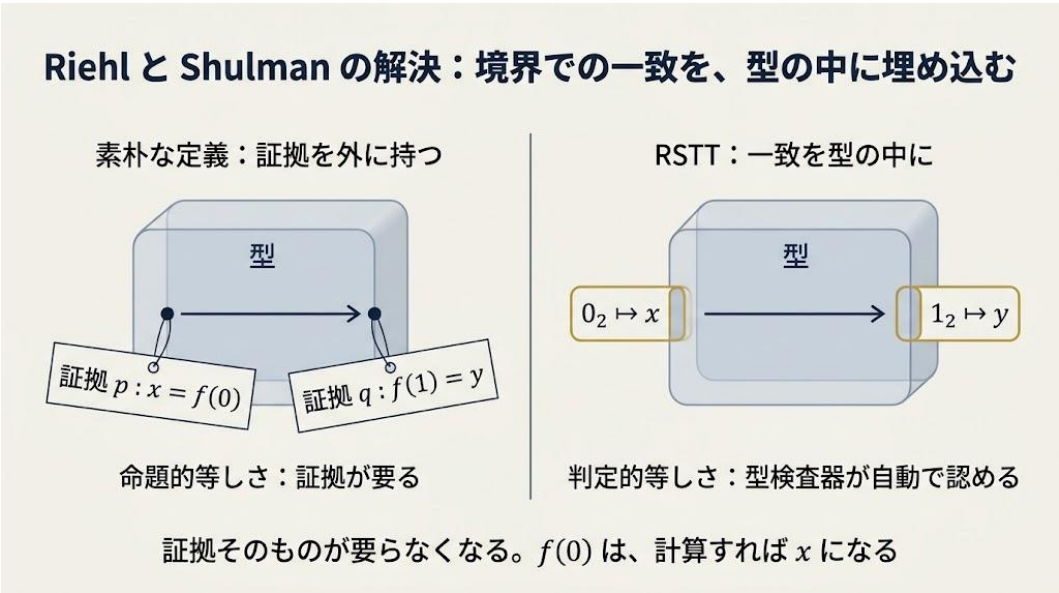
Riehl と Shulman の解決は、**発想の転換** でした。

- **素朴な定義**：境界での一致を、証拠として持つ
- **RSTT**：境界での **一致を、型の中に埋め込む**

「$f(0)$ は x である」を、**証拠の要る命題的等しさ** ではなく、**型検査器が自動で認める判定的等しさ** にしてしまうのです。

そうすれば、**証拠そのものが要らなくなります。**

$f(0)$ は、**計算すれば x** になるからです。

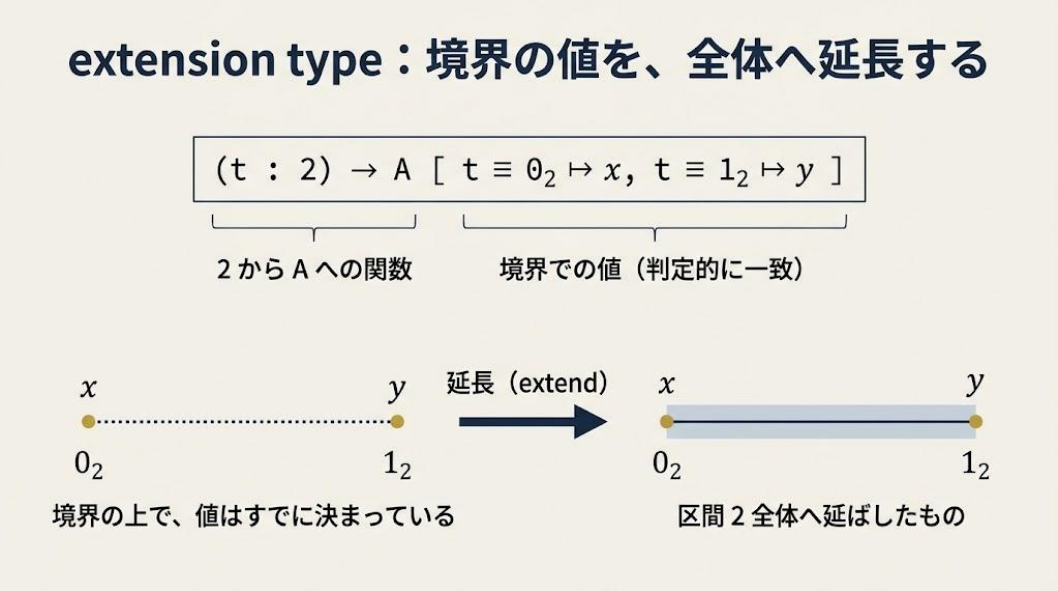


これを実現する型 を、**extension type**（*extension type*、定訳は確認できていないため、本稿では英語のまま使います）と呼びます。

$$(t{:}2) \to A\big[t \equiv 0_{_2} \mapsto x, t \equiv 1_{_2} \mapsto y\big]$$

> 「2 から A への関数で、しかも t が 0_2 のところでは x、t が 1_2 のところでは y に判定的に一致するもの」の型

名前の由来は、「境界（0_2 と 1_2）の上ですでに決まっている値 x、y を、全体（区間 2 全体）へ **延長**（*extend*）したもの」だからです。



なぜ判定的にすると、整合性の問題まで消えるのでしょうか？

命題的な等しさ の証拠は、データです。

データだから、「**どの証拠か**」**が問題になり、証拠どうしの整合性を確かめる必要** が生じました。

これに対して、**判定的な等しさ** は、型検査器が **計算で確かめる「事実」** です。

$f(0)$ を計算すると x になる、という **事実には、選び方も中身もありません。**

- **命題的な等しさ**
  - 証拠というデータがある
  - → どの証拠かが問題になる
  - → 証拠どうしの整合性が問題になる
- **判定的な等しさ**

- 計算で確かめる事実にすぎない
- → 選び方がない
- → 整合性の問題が、そもそも起きない

可換三角形の頂点 y で、「f の終点」と「g の始点」がどちらも y **に計算されるなら、それで終わり** です。

両者をつなぐ証拠を探す必要も、その証拠の整合性を確かめる必要もありません。

次元が上がっても、この事情は変わりません。

Rzk の論文 は、**この利点** をこう要約しています。

境界の条件が **判定的に成り立つ** ので、証明の中で **命題的な帳簿づけを一切必要としない**、と。

> 【補足コラム：extension type の規則】RSTT の論文は、extension type について 6 つの規則を定めています（Rzk 論文の図 9 にまとめられています）。ここでは雰囲気だけを伝えます。
>
> **形成**：形 ψ と、それに含まれる形 φ、ψ の上の型 A、φ の上の値 a があれば、extension type が作れる。
>
> **導入**：ψ の上の関数 b を書き、それが φ の上で a と判定的に一致すれば、b はこの型の要素になる。
>
> **適用**：この型の要素 f を、ψ の点 s に当てると、A の要素 f(s) が得られる。
>
> **β**：関数を書いてすぐ当てた結果は、本体に s を代入したものと同じ。
>
> **境界の計算**：s が φ に含まれるなら、f(s) は a(s) と判定的に等しい。この規則が、extension type の要です。
>
> **η**：f は、「s を受け取って f(s) を返す関数」と同じ。
>
> 第 8 章で見るとおり、Rzk はこれを 2 つの部品に分け、6 つの規則がすべて Rzk の中で成り立つことを、論文の付録で確かめています。

3-7　これが、Rzk の核になった

extension type は、RSTT の最も特徴的な道具 です。

そして、**Rzk の核**でもあります。

ただし、**RSTT** は **紙の上の理論** でした。

Riehl と Shulman の論文には、型理論の規則と、それを使った証明が書かれていますが、**それを機械で検査する処理系はありませんでした。**

紙の上の理論を動かすには、もう 1 つ壁があります。

紙の証明は、人間が読むことを前提に、いくつかの約束ごとや「黙った読み替え」を含んでいます。

型検査器は、それを補えません。 どこを補う必要があり、どう補ったのかは、第 8 章で詳しく見ます。

Rzk は、その処理系 です。

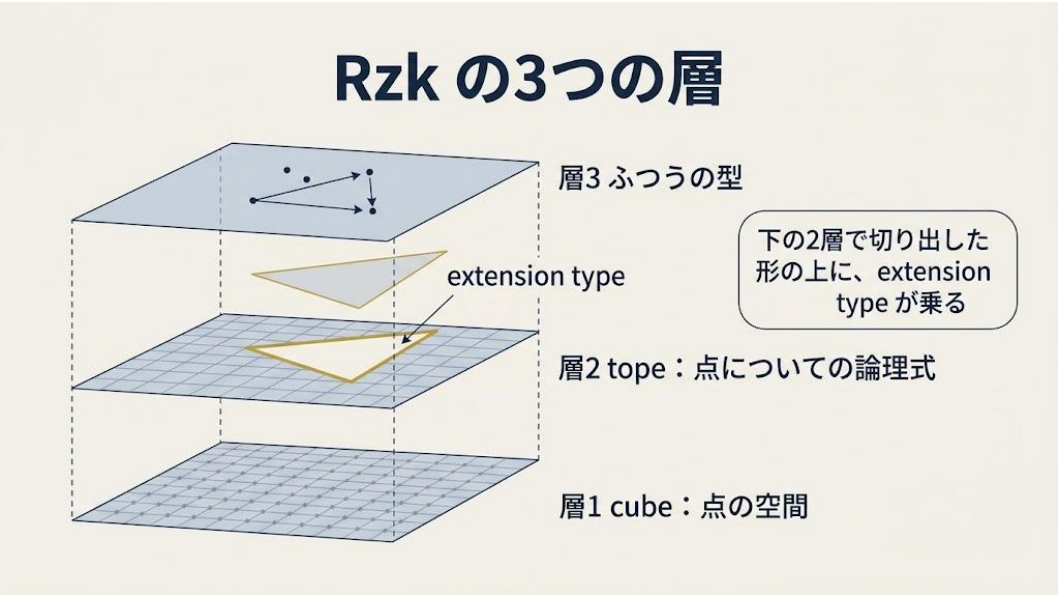
次章では、**Rzk が extension type をどういう仕組みで支えているか** を見ます。

> extension type そのものについては、別の記事で扱います。本稿は、その記事を読まなくても読めるように書いています。

第 4 章　Rzk の 3 つの層

この章の要点： **Rzk** は、**「点の空間（cube）」「点についての論理式（tope）」「ふつうの型」の 3 層** でできている。

下の 2 層で切り出した形の上に、**extension type が乗る** 。



4-1　Rzk は、3 層からできている

ふつうの依存型理論は、型と項だけで できています。

RSTT と Rzk は、その **下に 2 つの層を足しました。**

```
層 3  型の層       Martin-Löf 型理論（関数、Σ 型、等式型、宇宙 U など）
       ↑ 乗っている
層 2  tope の層    cube の点を制約する論理式
       ↑ 乗っている
層 1  cube の層    点の抽象的な空間
```

論文の言い方では、**RSTT は「3 層の型理論」** です。

下の 2 層は cube と tope からなる論理で、**3 層目は、その上に乗った Martin-Löf 型理論** です。

型検査器が確かめる主張を **判定**（*judgement*）と呼びます。

RSTT の判定は、3 つの文脈を持ちます。 論文の記法では、次のように書きます。

```
Ξ | Φ | Γ ⊢ J
```

記号	文脈	意味	例
Ξ（クサイ）	cube の文脈	どの点の変数があるか	t : 2
Φ（ファイ）	tope の文脈	どの条件を仮定しているか	t ≡ 0_2
Γ（ガンマ）	型の文脈	どの値の変数があるか	x : A
⊢	──	「〜のもとで、〜が成り立つ」	──
J	──	結論	──

論文に出てくる例を 1 つ読んでみます。

```
t : 2 | t ≡ 0 | x : A, y : A, f : hom_A(x, y) ⊢ f(t) ≡ x : A
```

「区間の点 t があり、t が 0 であると仮定し、A の要素 x , y と x から y への射 f があるとき、$f(t)$ は x と判定的に等しい」と読みます。

第 3 章で見た「$f(0)$ は計算すれば x になる」を、そのまま判定の形で書いたものです。

3 つの文脈が、それぞれ別の役割を担っていることが分かります。

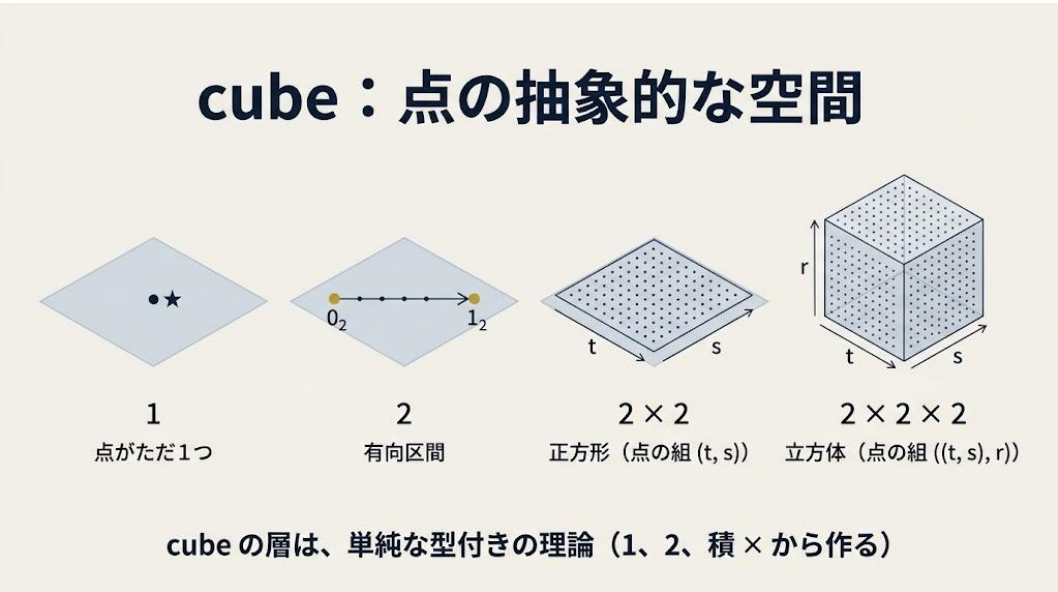
Rzk の **3 層構造** を、下から順に見ていきます。



4-2　cube とは何か

cube（*cube*、立方体。定訳は確認できていないため、本稿では cube と書きます）は、**点の抽象的な空間**です。

論文によれば、**cube の層** は、**単純な型付きの理論** です。



cube として使えるものは、**次の 3 種類から** 作られます。

cube	説明
`1`	終対象の cube。点がただ 1 つ（★）だけある
`2`	有向区間。2 つの特別な点 0_2 と 1_2 を持つ
`I × J`	2 つの cube の積。点は組 (s, t)

基本の cube は `2` です。

これは第 3 章で出てきた **有向区間** で、次の性質を持ちます。

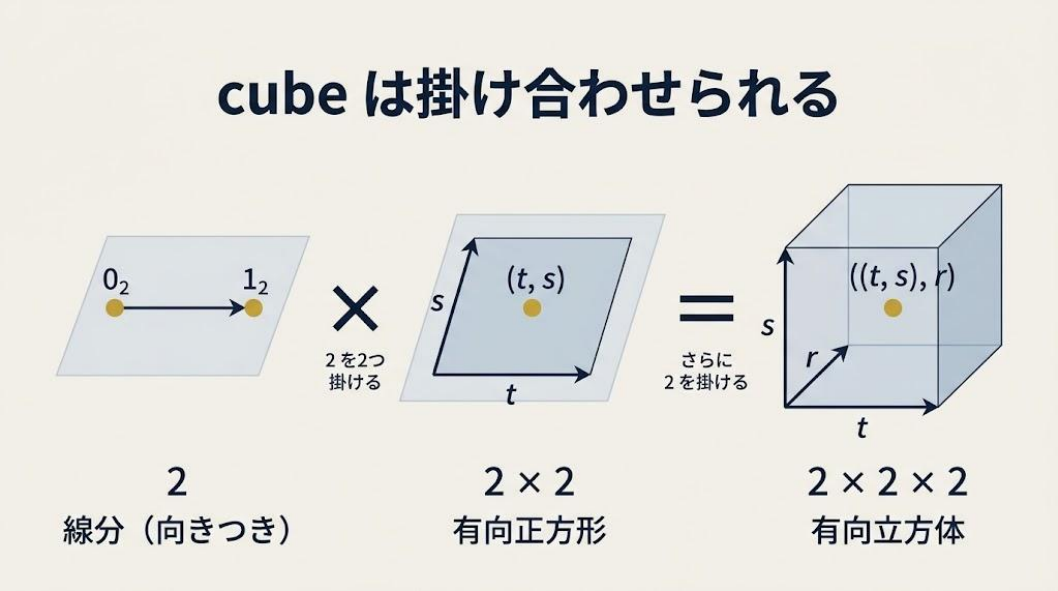
- **2 の点**：0_2（最小の点）、1_2（最大の点）、その間の点
- **順序**：全順序 ≤ を持つ。どの 2 点も比べられる
- **端点**：0_2 と 1_2 は異なる



cube は **掛け合わせられ** ます。

`2 × 2` は「点の組 (t, s)」からなる cube で、**有向正方形**にあたります。`2 × 2 × 2` は **有向立方体** です。

cube	形	点
`2`	線分（向きつき）	──
`2 × 2`	正方形	(t, s)
`2 × 2 × 2`	立方体	((t, s), r)



注意点が 2 つ あります。

1 つ目

cube は型ではありません。

2 の点 t を、ふつうの値のように引数として受け渡すことはできますが、2 は型の層には住んでいません。

cube は、図形の「座標」を与えるためだけの **別の層の存在** です。

では、**cube** は **どの層に** いるのでしょうか？

4-1 で見た 3 層のうち、**いちばん下の「層 1 cube の層」** です。

型 は **「層 3 型の層」** に住み、**その要素（値）** は **判定の型の文脈 Γ** に置かれます。

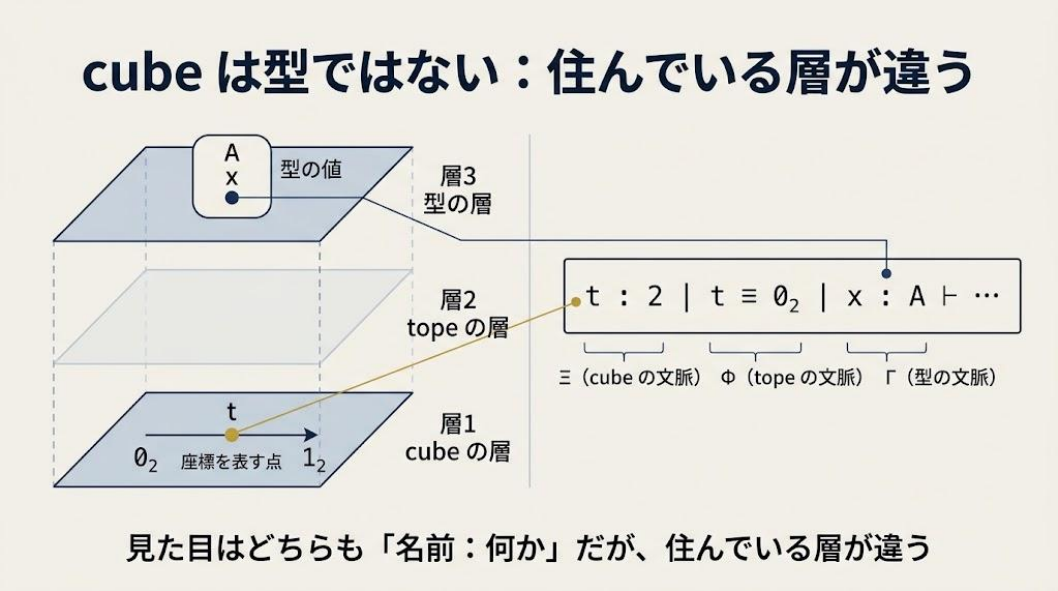
これに対して、**cube 2 の点 t** は、**cube の文脈 Ξ** に置かれます。

たとえば $t{:}\,2 | t \equiv 0_2 | x{:}\,A \vdash \cdots$ という **判定** では、

- $t{:}\,2$ は **Ξ の側**
- $x{:}\,A$ は **Γ の側**

に、**別々に** 書かれています。

見た目はどちらも「名前 : 何か」ですが、t は座標を表す点、x は型の値であり、住んでいる層が違う のです。

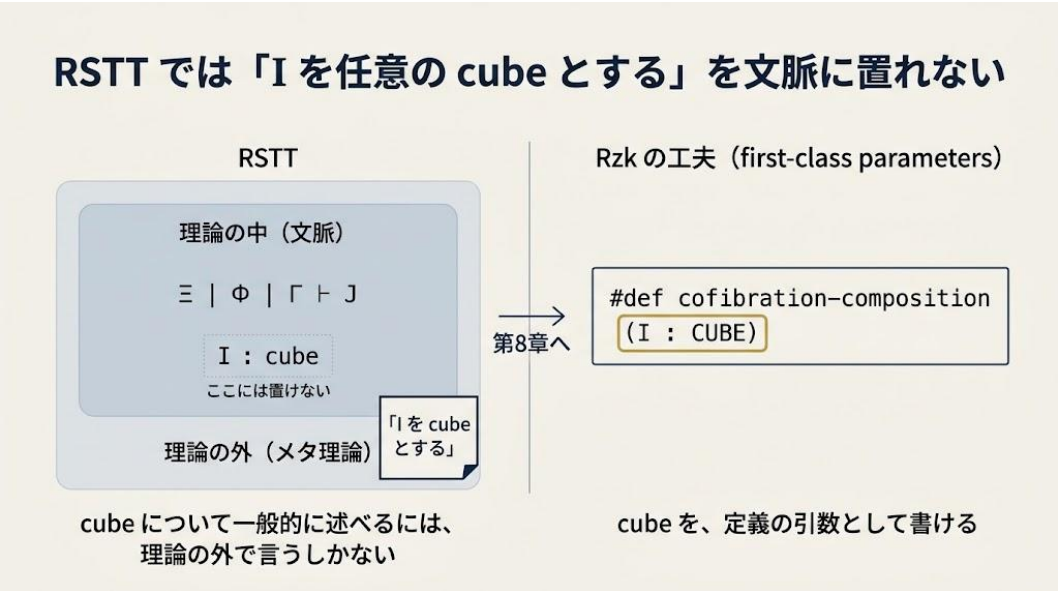


（注意点の）**2 つ目**

RSTT では、**文脈の中**で 「I **を任意の cube とする**」**と仮定することはできません** 。

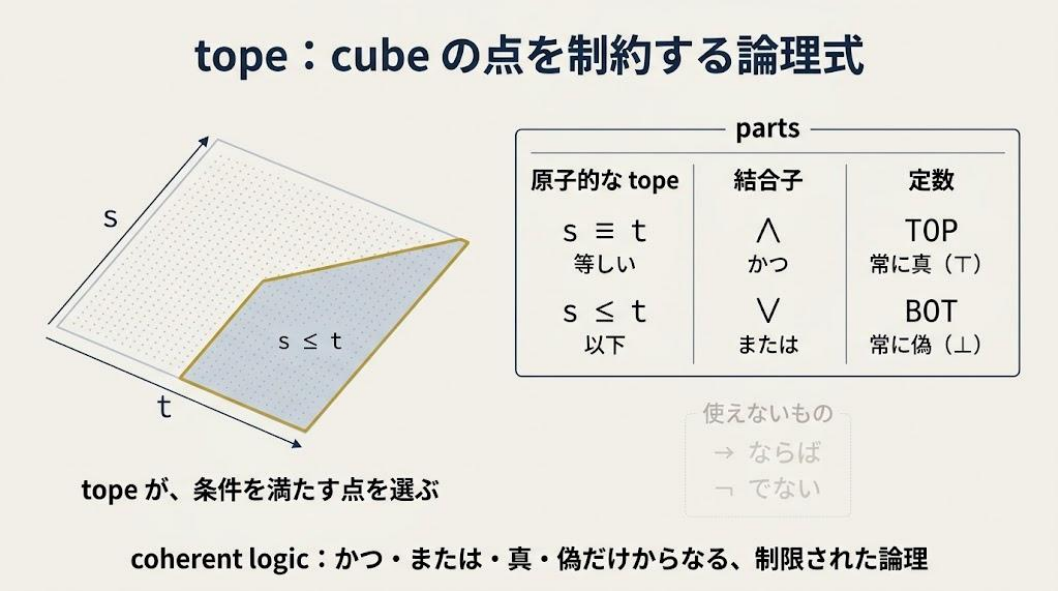
cube について **一般的に** 述べたいときは、**理論の外（メタ理論）で** 「I を cube とする」と言うしかありません。

この制約 が、第 8 章で見る **Rzk の工夫（first-class parameters）** につながります。



> 上図の右側は、第 8 章で扱う Rzk のコード `cofibration-composition` の引数 `(I : CUBE)` です。
>
> Rzk では、cube を定義の引数として直接書けます。

4-3　tope とは何か



tope（*tope*。定訳は確認できていないため、本稿では tope と書きます）は、**cube の点を制約する論理式**です。

原子的な tope

- `s ≡ t`：点 s と t が等しい
- `s ≤ t`：点 s が t 以下（2 の点どうし）

結合子

- `∧`：かつ
- `∨`：または

定数

- `TOP`：常に真（T）
- `BOT`：常に偽（⊥）

「ならば（→）」や「でない（¬）」はありません。

tope の論理は、かつ・または・真・偽だけからなる、制限された論理です 。

論文はこれを 「**coherent logic**」（**含意も否定も持たない論理**） と呼んでいます。

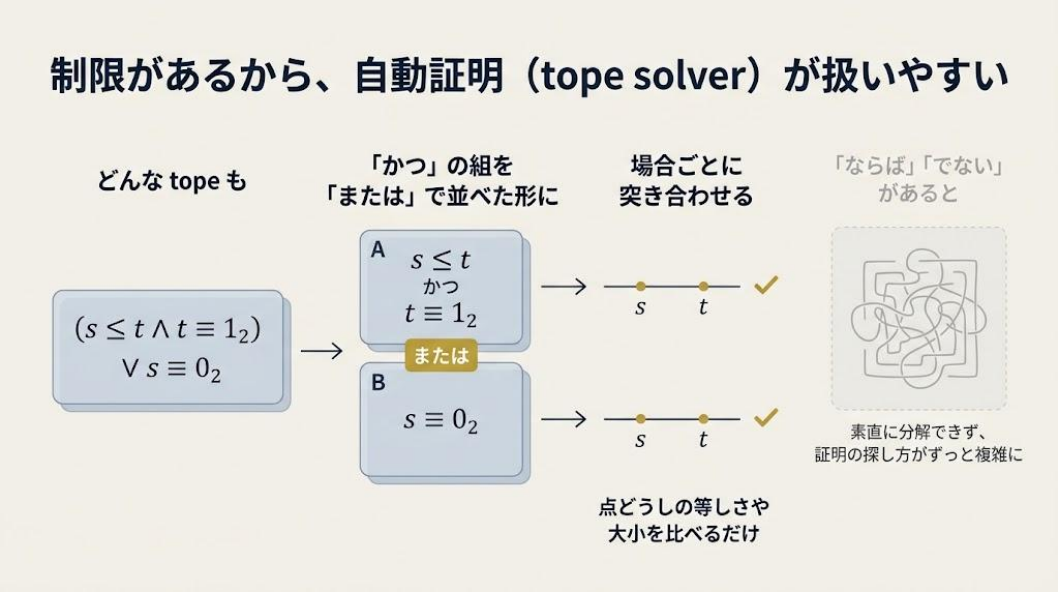
この制限 が、後で見る **自動証明（tope solver）を扱いやすくしています。**

なぜ　**制限** があると扱いやすいのでしょうか？

使える部品が「かつ」と「または」だけ なら、**どんな tope も「かつで結んだ条件」を「または」で並べた形に書き直すことができます。**

すると、**「この条件から、あの条件が導けるか」という問いは、場合ごとに「点どうしの等しさや大小を突き合わせる」作業に分解できます。**

「ならば」や「でない」があると、こうした素直な分解ができず、**証明の探し方がずっと複雑に** なります。



この分解を実際にどう行うか は、第 8 章 8-5 で見ます。

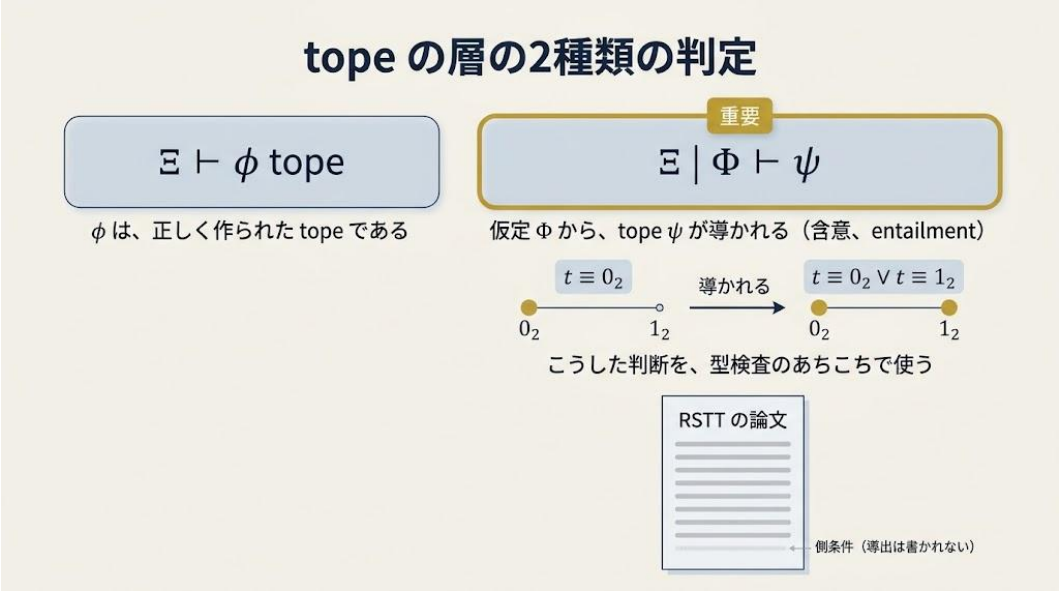
tope の層 には、**2 種類の判定** があります。

- Ξ ⊢ ϕ tope：φ は、点の文脈 Ξ のもとで正しく作られた tope である
- Ξ | Φ ⊢ ψ：仮定 Φ から、tope ψ が導かれる（含意、entailment）

2 つ目の **「導かれる」** が重要です。

「$t \equiv 0_2$ **ならば**、$t \equiv 0_2 \vee t \equiv 1_2$」のような **判断を型検査のあちこちで使う** からです。

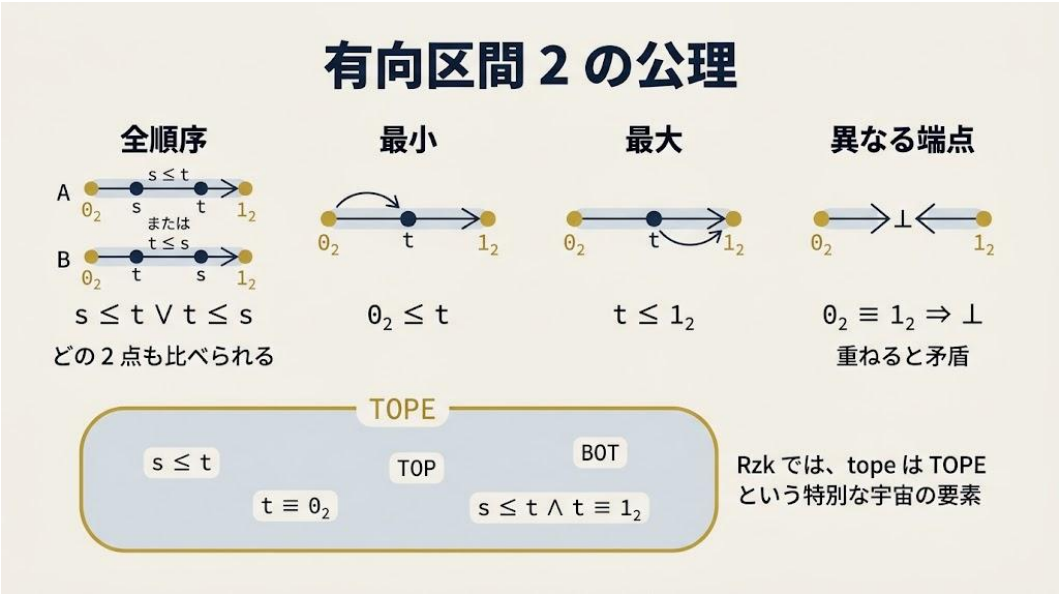
RSTT の論文 では、**こうした判断は「側条件」として扱われ、その導出は書かれずに済まされていました。**



有向区間 2 について、論文は次の **公理** を挙げています。

- **全順序**：どの 2 点 s, t についても $s \leq t \vee t \leq s$
- **最小**：どの t についても $0_2 \leq t$
- **最大**：どの t についても $t \leq 1_2$
- **異なる端点**：$0_2 \equiv 1_2$ から $\bot$（矛盾）が導かれる

Rzk では、**tope** は **`TOPE` という特別な宇宙の要素** として書きます。



4-4　shape（形）とは何か

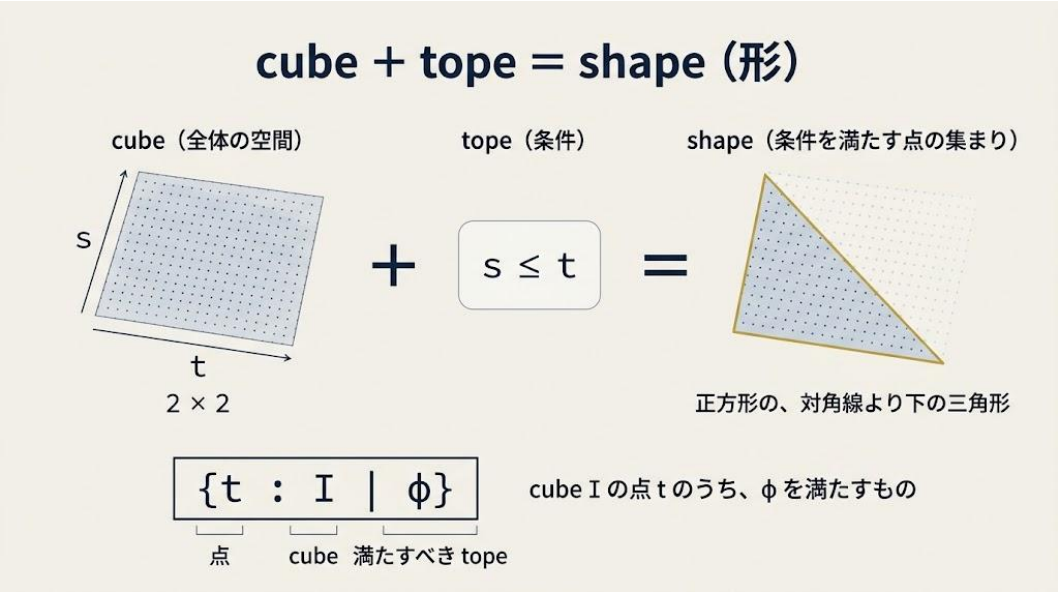
cube と **tope** を組み合わせると、**cube** の中から**形**が切り出されます。

```
cube（全体の空間）　＋　tope（条件）　＝　shape（条件を満たす点の集まり）

例　2 × 2　　＋　s ≤ t　　＝　正方形の、対角線より下の三角形
```

これを **shape**（*shape*、形）と呼びます。

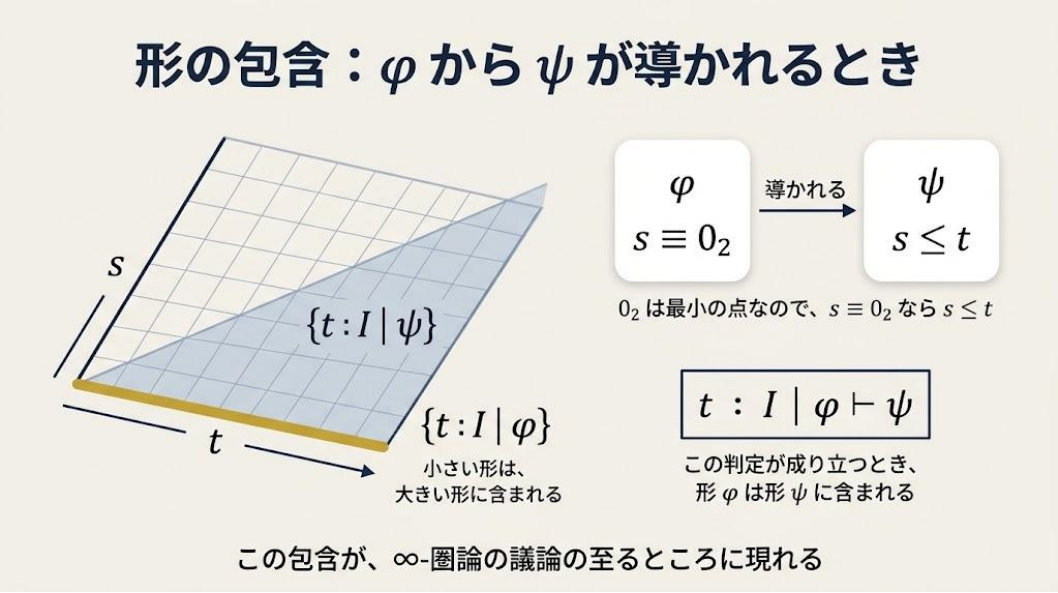
論文では {t : I | φ} と書きます。「cube I の点 t のうち、φ を満たすもの」という意味です。



形と形のあいだには、包含関係 があります。

形 {t : I | φ} が 形 {t : I | ψ} に含まれるのは、「φ から ψ が導かれる」とき、つまり t : I | φ ⊢ ψ のときです。

この包含が、∞-圏論の議論の至るところに現れます 。

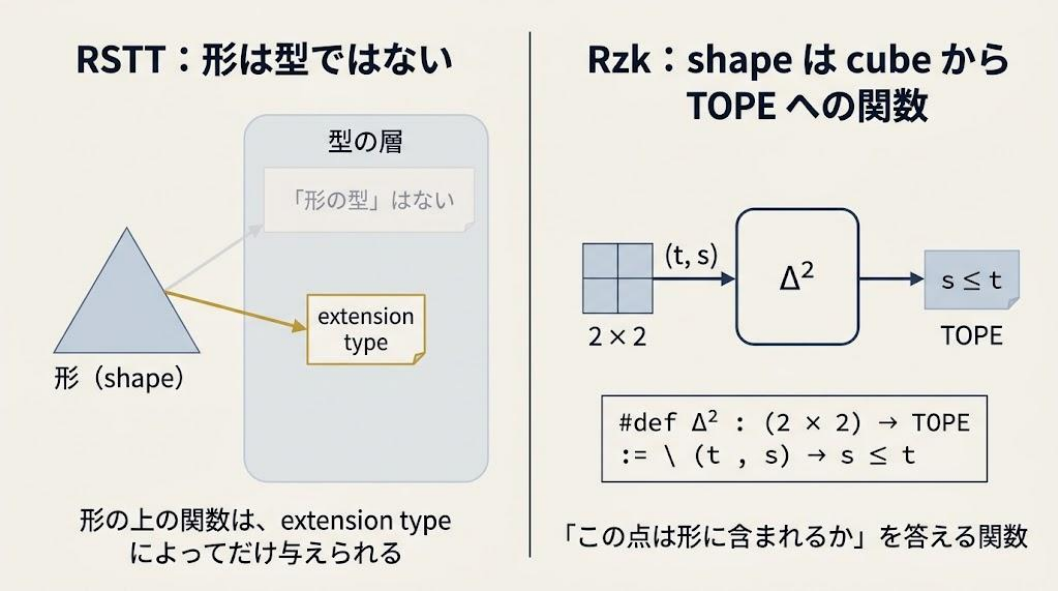


もう 1 つ注意すべき点 があります。

論文によれば、**RSTT** では **形は型ではありません。**

「形の型」というものはなく、**形の上の関数は、次に見る extension type によってだけ** 与えられます。

Rzk では、**shape** を **cube から TOPE への関数**として定義します。**「この点は形に含まれるか」を答える関数**、と考えると分かりやすいでしょう。



4-5　4 つの基本的な形

∞-圏論で繰り返し使う 4 つの形 を、**論文の図 1 の定義** どおりに示します。

ここで改めて、論文の URL を添えます。

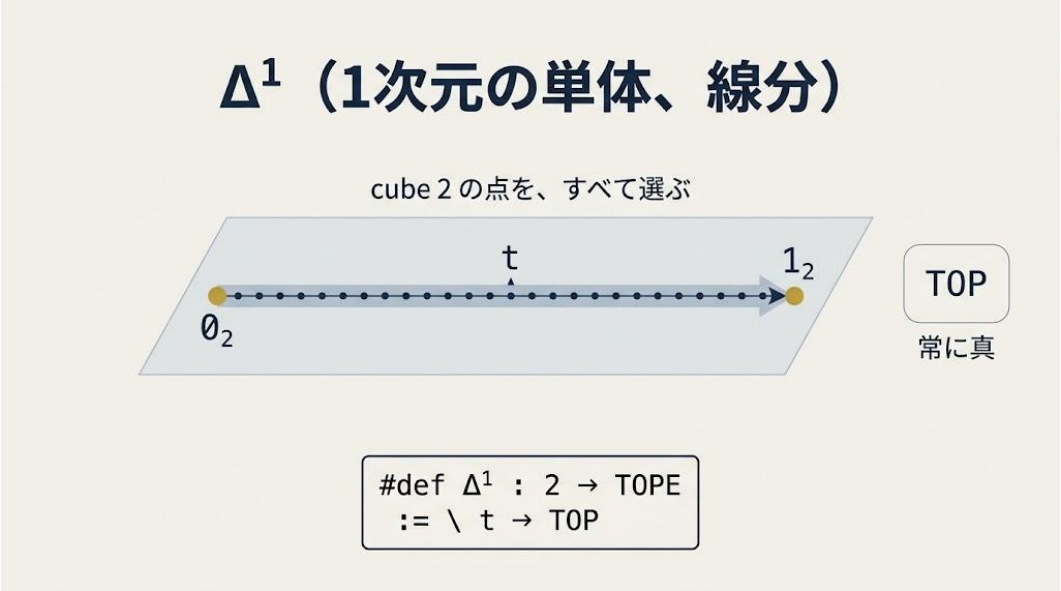
Kudasov, Sim, Ahrens「Rzk: a Proof Assistant for Synthetic ∞-Categories」（arXiv:2607.12207）

https://arxiv.org/abs/2607.12207

筆者は、これらがそのまま Rzk 0.11.4 の型検査を通ることを確認しました。

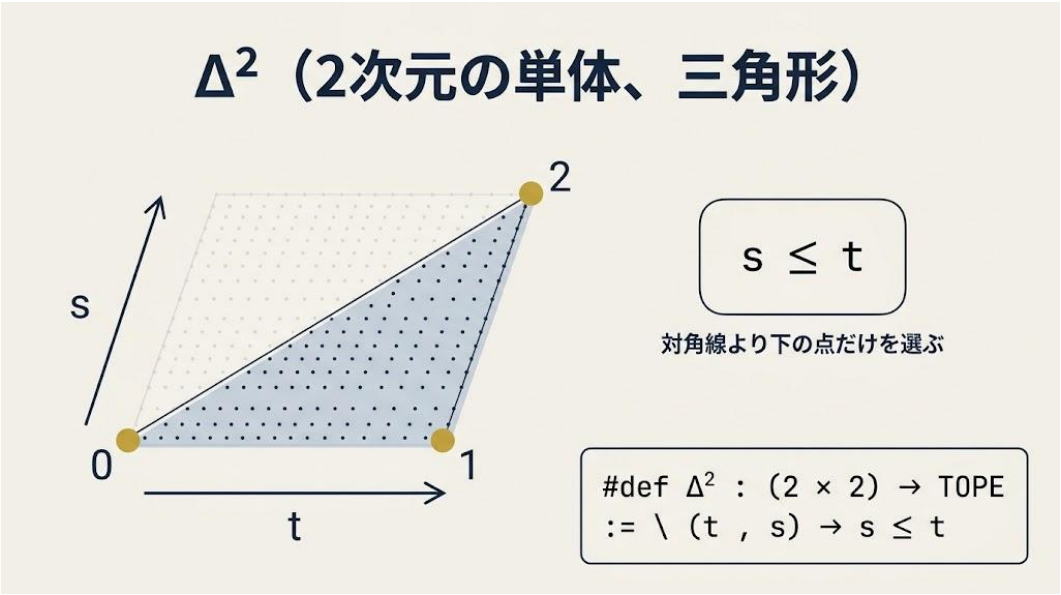
Δ¹（1 次元の単体、線分）：区間そのものです。条件は常に真（TOP）です。

```
#def Δ¹ : 2 → TOPE
  := \ t → TOP
```



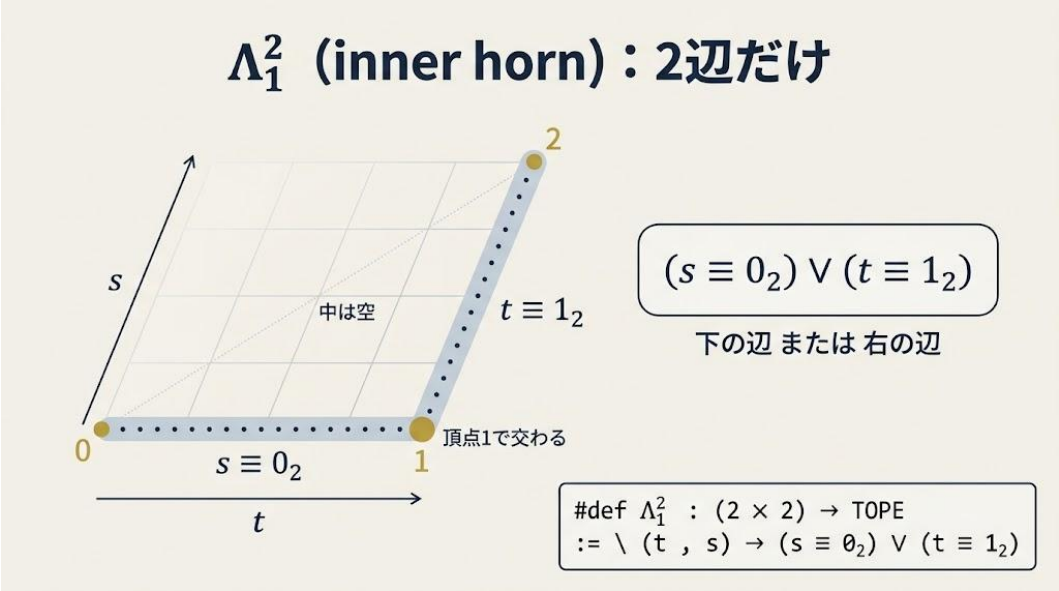
Δ²（2 次元の単体、三角形）：正方形 2 × 2 から、s ≤ t で切り出した三角形です。

```
#def Δ² : (2 × 2) → TOPE
  := \ (t , s) → s ≤ t
```



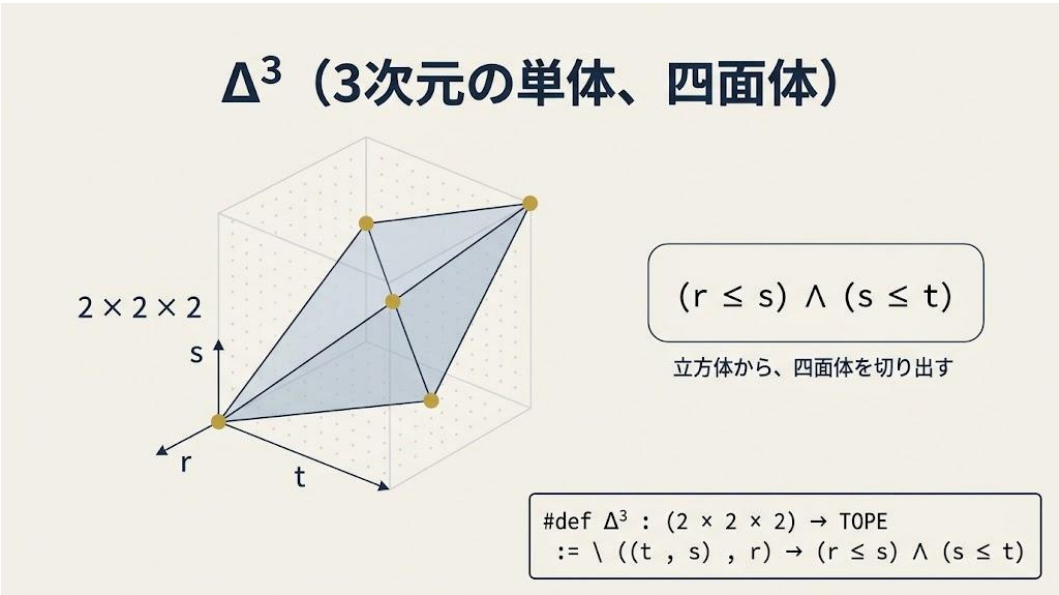
Λ²₁（inner horn）：Δ² のうち、頂点 1 で交わる 2 辺だけを残したものです。中身の詰まった三角形ではなく、「**くの字**」**の 2 辺**です。**inner horn**（*inner horn*、内側の角）と呼びます。

```
#def Λ²₁ : (2 × 2) → TOPE
  := \ (t , s) → (s ≡ 0₂) ∨ (t ≡ 1₂)
```



Δ³（3 次元の単体、四面体）：立方体 2 × 2 × 2 から、(r ≤ s) ∧ (s ≤ t) で切り出した四面体です。

```
#def Δ³ : (2 × 2 × 2) → TOPE
  := \ ((t , s) , r) → (r ≤ s) ∧ (s ≤ t)
```



1 行ずつ読むと、**それぞれの定義が「どの点を含むか」という条件だけで書かれていること** が分かります。

- Δ^2 の条件 $s \le t$ は、正方形のうち対角線の片側の三角形を表します。
- Λ^2_1 の条件は「s が 0_2 である辺」と「t が 1_2 である辺」の和です。
- Δ^3 の条件 $r \le s \le t$ は、立方体の中の四面体を表します。

四面体の 4 枚の面も、同じように条件で書けます。

不等式 $r \le s \le t$ のどこか 1 か所を「ぴったり」にすると、面になります。

条件	意味	面
`r ≡ 0₂`	いちばん下の座標が最小	→ 1 枚目の面
`r ≡ s`	下の 2 つの座標が等しい	→ 2 枚目の面
`s ≡ t`	上の 2 つの座標が等しい	→ 3 枚目の面
`t ≡ 1₂`	いちばん上の座標が最大	→ 4 枚目の面

どの面も三角形（Δ^2 と同じ形） です。

四面体の境界 は、**この 4 枚の面の「または」で** 表されます。

高次元の形の境界 を、**論理式の組み合わせとして書けること** が、**tope という仕組みの強み** です。

第 5 章で、四面体の面を可換三角形に固定する話が出てきますが、**そこで使われるのがこの 4 つの条件** です。

> 【入力について】Rzk は Unicode の記号（Δ、→、≤、≡、$_2$ など）を受け付けます。
>
> ASCII でも書けて、たとえば `->`、`<=`、`x` などが使えます。
>
> 筆者の検証では、Δ のような記号を名前に含める場合、 `Δ¹` のようにバックォートで囲む書き方も使いました（Rzk のバージョンや入力環境によって書き方が異なりうるので、playground で確かめてください）。

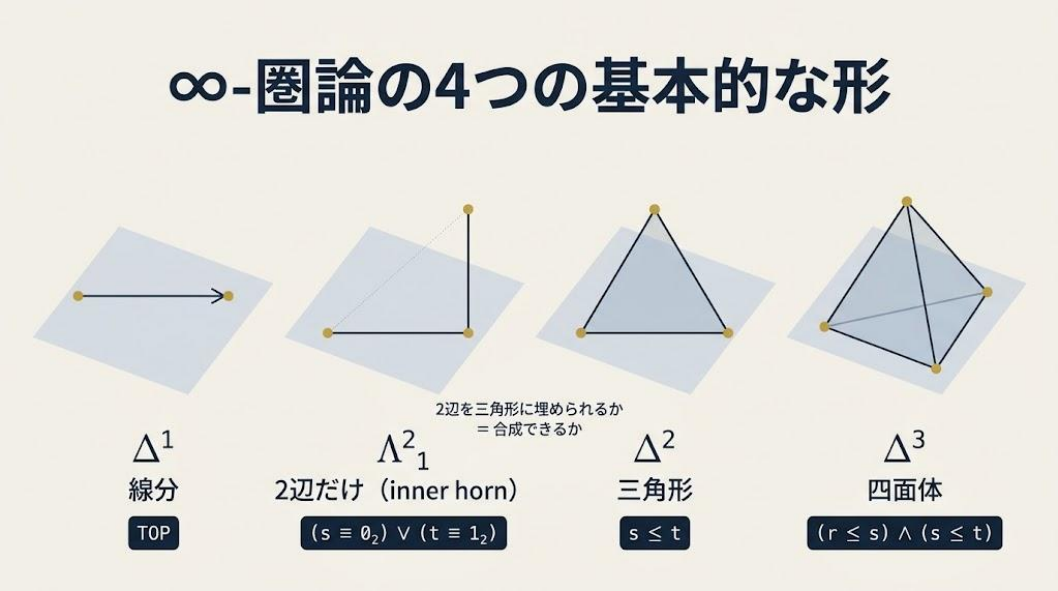
4 つの形の関係 を、図にしておきます。

```
Δ¹    0 ──────→ 1                          線分

Λ²₁         1                               2 辺だけ（中は空）
          ↗   ↘
        0       2

Δ²          1                               中まで詰まった三角形
          ↗ ■ ↘
        0 ──────→ 2

Δ³    4 つの頂点、6 本の辺、4 枚の面、中まで詰まった四面体
```



Δ^2 の 3 本の辺が、座標の条件でどう表されるかも見ておきます。

第 5 章の hom2 を読むときに必要になります。

三角形の頂点を、

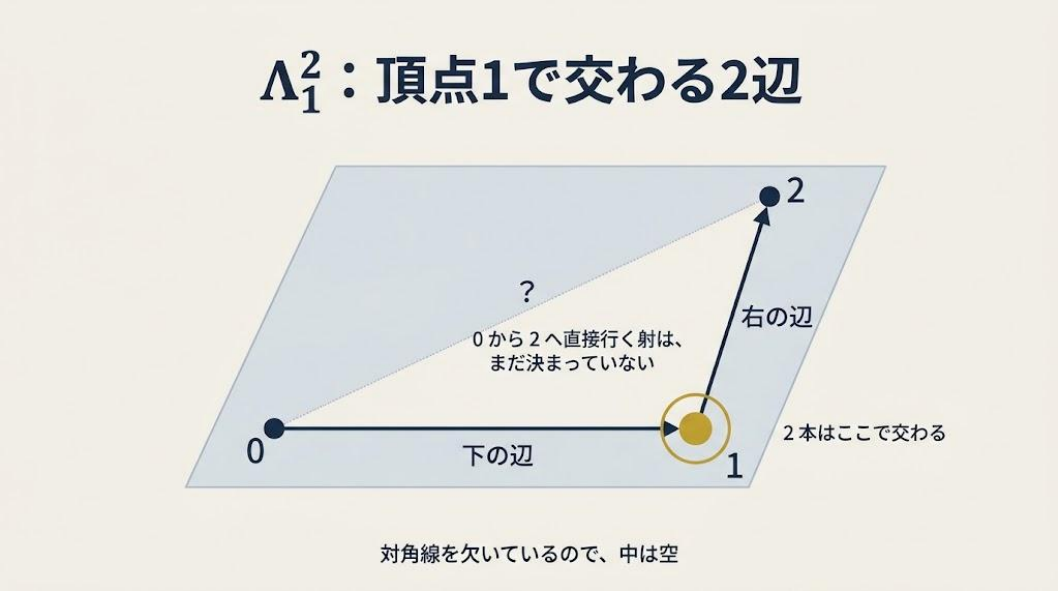
- $0 = (0_2, 0_2)$
- $1 = (1_2, 0_2)$
- $2 = (1_2, 1_2)$

と置きます（点は (t, s) の順）。

辺	条件	どの頂点からどの頂点へ	辺の上を動く座標
下の辺	$s \equiv 0_2$	頂点 0 → 頂点 1	t が 0_2 から 1_2 へ
右の辺	$t \equiv 1_2$	頂点 1 → 頂点 2	s が 0_2 から 1_2 へ
対角線	$s \equiv t$	頂点 0 → 頂点 2	t と s が一緒に動く

Λ^2_1 は、このうち「下の辺」と「右の辺」だけを残した形です。

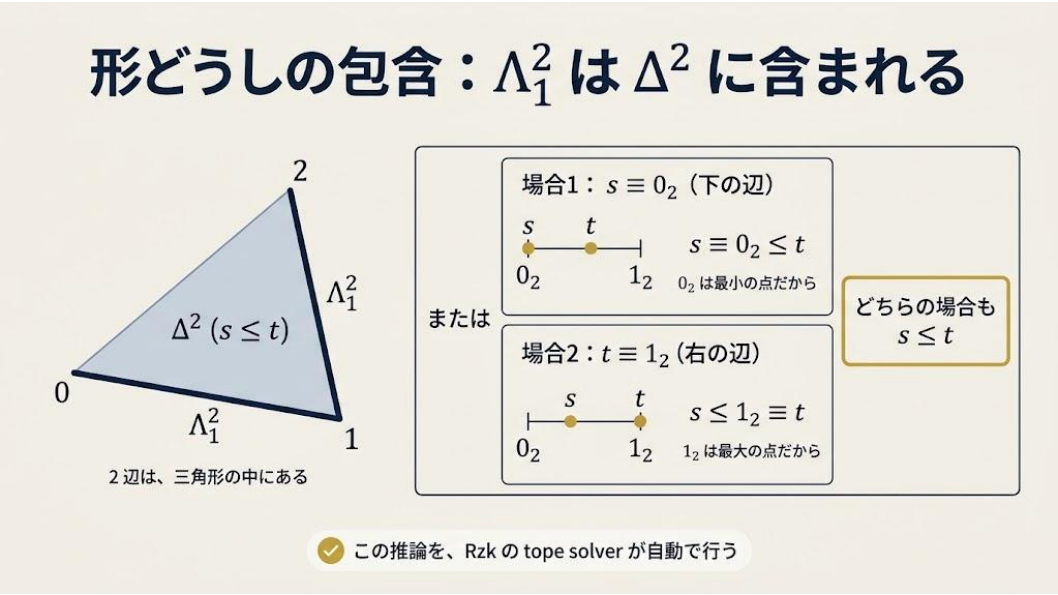
2 本は頂点 1 で交わります。だから「頂点 1 で交わる 2 辺」なのです。 対角線を欠いているので、「0 から 2 へ直接行く射」はまだ決まっていません。



形どうしの包含 も確かめておきます。

Λ^2_1 は Δ^2 に含まれます。 Λ^2_1 の点は $s \equiv 0_2$ **か** $t \equiv 1_2$ **を満たします** が、どちらの場合も、**区間の公理**（0_2 が最小、1_2 が最大）から $s \le t$ **が導かれるから** です。

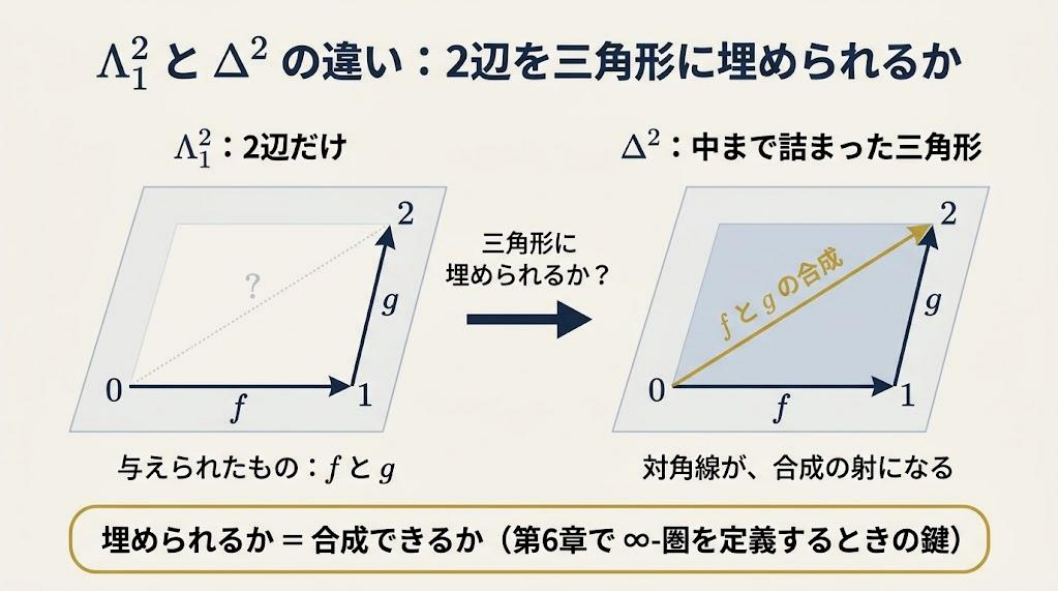
こうした推論 を、**Rzk** では **tope solver が自動で** 行います。



「Λ^2_1 と Δ^2 の違い」は、第 6 章で ∞-圏を定義するときの鍵になります。

2 辺が与えられたとき、それを三角形に埋められるか。

これが「合成できるか」という問いにあたるから です。



ここは大事なところですので、繰り返し述べます。

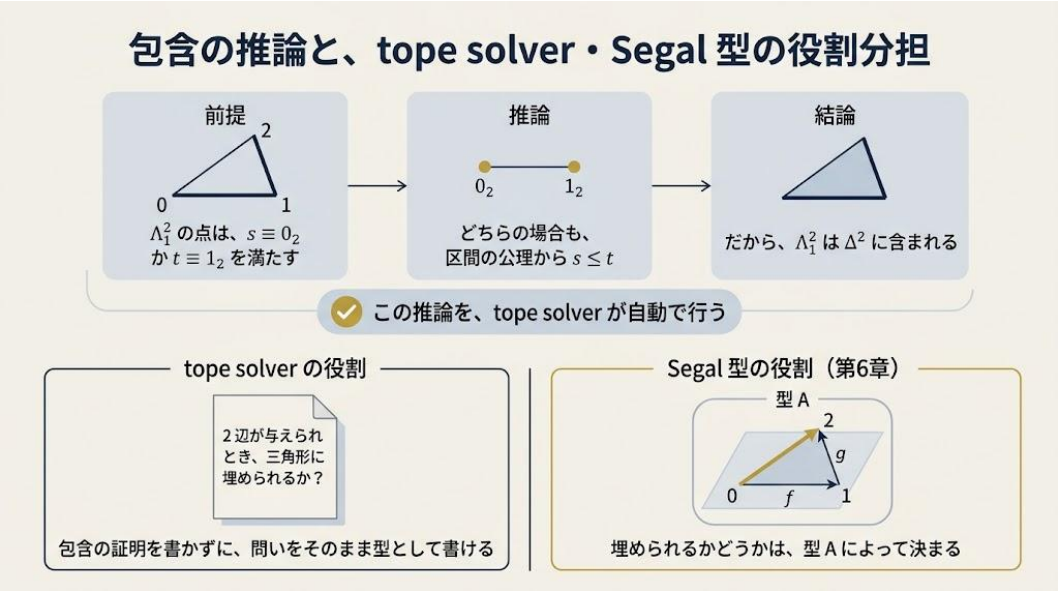
- Λ^2_1 の点は、$s \equiv 0_2$ **か** $t \equiv 1_2$ **を満たす**。
- どちらの場合も、**区間の公理**（0_2 が最小、1_2 が最大）から $s \leq t$ **が導かれる**。
- だから、Λ^2_1 **は** Δ^2 **に含まれる**。

Rzk では、**こうした推論** を、**tope solver が自動で** 行います。

このおかげで、利用者は包含の証明を自分で書かずに、**「2 辺が与えられたとき、それを三角形に埋められるか」という問いを、そのまま Rzk の型として書けます。**

ただし、「2 辺を三角形に埋められるか」という問いに答えるのは tope solver ではありません。

埋められるかどうかは型 A によって決まり、それを定めるのが、第 6 章で見る **Segal 型** の条件です。



4-6　extension type

いよいよ、**Rzk の核** にあたる **extension type** です。

次の 3 つが与えられたとします。

記号	意味	例
shape ψ	大きい形	例：Δ¹（線分全体）
部分形 φ	ψ に含まれる形	例：両端点 $\{0_2, 1_2\}$
切断 a	φ の上で決まった値	例：0_2 では x、1_2 では y

このとき、**extension type** は次のように書きます。

```
(t : ψ) → A [ φ ↦ a ]
```

意味は、「**ψ 全体の上の関数で、φ が成り立つところでは a に判定的に一致するもの**」の **型** です。

- `(t : ψ) → A`：ψ の上の関数（ふつうの関数型の **形版** ）
- `[ φ ↦ a ]`：ただし、φ の上では a と一致する（という **制約** ）

「**判定的に一致する**」 の意味を、もう一度確認します。

f が **この型を持つなら** 、φ を満たす点 t で `f t` を計算すると、**型検査器が自動で `a t` に書き換えます**。

証明は必要ありません。 強制変換（型を合わせるための変換関数）も要りません。

- **素朴な定義**： $f(0) = x$ の証拠 p を **自分で持ち回る**
- **extension type**： $f0_2$ は、**計算すれば** x になる。**証拠は存在しない**

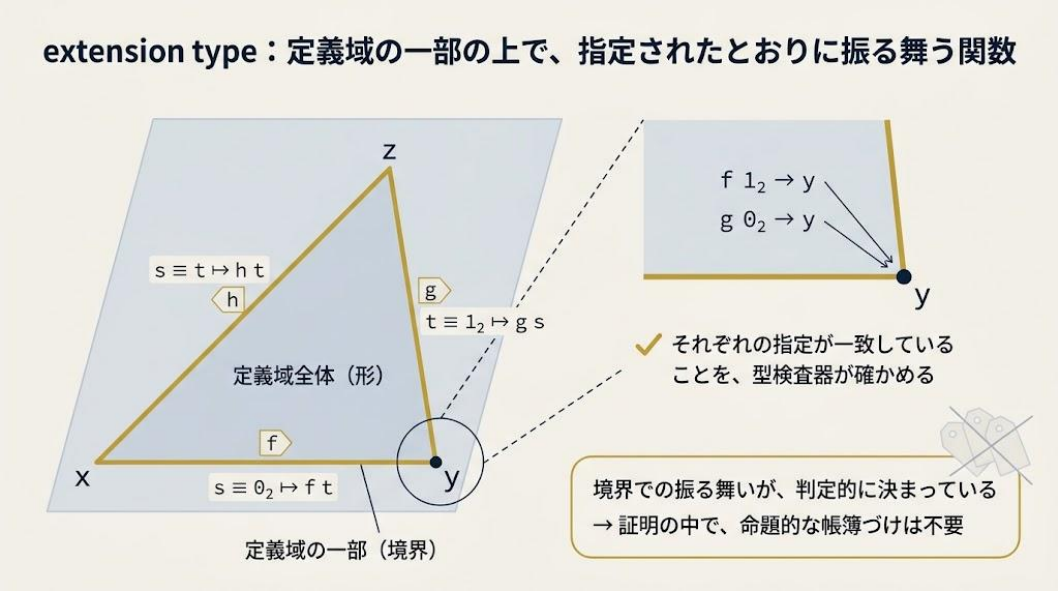
境界の指定は、いくつ並べても構いません。

`[ t ≡ 0₂ ↦ x, t ≡ 1₂ ↦ y ]` のように、カンマで区切って複数の条件を書けます。

複数の条件が同時に成り立つ点（たとえば三角形の角）では、**それぞれの指定が一致していることを、型検査器が** 確かめます。

論文の表現を借りれば、**extension type** は、**「定義域の一部の上で、指定されたとおりに振る舞う関数」の型** です。

境界での振る舞い が **判定的に決まっているの** で、**証明の中で命題的な帳簿づけを行うことは不要になる** のです。



> 【コラム】extension type の正確な定義
>
> RSTT では、shape {t : I | ψ}、部分形 {t : I | φ}（t : I | φ ⊢ ψ）、{t : I | ψ} 上の型族 A、φ 上の項 a が与えられたとき、extension type ⟨Π_{t:I|ψ} A |^φ_a⟩ が形成される。
>
> その要素 f は、φ(s) が成り立つ任意の点 s で f(s) ≡ a(s) を判定的に満たす。
>
> 規則は、形成・導入・適用・β・境界の計算（Ext-comp）・η の 6 つ（論文 Fig. 9）。
>
> 境界 φ が ⊥（矛盾）の場合は、制約のない「形の上のふつうの関数型」になり、RSTT の論文ではこれを Π_{t:I|ψ} A と略記する約束になっている。

4-7　Rzk のコードを、どこに書くか

次章からコードを本格的に読む前に、**Rzk のコードをどう書いて、どう検査するか** を押さえておきます。

Rzk 言語のソースコードを記述したファイルは、1 行目に Rzk 言語のバージョン（`rzk-1`。処理系のバージョン 0.11.4 とは別のもの）を宣言し、その後に `#def`（または `#define`）で定義を並べます。

```
#lang rzk-1

#def Δ¹ : 2 → TOPE
  := \ t → TOP
```

`#def 名前 : 型 := 値` が基本の形です。

Haskell の `名前 :: 型` と `名前 = 値` を、1 つにまとめたものと考えると近いでしょう。

引数は、名前の後ろに `(A : U)` のように並べられます。

筆者の検証で分かった注意点が 1 つあります。**定義の順序が重要です。**

後ろで定義する名前を、前で使うことはできません。使う前に定義しておく必要があります。

コードを試す方法は、主に 3 つあります。

- **playground**
  - ブラウザで開くだけで使える。インストール不要
  - https://rzk-lang.github.io/rzk/playground/
- **コマンドライン**
  - GitHub のリリースから実行ファイルを取得して、ファイルを検査する
  - （筆者は Linux x86_64 版の Rzk 0.11.4 で検証した）
- **VS Code**：拡張機能をインストールすると、書きながら検査結果が表示される

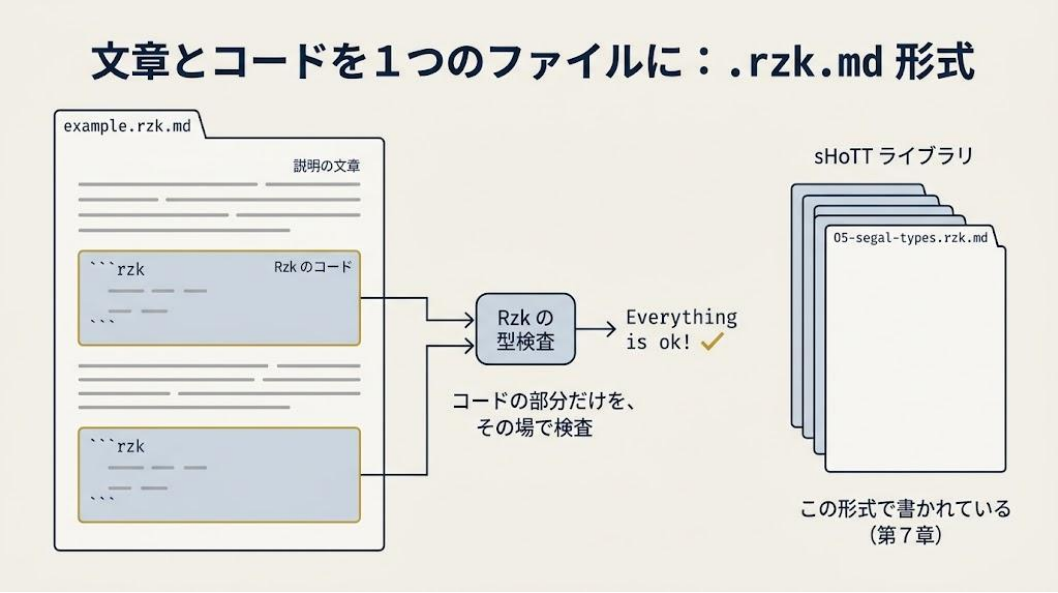
検査に成功すると、`Everything is ok!` と表示されます。

失敗すると、どこで何が合わなかったかが表示されます（第 1 章の bad の例）。

また、**Rzk** は Markdown の中にコードブロックとして **Rzk** のコードを埋め込んだファイル（拡張子 `.rzk.md`）も検査できます。

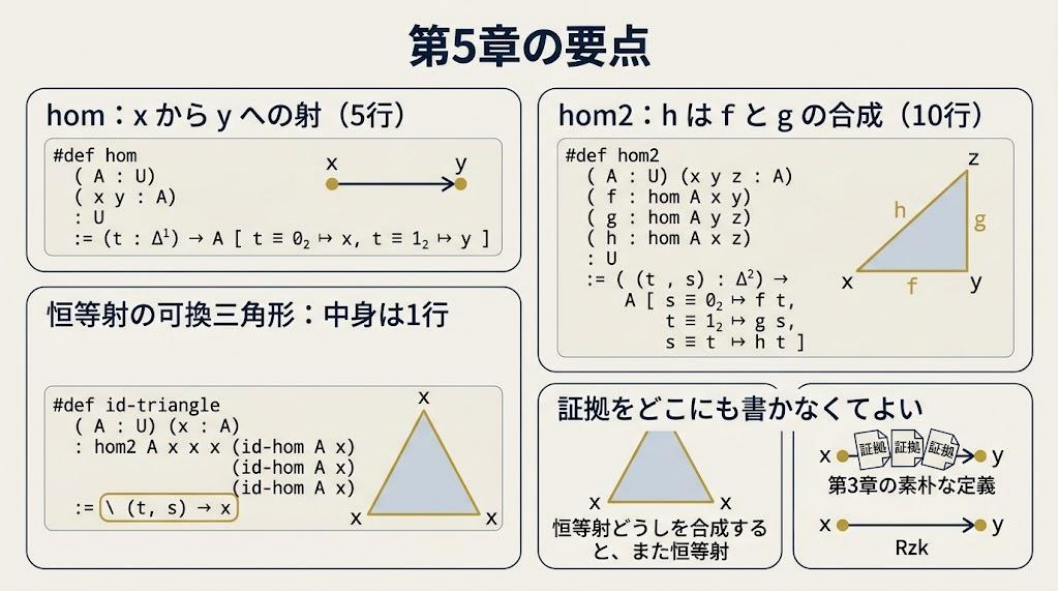
説明の文章とコードを 1 つのファイルに書き、コードの部分だけをその場で検査する形式です。

第 7 章で見る **sHoTT ライブラリ** は、**この形式** で書かれています。



次章では、この **extension type** を使って、実際に **射** と **可換三角形** を書きます。

第 5 章　射と可換三角形を書く



この章の要点：

- 「x から y への射」を表す型 `hom` は、Rzk ではわずか 5 行で定義できる。

```
#def hom
  ( A : U)
  ( x y : A)
  : U
  := (t : Δ¹) → A [ t ≡ 0₂ ↦ x, t ≡ 1₂ ↦ y ]
```

- 「h は f と g の合成である」ことを表す可換三角形の型 `hom2` も、10 行で定義できる。

```
#def hom2
  ( A : U) (x y z : A)
  ( f : hom A x y)
  ( g : hom A y z)
  ( h : hom A x z)
  : U
  := ( ( t , s) : Δ²) →
     A [ s ≡ 0₂ ↦ f t ,
         t ≡ 1₂ ↦ g s ,
         s ≡ t ↦ h t ]
```

- 「恒等射どうしを合成すると、また恒等射になる」という可換三角形の証明は、中身がたった 1 行で済む。

```
#def id-triangle
  ( A : U) (x : A)
  : hom2 A x x x (id-hom A x) (id-hom A x) (id-hom A x)
  := \ (t , s) → x
```

- 第 3 章の素朴な定義では、端点が正しいことの「証拠」を持ち回る必要があった。Rzk では、その証拠をどこにも書かなくてよい。

5-1　hom 型

論文の図 2 にある、**射の型の定義** です。

```
#def hom
  ( A : U)
  ( x y : A)
  : U
  := (t : Δ¹) → A [ t ≡ 0₂ ↦ x, t ≡ 1₂ ↦ y ]
```

1 行ずつ読んでいきましょう。

コード	意味
`#def hom`	「hom」という名前の定義を始める
`( A : U)`	引数 1：型 A（U は「型全体の宇宙」）
`( x y : A)`	引数 2、3：A の要素 x と y
`: U`	この定義は、型を返す
`:= (t : Δ¹) → A`	Δ^1（線分）の上の、A への関数で、
`[ t ≡ 0₂ ↦ x,`	t が 0_2 のところでは x に、
`t ≡ 1₂ ↦ y ]`	t が 1_2 のところでは y に、判定的に一致するもの

これが **射**（*morphism*）の型です。

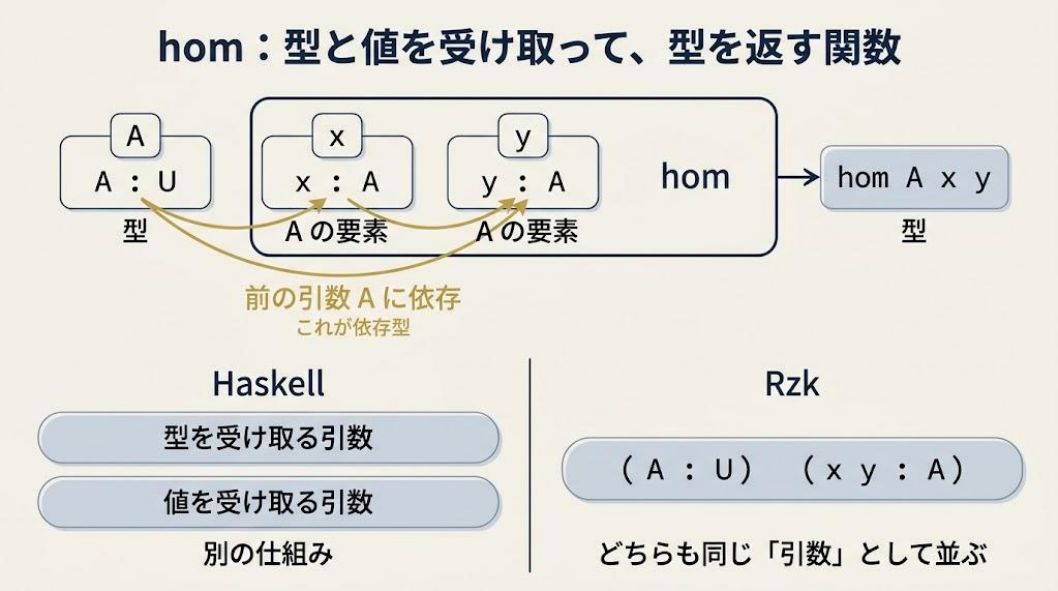
`hom A x y` の要素が、A の中の「x から y への射」です。

論文の言葉では、**hom 型** は「2 つの要素 x と y のあいだの射の型」であり、「1-単体から A への写像で、**端点が判定的に固定されたもの** 」です。

hom は、型 A と、A の要素 x 、y を受け取って、型を返す関数です。

Rzk では、引数 x と y の型 A が、前の引数 A に依存しています。これが **依存型** です。

Haskell では、型を受け取る引数と値を受け取る引数は別の仕組みですが、**Rzk** では **どちらも同じ「引数」として** 並べられます。



> 大学院で何年もかけて学ぶはずの ∞-圏論の「射」が、5 行で書けています。 この意味は、第 10 章で振り返ります。

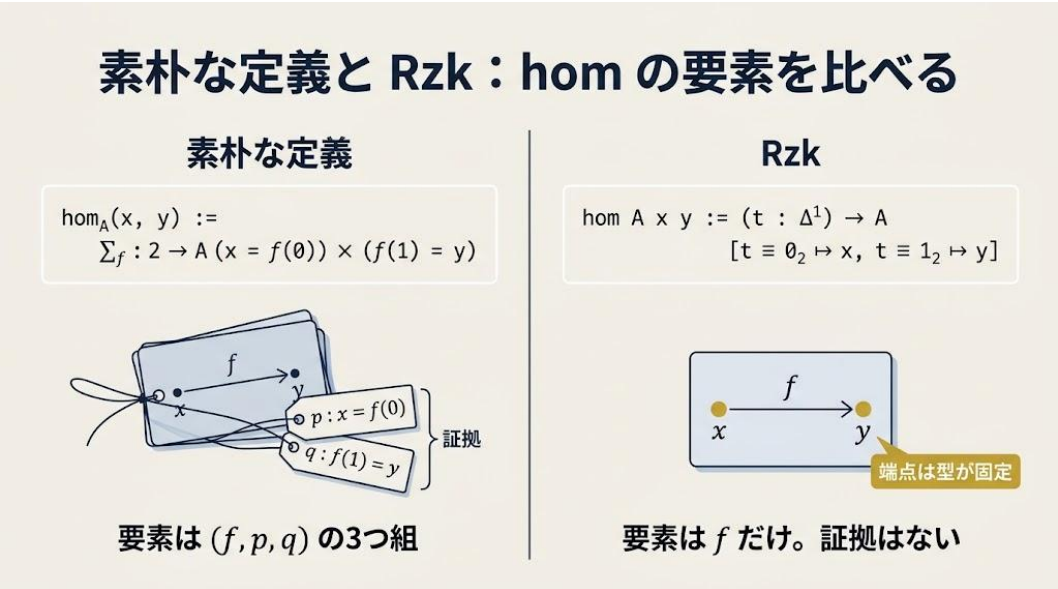
5-2　第 3 章の素朴な定義と比べる

素朴な定義

- `hom_A(x, y) := Σ_{f : 2 → A} (x = f(0)) × (f(1) = y)`
- 要素は (f, p, q) の **3 つ組** 。p と q は **証拠**

Rzk

- `hom A x y := (t : Δ¹) → A [ t ≡ 0₂ ↦ x, t ≡ 1₂ ↦ y ]`
- 要素は f だけ。**証拠はない**



証拠が、どこにもありません。

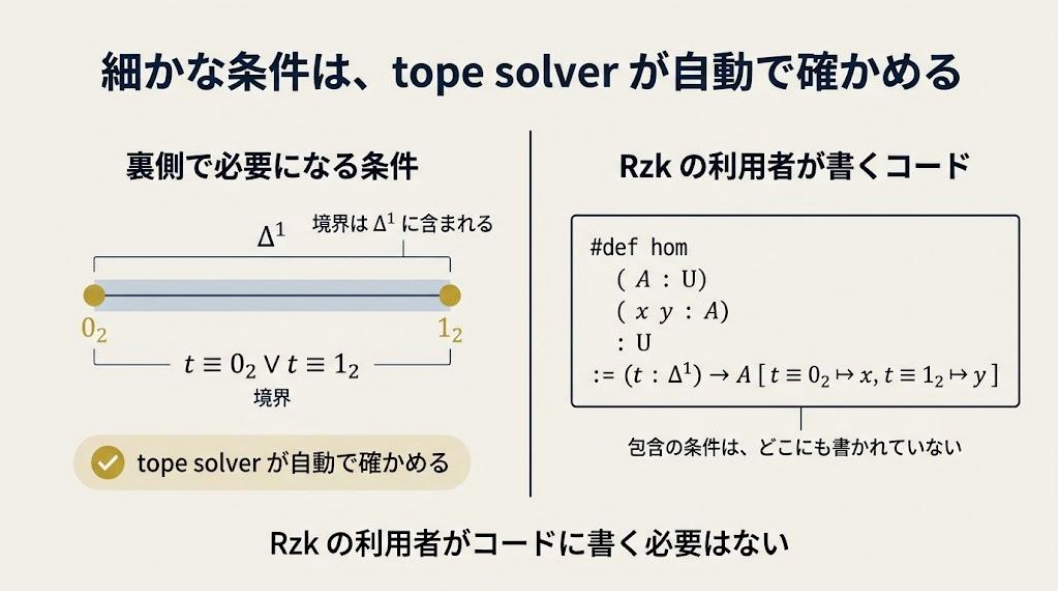
f の要素を 0_2 に適用すると、**Rzk** は `f 0₂` を `x` に書き換えます。1_2 なら `y` です。

f を使う側は、端点が x と y であることを、何もせずに知っています。

論文は、ここで **Rzk の裏側で起きていること** を説明しています。

hom の要素を端点 $t \equiv 0_2$ で使うと、**Rzk** は その適用を x **に書き換えます。**

そのとき、「**境界** $t \equiv 0_2 \vee t \equiv 1_2$ **は** Δ^1 **に含まれる**」**といった細かな条件** も **必要になりますが、それは tope solver が自動で確かめます。** Rzk 言語の利用者がコードに書く必要はありません。



最も簡単な射は、**恒等射**（*identity morphism*）です。

x から x への「何もしない」射で、時刻 t によらず常に x にとどまる道筋です。

```
#def id-hom
  ( A : U)
  ( x : A)
  : hom A x x
  := \ t → x
```

`\ t → x` は「t を受け取って x を返す関数」（ラムダ式）です。

Haskell の `\t -> x` と同じです。 Haskell なら `const x` と書くところです。

これが **型検査を通る理由** を、**型検査器の立場で** 追います。

目標：`\ t → x` が `hom A x x` の要素であること

確かめること	内容	結果

確かめること	内容	結果
1	本体 x が A の要素である	✓
2	$t \equiv 0_2$ のとき、本体 x が x と一致	✓（x と x）
3	$t \equiv 1_2$ のとき、本体 x が x と一致	✓（x と x）

第 1 章で見た `bad`（`hom A x y` に対して `\ t -> x` と書いた定義）が拒否されたのも、同じ確かめ方によるものです。

`hom A x y` では、終点（$t \equiv 1_2$ のところ）の値は `y` でなければなりません。ところが `\ t -> x` は、終点でも `x` を返します。

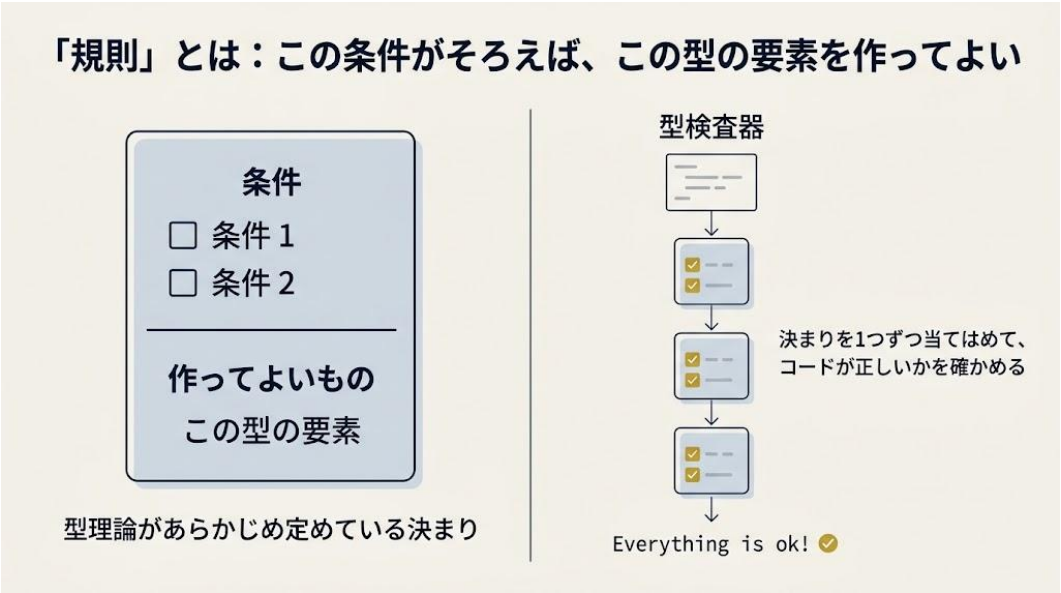
型検査器 は終点で `x` と `y` を **突き合わせ、一致しないので拒否** しました。

エラーメッセージの `cannot unify term y` は、まさにこの「`y` と一致させられなかった」という失敗を指しています。

Kudasov、Sim、Ahrens の論文「Rzk: a Proof Assistant for Synthetic ∞-Categories」の付録 C には、この **恒等射の型検査** を、**RSTT** の規則でたどった場合と、**Rzk** の規則でたどった場合が並べて示されています。

ここでいう「**規則**」とは、**型理論があらかじめ定めている「この条件がそろえば、この型の要素を作ってよい」という決まりのこと** です。

型検査器 は、**この決まりを 1 つずつ当てはめて、コードが正しいかを確かめていきます。**



RSTT では、恒等射を作るのに、「extension type の要素を作ってよい」という **決まりを 1 つ当てはめるだけで済んで** いました。

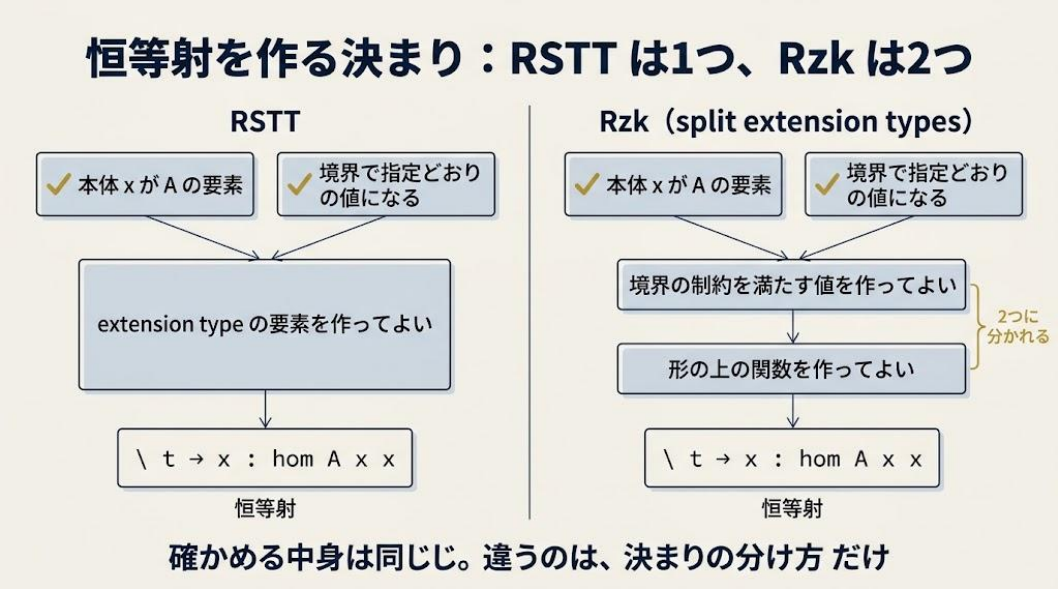
Rzk では、**これが 2 つの決まりに分かれます** 。

- 「境界の制約を満たす値を作ってよい」という決まり
- 「形の上の関数を作ってよい」という決まり

の **2 つ** です（第 8 章で見る split extension types）。

どちらの場合も、確かめる中身は本体 x が A の要素であることと、境界で指定どおりの値になること です。

違うのは、それを確かめる決まりの分け方だけ です。



5-3　射を関数として使う

hom の要素 f は、**区間の上の関数でも** あります。 だから、区間の任意の点 s に適用できます。

```
f : hom A x y
s : 2

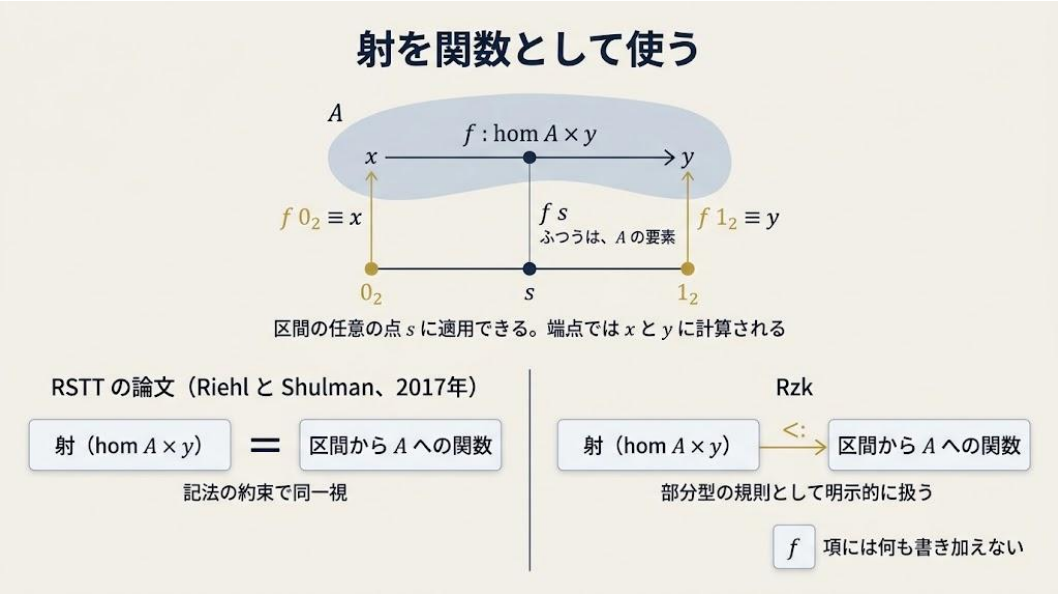
f s : A          ふつうは、A の要素
f 0₂ ≡ x         端点では、x に計算される
f 1₂ ≡ y         端点では、y に計算される
```

論文は、この **「射を関数として使う」読み方** を例 4.4 として取り上げています。

Riehl と Shulman の論文 *A type theory for synthetic ∞-categories*（2017 年） では、**射** と **「区間から A への関数」** の 2 つの読み方を、記法の約束で **同一視** していました。

Rzk では、**この読み替え** を、第 8 章で見る **部分型の規則として明示的に扱います。**

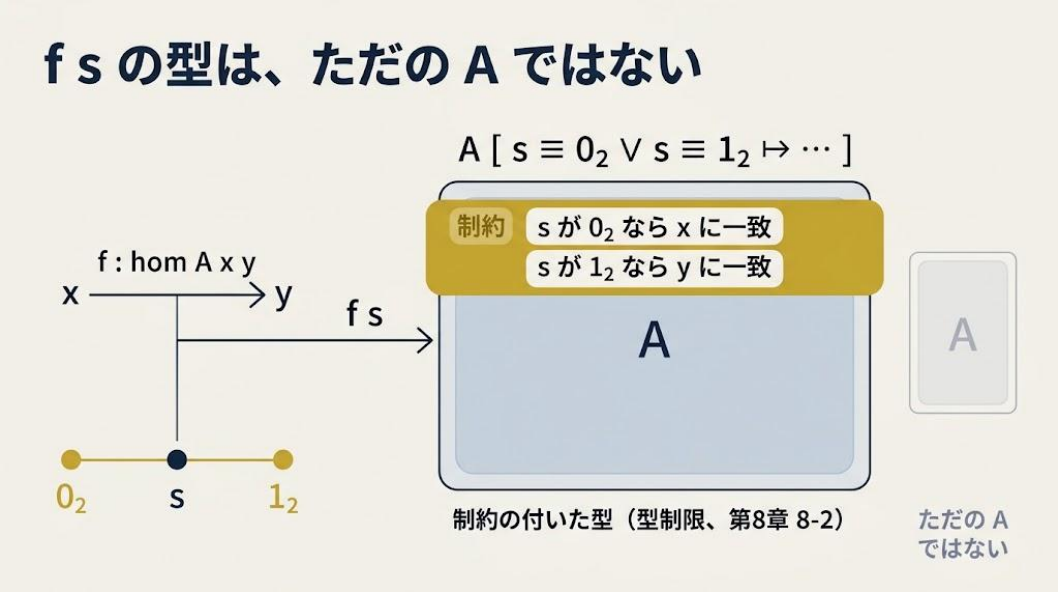
項（プログラム）には何も書き加えずに、**型検査の導出の中だけで読み替えが行われます。**



読み替え を、具体的に見てみます。

射 `f : hom A x y` を区間の点 `s` に当てると、`f s` の型は、**ただの `A` ではありません。**

「`s` が端点なら `x` か `y` に一致する」 **という制約が付いた** 、`A [ s ≡ 0₂ ∨ s ≡ 1₂ ↦ … ]` **という型** になります（第 8 章 8-2 で見る「型制限」です）。



ところが、`f s` を、**ふつうの `A` の要素として使いたい場面** がよくあります。

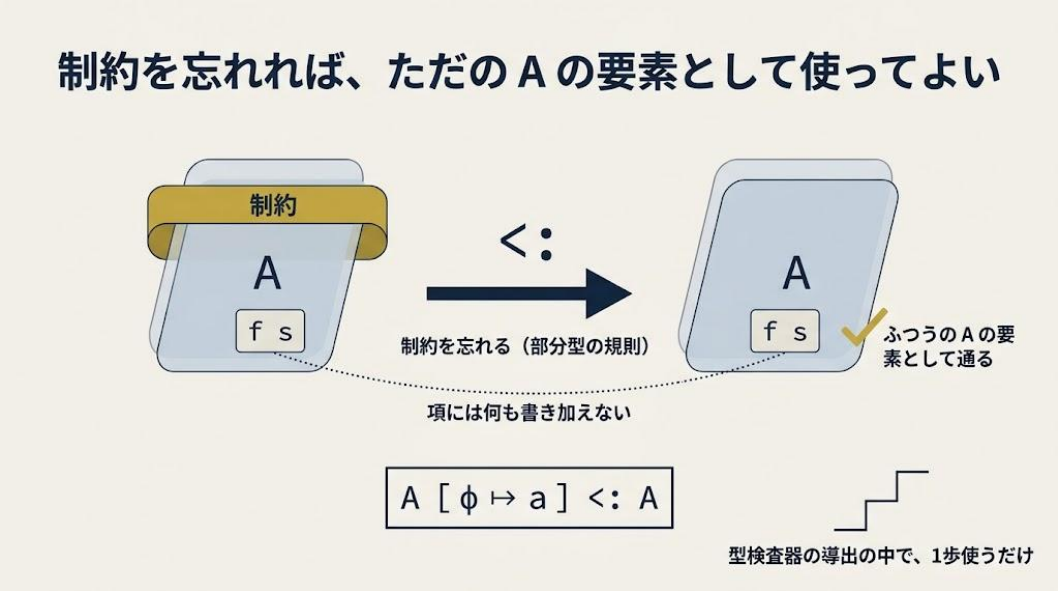
そこで **Rzk** は、「**制約の付いた** `A` **の要素は、制約を忘れれば、ただの** `A` **の要素として 使ってよい** 」という **部分型の規則** を用意しています。

記号で書けば `A [ ϕ ↦ a ] <: A` です。

この規則のおかげで、`f s` はそのまま `A` の要素として通ります。

このとき、`f s` という項には **何も書き加えられません** 。

「制約の付いた `A` から、ただの `A` へ」という読み替えは、**型検査器が導出の中で 1 歩使うだけ** です。



`f s` を、**ふつうの** `A` **の要素として使いたい場面** としては、たとえば、次のようなものがあります。

- **ふつうの関数に渡すとき。** `F : A → B` という関数があれば、`F (f s)` と書いて、射の上の点を `B` へ送りたくなります。 **関数** `F` **が受け取るのは、制約の付いていないふつうの** `A` **の要素** です。
- **射を、別の型へ写すとき。** 上の `F` を射の各点に当てて `\ t → F (f t)` と書くと、`B` の中の「`F x` から `F y` への射」が作れます。 関数が射を射へ写す、という圏論の基本的な操作です。 ここでも、`f t` は **ふつうの** `A` **の要素として** `F` **に渡されて** います。
- **等しさを述べるとき。** `f s = f s'` のように、射の上の 2 点が等しいかを、**ふつうの** `A` **の等式型で述べたい** 場面です。

どの場面でも、**利用者は「制約を外す」ための操作を何も書きません。**

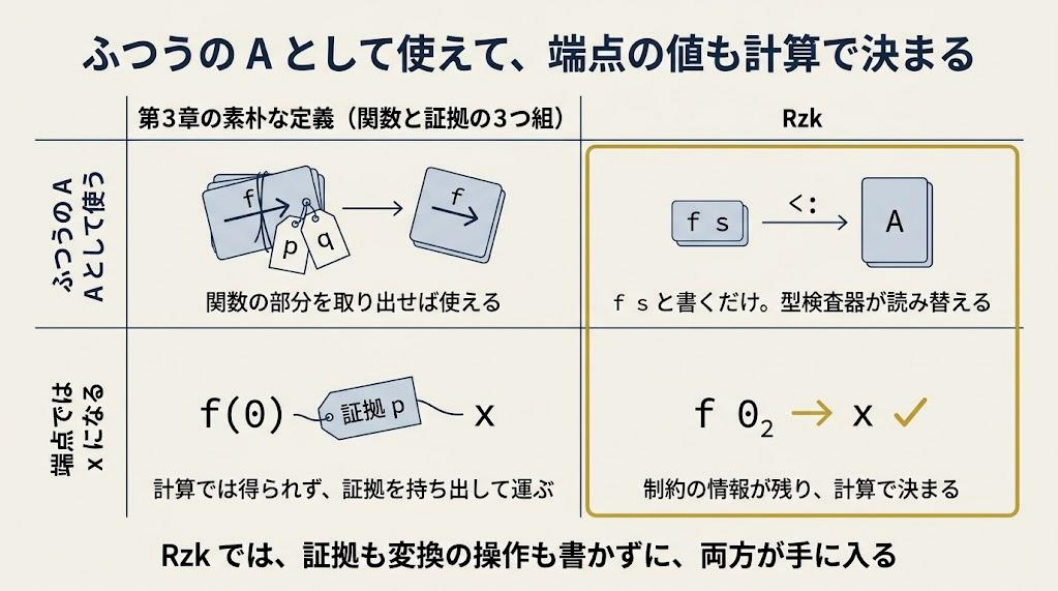
`f s` と書けば、**型検査器が部分型の規則で読み替えて** くれます。

しかも、`s` が端点のときには、`f 0₂` が `x` に計算されるという **制約の情報も、失われずに残っています。**

第 3 章の素朴な定義（関数と証拠の 3 つ組）でも、関数の部分を取り出せば、ふつうの `A` の要素は手に入ります。

ただし **その場合、「端点では `x` になる」という事実は計算では得られず、証拠 `p` を持ち出して運ぶ必要** がありました。

Rzk では、**「ふつうの `A` として使えること」と「端点の値が計算で決まること」の両方が、証拠も変換の操作も書かずに** 手に入ります。



5-4　射は逆向きにたどれない

ここで、**HoTT の「道」** と、**Rzk の「射」** の **違い** がはっきり見える例を考えます。

HoTT の道は、逆向きにたどることができます。

x から y への道があれば、y から x への道を作ることができます。

Haskell 風に言えば、区間を「時間を逆に流す」関数があるからです。

では、x から y への **射** f があったとき、y から x への **射** を作ることはできるでしょう？

素朴に考えると、次のように書きたくなります。

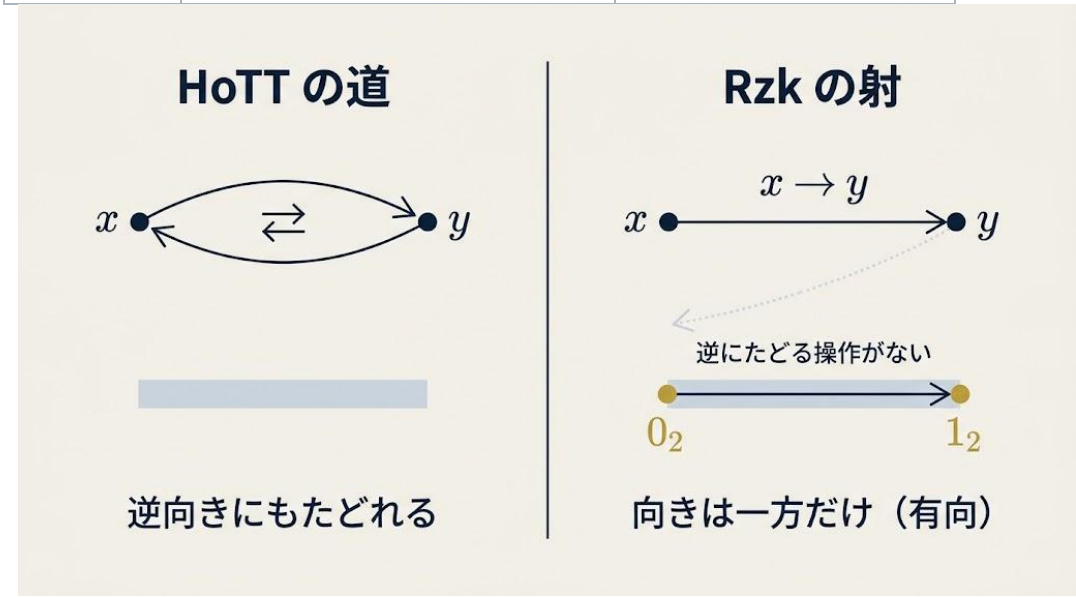
```
\ t → f (「t を逆にした点」)
```

ところが、**Rzk の cube 2** には、「t **を逆にした点**」**を作る操作がありません。**

第 4 章で見たとおり、**cube 2 の点** について使える道具は、0_2 と 1_2、等号 ≡ と順序 ≤ **だけ** です。

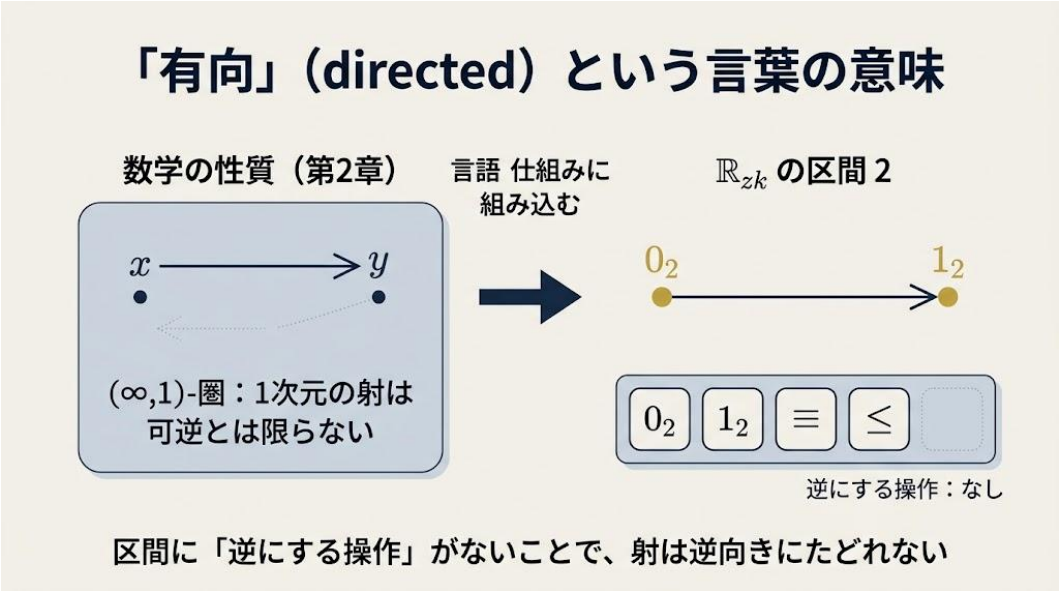
区間をひっくり返す関数は、どこにも用意されていません。

区間	向き	結果
HoTT の区間	向きがない。逆にたどる操作がある	→ 道は逆向きにたどれる
Rzk の区間 2	向きがある。逆にたどる操作がない	→ 射は逆向きにたどれない



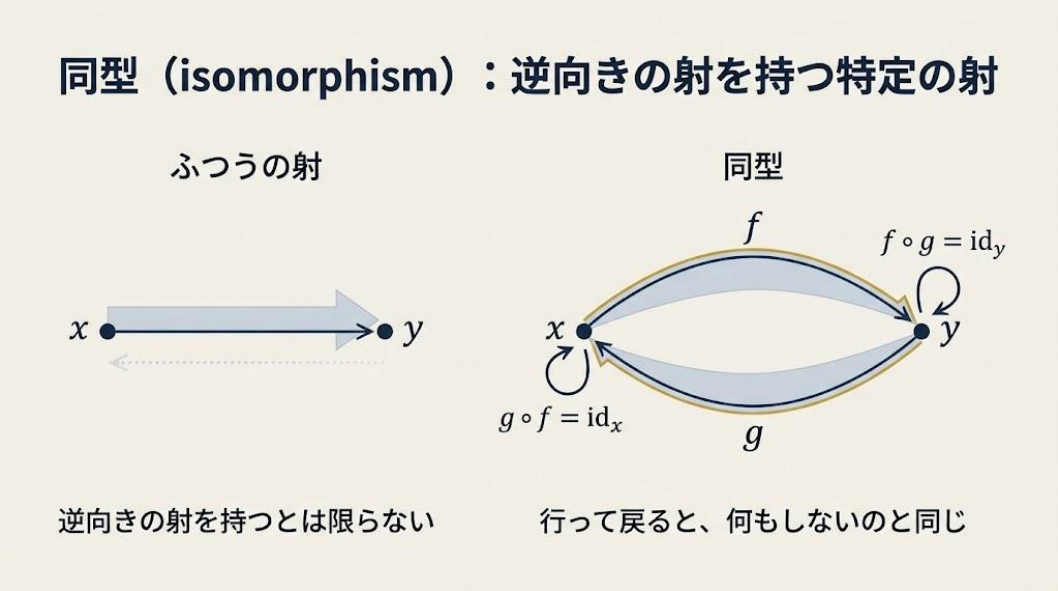
これが 「**有向**」（*directed*）という言葉の意味です。

第 2 章で見た **(∞,1)-圏の「1 次元の射は可逆とは限らない」という性質** が、**区間に「逆にする操作がない」という形で、言語の仕組みに組み込まれている** のです。



もちろん、**特定の射** が逆向きの射を持つことはあります。そういう射を **同型**（*isomorphism*）と呼びます。

同型は第 6 章の主役の 1 つです。



5-5　hom2 型 ── 可換三角形

同じ型を、1 次元上げます。

論文の図 2 の後半です。

```
#def hom2
  ( A : U) (x y z : A)
  ( f : hom A x y)
  ( g : hom A y z)
  ( h : hom A x z)
  : U
  := ( ( t , s) : Δ²) →
     A [ s ≡ 0₂ ↦ f t ,
         t ≡ 1₂ ↦ g s ,
         s ≡ t ↦ h t ]
```

このコードを読んでみましょう。

- **引数**：3 つの点 x, y, z と、3 本の射 $f: x \to y$, $g: y \to z$, $h: x \to z$
- **本体**：Δ^2（三角形）の上の、A への関数で、
  - 辺 $s \equiv 0_2$（下の辺）では f に、
  - 辺 $t \equiv 1_2$（右の辺）では g に、
  - 辺 $s \equiv t$（対角線）では h に、**判定的に** 一致するもの

第 4 章の表と照らし合わせると、対応がはっきりします。

辺	条件	どの頂点からどの頂点へ	辺の上を動く座標
下の辺	s ≡ 0_2	頂点 0 → 頂点 1	t が 0_2 から 1_2 へ
右の辺	t ≡ 1_2	頂点 1 → 頂点 2	s が 0_2 から 1_2 へ
対角線	s ≡ t	頂点 0 → 頂点 2	t と s が一緒に動く

下の辺は頂点 0 から頂点 1 へ、t が動きます。 だから、その辺の上の値は `f t` です。

右の辺は頂点 1 から頂点 2 へ、s が動きます。 だから `g s` です。

対角線は頂点 0 から頂点 2 へ、t と s が一緒に動きます。 だから `h t` です。

3 辺が、f、g、h に固定された三角形です。

三角形の中身を埋める関数があるということは、「f をたどってから g をたどる道」と「h をたどる道」が、**三角形の面を通じて変形し合えるということ** です。

名前の「2」は、この型が 2 次元の単体 Δ^2（三角形）の上の写像であることを表します。

論文は、この型の要素を「2-セル」（*2-cell*）と呼んでいます。

第 2 章の 2-射と同じく「射のあいだをつなぐ、1 つ上の次元のもの」ですが、**形が違います。**

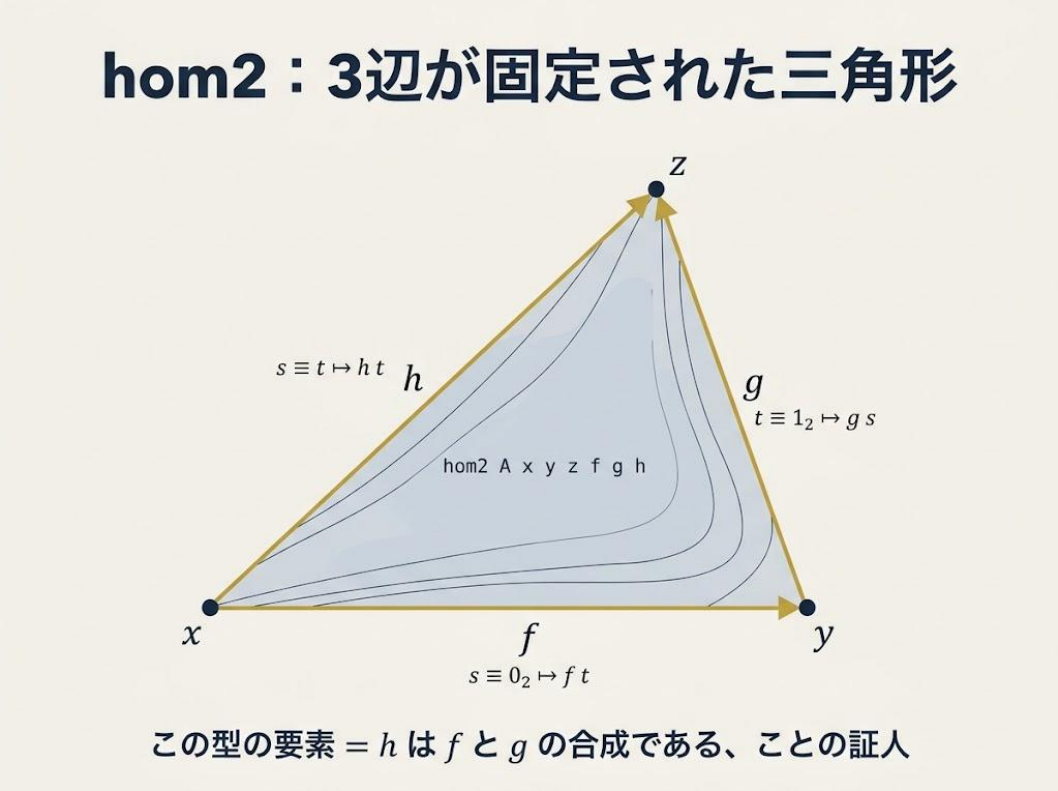
第 2 章の 2-射は平行な 2 本の射のあいだをつなぐものでしたが、hom2 の要素は三角形の 3 辺のあいだを埋めるものです。

つまり、`hom2 A x y z f g h` の要素は、**「h は f と g の合成である」ことの証人**です。

第 2 章の可換三角形が、そのまま型になっています。

論文の表現では、この型の要素は「f-g-h の三角形を埋める 2-セル」であり、「本質的に、h が f と g の合成であることの証人の型」です。

```
      y
  f ↗   ↘ g          hom2 A x y z f g h の要素
x ──────→ z            ＝ この三角形の内側を埋めるもの
      h
```



5-6　辺が交わる場所で、つじつまが合うこと

3 本の辺は、角で交わります。

角では、交わる 2 つの辺の指定が、**両方とも**適用されます。

そのため、**2 つの指定が矛盾しないかを確かめる必要** があります。

たとえば、角 $s \equiv 0_2 \wedge t \equiv 1_2$（辺 f と辺 g が交わる頂点 1 ）を見ます。

辺の指定	計算
辺 s ≡ 0_2 の指定	f t → t ≡ 1_2 なので f 1_2 → y
辺 t ≡ 1_2 の指定	g s → s ≡ 0_2 なので g 0_2 → y

どちらも y に計算される　✓ つじつまが合う

残りの 2 つの角も同じ です。

```
頂点 0（s ≡ 0₂ ∧ s ≡ t、つまり t ≡ 0₂ でもある）
  f t → f 0₂ → x
  h t → h 0₂ → x              ✓

頂点 2（t ≡ 1₂ ∧ s ≡ t、つまり s ≡ 1₂ でもある）
  g s → g 1₂ → z
  h t → h 1₂ → z              ✓
```

f の **終点** と g の **始点** が、どちらも y だからです。

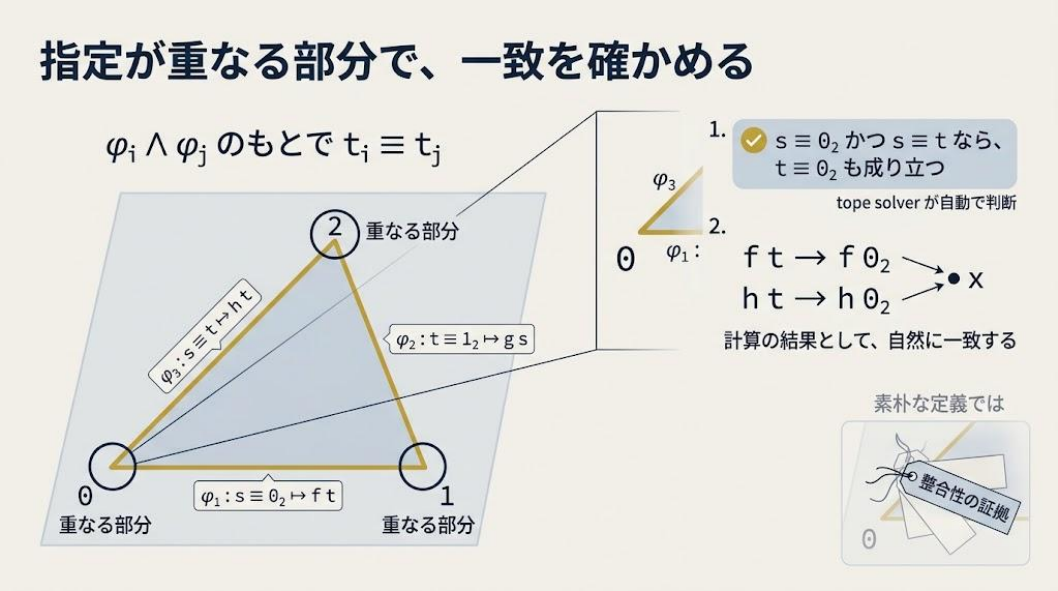
そして、それは f と g の**型**（`hom A x y` と `hom A y z`）**が保証しています。**

論文によれば、**Rzk** は、**境界の指定が複数あるとき、各指定どうしが重なる部分で一致すること**（$\varphi_i \wedge \varphi_j$ **のもとで** $t_i \equiv t_j$）**を確かめます。**

確かめるときに必要な「頂点 0 では $t \equiv 0_2$ **も成り立つ」といった論理の判断** は、第 8 章で見る **tope solver** が **自動で** 行います。

素朴な定義 では、ここに **「証拠どうしの整合性の証拠」が必要でした** 。

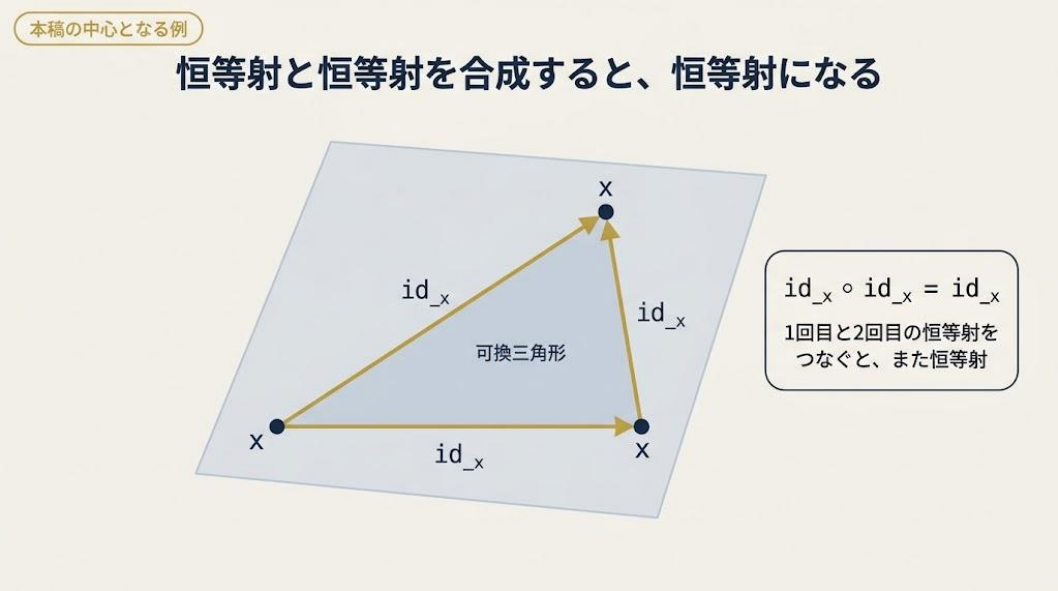
Rzk では、それは **計算の結果として自然に一致するものになっています。**



5-7　恒等射の可換三角形 ── 本稿の山場

いよいよ、**本稿の中心となる例** です。

「恒等射と恒等射を合成すると恒等射になる」という可換三角形 を書きます。




```
#def id-triangle
  ( A : U) (x : A)
  : hom2 A x x x (id-hom A x) (id-hom A x) (id-hom A x)
  := \ (t , s) → x
```

型の部分を読むと、「x, x, x の 3 点と、3 本とも恒等射である三角形」です。

証明の本体は、`\ (t , s) → x` の 1 行です。

「三角形のどの点 (t, s) でも x を返す」関数 です。

型検査器が確かめること は、次のとおりです。

辺	確かめること	結果
辺 s ≡ 0_2	本体 x と、id-hom A x を t に適用したもの（→ x）が一致	✓
辺 t ≡ 1_2	本体 x と、id-hom A x を s に適用したもの（→ x）が一致	✓
辺 s ≡ t	本体 x と、id-hom A x を t に適用したもの（→ x）が一致	✓

どの確認でも、`id-hom A x` を点に適用すると **x に計算される** ことが、**判断の根拠に** なっています。

`id-hom` の定義 `\ t → x` に点を代入すれば x になるからです。

筆者は、これを Rzk 0.11.4 で実際に検査し、**Everything is ok!** という出力を確認しました。

第 3 章で見た「素朴な定義」と並べてみます。

素朴な定義とは、汎用の言語で射を定義するやり方のことでした。 端点が正しいことを示す「証拠」を、関数と組にして持ち回ります。

- **素朴な定義**：端点の証拠だけで少なくとも 6 つ。加えて整合性の証拠

- **Rzk**：`\ (t , s) → x`

「ほとんど乗り越えられない」とされた可換三角形が、1 行になりました。

数学的な内容（恒等射を 2 回つないでも恒等射）は何も変わっていません。

消えたのは、内容とは無関係な帳簿づけだけ です。

> 【試してみる】playground で、`id-triangle` の本体を `\ (t , s) → y` のように別の値に変えてみてください（そのためには引数に y を足す必要があります）。
>
> 辺の指定と一致しなくなるので、型検査器が受け入れないはずです。
>
> どの辺で一致しなかったかが、エラーとして表示されます。

5-8　四面体も、同じ形で書ける

同じ考え方 は、**四面体** にもそのまま適用できます。

Δ^3 の上の関数で、4 枚の面をそれぞれ与えられた可換三角形に固定すればよいのです。

第 4 章で見た 4 つの条件（$r \equiv 0_2$、$r \equiv s$、$s \equiv t$、$t \equiv 1_2$）が、4 枚の面にあたります。

結合律の証人は、この型の要素として表されます。

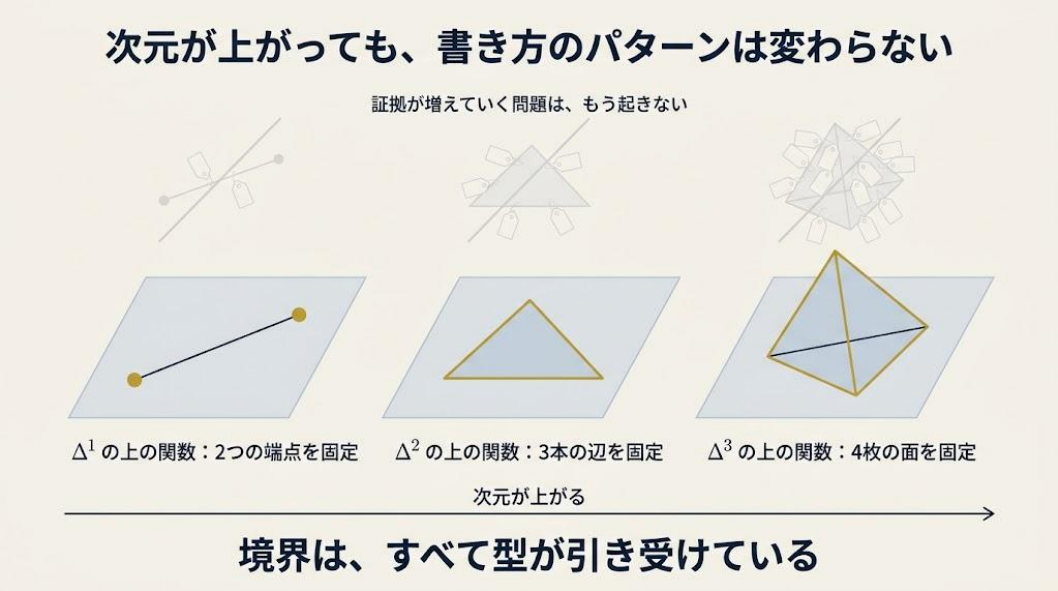
型	何の上の関数か	固定するもの
hom	Δ^1 の上の関数	2 つの端点を固定
hom2	Δ^2 の上の関数	3 本の辺を固定
hom3	Δ^3 の上の関数	4 枚の面を固定（そこから、6 本の辺と 4 つの頂点も決まる）

（ここで hom3 は説明のための呼び名です。sHoTT ライブラリにおける実際の定義名と形は、ライブラリで確認してください。）

次元が上がっても、書き方のパターンは変わりません。

証拠が次元とともに増えていく問題は、もう起きません。

境界は、すべて型が引き受けているから です。



5-9　この章で起きていたこと

ここまでの例で、**Rzk の裏側** では、**2 つの仕組み** が働いていました。

1. **境界の書き換え**
   - hom の要素を 0_2 に適用すると、x に書き換える
   - （型の情報を使って計算する。第 8 章で扱う）
2. **tope solver**
   - 「角 $s \equiv 0_2 \wedge t \equiv 1_2$ は、辺 $s \equiv 0_2$ に含まれるか」
   - 「境界 $t \equiv 0_2 \vee t \equiv 1_2$ は、Δ^1 に含まれるか」といった形の包含を、**自動で** 確かめる

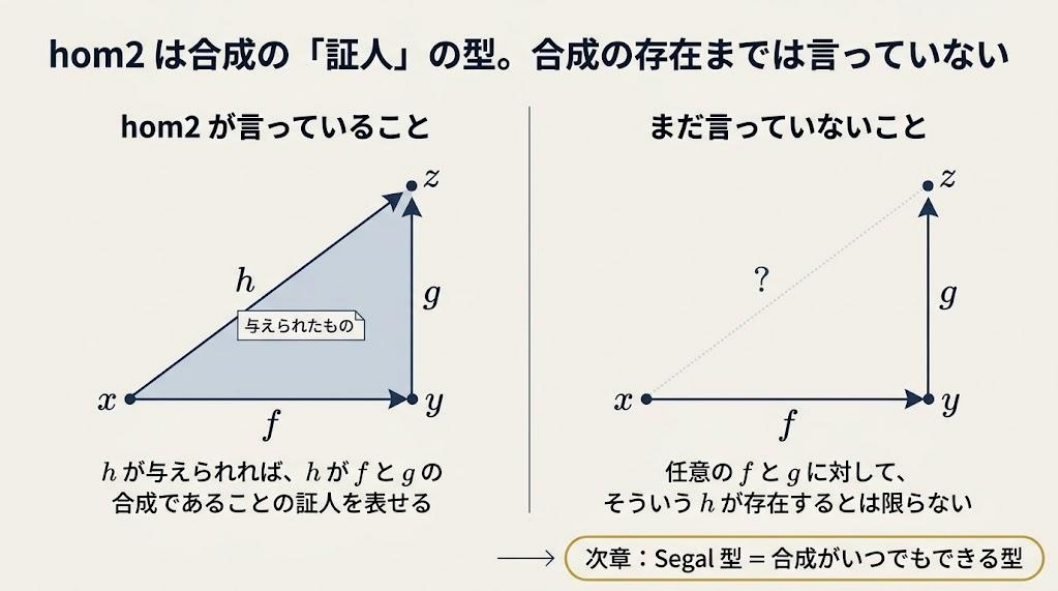
利用者のコードには、どちらも現れません。

利用者が書くのは数学の中身だけで、形の帳簿づけは処理系が引き受けています。

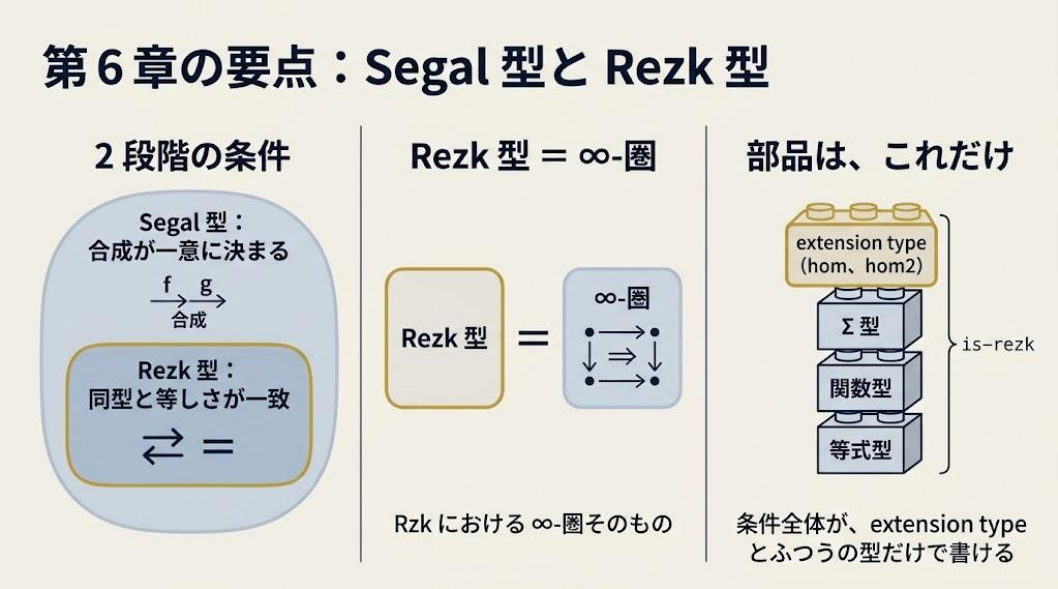
ただし、ここまでで「**射**」と「**可換三角形**」は書けましたが、「**任意の 2 本の射が合成できる**」**とはまだ言っていません。**

hom2 は「h が f と g の合成である」ことの証人の型であって、**そういう h が存在するとは主張していないか** らです。

合成がいつでもできる型を選び出すのが、次章の Segal 型 です。



第 6 章　Segal 型と Rezk 型 ── ∞-圏そのもの



この章の要点：

- 「**合成が一意に決まる型**」 を **Segal 型** 、さらに 「**同型と等しさが一致する型**」 を **Rezk 型** と呼ぶ。
- **Rezk 型が、Rzk における ∞-圏そのものである** 。
- **その条件全体** が、**extension type** と **ふつうの型** だけで書き下せる。

6-1　どの型でも、∞-圏になるわけではない

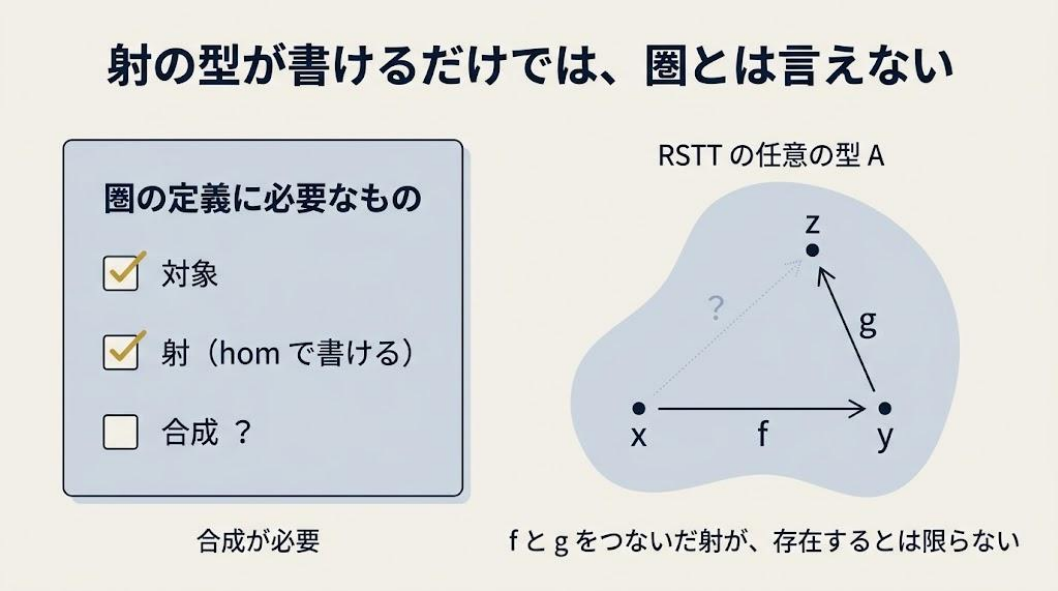
第 5 章で、どんな型 A に対しても `hom A x y` が書けることを見ました。

しかし、**射の型が書けるだけでは、圏とは言えません。**

圏 の定義を満たすためには **合成** が必要だからです。

RSTT の世界では、任意の型 A が自動的に合成を持つわけではありません。

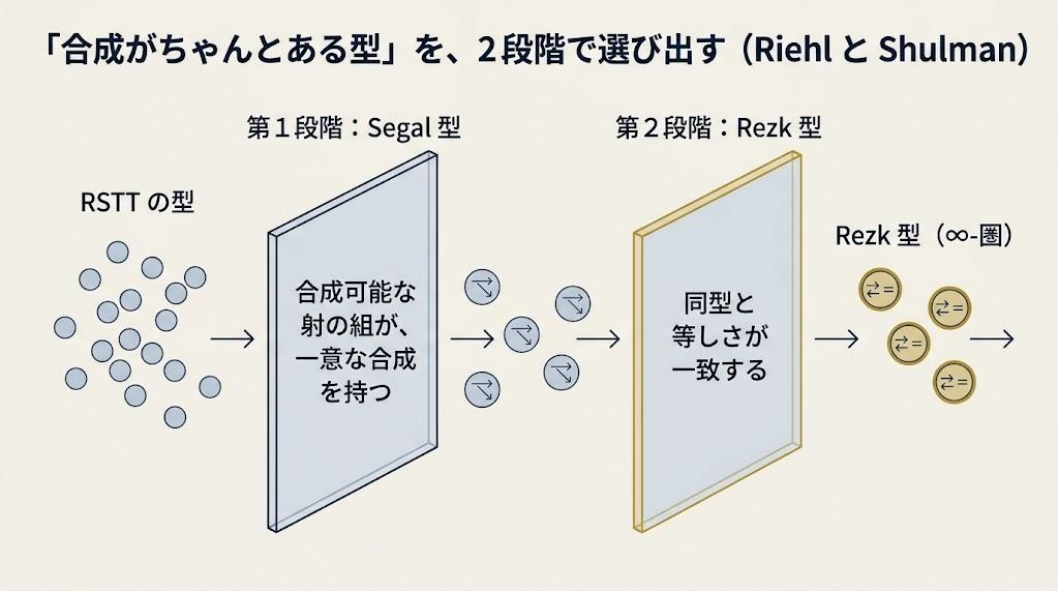
射 $f:x \to y$ と $g:y \to z$ があっても、**それらをつないだ射が存在するとは限らない** のです。



そこで、**「合成がちゃんとある型」を選び出す条件** を考えます。

Riehl と Shulman は、これを 2 段階で行いました。

段階	型	条件
第 1 段階	Segal 型	合成可能な射の組が、一意な合成を持つ型
第 2 段階	Rezk 型	Segal 型のうち、同型と等しさが一致する型



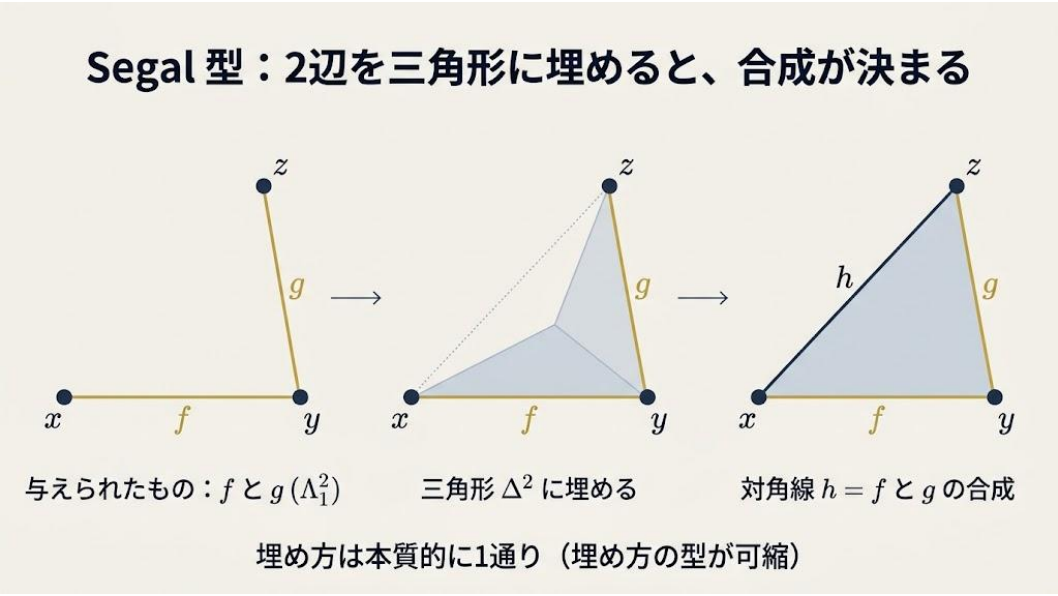
6-2　Segal 型とは

第 4 章で見た inner horn Λ^2_1 を思い出してください。

中の詰まっていない「くの字」の 2 辺でした。

- **与えられたもの**：f : x → y と g : y → z = Λ^2_1 の形（2 辺だけ）
- **欲しいもの**：三角形 Δ^2 全体に埋めること
- **埋めると得られる**：対角線 h : x → z と、三角形の中身（可換三角形の証人）

2 辺を三角形に埋められれば、対角線が「f と g の合成」になります。



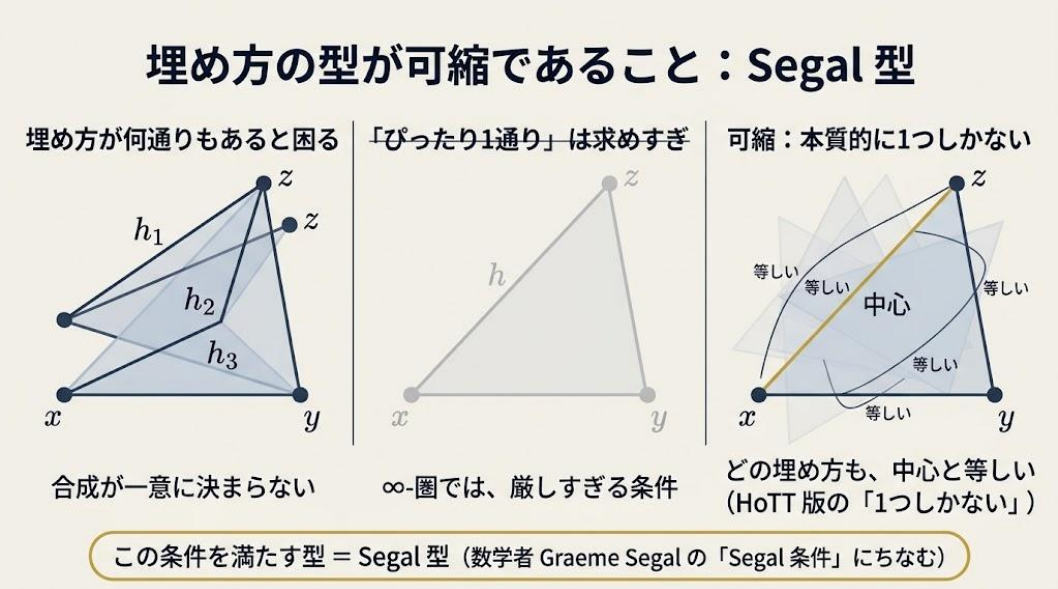
ただし、**埋め方が何通りもあると困ります。**

合成が一意に決まらないから です。

∞-圏では「ぴったり 1 通り」は求めすぎなので、代わりに「埋め方の型が可縮である」ことを求めます。

> 【用語】
>
> 型が **可縮**（*contractible*）であるとは、その型に要素が存在し、しかもどの要素も、その要素と（等式型の意味で）等しいこと。
>
> 「**本質的に 1 つしかない**」の、**HoTT 版** です。

この条件を満たす型を、**Segal 型**（*Segal type*）と呼びます。



名前は、**数学者 Graeme Segal に由来する「Segal 条件」** から来ています。

論文の言葉では、Segal 型は「合成的な前-∞-圏」です。

sHoTT ライブラリ では、**Segal 型であることを表す述語** `is-segal` が、次のように **定義** されています（Riehl と Shulman の定義 5.3 に対応）。

```
#def is-segal
  ( A : U)
  : U
  :=
    ( x : A) → (y : A) → (z : A)
  → ( f : hom A x y) → (g : hom A y z)
  → is-contr (Σ (h : hom A x z) , (hom2 A x y z f g h))
```

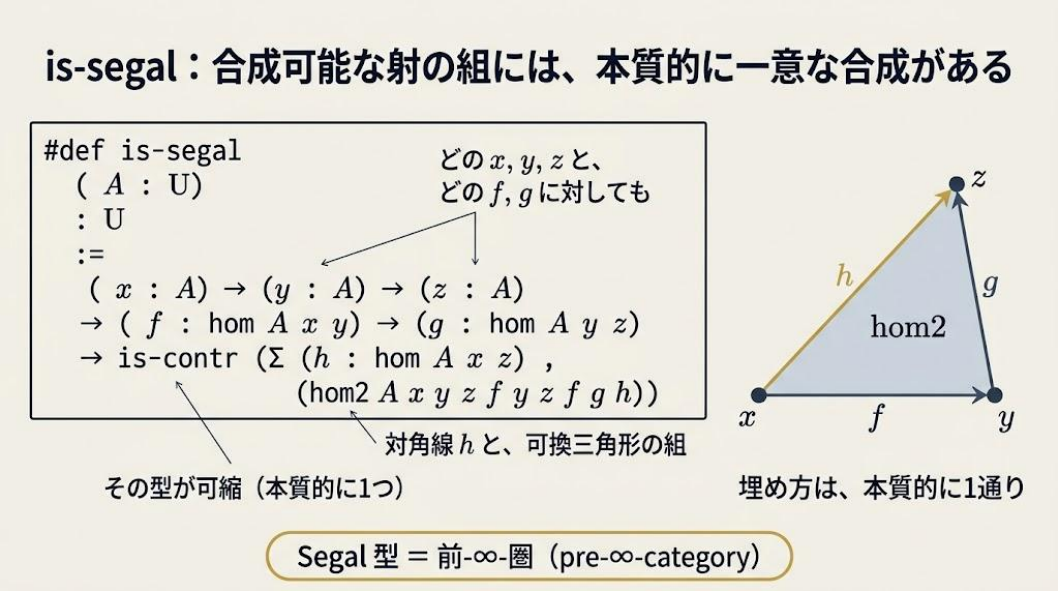
コードを読んでみましょう。

> どの x, y, z と、どの f : x→y、g : y→z に対しても、「対角線 h と、f-g-h の可換三角形の組」の型が、可縮である

つまり、**合成可能な射の組には、必ず、本質的に一意な合成がある。**

これが **Segal 型** です。

Segal 型 は **「前-∞-圏」（*pre-∞-category*）にあたります。**



> 【出典】上のコードは、sHoTT の `src/simplicial-hott/05-segal-types.rzk.md` にある定義を、そのまま引いたものです（2026 年 11 月 6 日時点）。

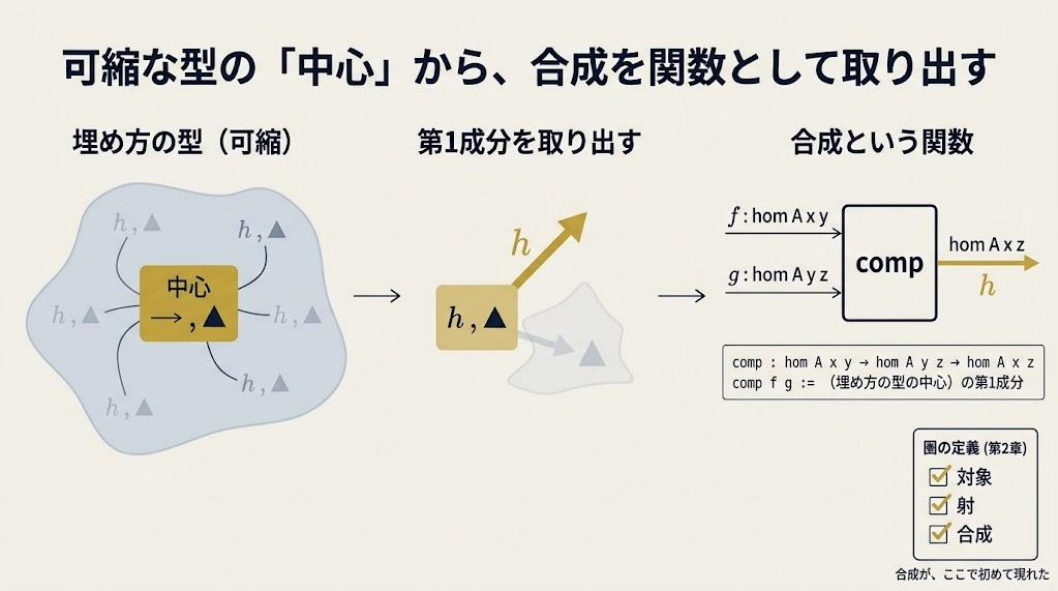
可縮な型 には、**「中心」と呼ばれる要素** があります。

Segal 型 A では、**この中心の第 1 成分を取り出すことで、合成を関数として定義できます。**

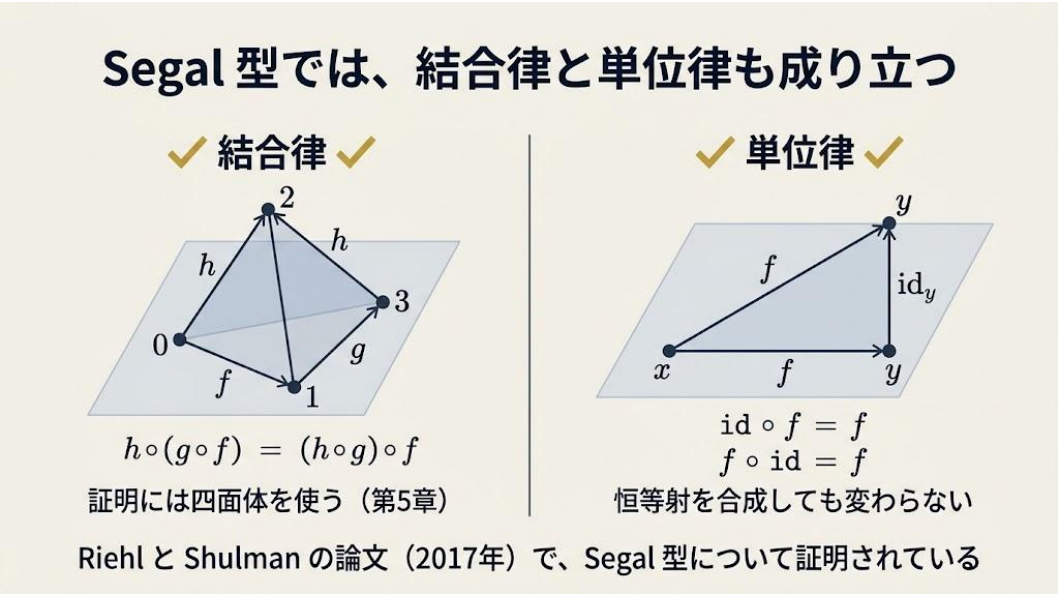
```
comp : hom A x y → hom A y z → hom A x z
comp f g := （埋め方の型の中心）の第 1 成分
```

こうして、**Segal 型の中では、射の合成が「関数」として手に入ります。**

第 2 章の **圏の定義で「合成」と呼んでいたものが、ここで初めて現れた** ことになります。



結合律や単位律も、Riehl と Shulman の論文で、Segal 型について証明されています（結合律の証明には、第 5 章で触れた四面体が使われます）。



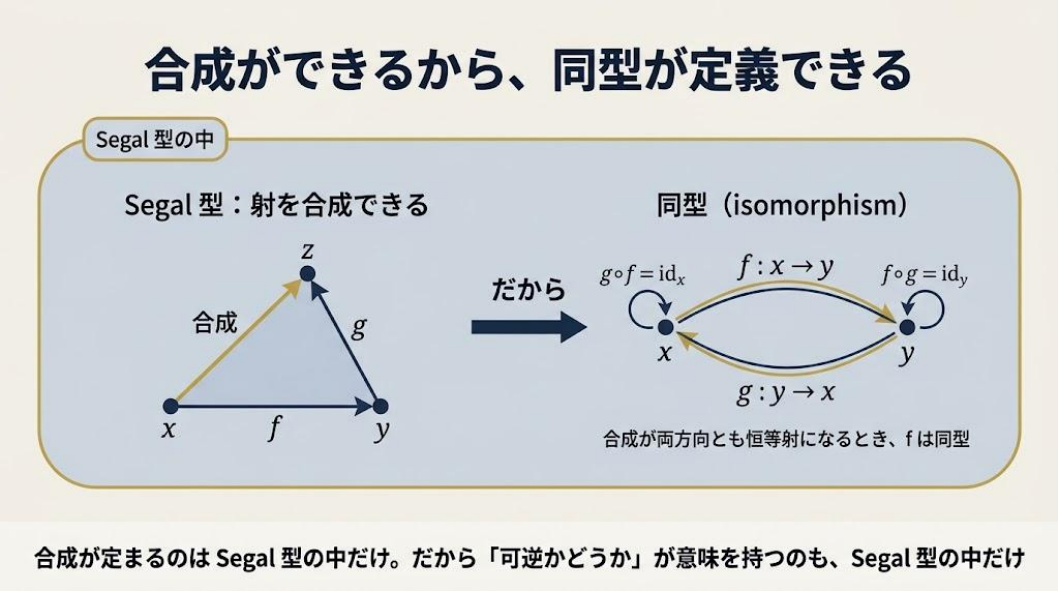
6-3　Rezk 型とは

Segal 型 では、**射を合成できます** 。

だから「逆向きの射を持つ射」、つまり **同型**（*isomorphism*）が定義できます。

$f : x \to y$ に $g : y \to x$ があって、**合成が両方向とも恒等射になるとき** 、f は **同型** です。

論文が指摘するとおり、**合成が定まるのは Segal 型であればこそなので、「可逆かどうか」が意味を持つのも Segal 型の中だけ** です。



ここで、型 A には、もともと 2 種類の「同じ」があります。

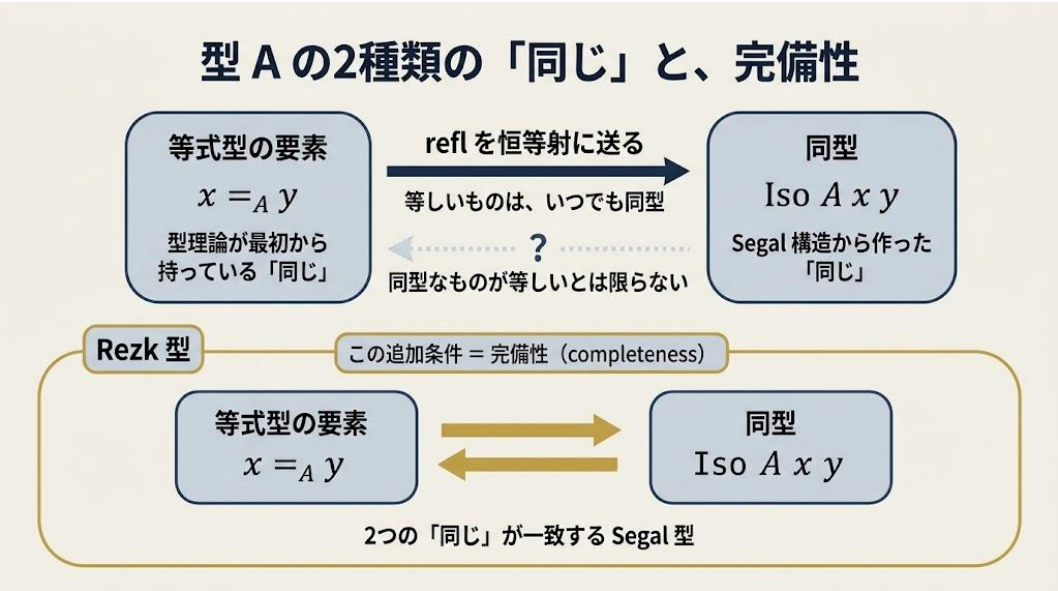
「同じ」の種類	書き方	由来
等式型の要素	x = y	型理論が最初から持っている「同じ」
同型	Iso A x y	Segal 構造から作った「同じ」

この 2 つは、一般には一致しません。

等しいものは同型ですが（等しさの証拠 refl を恒等射に送ればよい）、同型なものが等しいとは限りません。

この 2 つが一致する Segal 型を、Rezk 型（*Rezk type*）と呼びます。

この追加条件を **完備性**（*completeness*）と呼びます。



論文の説明では、**完備性** とは「型理論の等しさ x = y が、すべて Segal 構造における一意な同型から来る」ことです。

Riehl と Shulman は、**Rezk 型** を **(∞,1)-圏の合成的な対応物** として位置づけています。**Rzk という名前の由来も、ここ** でした（1-3）。

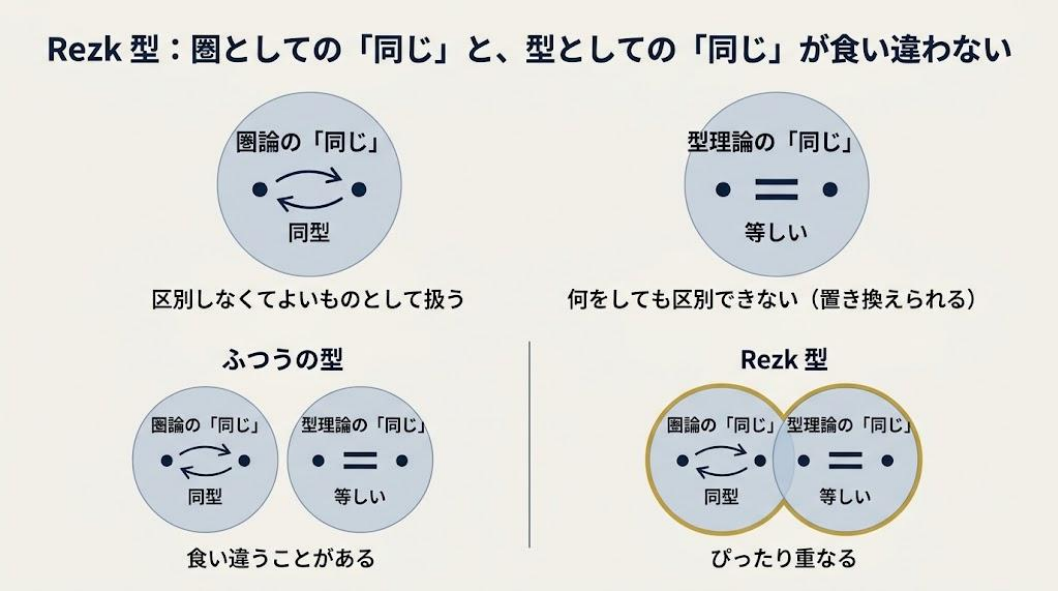
なぜ、**この条件** が **「∞-圏らしさ」** にとって大事なのでしょうか。

圏論 では、**同型な対象は「区別しなくてよい」ものとして扱うのが普通** です。

一方、**型理論** では、**等しいものは「何をしても区別できない」もの** です。

この 2 つが一致していれば、圏論の「同型なら同じとみなす」という習慣が、型理論の「等しいものは置き換えられる」という原理と、**ぴったり重なります。**

Rezk 型は、圏としての「同じ」と、型としての「同じ」が食い違わない型なのです。



> 【コラム】Rezk 型の正確な定義 型 A が Rezk 型であるとは、A が Segal 型であり、かつ任意の x, y : A について、等式型 x = y から同型の型 Iso(x, y) への標準写像（等式型の帰納法で、refl を恒等同型に送るもの）が同値（equivalence）であること（Riehl と Shulman の論文の第 10 節で導入される）。
>
> 同値は HoTT における「同型」であり、両側の逆写像を持つ写像として定義される。

6-4　Rzk での定義

sHoTT のモジュール `simplicial-hott/10-rezk-types` にある定義 を、論文から引きます。

```
#def is-rezk (A : U) : U
  := Σ ( is-segal-A : is-segal A)
   , ( x : A) → (y : A)
     → is-equiv (x = y) (Iso A is-segal-A x y) (iso-eq A is-segal-A x y)
```

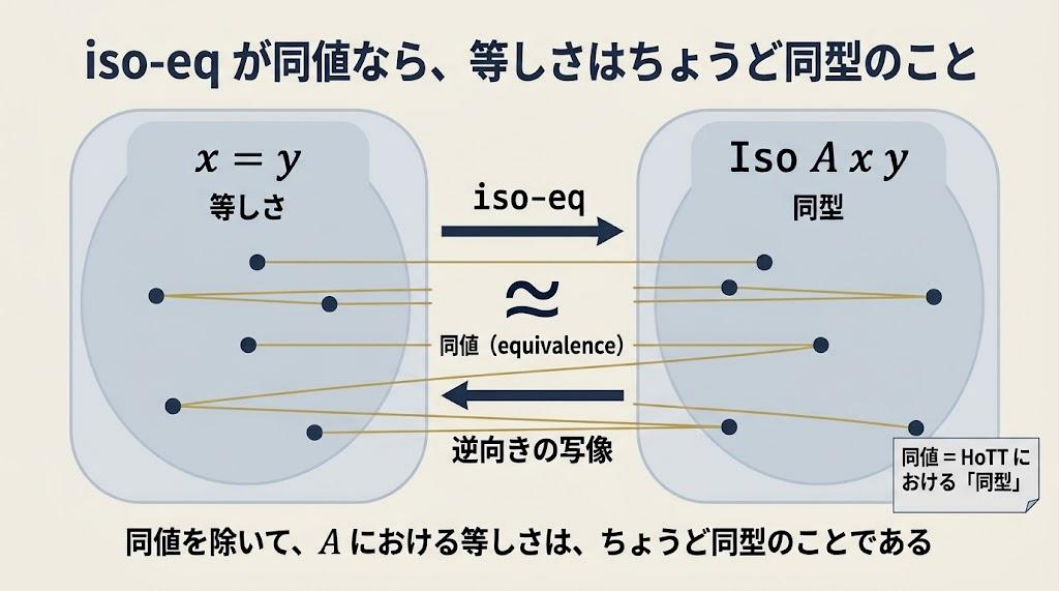
コードから意味を読み取ってみます。

- `Σ ( is-segal-A : is-segal A) , …`
  - A が Segal 型であることの証拠と、次のものの組
- `( x : A) → (y : A) → …`
  - どの x, y に対しても
- `is-equiv (x = y) (Iso A is-segal-A x y) (iso-eq A is-segal-A x y)`
  - 「$x = y$」から「x と y の同型の型」への標準的な写像 iso-eq が、同値（equivalence）である

`iso-eq` は、**等しさの証拠 refl** を **恒等同型** に送る **写像** です（等式型の帰納法で定義されます）。



それが **同値**（*equivalence*、HoTT における「同型」）であるということは、**同値を除いて、A における等しさは、ちょうど同型のことである** という意味です。



> 【用語】「同値を除いて」とは
>
> 数学者は「◯◯を除いて同じ」という言い方をよくします。
>
> これは、「◯◯の違いは気にしなければ、同じとみなしてよい」という意味です。
>
> たとえば、「1 時間」と「60 分」は、書き方は違いますが、日常の場面では同じ長さとして扱って困りません。
>
> 「時間」から「分」へは 60 を掛け、「分」から「時間」へは 60 で割れば、何も失わずに行き来できるからです。
>
> この例を、言葉に当てはめると次のようになります。

- **同値**：「60 を掛ける」と「60 で割る」という、行きと帰りの換算がそろっていること
- **除いて**：「時間」と「分」という書き方の違いを、気にしないこと
- **同じ**：「1 時間」と「60 分」を、同じ長さとして扱うこと

> つまり、「1 時間と 60 分は、換算を除いて同じ」と言えます。
>
> 本文の「同値を除いて」も同じ形です。

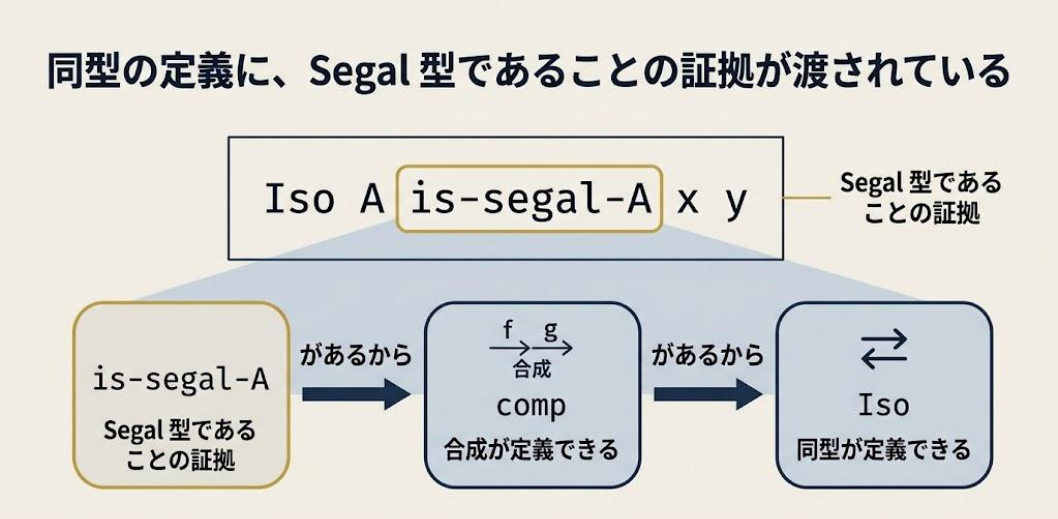
- 「1 時間」にあたるのが、型 `x = y`（等しさ）
- 「60 分」にあたるのが、型 `Iso A x y`（同型）
- 「60 を掛ける」にあたるのが、写像 `iso-eq`
- 「60 で割る」にあたるのが、その逆向きの写像

> 行きと帰りがそろっている（同値である）ので、書き方の違いを気にしなければ、等しさと同型は同じもの、と言えるのです。

`Iso A is-segal-A x y` の **引数** に、**Segal 型であることの証拠 `is-segal-A` が渡されていること** にも注目してください。

同型を定義するには合成が要り、合成を定義するには Segal 型であることの証拠が要るから です。

6-3 で見た **「Segal 型であればこそ同型が意味を持つ」ことが、コードの形に現れています。**



「Segal 型であればこそ同型が意味を持つ」ことが、コードの形に現れている（6-3）

6-5　この述語が、何でできているか

`is-rezk` を分解していくと、最後に残るのは次のものだけです。

- **extension type**：hom、hom2（cube と tope の層の上に乗る）
- **Σ 型**：組
- **関数型**：「どの x, y に対しても」
- **等式型**：$x = y$

つまり、**「これは (∞,1)-圏である」という条件全体が、extension type と、ふつうの型の組み合わせで書けている** のです。

その裏側では、第 5 章と同じく 2 つの仕組みが働いています。

- **境界の書き換え**：is-rezk の証拠を端点で使うたびに、型の情報を使って計算する
- **tope solver**：Segal 部分に出てくる「単体」「horn」の包含の条件を、自動で確かめる

6-6　つまり、Rzk は何ができるのか

ここまでの話を、1 文にまとめます。

> **Rzk は、「この型は (∞,1)-圏である」という条件そのものを書き下し、それを満たすことの証明を機械で検査できる。**

第 3 章の **素朴なやり方** では、**可換三角形の段階ですでに「ほとんど乗り越えられない」と言われていました。**

Segal 条件 は三角形を、その先の議論は四面体やさらに高次の形を使います。

Rzk は、その全体を、書けて検査できるものにしました。

ここで、少し立ち止まってみます。

∞-圏論は、大学院で何年もかけて学ぶ分野 です。

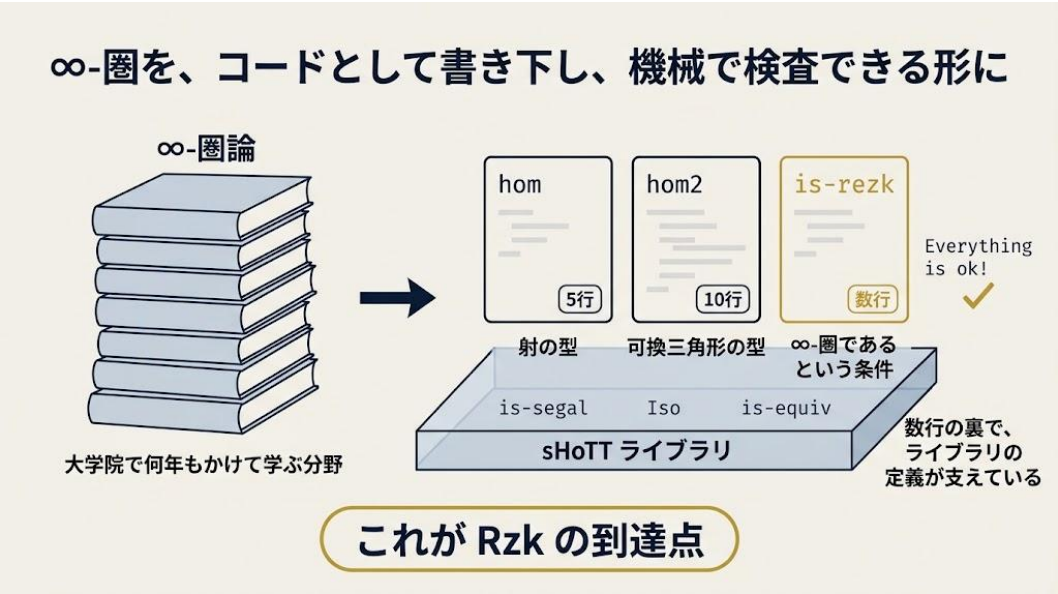
その **基本の道具** である **「射」の型** `hom` は 5 行、 **「可換三角形」の型** `hom2` は 10 行のコードで定義できました。

そして、**「この型は ∞-圏である」という条件** `is-rezk` も、**定義そのものは数行** です。

もちろん、その数行の裏では、`is-segal` や `Iso` といった **sHoTT ライブラリの定義** が支えています。

それでも、**∞-圏という対象を、コードとして書き下し、機械で検査できる形にした。**

これが Rzk の到達点 です。

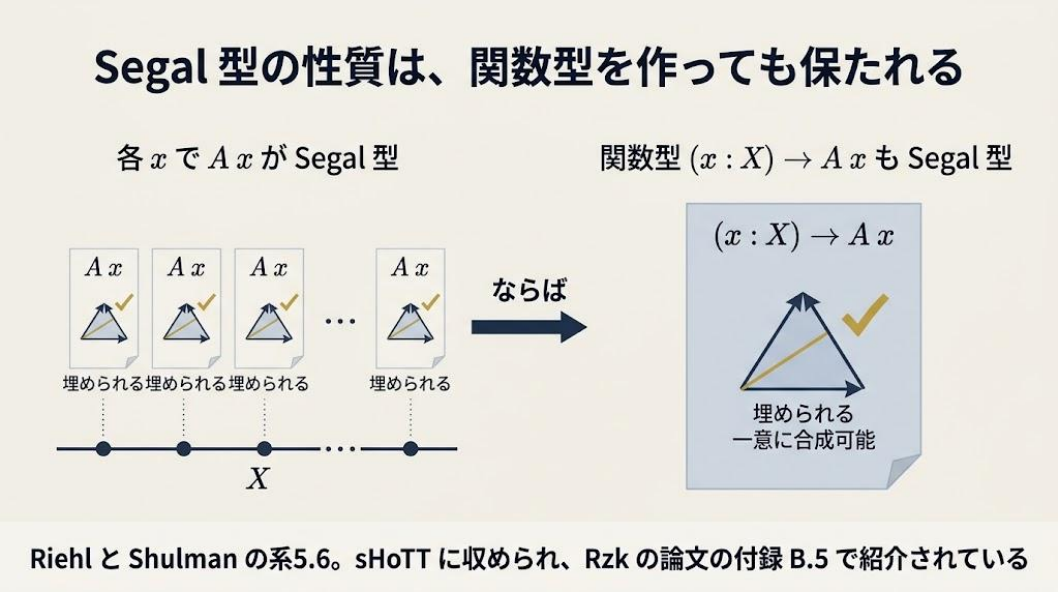


6-7　関数型も Segal 型になる

最後に、**sHoTT にある定理** を 1 つ紹介します。 **論文の付録 B.5** で取り上げられているものです。

Segal 型の性質は、関数型を作っても保たれます。

「各 x で Ax が **Segal 型** なら、**関数型** $(x{:}X) \to Ax$ も **Segal 型** 」という **定理** です（Riehl と Shulman の系 5.6）。



Riehl と Shulman の系5.6。sHoTT に収められ、Rzk の論文の付録 B.5 で紹介されている

sHoTT では、次のように **宣言** されています。

```
#def is-segal-function-type uses (funext)
  ( X : U)
  ( A : X → U)
  ( fiberwise-is-segal-A : (x : X) → is-segal (A x))
  : is-segal ((x : X) → A x)
  := ...
```

（本体は省略しています。論文によれば、horn の包含についての補題を経由して証明されます。）

ここで目を引くのが、`uses (funext)` という書き方です。

funext は **「関数外延性」という公理** で、**「すべての入力で値が等しい 2 つの関数は等しい」と主張** します。

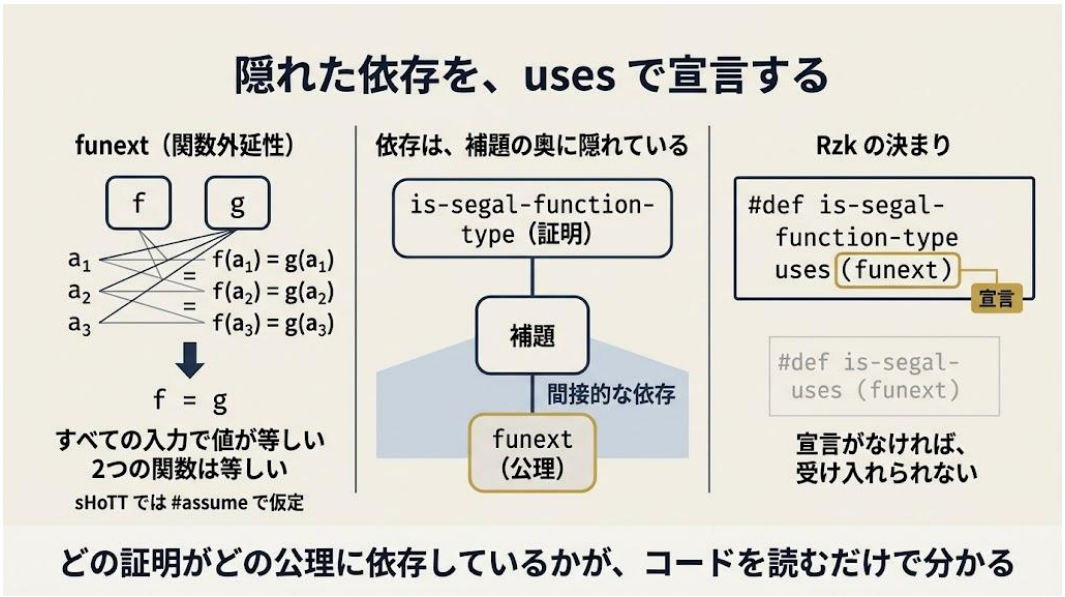
sHoTT では、この公理を `#assume` で **仮定** しています。

論文の説明によれば、この証明は funext を直接は使っていません。

使っている補題の奥で、間接的に使っています。

Rzk は、**こうした「隠れた依存」を、`uses` で宣言しない限り受け入れません。**

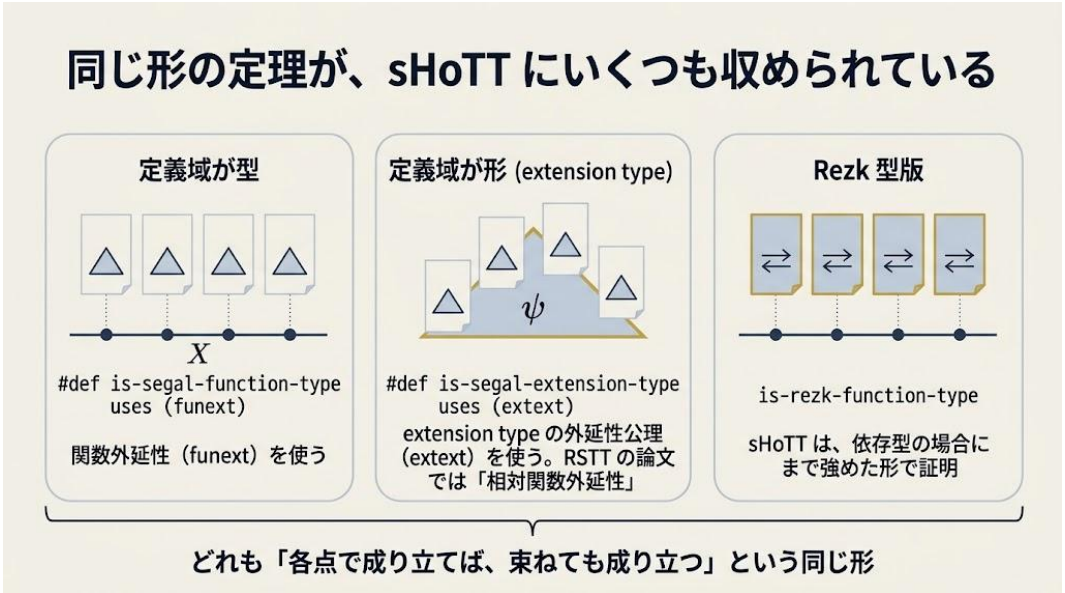
どの証明がどの公理に依存しているかが、コードを読むだけで分かる仕組み です。



同じ定理は、**型ではなく形を定義域とする場合（extension type の場合）にも成り立ち** 、**sHoTT** には **`is-segal-extension-type` として収められています。**

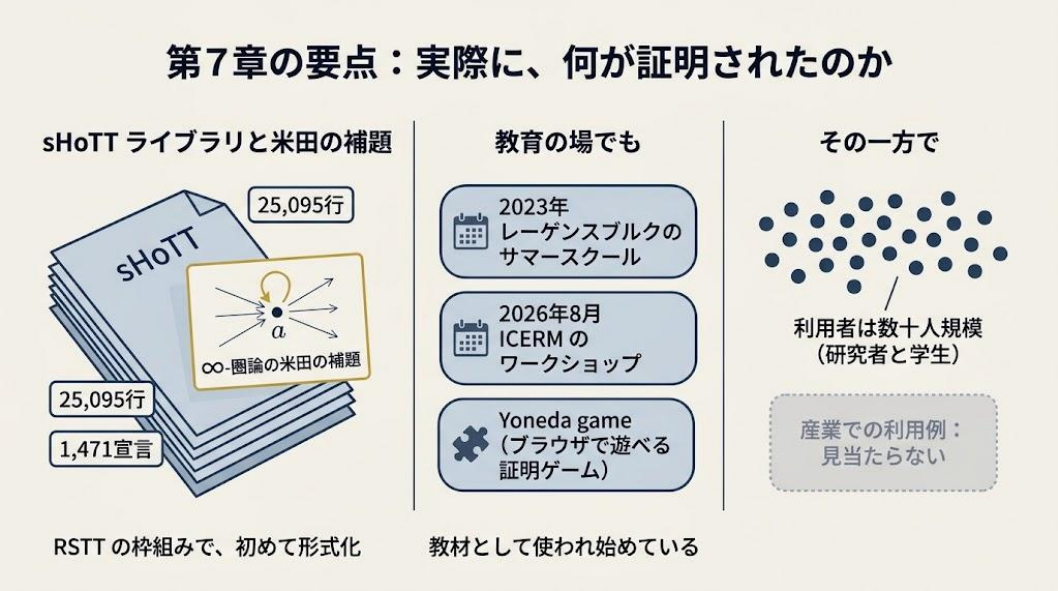
こちらは、**extension type についての外延性公理（RSTT の論文では「相対関数外延性」と呼ばれるもの）** を **`uses (extext)` で宣言** しています。

Rezk 型についても同様の定理 があり、**sHoTT は依存型の場合にまで強めた形で証明** しています。



次章では、**この土台の上で、実際に何が証明されたのか** を見ます。

第 7 章　実際に、何が証明されたのか



この章の要点：

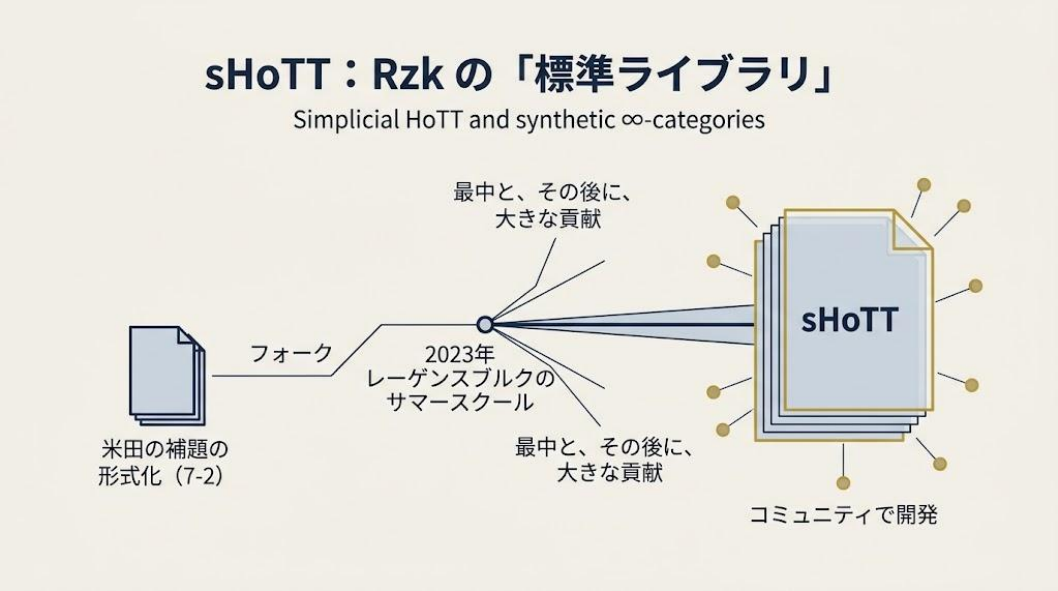
- Rzk の上に、25,095 行・1,471 宣言の **sHoTT ライブラリ** が築かれ、**合成的 ∞-圏論（RSTT）の枠組みで、∞-圏論の米田の補題が初めて形式化された。**
- 教育の場でも使われ始めている。
- その一方で、利用者は数十人規模の研究者と学生で、産業での利用例は見当たらない。

7-1　sHoTT ライブラリ

Rzk の「標準ライブラリ」にあたる のが、**sHoTT**（*Simplicial HoTT and synthetic ∞-categories*）です。

コミュニティで開発されています。

論文によれば、**sHoTT は、米田の補題の形式化（7-2）のフォークとして始まり、2023 年のレーゲンスブルクのサマースクールの最中と、その後に、大きな貢献を受けて育ちました。**

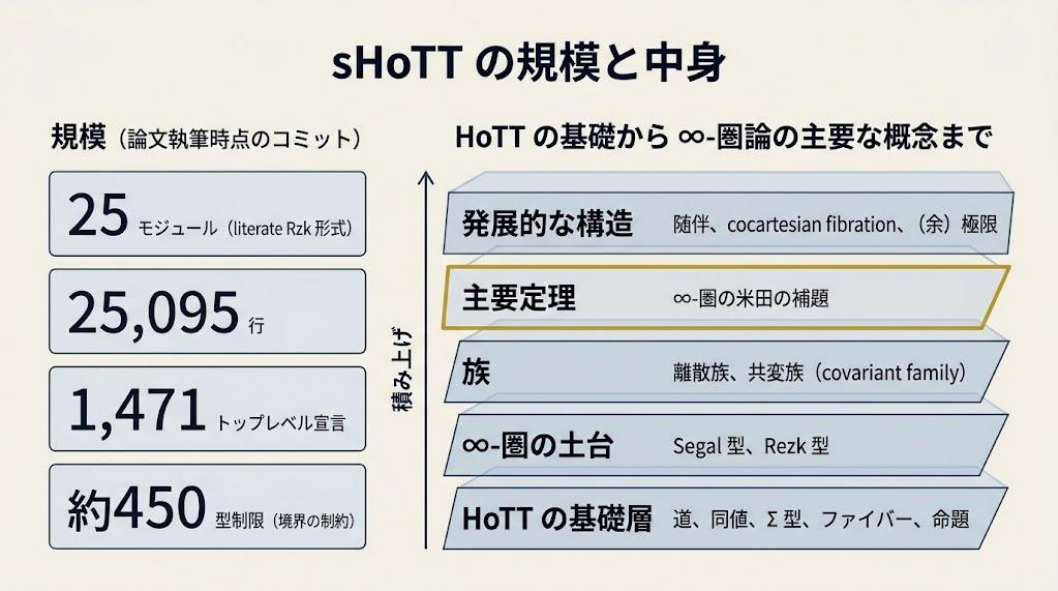


論文が報告する、論文執筆時点のコミットでの規模は次のとおりです。

項目	値
モジュール数	25（literate Rzk 形式）
行数	25,095 行
トップレベル宣言	1,471 個
restriction 型（境界の制約）の数	約 450 個

内容は、**HoTT の基礎から ∞-圏論の主要な概念まで、層をなしています。**

層	内容
HoTT の基礎層	道、同値、Σ 型、ファイバー、命題
∞-圏の土台	Segal 型、Rezk 型
族	離散族、共変族（*covariant family*）
主要定理	∞-圏の米田の補題
発展的な構造	随伴、cocartesian fibration、（余）極限



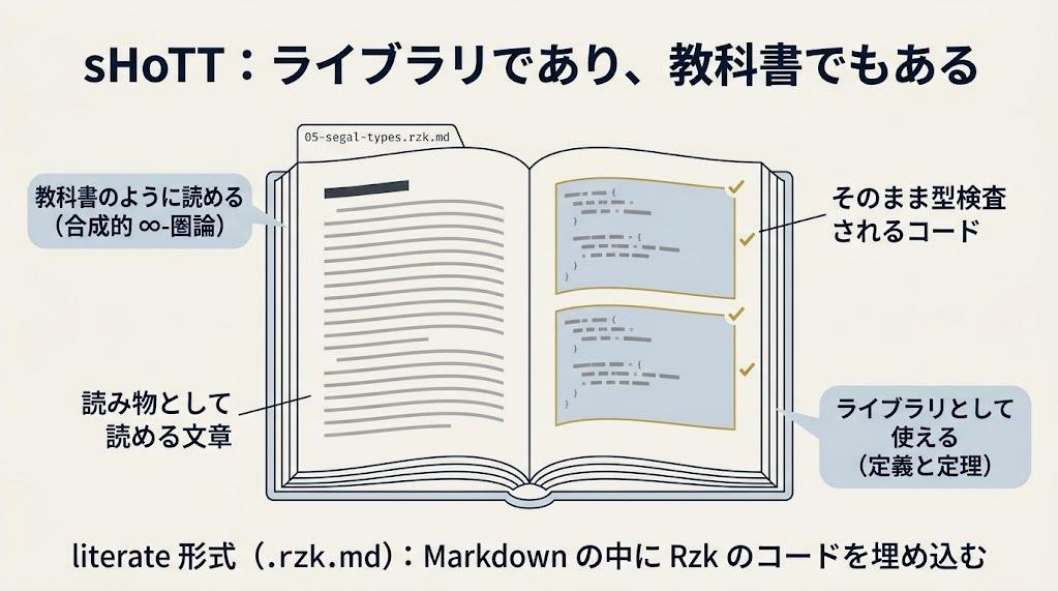
いくつかの用語 を、ひとことずつ説明しておきます。

- **共変族**
  - 型 A の上の型の族で、A の射に沿って「前へ運べる」もの
  - （A の射 f : x → y があれば、x の上の要素を y の上へ送れる）
- **関手**：圏から圏への、射のつなぎ方（合成）を保つ対応
- **随伴**：2 つの関手のあいだの、圏論で最も重要な関係の 1 つ
- **cocartesian fibration**：圏の族を、射に沿って運べるようにまとめた構造
- **（余）極限**：図式を「まとめた」普遍的な対象（積、和、引き戻しなどの一般化）

sHoTT は、Markdown の中に Rzk のコードを埋め込む **literate** 形式（拡張子 `.rzk.md`）で書かれています。

読み物として読める文書が、そのまま型検査されます。

sHoTT は、ライブラリであると同時に、合成的 ∞-圏論の教科書のような性格も持っています。



7-2　∞-圏論の米田の補題



Rzk の最も大きな成果 は、**合成的 ∞-圏論（RSTT）の枠組みにおける、∞-圏論の米田の補題の初めての形式化** です。

Rzk の論文は、この成果を 「**∞-圏論の米田の補題の、最初の形式化**」 と位置づけています。

> Kudasov, Riehl, Weinberger "Formalizing the ∞-Categorical Yoneda Lemma" CPP 2024（Certified Programs and Proofs）

米田の補題（*Yoneda lemma*）は、圏論で最も基本的で、最も重要な定理のひとつです。圏論を学んだ人なら、必ず出会います。

> 【コラム】：米田の補題とは何か

> 大ざっぱに言えば、「ある対象 a がどういうものかは、a から出ていく射の全体を見れば完全に分かる」という定理です。
>
> 正確には、表現可能関手についての定理です。本稿では、内容にはこれ以上立ち入りません。

通常の圏（1-圏） の米田の補題は、汎用の定理証明言語でとうに形式化されています。

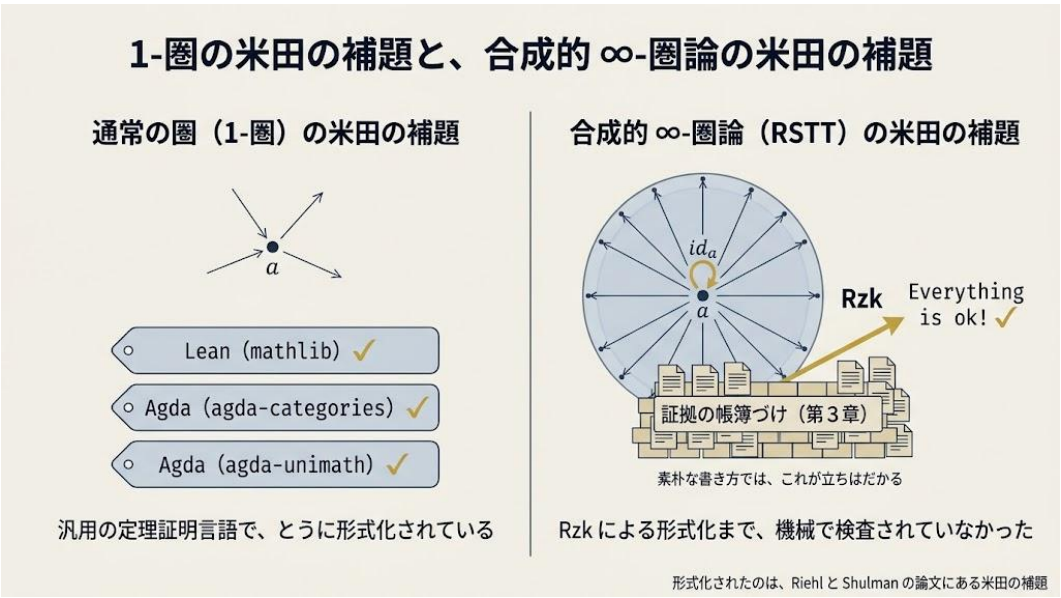
たとえば、次のライブラリに収められています。

- **Lean（mathlib）**：ライブラリに、米田埋め込みとともに米田の補題が収められています。
- **Agda（agda-categories）**：圏論ライブラリの構成に、Yoneda のディレクトリがあります。
- **Agda（agda-unimath）**：∞-圏論の米田の補題を形式化したプロジェクト（Kudasov、Riehl、Weinberger）の一環として、前圏（precategory）に対する米田の補題の形式化が、このライブラリに寄贈されました。

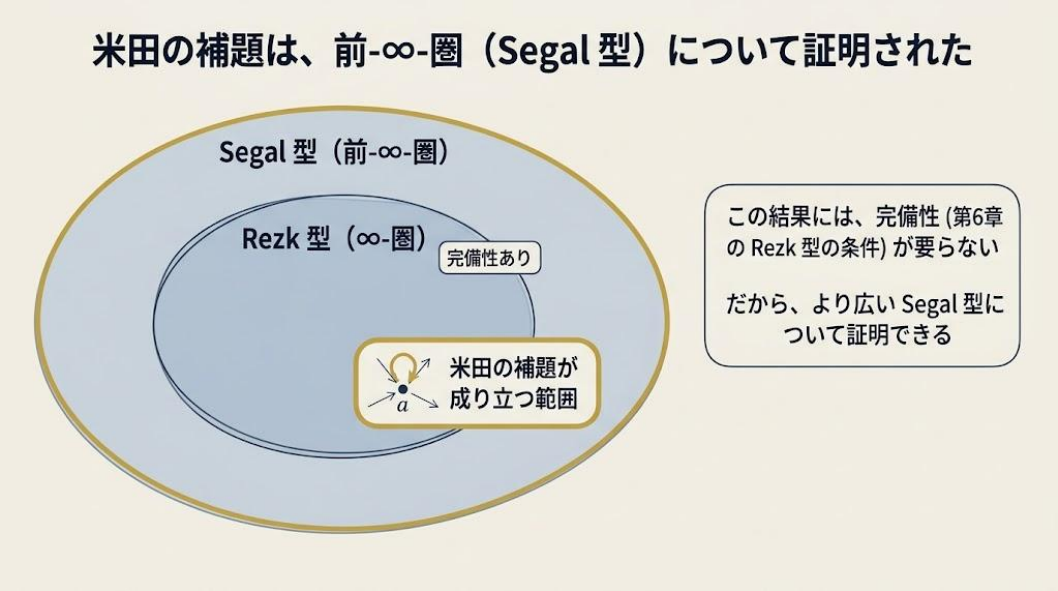
しかし、本稿で扱う **合成的 ∞-圏論（RSTT）における米田の補題** は、**Rzk による形式化までは、機械で検査されていませんでした。**

素朴な書き方では、第 3 章で見た **証拠の帳簿づけが立ちはだかるから** です。

形式化されたのは、Riehl と Shulman の論文にある、**合成的 ∞-圏論の米田の補題** です。



形式化を担った研究者らのプロジェクトのページによれば、**この結果には完備性（第 6 章の Rezk 型の条件）が要らないため、前-∞-圏（Segal 型）について証明** されています。



https://emilyriehl.github.io/yoneda/

> Yoneda for ∞-categories¶
>
> This is a formalization library for simplicial Homotopy Type Theory (HoTT) with the aim of proving the Yoneda lemma for ∞-categories following the paper "A type theory for synthetic ∞-categories" 1. This formalization project could be regarded as a companion to the article "Could ∞-category theory be taught to undergraduates?"

https://rzk-lang.github.io/sHoTT/

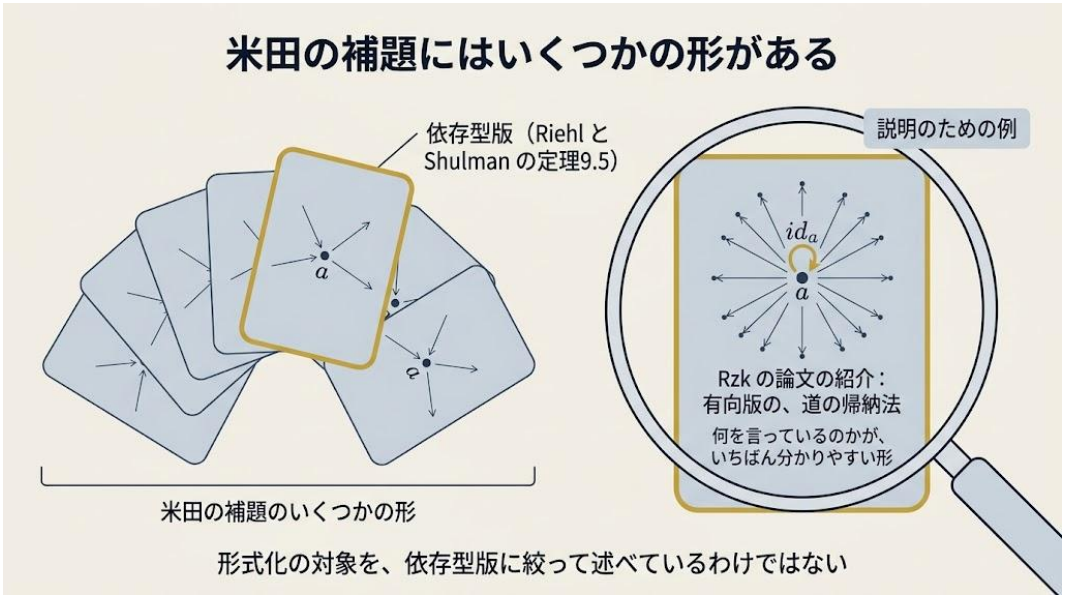
この **米田の補題にはいくつかの形** があります。

そのうちの 1 つ、依存型版 の米田の補題（Riehl と Shulman の定理 9.5）を、**Rzk の論文は「有向版の、道の帰納法」として紹介** しています。

米田の補題が何を言っているのかをつかむには、この形がいちばん分かりやすい ので、ここではこれを例に説明します。

ここで **依存型版を取り上げるのは、説明のための選択** です。

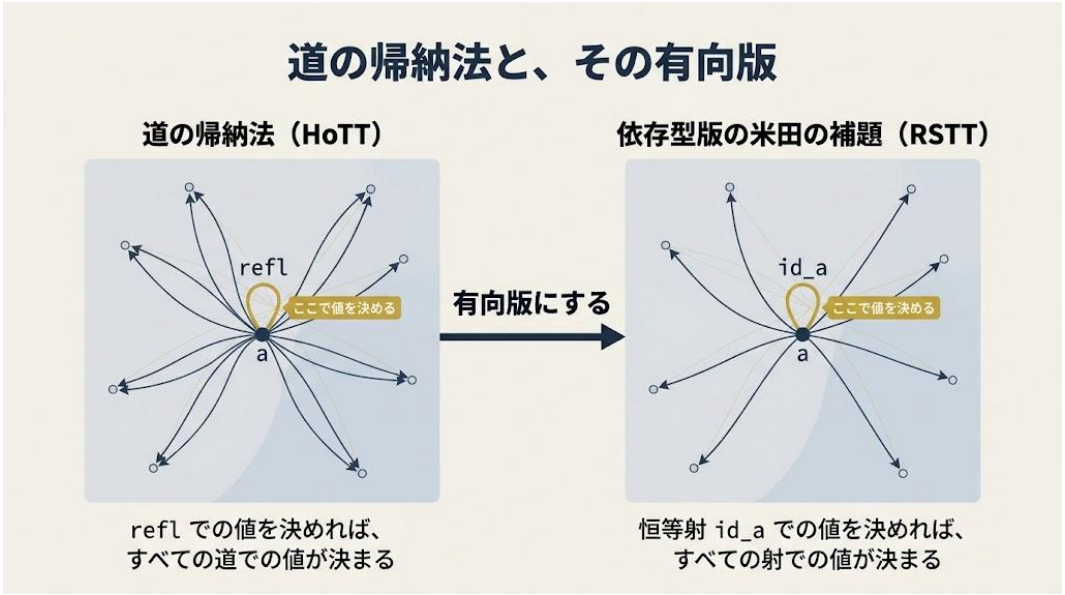
形式化の対象を依存型版に絞って述べているわけではありません。



HoTT では、**等式型** について「**refl での値を決めれば、すべての道での値が決まる**」**という原理** があります。

これを **道の帰納法** と呼びます。

依存型版の米田の補題は、その有向版 です。



論文による要約は、次のとおりです。

> Segal 型 A と、その要素 a、 共変な型族 C : (x : A) → hom A a x → U について、
>
> 「すべての x と、すべての射 f : a → x にわたる C(x, f) の要素の族」は、 恒等射 id_a での値によって、同値を除いて決まる

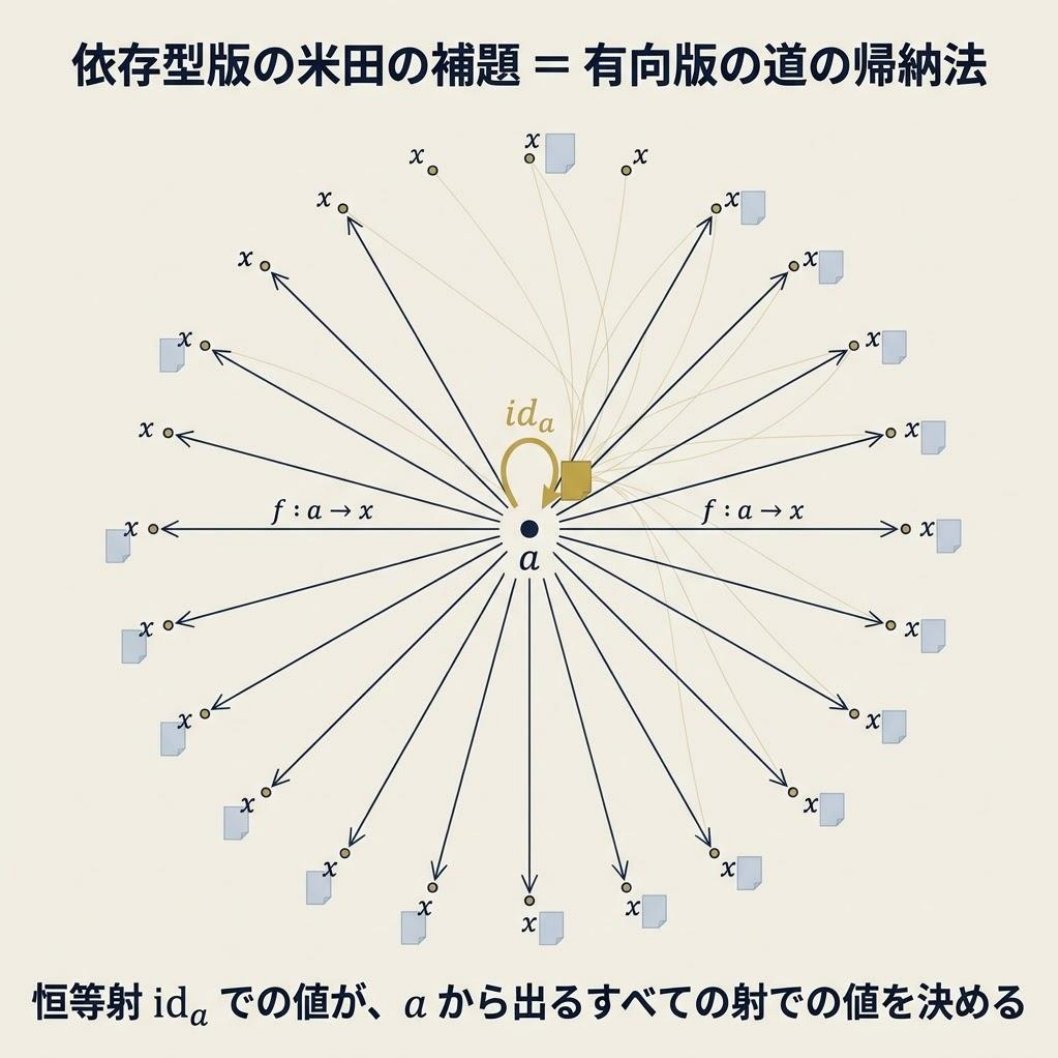
道の帰納法との対応を並べます。

	道の帰納法（HoTT）	依存型版の米田の補題（RSTT）
出発点	点 a	点 a
たどるもの	a から出る道	a から出る射
決め手	refl での値	恒等射 id_a での値
必要な条件	なし	A が Segal 型、C が共変

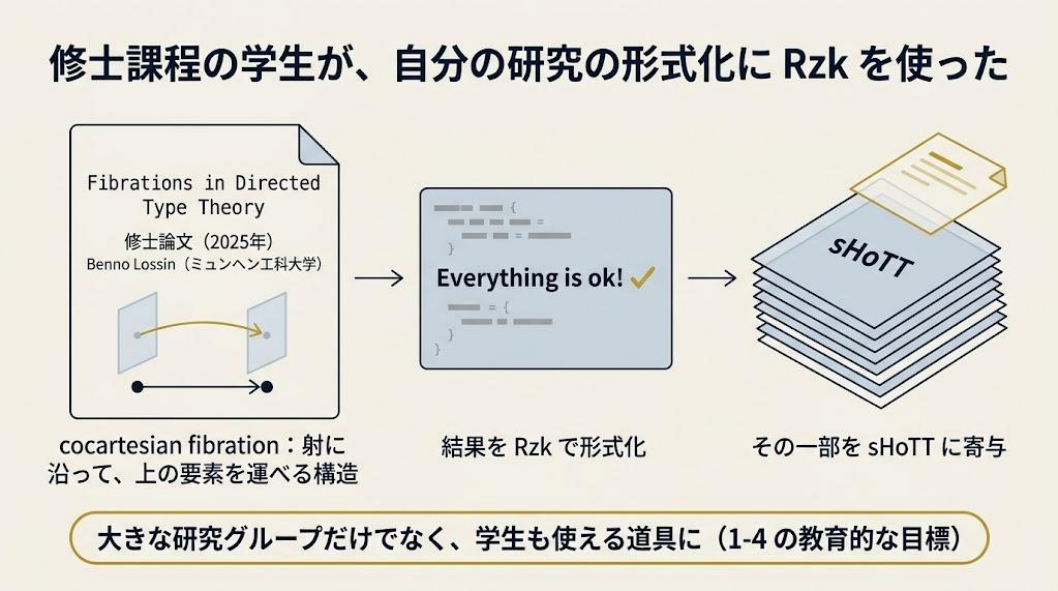
数学の中心的な定理が、その数学のために作られた言語で、初めて機械検査された。

これが、**Rzk の到達点を示す最もはっきりした事実** です。

論文が関連研究の節で述べているとおり、各種の定理証明言語における圏論ライブラリと米田の補題の形式化の概観は、この CPP 2024 の論文にまとめられています。



7-3　Lossin の修士論文（2025 年）



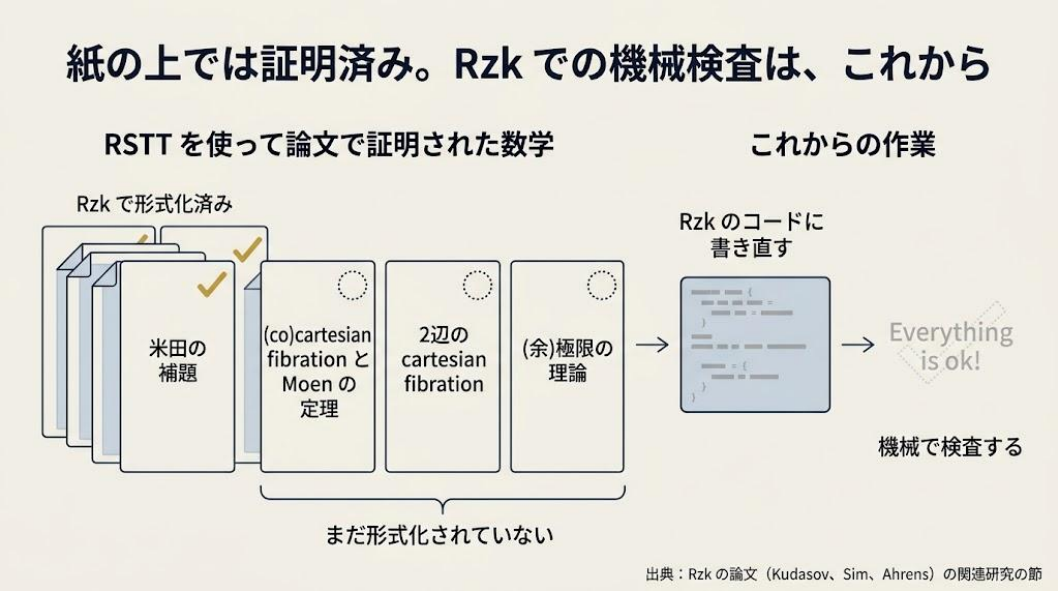
ミュンヘン工科大学の Benno Lossin は、2025 年の修士論文 *Fibrations in Directed Type Theory* で、**合成的 (∞,1)-圏論** における **cocartesian fibration** を展開しました。

結果を Rzk で形式化し、その一部を sHoTT に寄与しています。

大きな研究グループだけでなく、**修士課程の学生が、自分の研究の形式化に Rzk を使った**という点は、教育的な目標（1-4）との関係で重要です。

Rzk の論文（Kudasov、Sim、Ahrens *Rzk: a Proof Assistant for Synthetic ∞-Categories*） の関連研究の節によれば、RSTT を使って論文で証明された数学のうち、**まだ Rzk で形式化されていないもの** もあります。

紙（論文）の上では証明済みでも、それを **Rzk** のコードに書き直し、機械（定理証明支援系の処理系）で検査する作業は、これからだということです。



たとえば、次の研究がそうです。

- **Weinberger（2022 年）、Buchholtz と Weinberger（2023 年）**：(co)cartesian fibration の理論と、Moen の定理の証明
- **Weinberger（2024 年）**：2 辺の cartesian fibration
- **Bardomiano Martínez（2025 年）**：(余)極限の理論

紙の上で展開された数学を、Rzk で形式化していく作業は、まだ続いています。

7-4　教育での使用

Rzk は、研究の道具としてだけでなく、∞-圏論を教える場でも使われ始めています。

第 1 章で紹介した Riehl の問い「∞-圏論を、学部生に教えられるか」に、実際の教室で答えようとする試みです。

これまでに、次の 2 つの場で使われています。

- **2023 年**
  - レーゲンスブルクのサマースクール
  - Rzk と sHoTT を教材として使用
  - 参加者が初期の Rzk を試し、sHoTT への貢献が始まった
- **2026 年 8 月**
  - ICERM のワークショップ
  - *Teaching Higher Category Theory with Computers*

- （コンピュータで高次圏論を教える）
- 実習が Rzk にもとづく

ワークショップの名前そのものが、「∞-圏論を教えられるか」という問いと響き合っています。

論文の謝辞には、レーゲンスブルクのサマースクールの参加者が初期の **Rzk** を試したこと、そして Emily Riehl と Jonathan Weinberger が **Rzk** の「最初の外部の利用者」だったことが記されています。

7-5　Yoneda game

Yoneda game は、Lean 4 の「ゲーム」形式（Natural Number Game などで知られる、証明をパズルとして解く形式）にならった、Rzk 版の証明ゲームです。論文によれば、Lean 4 のゲームのような証明ゲームのエンジンが Rzk の上に作られており、その最初のゲームが、開発中の Yoneda game です。ブラウザで遊べます。

https://rzk-lang.github.io/yoneda-game/

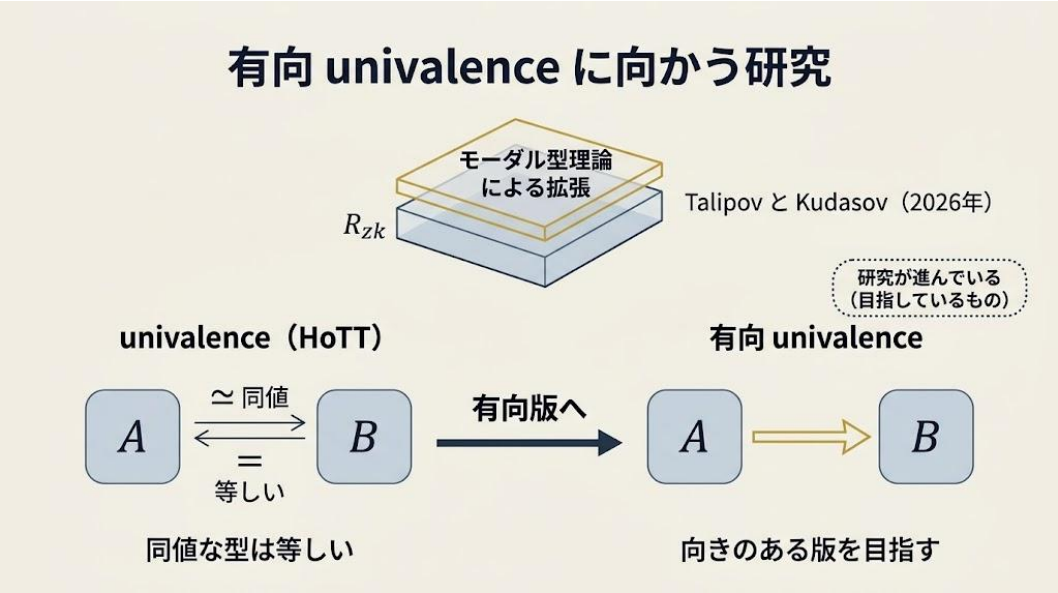
ゲームの題材が米田の補題であることにも、注目してください。

7-2 の到達点を、学習者がたどれる道筋にしようとしています。

このほか、**Rzk** を **モーダル型理論で拡張** し、**有向 univalence**（*directed univalence*）に向かう研究も進んでいます（Talipov と Kudasov、2026 年）。

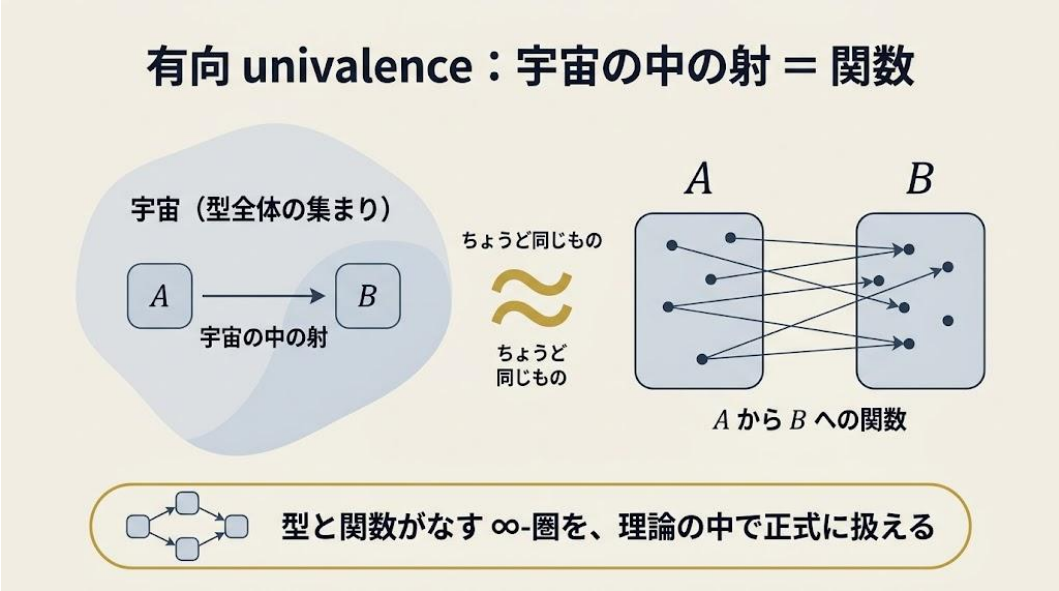
univalence は、**HoTT で「同値な型は等しい」と主張する公理** でした。

有向 univalence は、その有向版を目指すもの です。

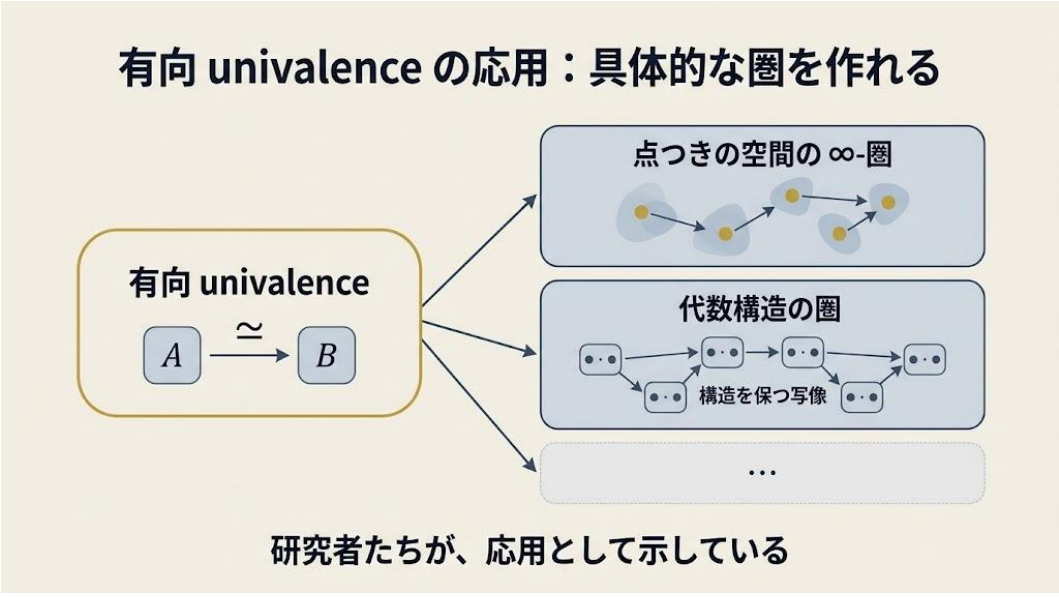


有向 univalence は、型全体の集まり（宇宙）の中で、**「型 A から型 B への射」が、「A から B への関数」とちょうど同じものになる、という原理** です。

有向 univalence が成り立つと、「型と関数がなす ∞-圏」を、理論の中で正式に扱えるようになります。



研究者たちは、**その応用** として、**点つきの空間の ∞-圏や、代数構造の圏といった、具体的な圏を作れること** を示しています。

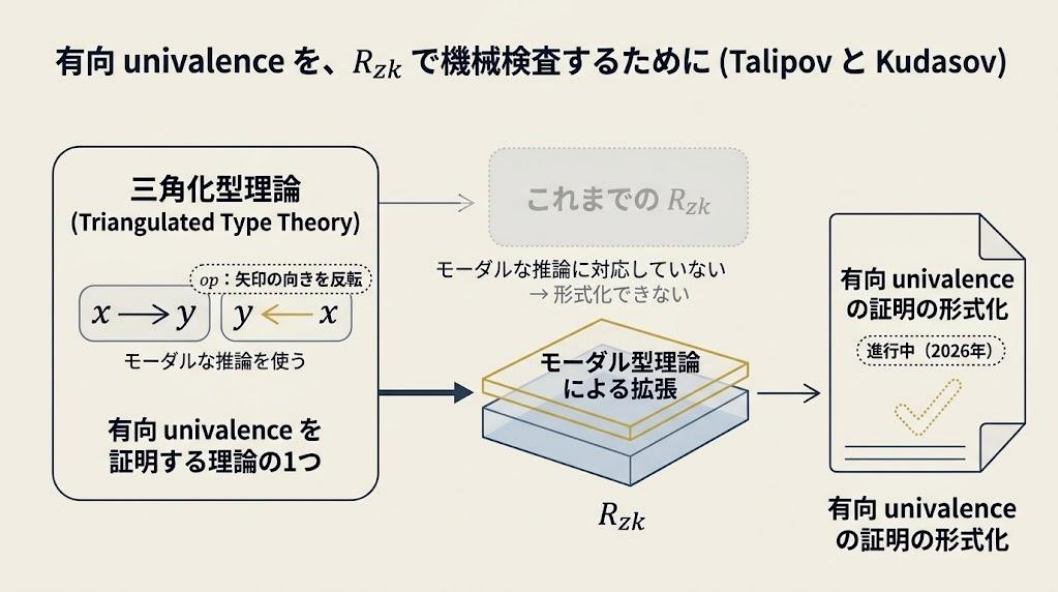


Talipov と Kudasov の **研究の狙いは、この有向 univalence を、Rzk の上で機械検査できるようにすること** です。

有向 univalence を証明する理論の 1 つ（三角化型理論、*Triangulated Type Theory*） は、**矢印の向きを反転させる操作などの「モーダルな」推論** を使います。

これまでの Rzk は **モーダルな推論に対応していなかったため、この理論を形式化できません** でした。

そこで、**Rzk をモーダル型理論で拡張し、その上で有向 univalence の証明を形式化しようとしています。**



2026 年の時点では、形式化はまだ進行中です。

「〇〇として同じ」の 3 つの話を整理する

ここまでで、「〇〇として同じ」ものを **同一視する** という話が、**3 つ登場** しました。

似た言い回しなので混同しやすいため、ここで関係を整理します。

1. 射を関数として読む

射 を 区間の点に当てる話で、**1 本の射** を **「区間の上の関数」としても読める** 、と説明しました。

Riehl と Shulman の論文では、この 2 つの読み方を、記法の約束で同一視していました。

Rzk は、この読み替えを、**部分型の規則** として扱っています。

これは、**「同じ」の意味が一致するという話ではありません。**

同じ 1 つのものを、2 通りに読める という話です。

2. 圏として「同じ」と、型として「同じ」が一致する（Rezk 型）

Rezk 型 は、**圏としての「同じ」**（逆向きの射で行き来できる、つまり同型）と、**型としての「同じ」（等しい）** が **食い違わない型** でした。

本稿の冒頭で挙げた、**∞-圏を扱うために言語に求められる 4 つのことのうち、「『同じ』の意味が食い違わないこと」は、これにあたります。**

∞-圏を扱うための、欠かせない条件です。

3. 宇宙の中の射と、関数が一致する（有向 univalence）

有向 univalence は、**型全体の集まり（宇宙）** の中で、**「型 A から型 B への射」が、「A から B への関数」とちょうど同じものになる、という原理** でした。

「射として同じ」と「関数として同じ」が一致する、という話 は、ここに当たります。

ただし、これは **まだ研究中の目標** で、**Rzk** の上での形式化も進行中です。

∞-圏を扱うための前提条件ではなく、その先の研究の目標 です。

3 つの話の違い

話	何が「同じ」になるか	位置づけ
射を関数として読む	射と、区間の上の関数	同じものを 2 通りに読める（読み替え）
Rezk 型	圏としての「同じ」と、型としての「同じ」	**∞-圏を扱うための、欠かせない条件**
有向 univalence	宇宙の中の射と、関数	**その先の研究の目標**

Rzk は、∞-圏論の議論を書き下すための言語として生まれました。

ただし、いまの **Rzk** が得意なのは、抽象的な ∞-圏論です。

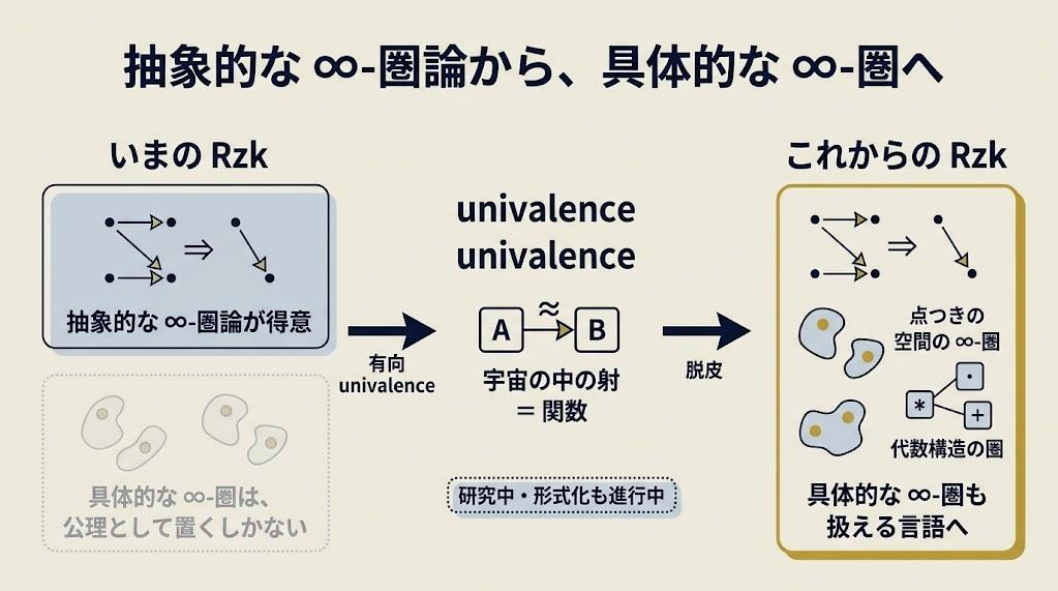
点つきの空間の ∞-圏のような **具体的な ∞-圏** は、いまの **Rzk** では、性質を公理として置くしかありません。

上の表の最後の行にある有向 univalence は、この状況を変えうるものです。

有向 univalence を使えば、具体的な ∞-圏を、理論の中で作れるようになるからです。

Rzk は、抽象的な ∞-圏論を書く言語から、**具体的な ∞-圏も扱える言語へと、脱皮を遂げようとしている** のです。

ただし、これはまだ研究中の段階で、**Rzk** の上での形式化も進行中です。



7-6　型検査の性能 ── tope solver の実測

この節では、**Rzk のプログラムが実際にどれくらいの速さで動き、どこに時間がかかっているのかを測った結果** を紹介します。

数学の内容ではなく、道具としての性能の話なので、興味がなければ後回しにしても構いません。

論文は、**tope solver**（形の包含を自動で確かめる仕組み。第 8 章で扱う）の**性能** を、**sHoTT 全体を型検査して** 測っています。

論文自身が、この節の目的を「tope solver の不完全性が、実践では大きな制約にならないことを示す」ことだと述べています。

測定は、2 つの方法で行われました。

1 つは、GHC（Haskell のコンパイラ）のプロファイラで、関数ごとにかかった時間を測る方法です。

もう 1 つは、solver への問い合わせ（クエリ）を 1 件ずつ記録し、それぞれ何段の探索を要したかを数える方法です。

探索段数は、計算機の速さに左右されない「難しさ」の指標です。

測定環境は、**Rzk v0.7.8、GHC 9.10.2（Stackage LTS 24.4）** です。

項目	値
solver へのクエリ数	25,615

項目	値
型検査時間に占める solver の割合	約 10%
1 クエリの探索段数（中央値／99 パーセンタイル／最大）	1 ／ 35 ／ 19,507
1 段で解けたクエリ	15,133（59.1%）
False を返したクエリ	9,125（35.6%）
飽和上限に達した回数	0

分布は、極端に偏っています。

59% が 1 段で片づき、99% が 35 段以内に収まります。

区間の線形性による場合分けを使ったのは、82 クエリ（0.3%）だけでした。

一方、最も難しい 2 つのクエリだけで、全探索段数の 22%を占めています。 プロファイラの結果も、同じ偏りを示しました。

典型的なクエリは自明で、全体の時間は一握りの難しいクエリに集中しています。

最も難しいクエリは、sHoTT の `05-segal-types.rzk.md` にある定義 `h^` で現れました。

6 分岐の場合分け（recv）を検査するにあたり、分岐の条件が「$\Lambda^3_1 \boxtimes \Lambda^2_1 \subseteq \Delta^3 \times \Delta^2$」という複雑な形（pushout-product 形）を覆うことを確かめる必要があり、19,507 段の探索（うち 825 回の場合分け）を要しました。

確かめる目標は、5 次元の cube の点の 8 つの射影についての、≤ 不等式の「または」でした。

失敗についても、論文は報告しています。

solver には、導出する事実の数に上限（飽和の上限）がありますが、sHoTT 全体で、この上限に達したことは一度もありませんでした。

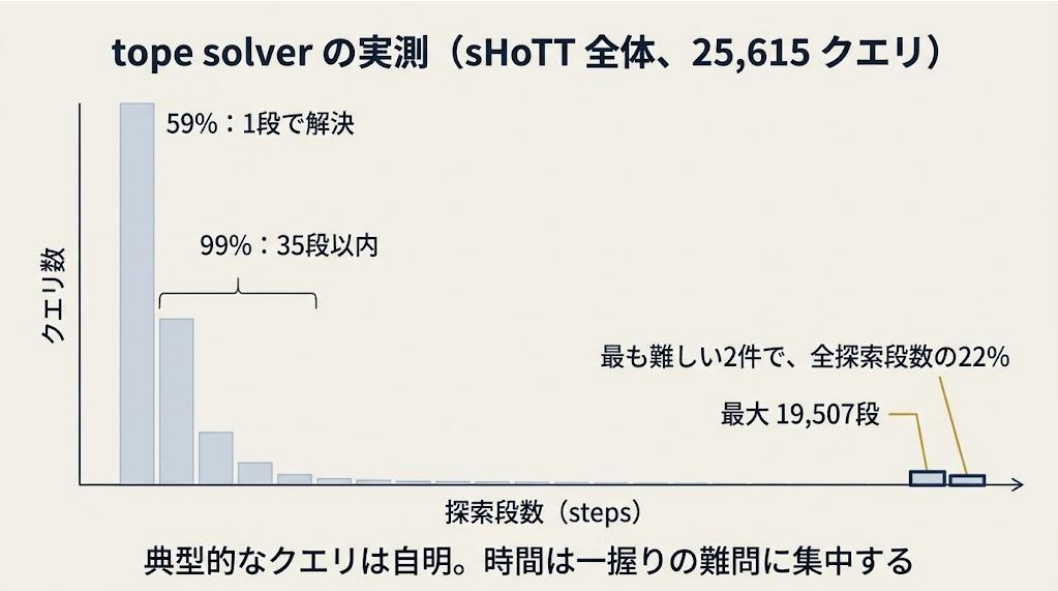
False を返した 9,125 件は、型検査器が側条件を試しに調べ、別の経路を取った結果であり、正当な「いいえ」です。

sHoTT 全体の型検査は成功し、**solver のために手で回避策を書く必要が生じた開発は、1 つもありませんでした。**

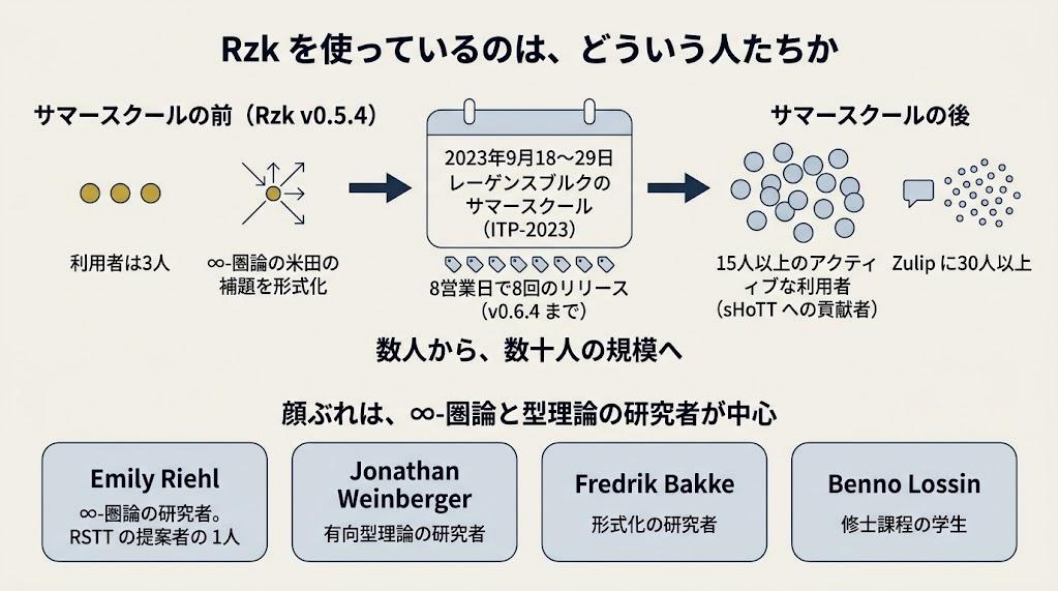
そのうえで、論文は自らの実装に厳しい評価も下しています（筆者訳）。

> とはいえ、ほとんど自明なクエリに答える solver が 10%を占めるということは、実装が非効率であることを示している。normalization-by-evaluation のバックエンドとともに、solver は最適化の最初の候補のひとつである。

自分の道具の弱点を、数字とともに正直に書いている点は、この論文の美点の 1 つです。



7-7　利用者は、誰なのか



Rzk を使っているのは、どういう人たちでしょうか。

手がかりになるのが、Kudasov が 2023 年 10 月に行った発表のスライドです。

このスライドは、2023 年 9 月 18〜29 日にレーゲンスブルクで開かれたサマースクール（*Interactions between Proof Assistants and Mathematics*、略称 ITP-2023）の前後で、Rzk の利用がどう変わったかを報告しています。

サマースクールの前（Rzk v0.5.4）

- 利用者は 3 人（Emily Riehl、Jonathan Weinberger、Fredrik Bakke）
- この 3 人と Kudasov で、∞-圏論の米田の補題を形式化していた

サマースクールの後

- Rzk は v0.6.4 まで進み、8 営業日で 8 回のリリースを重ねた
- 15 人以上のアクティブな利用者（sHoTT への貢献者）
- Rzk の Zulip（チャット）に 30 人以上の参加者

サマースクールをきっかけに、利用者が数人から数十人の規模へ広がったことが分かります。

顔ぶれは、∞-圏論と型理論の研究者が中心 です。

人物	立場
Emily Riehl	ジョンズ・ホプキンス大学。∞-圏論の研究者。RSTT の提案者の 1 人
Jonathan Weinberger	有向型理論の研究者。米田の補題の形式化の共著者
Fredrik Bakke	形式化の研究者
Benno Lossin	修士課程の学生（2025 年に修士論文）

処理系そのものの開発にも、複数の人が関わっています。

論文の謝辞によれば、言語サーバーと VS Code 拡張を Abdelrahman Abounegm が、Nix による開発環境と playground を Danila Danko が、タプルのパターンや入れ子の Σ 型の糖衣構文などを Ekaterina Maximova が手がけました。

バグの報告や提案を寄せた人として、Fredrik Bakke、Benno Lossin、Kenji Maillard、Tashi Walde の名前も挙がっています。

リポジトリの規模 も挙げておきます（いずれも 2026 年 11 月 6 日時点の GitHub の表示）。

Rzk 本体のリポジトリ は、スターが 311、フォークが 31、リリースが 40 回で、最新版は v0.11.4 です。

実装言語は Haskell です。

sHoTT のリポジトリ は、スターが 73、GitHub 上の貢献者が 23 人です。

利用者は、数十人の規模です。

Lean の mathlib4 の GitHub 上の貢献者が 835 人（2026 年 11 月 6 日時点）であることと比べると、きわめて小さな共同体です。

7-8　研究成果と、産業での成果

研究面の成果は、ここまでに見たとおりです。

年	成果
2023 年〜2024 年	∞-圏論の米田の補題の、初の形式化（形式化は 2023 年春、論文は 2024 年 1 月の CPP 2024）
2025 年	Lossin の修士論文（cocartesian fibration）
2026 年	Rzk そのものの論文（POPL 2027）
継続中	sHoTT ライブラリ（25,095 行）

産業での利用例は、筆者が確かめた範囲では見当たりません。

確認したもの

- 論文の記述：学術的な利用のみ
- リポジトリの README：形式化プロジェクトのみ
- Zulip の参加者：研究者と学生

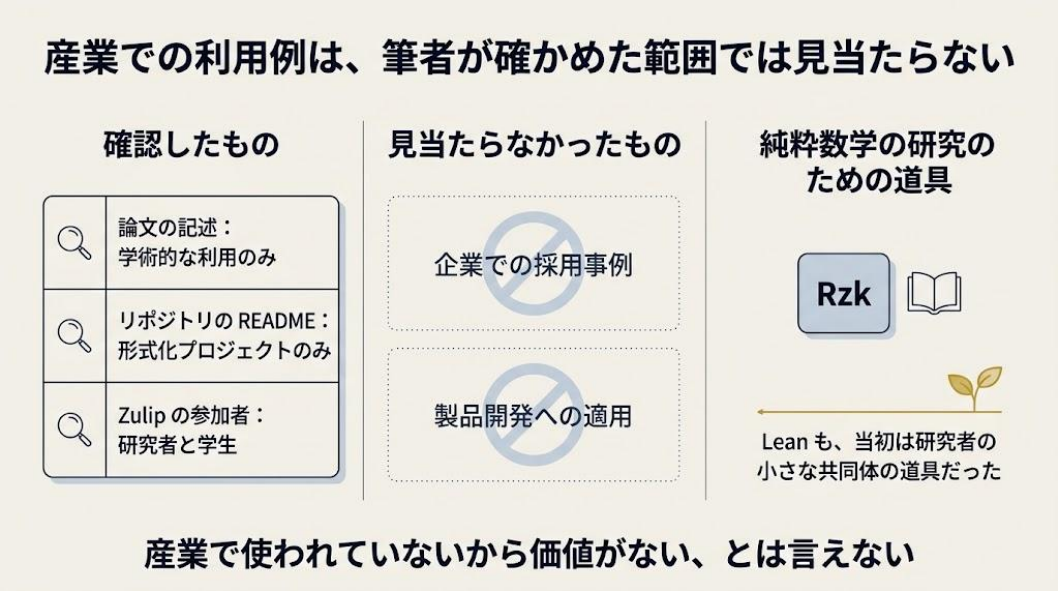
見当たらなかったもの

- 企業での採用事例
- 製品開発への適用

Rzk は、産業で使われている言語ではありません。純粋数学の研究のための道具です。

ただし、「産業で使われていないから価値がない」とは言えません。

Lean も、当初は研究者の小さな共同体の道具でした。



7-9　周辺のツール

研究用の実験的な言語でありながら、**周辺のツールはよく揃っています。**

ツール	役割
Rzk Playground	ブラウザだけで動く。GHCJS で JavaScript にコンパイルされている
`rzk lsp`	Language Server Protocol の実装。複数ファイルを増分で再検査する
vscode-rzk	VS Code 拡張
rzk-action	GitHub Action。形式化を CI で検査する
mkdocs-plugin-rzk	図を自動で描く MkDocs プラグイン
pygments-rzk	構文強調
hottbook	HoTT 本の形式化

論文は、**ブラウザで動く playground が「入門の教育に便利だと分かった」と述べています** 。

言語サーバーについては、複数ファイルにまたがる開発を増分で再検査できる一方、より豊かな対話機能はまだ開発中だとしています。

本稿の読者にとって、最も重要なのは **playground** でしょう。

インストールせずに、本稿のコードをそのまま試せます。

第 8 章　Rzk が RSTT に加えた改良

この章の要点：

- 紙の理論 RSTT を動く型検査器にするために、Rzk は 5 つの改良を加えた。
- 2 つは理論そのものの改良、3 つは実装上の工夫である。
- どれも、紙の証明が「黙って」済ませていたことを、機械が扱える形にするためのものである。

8-1　なぜ、改良が必要だったのか

紙の証明を読むのは人間です。 人間は、多少の省略や「言わずもがな」を補って読めます。

しかし、型検査器は補えません。

RSTT の論文（紙の上の理論） も、人間が読むことを前提に書かれているので、「言わなくても分かるだろう」という省略や、黙った読み替えをいくつも含んでいます。

そこで Rzk は、その省略や読み替えを、型検査器が扱える「規則」として言語に組み込みました。

Rzk の論文（Kudasov、Sim、Ahrens *Rzk: a Proof Assistant for Synthetic ∞-Categories*） は、その対応を表にまとめています（論文の表 2）。

次の表は、その表に、「Rzk の利用者がコードに書く必要があるか」の列を加えたものです。

RSTT の論文で黙って済ませていたこと	Rzk での対応	利用者がコードに書く必要は	本章
extension type	2 つの部品（shape-Π と型制限）に分ける	書き方は RSTT とほぼ同じ。分け方は型検査器の内部の話	8-2
黙った同一視	強制変換を挿入しない部分型。型検査器が読み替える	ない	8-3
形とその包含	tope family。包含は引数の型として書き、定義域の条件は型検査器が自動で補う	包含を引数の型として短く書くだけ	8-4
理論の外で述べていた cube や tope	定義の引数（first-class parameters）として書く	ある（`(I : CUBE)` のように短く書ける）	8-4
「明らかな側条件」として	tope solver が自動で確かめる	ない	8-5

RSTT の論文で黙って済ませていたこと	Rzk での対応	利用者がコードに書く必要は	本章
省略していた包含の確認			

「ない」の行は、型検査器が自動で処理するので、利用者は何も書かなくてよいものです。

それ以外の行は、自動化ではなく、言語の仕組みとして整理し直したものです。

省略されていたことを、短く、はっきり書けるようにしています。

Rzk の論文は、これらの工夫を、次の 2 つに分けて整理しています。

RSTT に足したもの	分類
split extension types	宣言的 Rzk（理論の改良）
coercion-free subtyping	宣言的 Rzk（理論の改良）
first-class parameters	アルゴリズム的 Rzk（実装の工夫）
bidirectional typing	アルゴリズム的 Rzk（実装の工夫）
automated tope solver	アルゴリズム的 Rzk（実装の工夫）

宣言的（*declarative*）とは、「何が正しいか」を規則として述べる層です。

アルゴリズム的（*algorithmic*）とは、「それをどう確かめるか」を手続きとして述べる層です。

論文は、宣言的な Rzk を dRzk、アルゴリズム的な Rzk を aRzk と略しています。

順に見ていきます。



8-2　split extension types ── extension type を 2 つに分ける

第 5 章の hom の型を、もう一度見てみます。

```
(t : Δ¹) → A [ t ≡ 0₂ ↦ x, t ≡ 1₂ ↦ y ]
```

この **型** は、**2 つの部分** からできています。

- 前半の `(t : Δ¹) → A`：線分 Δ^1 の上の関数である、という部分
- 後半の `[ t ≡ 0₂ ↦ x, t ≡ 1₂ ↦ y ]`：両端では x と y に一致する、という部分

RSTT では、**この 2 つ** は **切り離せない 1 つの型（extension type）として扱われて** いました。

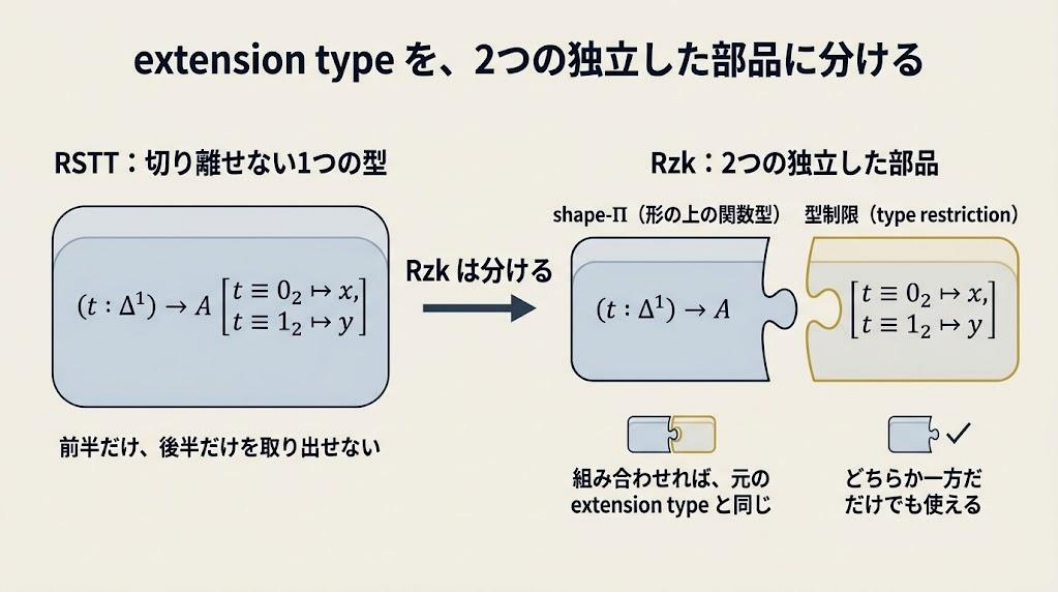
前半だけ、後半だけを取り出して使うことはできないのです。

Rzk は、これを **2 つの独立した部品に分けました。**

- 前半にあたる部品を **shape-Π（ 形の上の関数型 ）**と呼びます。
- 後半にあたる部品を **型制限（ *type restriction* ）**と呼びます。

2 つを組み合わせれば、元の extension type と同じものになります。

そのうえで、**必要に応じて、どちらか一方だけを** 使うこともできます。



部品	書き方	意味
shape-Π	`(t : ψ) → B`	形 ψ の上の関数型
型制限	`B [ ϕ ↦ b ]`	任意の型 B に、境界の制約を付ける
組み合わせると	`(t : ψ) → A [ ϕ ↦ a ]`	＝ RSTT の extension type

shape-Π は、形の上のふつうの関数型です。

RSTT で「境界が空の extension type」と呼ばれていたものにあたります。

型制限（*type restriction*）は、**Rzk 独自の構成要素** です。

「型 B の要素で、φ が成り立つところでは b に一致するもの」の型です。

論文は、**型制限について 3 つの規則** を定めています。

- **形成**：B が型で、φ のもとで b が B の要素なら、B [φ ↦ b] は型
- **導入**：x が B の要素で、φ のもとで x ≡ b なら、x は B [φ ↦ b] の要素
- **計算**：x が B [φ ↦ b] の要素で、いま φ が成り立つなら、x ≡ b

3 つ目の「計算」が、第 5 章で見た「端点で x に書き換える」動きの正体です。

分けることが必要になる場面を、1 つ紹介します。

第 5 章で、射 `f : hom A x y` を区間の点 `s` に当てると、`f s` の型は「ただの `A`」ではなく、「境界の制約が付いた `A`」になる、という話をしました。

この型を、2 つの部品で見直してみます。

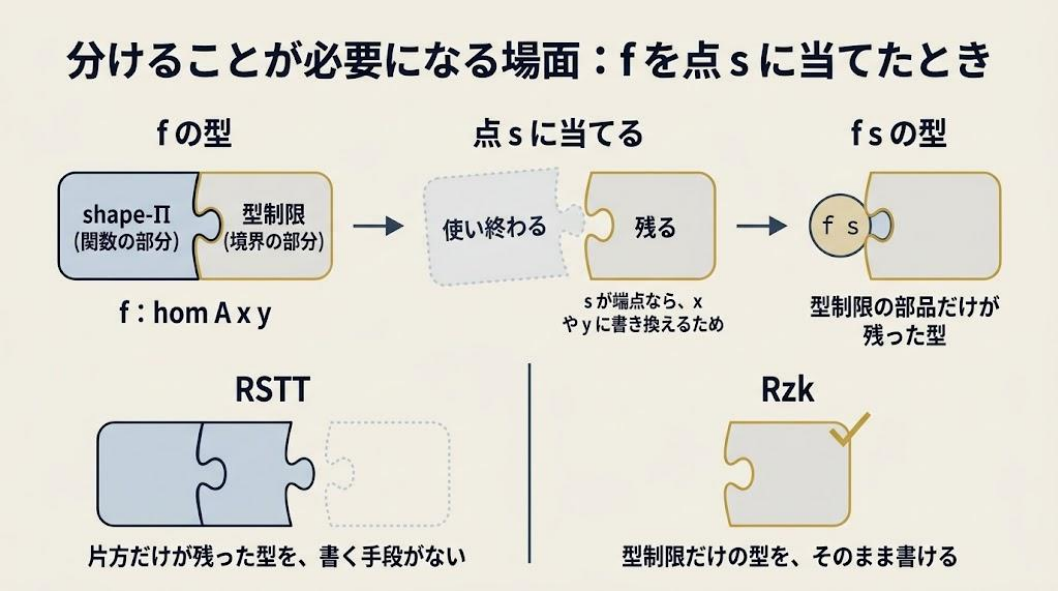
- `f` の型には、shape-Π（関数の部分）と型制限（境界の部分）の両方がありました。
- `f` を点 `s` に当てると、関数の部分は使い終わります。
- しかし、境界の部分は残ります。`s` が端点だと分かったときに、`f s` を `x` や `y` に書き換える必要があるからです。

つまり、`f s` の型は、**型制限の部品だけが残った型** です。

RSTT では、2 つの部品が 1 つにくっついていたので、片方だけが残った型を書く手段がありませんでした。

Rzk では、**部品が分かれているので、型制限だけの型をそのまま記述することができます。**

こうして、境界の情報は、`f s` の型にそのまま付いて回ります。



8-3　coercion-free subtyping ── 黙った同一視を、規則にする

分けたことで、新しい問いが生まれます。

たとえば、型 B と、型 `B [ ⊥ ↦ … ]`（決して成り立たない条件での制約）は、どういう関係にあるのでしょうか。

紙の証明では、こうした読み替えを黙って行います。

論文によれば、Riehl と Shulman の定理 4.4 の証明は、項を「別の extension type に詰め替える」と書き、命題 6.3 の後の注意では、自然変換とその成分の区別を「ぼかす」と書いています。

いずれも、RSTT の η 規則によって正当化される読み替えです。

人間にはそれで十分です。

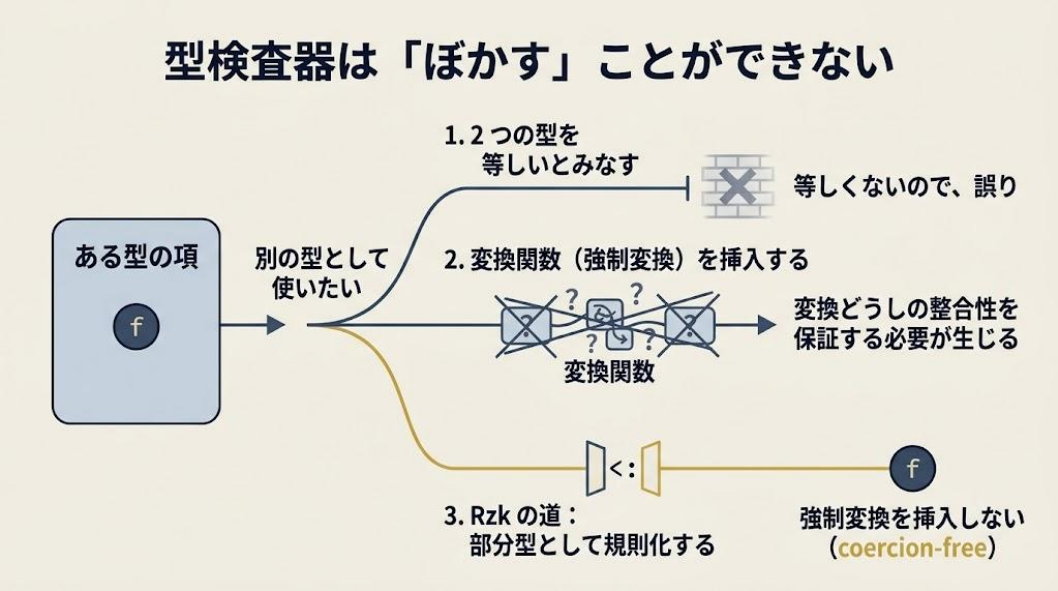
しかし、**型検査器は「ぼかす」ことができません。**

ある型の項が別の型として使われたとき、選択肢は 2 つしかありません。

1. 2 つの型を等しいとみなす → 等しくないので、**誤り**
2. 変換関数（強制変換）を挿入する → **挿入した変換どうしの整合性を保証する必要が生じる**

Rzk は、第 3 の道を取りました。

読み替え を **部分型**（*subtyping*）の関係として **明示的に規則化** し、しかも **強制変換を挿入しない**（*coercion-free*）ことにしたのです。



- `A <: B`
  - 「A は B の部分型である」
  - A の項は、そのまま（何も変えずに）B の項として通る

Haskell に馴染みのある方は、Haskell に部分型がないことをご存じかもしれません。

オブジェクト指向言語の「サブクラスのインスタンスは、親クラスの変数に入れられる」に近い考え方です。

Rzk の論文（Kudasov、Sim、Ahrens *Rzk: a Proof Assistant for Synthetic ∞-Categories*）が挙げる、**部分型の主な規則** は次の **5 つ** です。

- `S-Restr`
  - 制約を弱める
  - $A[\varphi \mapsto t] <: B[\psi \mapsto u]$（$A <: B$ 、ψ から φ が導かれ、重なりで $t \equiv u$ のとき）
- `S-Restr'`
  - 制約を捨てる
  - $A[\varphi \mapsto a] <: A$
- `S-Pi-Shape`
  - 形の上の関数型。定義域は逆向き（反変）
  - 広い形の上の関数は、狭い形の上の関数として使える
- `S-Restr-Bot`
  - 矛盾した条件の制約を付ける
  - $B <: B[\bot \mapsto \cdots]$

- `S-Split`：「または」で場合分けして確かめる

最初の問いへの答えは、S-Restr-Bot と S-Restr′ の組み合わせです。

B と `B [ ⊥ ↦ … ]` は、互いに部分型の関係 にあります。

強制変換を挿入しないことには、**大きな利点** があります。

項が変わらないので、「別々の場所で挿入した変換が、互いに整合しているか」という問題が、項のレベルでは起きません。

論文は、この方式を、**Luo の「強制変換つき部分型」（部分型関係が、挿入される変換関数の省略記法になっている方式）と対比** しています。

論文の cofibration composition の例（Riehl と Shulman の定理 4.4）で、この働きが見えます。

「形の包含」 を **cofibration**（*cofibration*）と呼び、この定理は「φ から χ への延長は、φ から ψ への延長と、ψ から χ への延長に分解できる」と述べます（$\varphi \subseteq \psi \subseteq \chi$）。

```
#def cofibration-composition (I : CUBE)
  ( χ : I → TOPE)          -- cube I の上の tope
  ( ψ : χ → TOPE)          -- χ の部分 tope
  ( φ : ψ → TOPE)          -- ψ の部分 tope
  ( X : χ → U)             -- χ の上の型族
  ( a : (t : φ) → X t)     -- 部分形 φ の上で決まった値
  : Equiv
    ( ( t : χ) → X t [φ t ↦ a t])
    ( Σ ( f : (t : ψ) → X t [φ t ↦ a t])
      , ( ( t : χ) → X t [ψ t ↦ f t]))
  :=
  ( ( \ h → (\ t → h t , \ t → h t))
  , ( ( \ (_ , g) → g , \ _ → refl)
    , ( \ (_ , g) → g , \ _ → refl)))
```

注目は、逆向きの写像 `\ (_ , g) → g` です。

g の型は「ψ の上で f に一致する」ものですが、期待されているのは「ϕ の上で a に一致する」ものです。

型が違います。

それでも通るのは、$\phi \subseteq \psi$ であり、ϕ の上では $gt = ft = at$ が **計算で成り立つから** です。

この **読み替え** を、**Rzk** は **部分型（S-Restr）** として受け入れます。**証明の項には、何も挿入されていません。**

筆者も、この前向きと逆向きの写像を Rzk 0.11.4 で別々に検査し、どちらも通ることを確認しました（`Equiv` を使う全体の検査には、sHoTT の定義が必要です）。

一方、前向きの写像は `\ h → (\ t → h t , \ t → h t)` と、わざわざ η 展開した形で書かれています。単に `h` とは書けません（論文の宣言的な体系では、この形しか受け入れられません）。

論文の説明によれば、この箇所は、**「h という特定の項については正しいが、h の型のすべての項については正しくない」読み替えであり、部分型（型のすべての項について成り立つ関係）では表せないから** です。

Riehl と Shulman の紙の証明も、sHoTT も、同じく展開した形で書いています。

論文は、この点を解消する規則を開発版の型検査器で試作したが、この論文の体系には入れていない、とも述べています。

8-4　first-class parameters ── メタ理論の前提を、引数にする

Riehl と Shulman は、定理を「I を cube とし、φ、ψ を tope とし、φ ⊢ ψ とする。このとき…」という形で述べます。

ところが **RSTT** では、**文脈に「抽象的な cube」や「抽象的な tope」を置くことができません。**

こうした前提は、**理論の外、つまりメタ理論** で述べられています。

Rzk の論文は、まずこの **メタ理論** の前提を **メタ理論的パラメータ層**（*meta-theoretic parameter layer*、MPL）として形式化しました。

MPL の文脈には、5 種類の項目が並びます。

- **cube パラメータ**：「I を cube とする」
- **tope パラメータ**：「φ を tope とする」
- **tope の包含の仮定**：「φ から ψ が導かれるとする」
- **項パラメータ**：「a を A の要素とする」
- **型族パラメータ**：「X を形の上の型族とする」

RSTT の定理は、この文脈に対して「図式的に」述べられたもの、と捉え直されます。具体的な cube や tope を代入すれば、個々の定理が得られます。

そのうえで **Rzk** は、これを **定義の引数として内部化**しました。

上の cofibration-composition で、`(I : CUBE)` や `(χ : I → TOPE)` が引数になっているのが、それです。論文の第 2 章の注で触れられているとおり、2 つの包含の仮定（$\phi \subseteq \psi$、$\psi \subseteq \chi$）は、引数 ψ と ϕ の型の中に入り込んでいます。

その仕組みが **tope family** です。`(ψ : χ → TOPE)` と書くと、「ψ は χ の上で定義された tope である」という包含の前提まで、引数の型として表せます。

Rzk は、tope family を使う箇所で、定義域の条件を自動で補います。

$\psi(t)$ と書かれたところを、内部では $\chi(t) \wedge \psi(t)$ と読むのです。

cofibration-composition の例 では、$\phi(t)$ は $\chi(t) \wedge \psi(t) \wedge \phi(t)$ と読まれます。

これにより、包含の前提が tope solver から使えるようになります。

この「自動で補う」仕組みには、もう 1 つ効用があります。

論文の例では、ϕ に「常に真」の tope（λt. ⊤）を代入すると、一見、形が cube 全体に広がりそうに見えます。

しかし実際には、$\phi(t)$ はすでに $\psi(t) \wedge \phi(t)$ と読まれているので、代入の結果は $\psi(t) \wedge \top$、つまり ψ の形そのものになります。

tope family に ⊤ を代入すると、「定義域の全体を選ぶ」ことになり、それ以上には広がりません。

論文は、この仕組みを「実用的な答え」と位置づけ、より整った設計（定義域を型で追跡する、tope の部分宇宙）を今後の課題に挙げています（第 10 章）。

8-5　automated tope solver ── 形の包含を、自動で確かめる

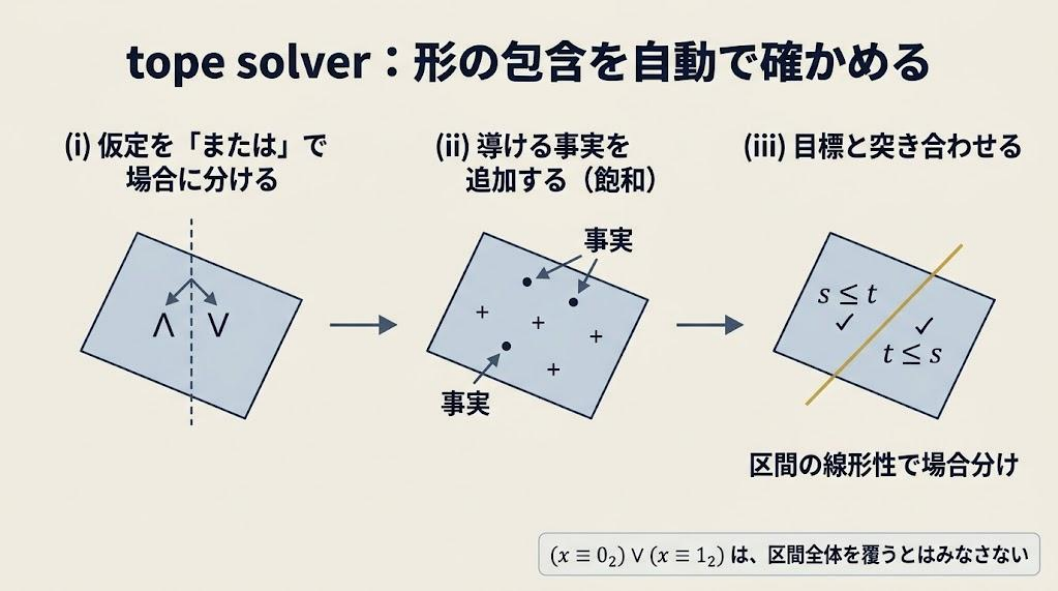
型検査の途中では、「いまの仮定から、この tope が導けるか」という問いが、繰り返し現れます。

- **形の包含**：「境界 $t \equiv 0_2 \vee t \equiv 1_2$ は、Δ^1 に含まれるか」
- **制約の発動**：「いま $s \equiv 0_2$ が成り立つので、fs を x に書き換えてよいか」
- **場合分けの網羅**：「この場合分けは、形全体を覆っているか」

RSTT の論文では、これらは「明らかな側条件」として省略されていました。

Rzk では、**tope solver**（*tope solver*）が自動で確かめます。

利用者が包含を手で証明することはありません。



solver のアルゴリズム は、**3 段階** です。

- **(i) 仮定を選言標準形（「または」で区切られた場合の並び）に直す**
  - 目標は、すべての場合で導けなければならない
- **(ii) 各場合で、仮定から導ける事実を追加していく（飽和）**
  - 等号・順序の法則、等しいものの代入、$x \leq 0_2$ なら $x \equiv 0_2$、など
  - ただし、固定の上限まで
- **(iii) 目標を、飽和した仮定と突き合わせる**
  - 必要なら、区間の線形性 $(x \leq y) \vee (y \leq x)$ で場合分けする

Rzk の論文（Kudasov、Sim、Ahrens *Rzk: a Proof Assistant for Synthetic ∞-Categories*）は、1 つの注意点を強調しています。

solver は、$(x \equiv 0_2) \vee (x \equiv 1_2)$ を「区間全体を覆う」とはみなさない。

有向区間は「0 と 1 の 2 点からなる集合」ではなく、間に点を持つ線形順序だからです。

solver が場合分けに使う「または」は、**区間の公理** である **線形性** $(x \leq y) \vee (y \leq x)$ だけです。公理なので、この場合分けは健全です。

solver は **健全**（正しいと答えたものは本当に正しい）ですが、**不完全**（正しいのに答えられないことがありうる）です。

論文は、健全性を補題 6.1 として述べ、付録で 3 つの段階のそれぞれが区間の公理からの正しい推論であることを確かめています。

試作品として、健全性を確かめやすい単純な規則で作られ、実践では十分だったことが分かりました。その実測が、7-6 で見た数字です。

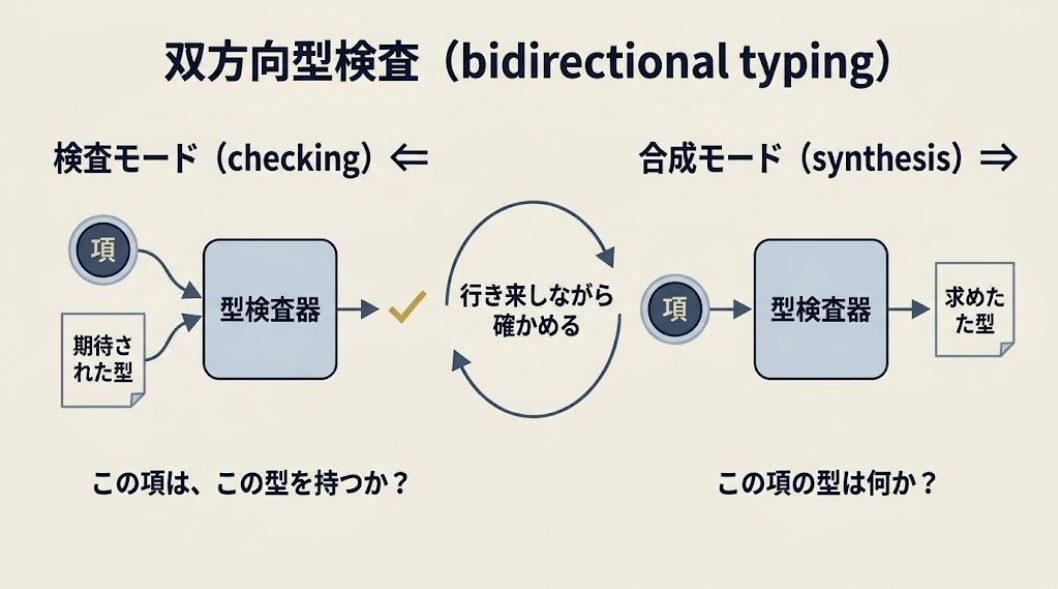
tope の論理で、含意が判定できるかどうか（決定可能性）について、論文は「問い合わせに現れる点は有限個なので、決定可能だと予想しているが、証明はない」としています。

8-6　bidirectional typing ── 双方向の型検査

Rzk の型検査 は、**双方向型検査**（*bidirectional typing*）という **アルゴリズム** で行われます。

- **検査モード（checking）**：「**この項は、この型を持つか**」 を確かめる
- **合成モード（synthesis）**：「**この項の型は何か**」 を求める

2 つのモードを行き来しながら、型を確かめていきます。

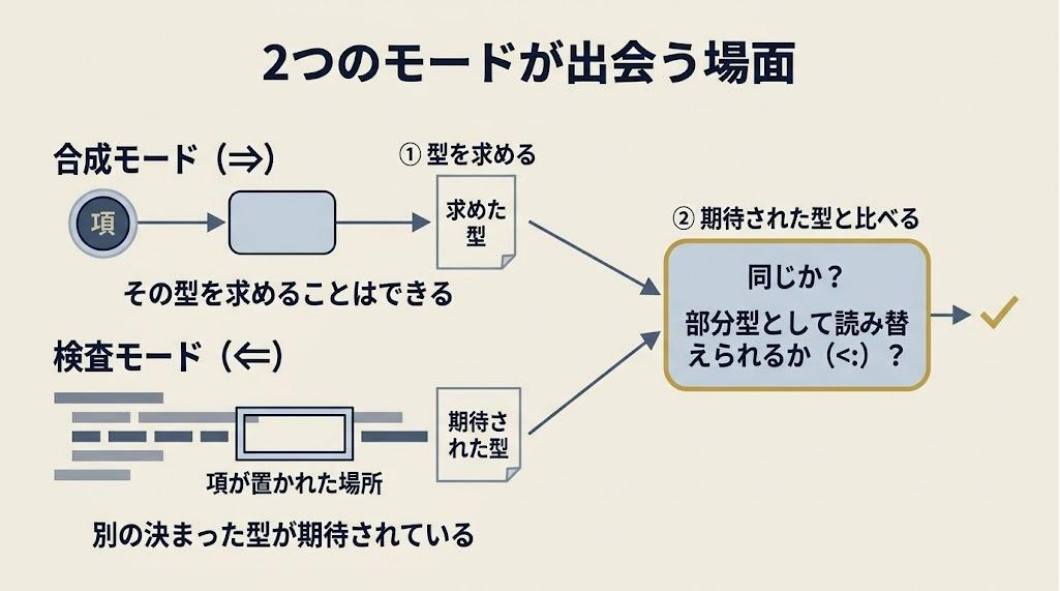


2 つのモードは、次のような場面で出会います。

ある項について、その型を求めることはできる（ **合成モード** ）。

しかし、その項が置かれた場所では、別の決まった型が期待されている（ **検査モード** ）。

このとき型検査器は、まず項の型を求め、次に、それが期待された型と同じか（または部分型として読み替えられるか）を比べます。



たとえば、第 5 章の `f s` がそうです。

`f s` の型は **「境界の制約が付いた `A`」と求められますが、ふつうの `A` が期待される場所に置かれることがあります。**

そのとき **型検査器は、2 つの型を比べ、部分型の規則で読み替えてよいかを確かめます。**

このとき、**合成した型** と **期待された型** を、**定義的等しさと部分型の関係で比べます。**

これ自体は、**依存型の言語で標準的な方法** です。

Rzk の型検査で、**とくに注意が必要な点は、次の 4 つ** です。

1. **計算が型に依存する**
2. **等しさが、tope の仮定に依存する**
3. **矛盾した仮定のもとでは、すべてが潰れる**
4. **部分型の検査は、等しさの検査と同じ手続きで行われる**

順に見ていきます。

特徴 1　計算が型に依存する

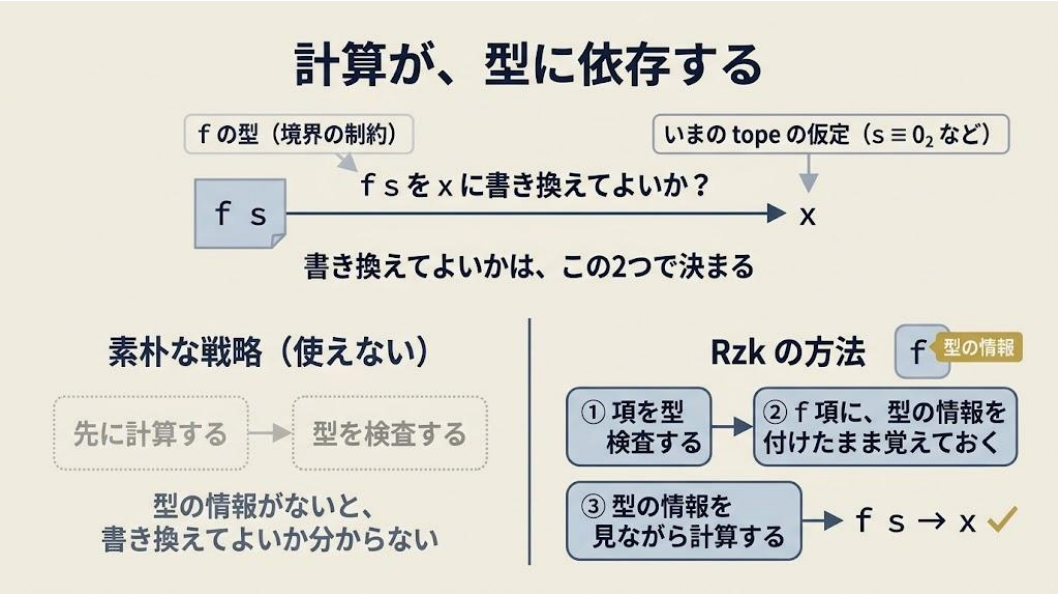
第 5 章で見たとおり、fs を x に書き換えてよいかは、f の型（境界の制約）と、いまの tope の仮定によって決まります。

そのため、**Rzk では「先に計算してから、型を検査する」という素朴な戦略が使えません。**

項を検査してから、**型の情報を使って** 計算する必要があります。

そこで **Rzk** は、型検査が済んだ式（項）に、**その型の情報を付けたまま** 覚えておきます。

そして、計算するときは、**その型の情報を見ながら** 計算します。



たとえば、`f s` を計算するときには、**`f` に付いている型（境界の制約）を見て、`s` が端点なら `x` や `y` に書き換えます。**

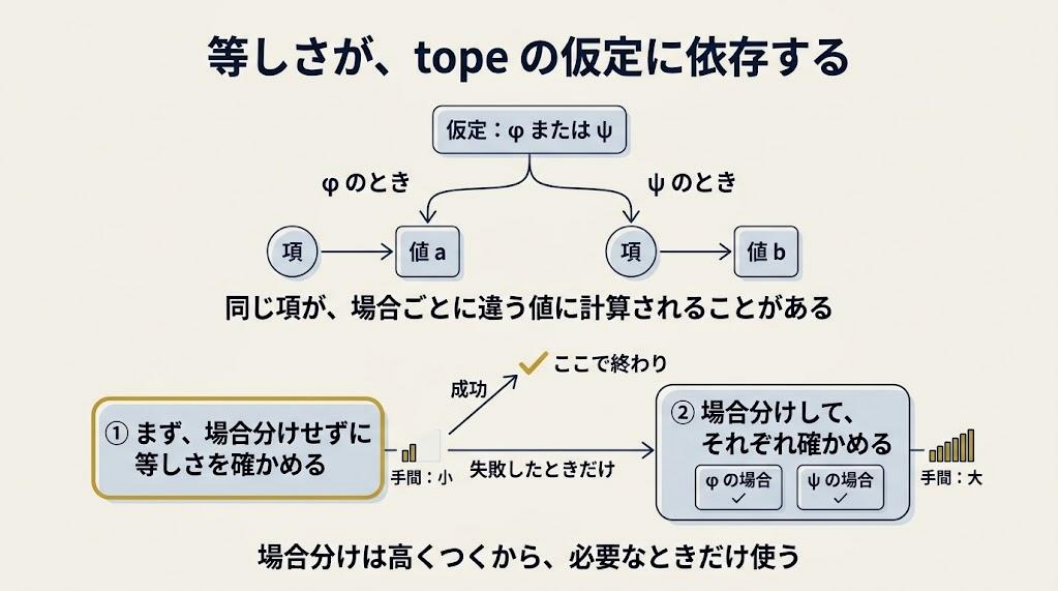
> 【コラム】キュービカル型理論の処理系も、同じやり方なのか
>
> **「計算が型に依存する」という難しさ** は、**Rzk** だけのものではありません。
>
> **キュービカル型理論の処理系（cubical Agda など）でも、計算は同じように型に依存します。**
>
> では、キュービカル型理論の処理系も、Rzk と同じやり方をしているのでしょうか？
>
> **目的は同じですが、やり方には違いがあります。**
>
> どちらも、「計算するときに、境界（端点）の情報が手元にある」ようにする必要があります。
>
> **Rzk** は、型検査が済んだ式に、**その型の情報をまるごと付けたまま** 覚えておきます。
>
> 一方、**Cubical Agda** は、道を区間の点に当てる操作を内部で表すとき、**その道の両端の値を、操作の中に一緒に記録** しておきます。
>
> 型の情報をまるごと持つか、計算に必要な端点の値だけを持つか、という違いです。

特徴 2　等しさが tope の仮定に依存する

仮定に 「または」があると、同じ項が場合ごとに違う値に計算されることがあります。

Rzk は、まず場合分けせずに等しさを確かめ、失敗したときだけ場合分けして確かめます。

場合分けは高くつくから です。



「高くつく」とは、**型検査器の手間（計算の量と時間）が大きく増える** ということです。

場合分けをすると、同じ等しさの確かめを、**場合の数だけ繰り返す** ことになります。

「または」が 1 つなら 2 つの場合で済みますが、仮定の中に「または」がいくつもあると、**場合の数は掛け算で増えていきます。**

たとえば、**「または」が 3 つ** あれば、$2 \times 2 \times 2 = 8$ 通りの場合を、それぞれ確かめなければなりません。

しかも場合ごとに、項をあらためて計算し直す必要があります。

では、場合分けをせずに確かめるとは、具体的に何をすることなのでしょうか。

「場合分けをせずに確かめる」とは、**仮定の「または」をいったん脇に置いて、2 つの式を、そのままの形で見比べる** ことです。

例を挙げてみます。

射 `f` と `g` は、どちらも `x` から `y` への射（`hom A x y`）だとします。

そして、仮定は「$t \equiv 0_2$ または $t \equiv 1_2$」（t は区間の端点のどちらか）だとします。

例 1：`f t` と `f t` は等しいか

2 つの式は、文字どおり同じ形です。

どちらの場合かを考えるまでもなく、等しいと分かります。

場合分けをせずに、1 回で決着がつきます。

例 2：ｆ ｔ と ｇ ｔ は等しいか

そのままの形で見比べると、ｆ と ｇ は違う射なので、等しいかどうか分かりません。

そこで、場合分けに進みます。

- $t \equiv 0_2$ の場合：ｆ ｔ も ｇ ｔ も、計算すると ｘ になる。等しい
- $t \equiv 1_2$ の場合：ｆ ｔ も ｇ ｔ も、計算すると ｙ になる。等しい

どちらの場合も等しいので、**場合分けをして初めて、等しいと分かります。**

例 1 のように、そのままの形で見比べるだけで決着がつくことは、よくあります。

そのときは、場合分けという高い確かめをせずに済みます。

そこで **Rzk** は、**まず、そのままの形で見比べ、それで決着がつかないときだけ、場合分けに進む** のです。

特徴 3　矛盾した仮定のもとでは、すべてが潰れる

たとえば、「$t \equiv 0_2$ かつ $t \equiv 1_2$」という仮定を考えます。

区間の公理では、0_2 と 1_2 は異なる点なので、**この 2 つが同時に成り立つことはありません。**

このように、tope の仮定から **矛盾（⊥）が導かれる** 場面が、型検査の途中で現れることがあります。

論理学には、**「矛盾からは、何でも導ける」という原理** があります。

型検査器もこれに従います。

矛盾した仮定のもとでは、「この 2 つの項は等しい」という主張も、どんな 2 つの項について導けてしまいます。

同じように、**「この 2 つの型は等しい」という主張も、どんな 2 つの型について導けてしまいます。**

つまり、矛盾した仮定のもとでは、**どの項も互いに等しく、どの型も互いに等しい** のです。

区別する意味がなくなる ので、型検査器は、**すべての型を 1 つの代表（⊥ の除去子 rec⊥）にまとめて** 扱います。

ただし、ここには落とし穴があります。

「どの項も等しい」のなら、**でたらめに書いた壊れた項でも、検査を通ってしまう** おそれがあります。

そこで **Rzk** は、矛盾した仮定のもとでも、**項がそれ自体として正しく作られているかどうかは、きちんと確かめます。**

矛盾した仮定を隠れみのにして、壊れた項が受け入れられないようにするため です。

特徴 4　部分型の検査は、等しさの検査と、同じ手続きで行われる

型検査器は、2 種類の確かめをします。

- **等しさの確かめ**：型 A と型 B は、同じ型か
- **部分型の確かめ**：型 A の値を、型 B の値として使ってよいか（$A <: B$）

Rzk では、ある型の値を別の型の値として使うときに、**値には何も手を加えません**（8-3）。

値は、書かれたままの形で、そのまま通ります。

そのため、部分型の確かめも、等しさの確かめと同じく、**2 つの型を部品ごとに見比べていく** 作業になります。

違いは 1 つだけです。

等しさには **向きがありません**（A と B が同じなら、B と A も同じ）。

部分型には **向きがあります**（A の値を B として使えても、その逆ができるとは限らない）。

そこで **Rzk** は、2 つを別々の手続きにせず、**1 つの手続きに「向き」の目印（旗）を持たせて** 確かめます。

旗が立っていなければ等しさを、旗が立っていれば部分型を確かめる、というしくみです。

この旗には、もう 1 つ大事な使い方があります。

関数の引数の型を見比べるときは、向きが逆になる のです。

たとえば、「動物を受け取る関数」は、「犬を受け取る関数」の代わりに使えます。

犬も動物なので、犬を渡しても問題なく動くからです。

しかし、「犬を受け取る関数」を、「動物を受け取る関数」の代わりに使うことはできません。

猫を渡されたら、動かないからです。

そこで型検査器は、関数の引数の型を見比べるときに、**旗の向きを反転させます。**

> 【コラム】Rocq にも、同じ工夫がある
>
> Rzk の論文（Kudasov、Sim、Ahrens *Rzk: a Proof Assistant for Synthetic ∞-Categories*）は、この方式には先例があると述べています。
>
> Rocq（旧 Coq）には、「小さい型の集まりに属する型は、より大きい型の集まりに属する型としても使ってよい」というしくみがあります。
>
> Rocq の型検査器も、このしくみを、等しさを確かめる手続きに旗を持たせることで判定しています。

最後の点を、覚えておいてください。

第 9 章のバグの話に、直接つながります。

第 9 章　理論と実装の往復

この章の要点：論文は、RSTT から Rzk への翻訳が「忠実」で、一定の条件のもとで「保存的」であることを証明した。

さらに、型検査のアルゴリズムが宣言的な Rzk に対して健全であることを示した。

そして、規則を正確に書き下して実装と照合する作業が、公開版に潜んでいた 2 つの健全性バグを見つけた。

9-1　翻訳、忠実性、保存性

第 8 章で見たとおり、Rzk は RSTT に改良を加えています。

すると、当然の疑問が生じます。

- **疑問 1**：RSTT で書けた証明は、Rzk でも書けるのか？
- **疑問 2**：Rzk で証明できたことは、RSTT でも本当に成り立つのか？

Rzk の論文（Kudasov、Sim、Ahrens *Rzk: a Proof Assistant for Synthetic ∞-Categories*） は、この 2 つに答えるために、RSTT から Rzk への **翻訳** τ（タウ）を定義しました。

翻訳はほとんど自明です。

cube と **tope** の **層**、そして **共通の Martin-Löf 型理論の部分** では、τ は何もしません。

extension type だけ を 「**shape-Π と型制限の組み合わせ**」**に書き直します。**

```
RSTT の extension type   ⟨Π_{t:I|ψ} A |^ϕ_a⟩
        ↓ τ
Rzk                      Π_{t:{I|ψ}} ( A [ ϕ(t) ↦ a(t) ] )
```

第 8 章で見たメタ理論的パラメータ層（MPL）の文脈も、τ はそのまま保ちます。

項パラメータや型族パラメータの型に **extension type が含まれていれば、その部分だけを書き直します。**

だから、「任意の cube と tope について」述べられた RSTT の定理は、同じ形のまま Rzk の定理に翻訳されます。

そのうえで、**2 つの性質** を示しました。

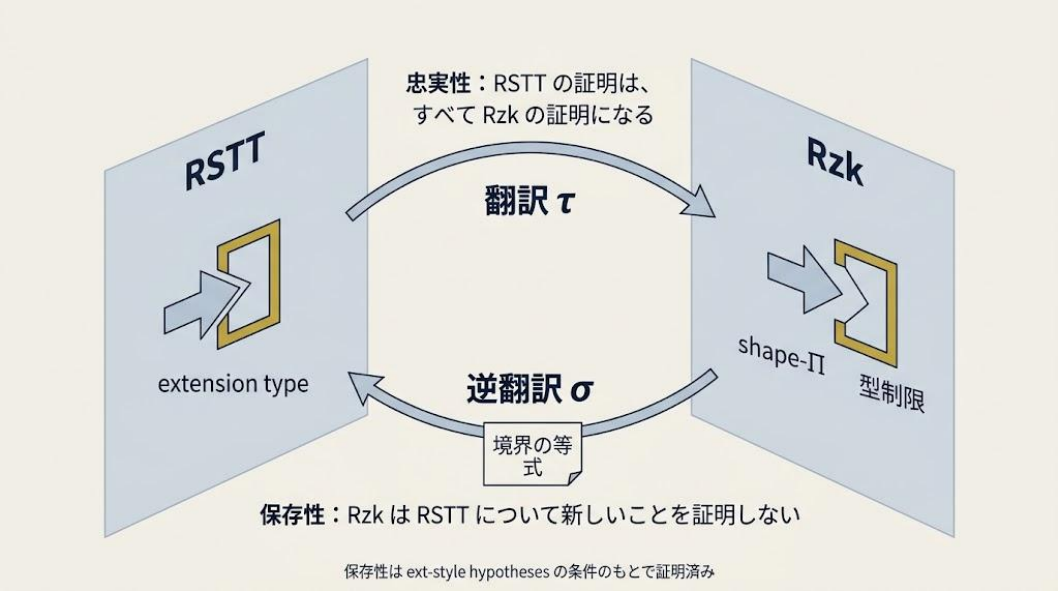
性質	英語	意味	答える疑問
忠実性	*faithfulness*	RSTT のすべての証明が、翻訳すると Rzk の証明になる	疑問 1
保存性	*conservativity*	Rzk が RSTT の型について証明できることは、RSTT でも証明できる	疑問 2

忠実性（論文の命題 5.4）の証明の要は、「RSTT の extension type の 6 つの規則が、Rzk の中ですべて成り立つ」ことです（付録の命題 C.1）。

第 3 章のコラムで紹介した、形成・導入・適用・β・境界の計算・η の 6 つです。

どれも、**型制限と shape-Π の規則を組み合わせれば導けます** 。

翻訳は部分型を一切使わないこと も、論文は強調しています。



9-2　なぜ、この 2 つが重要なのか

どちらかが欠けると、Rzk は RSTT の処理系として信頼できなくなります。

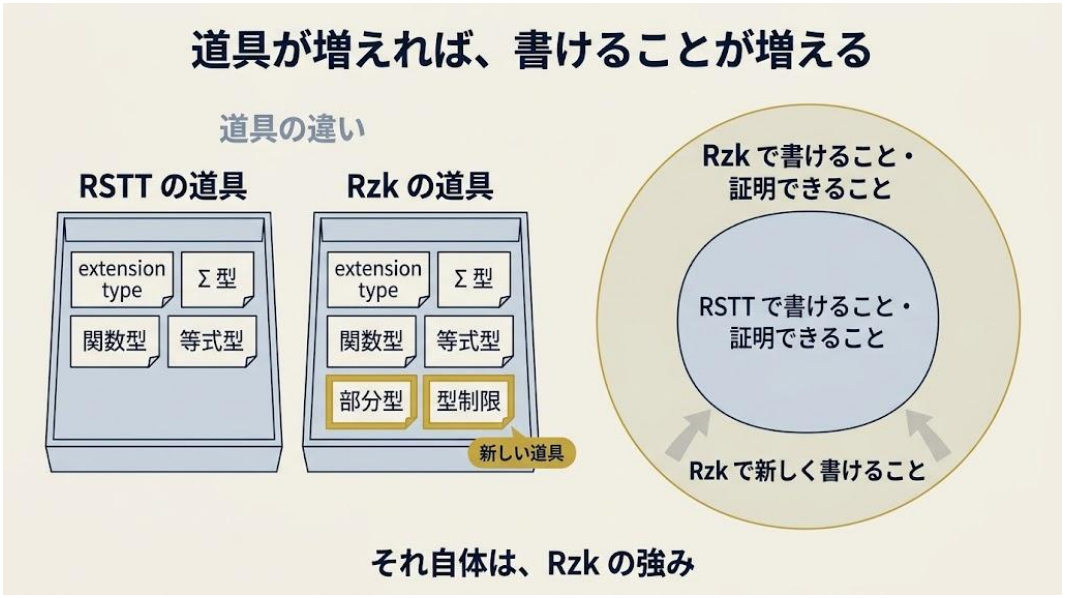
- **忠実性がなければ**
  - RSTT の論文で書けた証明が、Rzk で書けないかもしれない
  - （Rzk は RSTT より弱い）
- **保存性がなければ**
  - Rzk で証明できたことが、RSTT では成り立たないかもしれない
  - （Rzk は RSTT と違う数学をしている）

保存性は、とくに重要です。

Rzk は **部分型や型制限という新しい道具** を持っています。

道具が増えれば、RSTT では書けなかったことを、Rzk で書いて証明できるようになります。

それ自体は、**Rzk** の強みです。



ただし、**1 つだけ守らなければならないことがあります。**

RSTT の言葉でもともと書ける問いについては、Rzk と RSTT で答えが食い違ってはいけない ということです。

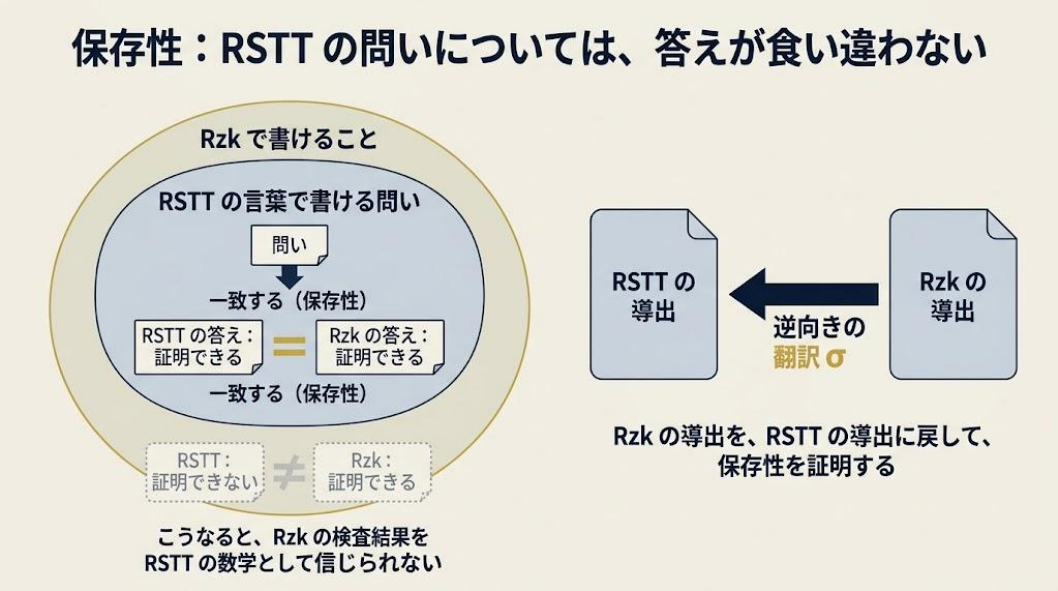
もし、RSTT の言葉で書ける問いなのに、RSTT では証明できず、Rzk でだけ証明できてしまうとしたら、どうでしょう。

そのときは、**Rzk** で検査に通った証明を見ても、それが **RSTT** の数学として正しいとは言えなくなります。

保存性 は、この食い違いが起きないことを保証する性質です。

保存性の証明 は、**逆向き** の翻訳 σ（シグマ）によります。

Rzk の導出を、RSTT の導出に戻す翻訳 です。



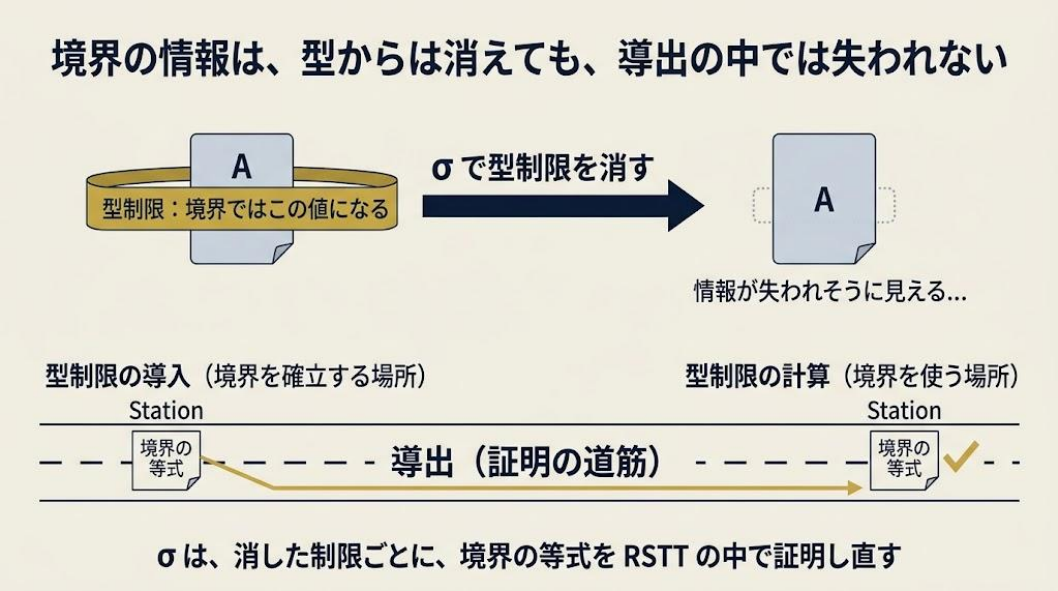
型制限の場所	σ での扱い
shape-Π の余域にある型制限	→ RSTT の extension type に戻す
それ以外の場所にある型制限	→ 消す（RSTT には受け皿がない）
部分型の段階	→ 項をそのまま使う、または η 展開を挟む

型制限を消すと、その制限が保証していた「境界ではこの値になる」という情報が失われそうに見えます。

そこで σ は、**消した制限ごと** に **境界の等式**（*boundary equation*）を、RSTT の中で証明し直します。

等式は、Rzk が境界を確立した場所（型制限の導入）で作られ、境界を使う場所（型制限の計算）で消費されます。

境界の情報は、型からは消えても、導出の中では失われないのです。



9-3　保存性には、条件が付いている

ただし、**論文が証明した保存性（定理 5.8）には、条件が付いています。**

ext-style hypotheses と呼ばれる **条件** です。

直観的に言えば、こうです。

> 自由な位置に置かれた型制限を、**結論として導く**ことはできるが、**仮定として置く**ことはできない。

ここで **「自由な位置」とは、*shape-Π* の余域以外の場所のこと** です。

型制限が **shape-Π の余域** にあれば、**RSTT の extension type** に **戻せます。**

それ以外の場所にある型制限は、RSTT に受け皿がありません。

なぜ、場所によって違いが出るのでしょうか？

これには、**Rzk** が **extension type** を **2 つの部品に分けた** という話が関わっています。

extension type `(t : ψ) → A [ ϕ ↦ a ]` は、もともと次の 2 つがくっついたものでした。

- **前半部分の部品** ：形 ψ の上の関数（**shape-Π**）
- **後半部分の部品** ：その関数の返す値に付いた、「境界ではこの値になる」という制約（**型制限**）

RSTT では、この 2 つは切り離せない 1 つの型でした。

Rzk では、この 2 つが別々の部品になっています。

ここで、言葉を 1 つ確認します。

関数の **余域** とは、**その関数が返す値の型のこと** です。

たとえば、Haskell の `f :: Int -> Bool` なら、**余域は `Bool`** です。

shape-Π `(t : ψ) → A` の余域は、`A` です。

型制限が shape-Π の余域にある とは、型制限が「形の上の関数の、返す値」に付いている、ということです。

`(t : ψ) → A [ ϕ ↦ a ]` のような形です。

これは、RSTT の extension type と、まったく同じ組み合わせです。

そのため、**2 つの部品をくっつけ直せば、そのまま RSTT の extension type に戻せるのです。**

一方、型制限が **それ以外の場所** にあることもあります。

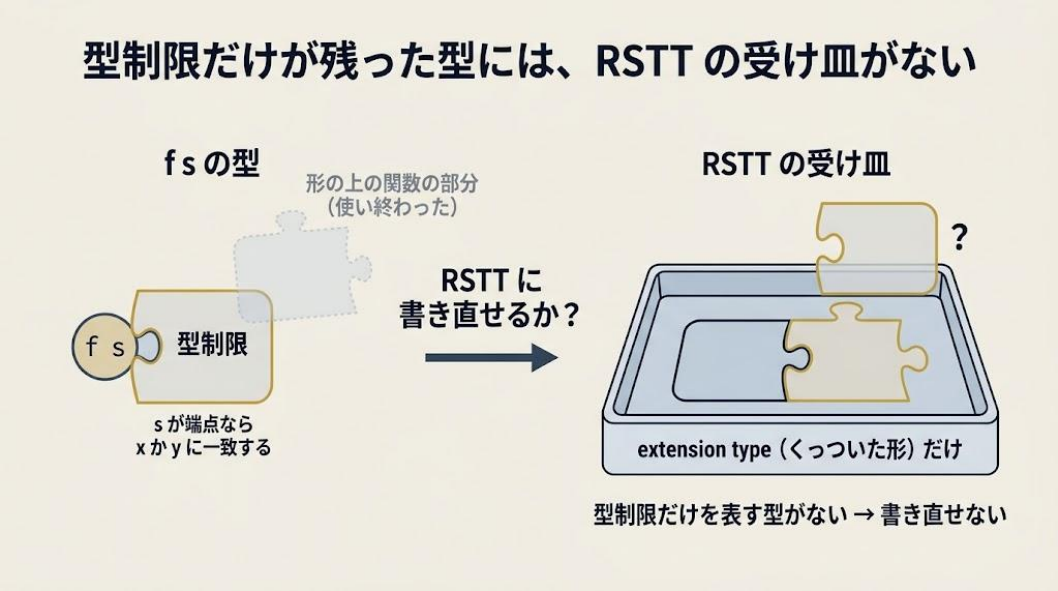
たとえば、射 `f : hom A x y` を区間の点 `s` に当てたときの `f s` の型を思い出してください。

`f s` の型は、「`s` が端点なら `x` か `y` に一致する」という制約だけが付いた `A` でした。

形の上の関数の部分は使い終わっていて、型制限だけが残った型です。

RSTT には、くっついた形の extension type しかないので、**型制限だけ** を**表す型がありません。**

そのため、こうした型制限は、RSTT の型には書き直せない（受け皿がない）のです。



では、受け皿がない型制限は、**RSTT** の証明に書き戻すときに、どうなるのでしょうか。

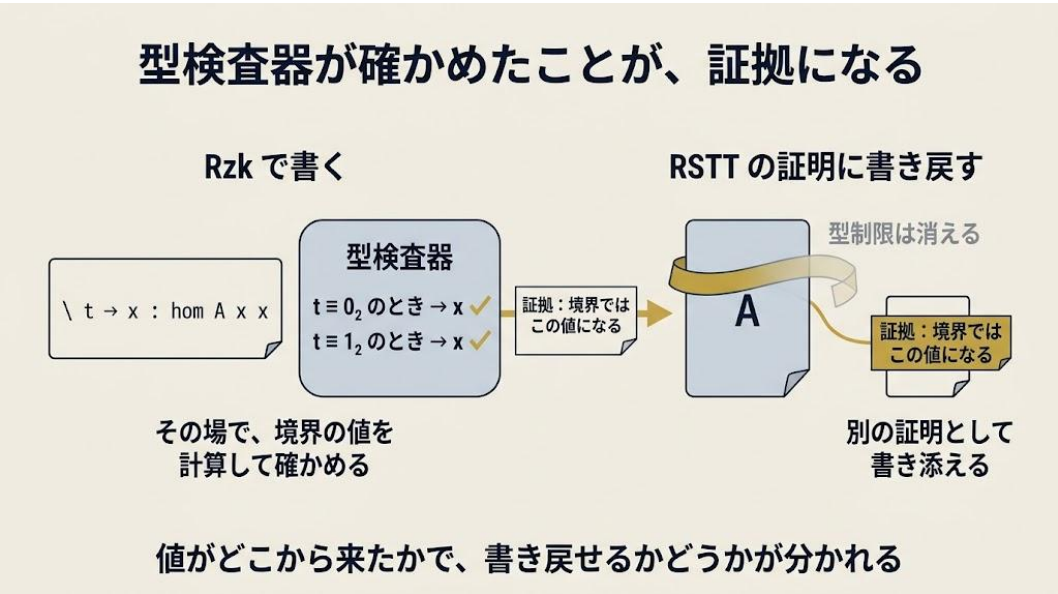
実は、**その値がどこから来たか** によって、書き戻せる場合と、書き戻せない場合に分かれます。

Rzk で、境界の制約付きの値を書くと、**型検査器がその場で、境界の値を確かめます。**

たとえば、恒等射 `\ t → x` を `hom A x x` として書くと、型検査器は「$t \equiv 0_2$ のとき x になるか」「$t \equiv 1_2$ のとき x になるか」を、計算して確かめます。

型検査器が計算して確かめたことが、そのまま「境界ではこの値になる」ことの証拠になります。

RSTT の証明に書き戻すときは、型から型制限が消えるので、この証拠を使って、**「境界ではこの値になる」ことを、別の証明として書き添えます。**



ところが、値を **外から受け取った** 場合（関数の引数など）は、事情が違います。

外から受け取った値については、型検査器は何も計算していません。

「境界ではこの値になる」は、型に書かれた約束として受け入れているだけ です。

Rzk の中では、これで何の問題もありません。

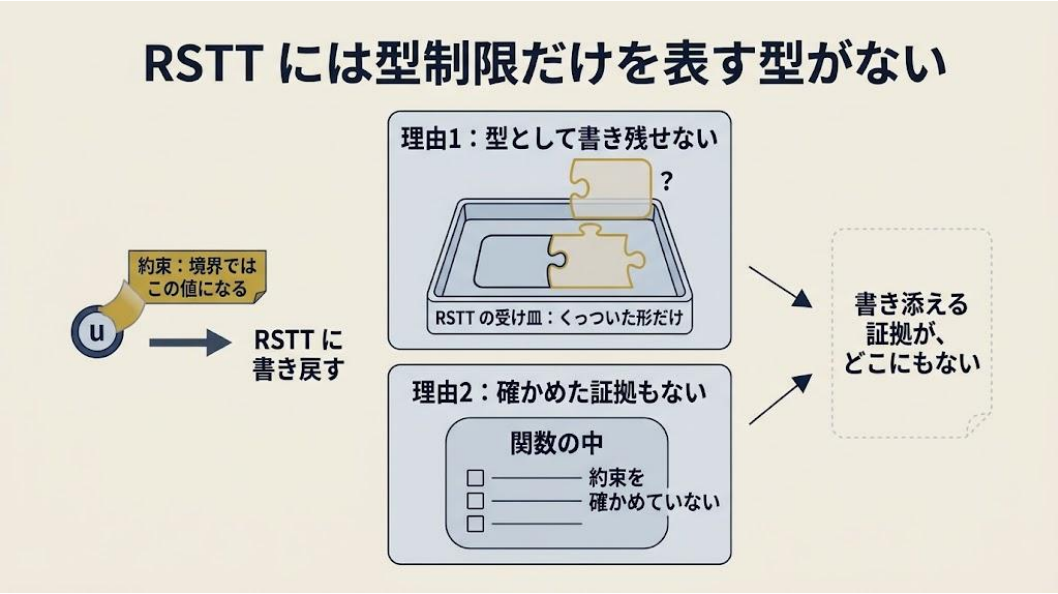
その関数を呼び出す側で値を渡すときに、型検査器が約束を確かめるからです。

困るのは、**RSTT** の証明に書き戻すときです。

RSTT には **型制限だけを表す型がないため、** 約束を型として書き残すことができません。

しかも、関数の中には、約束を確かめた証拠もありません。

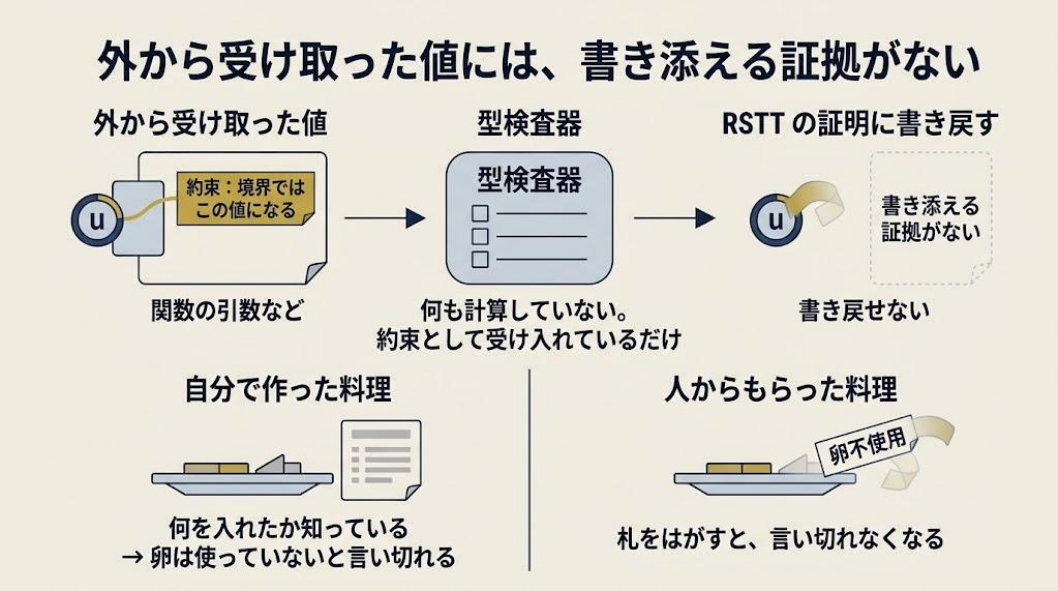
そのため、**書き添えるための証拠が、どこにもないのです。**



たとえるならば、自分で作った料理なら、何を入れたか知っているので「卵は使っていない」と言い切れます。

しかしながら、人からもらった料理に「卵不使用」の札が付いていただけなら、その札をはがしたとたん、卵を使っていないとは言い切れなくなります。

そのため、**外から受け取った値に付いた型制限は、RSTT の証明に書き戻すことができないのです。**



これが、**保存性の証明** に付いている条件の中身です。

保存性が保証されるのは、**外から受け取る値に、受け皿のない型制限が付いていない場合** に限られます。

もっとも、この条件は、実践ではほとんど制約になりません。

論文によれば、**sHoTT ライブラリにある約 450 個の型制限は、すべて shape-Π の余域に乗っており、境界の tope は形の tope に含まれている。**

つまり、ライブラリ全体がこの条件の範囲に収まっています。

論文の付録 D には、保存性を具体的にたどった例があります。

Riehl と Shulman の定理 4.1 です。

```
#def flip-ext-fun
  ( I : CUBE) ( ψ : I → TOPE) ( φ : ψ → TOPE)
  ( X : U)
  ( Y : ψ → X → U)
  ( f : (t : φ) → (x : X) → Y t x)
  : Equiv
    ( ( t : ψ) → ((x : X) → Y t x) [φ t ↦ f t])
    ( ( x : X) → (t : ψ) → Y t x [φ t ↦ f t x])
  := ( \ g x t → g t x ,
    ( ( \ h t x → (h x) t , \ g → refl) ,
      ( \ h t x → (h x) t , \ h → refl)))
```

（論文の図 12 のコードです。）

「形の上の関数で、値がまた関数であるもの」と、「関数で、値が形の上の関数であるもの」が同値である、という定理です。

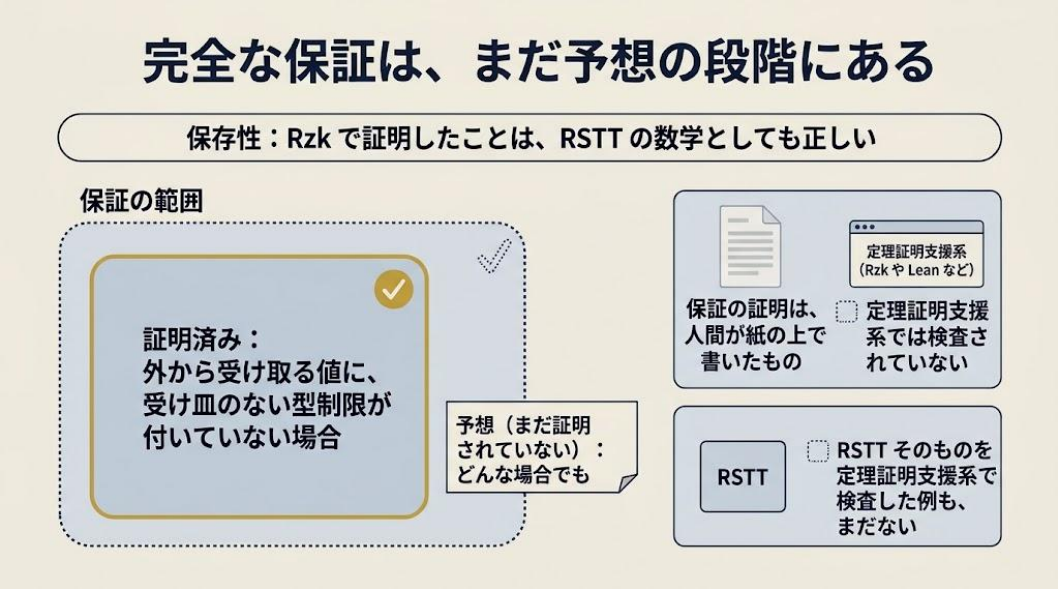
引数の順番を入れ替えるだけで、境界の条件もそのまま運ばれます。

論文は、この Rzk の証明に σ を当てると、同じ項がそのまま RSTT の証明になることを示しています。

4 か所の refl は、Rzk では型制限の計算で、RSTT では extension type の境界の計算で検査されます。

使われる等しさは同じです。

9-4　完全な保証は、まだ予想の段階にある



ここまでの話をまとめると、次のようになります。

Rzk で証明したことは、**RSTT** の数学としても正しい。

この保証（保存性）は、論文の中で証明されています。

ただし、証明されているのは、**外から受け取る値に、受け皿のない型制限が付いていない場合だけ**です。

この条件を外しても、同じ保証が成り立つのか。

著者たちは「成り立つはずだ」と考えていますが、まだ証明できていません。

論文では、これを **予想**（まだ証明されていない主張）として残しています。

- **証明済み**：外から受け取る値に、受け皿のない型制限が付いていない場合
- **予想**：どんな場合でも

もう 1 つ、論文が正直に書いていることがあります。

この保証の証明は、**人間が紙の上で書いたもので、定理証明支援系（Rzk や Lean のように、証明が正しいかを検査するプログラム）では検査されていません。**

細かい部分の中には、論文で説明はしたものの、定理証明支援系で検査するところまでは行っていない箇所が残っています。

そもそも、**RSTT** という理論そのものを、定理証明支援系の中に書き下して検査した例も、著者たちの知る限り、まだありません。

9-5　型検査器が受け入れたものは、正しい

ここまでの話は、論文の中に書かれた **Rzk の規則** についてでした。

規則とは、「何が正しい証明か」を定めた決まりのことです。

しかし、実際に証明を検査しているのは、その規則をもとに作られた **型検査器というプログラム** です。

規則が正しくても、プログラムがその規則どおりに動いていなければ、意味がありません。

そこで論文は、両者の関係も証明しています。

型検査器が受け入れた証明は、規則に照らしても正しい。

これを **健全性** と呼びます。

さらに、これまで見てきた保証（**Rzk** で証明したことは、**RSTT** の数学としても正しい）と組み合わせると、次のことが言えます。

型検査器が受け入れた証明は、RSTT の数学としても正しい。

ただし、さきほど見た条件（外から受け取る値に、受け皿のない型制限が付いていないこと）は、ここでも必要です。

論文は、これを「この論文の端から端までの保証」と呼んでいます。

一方で、論文は逆向きのことは主張していません。

つまり、**「正しい証明なら、型検査器は必ず受け入れる」とは言っていない** のです。

形の包含を自動で確かめる仕組み（tope solver）が、正しい包含を見落とすことがありえます。

また、型検査器の計算が必ず終わるかどうかも、まだ分かっていません（この話は、この章の最後で扱います）。

9-6　規則を書き下したことが、2 つの健全性バグを見つけた

ここからが、本稿の見どころです。

論文は、Rzk の型検査器が使っている規則を、付録にすべて書き下しました。

しかも、規則の 1 つ 1 つに、型検査器のプログラムの該当箇所へのリンクを付けています。

規則とプログラムを、1 つずつ突き合わせたのです。

その結果を、論文はこう報告しています（筆者による要約）。

> 規則に照らして型検査器を点検したところ、v0.7.7 までのリリース版に、2 つの健全性バグが見つかった。

健全性バグ とは、**本当は正しくない証明を、正しいと受け入れてしまうバグ** です。

証明を検査する道具にとって、最も深刻な種類のバグです。

バグ 1　「どの型の中での等しさか」を比べていなかった

Rzk には、「2 つのものが等しい」ことを表す型（等式型）があります。

等式型には、**何と何が等しいか** に加えて、**どの型の中での等しさか** という情報が含まれています。

型検査器は、2 つの等式型が同じかどうかを判定するとき、「何と何が等しいか」は比べていました。

ところが、**「どの型の中での等しさか」を比べ忘れていた** のです。

プログラムには、この比較が「TODO（あとでやる）」というメモとして残されていました。

その結果、次の 2 つを取り違えるおそれがありました。

- **射としての等しさ**
  - 射 f と g を、両端を固定したまま変形してつなぐ
- **ただの関数としての等しさ**
  - 射 f と g を、両端も動かしてよい自由な変形でつなぐ

両端を固定するかどうかで、数学的にはまったく別のもの になります。

本来なら受け入れてはいけない取り違えを、型検査器が見逃していたのです。

この比較は、v0.7.8 で追加されました。

ただし、修正には代償がありました。

sHoTT ライブラリの中に、この見逃しに頼って書かれた箇所があった のです。

どれも、数学としては問題のない使い方でした。

それでも、新しい型検査器を通すために、25 か所で「どの型の中での等しさか」を明記する修正が必要になりました。

バグ 2　関数の引数で、検査の向きが逆になっていた

型検査器は、「型 A の値を、型 B の値として使ってよいか」を確かめるとき、「向き」の目印を持っている、という話をしました。

そして、**関数の引数の型を見比べるときは、その向きを反転させる** 必要がありました。

「動物を受け取る関数」は「犬を受け取る関数」の代わりに使えるが、その逆はできない、という話です。

v0.7.7 までの型検査器は、一部の検査で、**この反転を忘れていました。**

そのため、関数の引数の位置では、検査の向きが逆になっていました。

その結果、論文が挙げる例では、**ある範囲でしか定義されていない関数が、その範囲の外でも使えてしまう** ことがありました。

「犬しか受け取れない関数」を、「動物を受け取る関数」として通してしまうようなものです。

この誤りも、v0.7.8 で修正されました。

こちらは、sHoTT ライブラリの修正を必要としませんでした。

これは、Rzk を貶める話ではない

ここで、誤解のないように書いておきます。

- **バグがあった**：実験的な言語だから、当然ありうる

- **見つかった**：論文を書く過程で、規則とプログラムを突き合わせたから
- **直った**：v0.7.8 で修正。ライブラリの修正も行われた
- **公表された**：論文自身が、バグの存在と内容を明記した

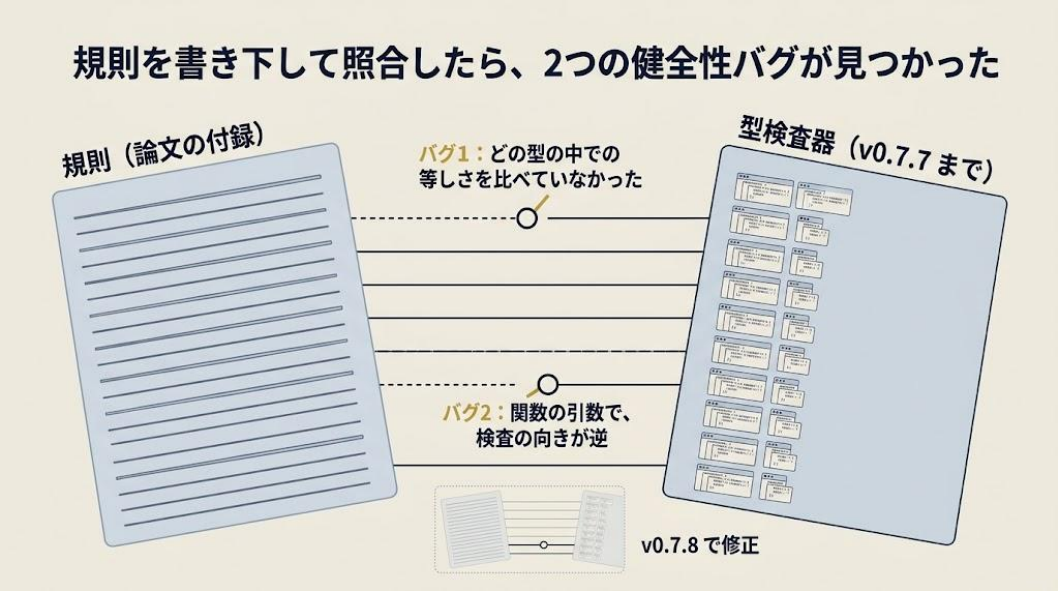
むしろ、**理論と実装を突き合わせる作業が、きちんと機能した証拠** です。

論文の言葉を借りれば、「アルゴリズムを正確に述べたことが、実践で報われた」のです。

本稿の冒頭で、専用の言語には「処理系そのものの正しさを、一から確かめる必要がある」という代償がある、と書きました。

Rzk は、その代償に正面から向き合ったと言えます。

> 【筆者の検証について】筆者が動かした Rzk 0.11.4 は、論文が報告する v0.7.8 より新しい版です。
>
> バグの修正はどちらにも含まれています。
>
> 本稿で「筆者が確認した」と書いたコード例は、すべて 0.11.4 で検査したものです。



9-7　まだ分かっていないこと ── 型検査は必ず終わるのか

論文は、未解決の問題を正直に列挙しています。

その中で、最も根本的なものがこれです。

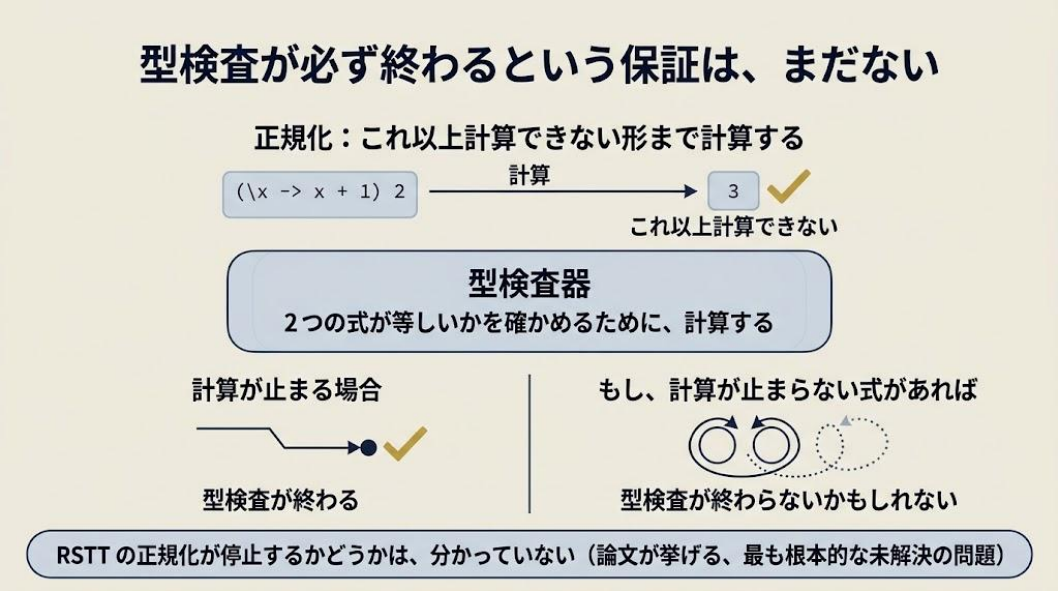
> RSTT の正規化が停止するかどうかは、分かっていない。

正規化（*normalization*）とは、項を「これ以上計算できない形」まで計算することです。

型検査器は、2 つの項が等しいかを確かめるために、項を計算します。

もし計算が止まらない項があれば、型検査そのものが終わらない可能性があります。

つまり、**Rzk は、型検査が必ず終わるという理論的な保証を、まだ持っていません。**



論文は、この点を踏まえて、**アルゴリズムの「健全性」（受け入れたものは正しい）は主張しますが、「完全性」（正しいものはすべて受け入れる）は主張していません。**

論文の言葉では、健全性には停止性も完全性も要らない。

失敗する実行や止まらない実行は、何も受け入れないからです。

筆者は、別の記事で、ちょうど「型検査が必ず終わる」ことの証明を扱いました。

【前層トポスと定理証明支援系】定義展開の制御で表示の膨張を抑えながら、型検査の停止性をどう保証するか

そこで扱った Gratzer らの研究（2025 年）は、定義展開を制御する型理論（*controlled unfolding*）について、正規化定理を証明しました。

一方、RSTT では、同じ種類の問いがまだ開いています。

- **Gratzer ら（2025 年）**：controlled unfolding について、正規化定理を証明した
- **Rzk の論文（2026 年）**：RSTT の正規化が停止するかは、分かっていない

同じ問いが、片方では解かれ、もう片方では未解決である。

理論を正確に書くことには、実務上の意味があります。

バグを見つけるという意味でも、どこまでが保証されているかを正直に示すという意味でも。

Rzk は、まだ発展途上の言語である

ここまで見てきた未解決の問題を、まとめておきます。

- **保証の範囲**：**Rzk** で証明したことが **RSTT** の数学としても正しい、という保証は、条件付きでしか証明されていない
- **保証の検査**：その保証の証明は、人間が紙の上で書いたもので、定理証明支援系では検査されていない
- **見落とし**：形の包含を自動で確かめる仕組み（tope solver）が、正しい包含を見落とすことがありうる
- **計算が終わるか**：型検査の計算が必ず終わるかどうかが、分かっていない

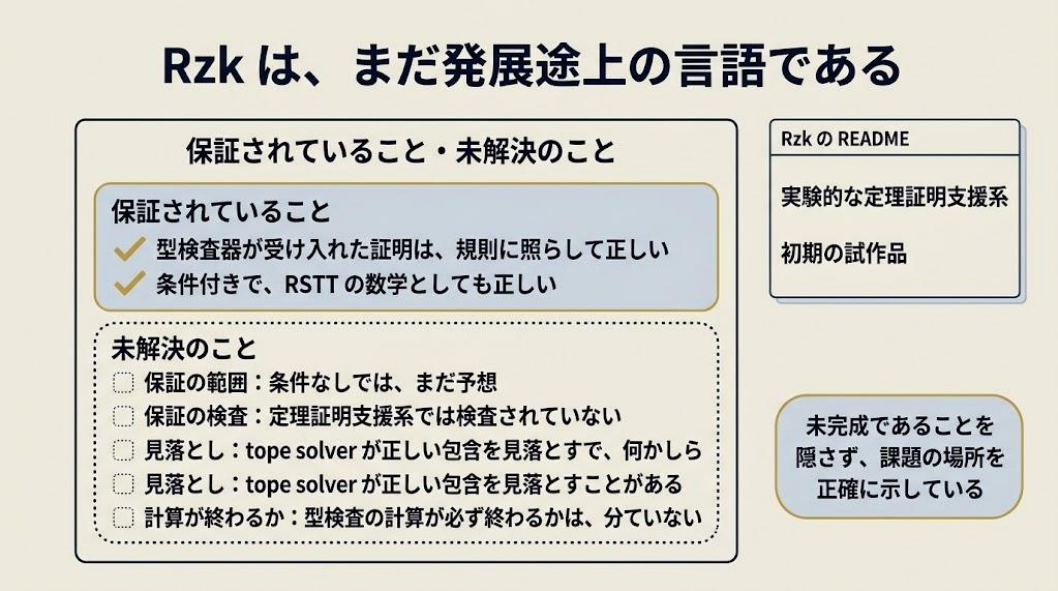
つまり、**Rzk は、まだ完成した言語ではありません。**

Rzk のリポジトリの README も、**Rzk** を「実験的な定理証明支援系」と名乗り、「初期の試作品」と説明しています。

それでも、**どこまでが保証されていて、どこからが未解決なのか** は、論文の中ではっきりと示されています。

未完成であることを隠さず、残っている課題の場所を正確に示していること。

それが、この論文と **Rzk** という言語の現状です。



第 10 章　まとめ ── 分野専用の言語という設計

この章の要点：

- **Rzk** は、**∞-圏論の「形の帳簿づけ」を言語の核に引き受けることで、∞-圏の条件そのものを書き下せるようにした。**
- ただし、**専用言語を作るべきか汎用言語の上でやるべきかという問いには、まだ答えが出ていない。**

10-1　Rzk は何ができるようになったのか

本稿の 3 つの問い に、順に答えます。

なぜ、圏論のために専用の言語が要るのか。

汎用の言語で ∞-圏を素朴に書くと、境界での一致を証拠として持ち回ることになり、可換三角形の段階で「ほとんど乗り越えられない」手間が生じるからです（第 3 章）。

Rzk は、具体的に何ができるのか。

cube と tope の層で形を切り出し、extension type で境界を型に埋め込むこと で、**射は 5 行、可換三角形の証明は 1 行で書けます**（第 4 章・第 5 章）。

そして、「**この型は (∞,1)-圏である**」**という条件そのもの（Rezk 型）を書き下し、機械で検査できます**（第 6 章）。

実際に、何が証明されたのか。

合成的 ∞-圏論（RSTT）の枠組みで、∞-圏論の米田の補題が初めて形式化 され、**25,095 行の sHoTT ライブラリ** が築かれました（第 7 章）。

そして、その土台は、紙の理論を機械で扱えるようにする 5 つの改良（第 8 章）と、その改良が数学を変えていないことの証明（第 9 章）に支えられています。

10-2　汎用の言語との比較

	Lean 4	Agda	Rocq	**Rzk**
目的	汎用	汎用	汎用	**∞-圏論**
実装言語	C++ / Lean	Haskell	OCaml	**Haskell**
処理系の GitHub スター数	9,895	2,965	5,661	**311**
主要ライブラリの GitHub 貢献者数	835（mathlib4）	165（agda-stdlib）	69（math-comp）	**23（sHoTT）**
主要ライブラリ	mathlib（百万行超）	agda-stdlib など	Mathematical Components など	**sHoTT（約 25,000 行）**
産業利用	あり	あり	あり	**見当たらない**
∞-圏の扱い	ライブラリとして書く	同左	同左	**言語の核**
射の定義	等式の証拠を持ち回る	同左	同左	**extension type で、証拠なし**
暗黙の引数	あり	あり	あり	**なし（実装予定）**

スター数と貢献者数は、いずれも 2026 年 11 月 6 日時点の GitHub の表示によります（処理系は leanprover/lean4、agda/agda、rocq-prover/rocq、rzk-lang/rzk の各リポジトリ）。

スター数は関心の目安であり、利用者数そのものではありません。

貢献者数は、GitHub のアカウントと結びついたものだけが数えられます。

10-3　どちらが優れているという話ではない

表を見ると、Rzk はほとんどの行で「小さい」「ない」と書かれています。

しかし、それは **Rzk が劣っているという意味ではありません。**

目的が違う のです。

汎用の言語は、「どんな数学でも書ける」ことを目指し、そのために言語の核を小さく一般的に保ちます。

Rzk は、「∞-圏論を自然に書ける」ことを目指し、そのために ∞-圏論の形を言語の核に入れました。

目的が違えば、比べる尺度も違います。

- **汎用の言語を測る尺度**：どれだけ多くの分野で、どれだけ多くの数学が書かれたか
- **専用の言語を測る尺度**：その分野の数学が、どれだけ自然に書けるようになったか

後者の尺度で見れば、Rzk は「合成的 ∞-圏論の枠組みで、∞-圏論の米田の補題を初めて形式化する」ところまで来ました。

前者の尺度で見れば、Rzk はまだ小さな共同体の道具です。

どちらも事実です。

10-4　強みと弱み

強み

	内容
1	∞-圏論の記法が、ほぼそのまま書ける。hom、hom2、Δ^2、Λ^2_1 が論文の記法どおりに定義できる
2	境界条件が判定的。証拠を持ち回らなくてよい
3	tope solver が、形の包含を自動で確かめる。sHoTT 全体で手作業の回避策は不要だった
4	ブラウザで動く。インストールせずに試せる
5	理論と実装が、論文で厳密に対応づけられている（忠実性、条件付きの保存性、アルゴリズムの健全性）

弱み

	内容
1	利用者が少ない。数十人の規模
2	ライブラリが小さい。sHoTT の約 25,000 行が中心
3	∞-圏論以外には向かない。利用者定義の（高次）帰納型がなく、**具体的な ∞-圏は公理として置くしかない**

	内容
4	実験的である。v0.7.7 までに健全性バグがあった。型検査の停止性も保証されていない
5	産業での利用例がない

論文自身も、2 つの機能の欠如を「形式化を制限している」と認めています。

1 つ目は、**暗黙の引数**（*implicit arguments*）がないことです。

暗黙の引数とは、型から推論できる引数を、書かずに済ませる仕組みです。

たとえば `hom2 A x y z f g h` の A, x, y, z は、f, g, h の型から分かるはずですが、Rzk では毎回書く必要があります。

論文によれば、このために Rzk の形式化は、Rocq や Agda と比べてかなり冗長になります。

著者らは、Mazzoli と Abel の方式にならって、Rzk の項の表現の土台である 2 階の抽象構文の技術を使って実装する予定だとしています。

2 つ目は、**利用者が定義できる（高次）帰納型**がないことです。

そのため Rzk は、抽象的な合成的 (∞,1)-圏論には向いていますが、具体的な ∞-圏を扱うには、その計算規則を公理として置くしかありません。

論文は、有向・単体的な高次帰納型の満足な理論そのものが、著者らの知る限り、まだ存在しないと述べています。

10-5　関数型言語・定理証明言語における位置づけ

関数型言語としての Rzk

- Haskell 製だが、Haskell とは別の言語
- プログラムを書いて実行する言語ではない
- 証明を書いて検査する言語

定理証明言語としての Rzk

- 汎用（Lean、Agda、Rocq、Isabelle/HOL）ではない
- **分野専用（∞-圏論）**
- 著者らの知る限り、**RSTT の唯一の処理系**

系譜

1. Martin-Löf 型理論
2. ホモトピー型理論（HoTT）
3. 有向型理論（directed type theory）
4. Riehl と Shulman の単体的型理論（RSTT、2017 年）
5. Rzk（2023 年〜）

（第 2 章 2-6 の図の再掲です。）

論文の関連研究の節から、近い位置にある処理系を挙げておきます。

処理系	扱う型理論	Rzk との関係
cubicaltt	キュービカル型理論	キュービカル型理論の最初の型検査器
Cubical Agda	キュービカル型理論	Agda のモード。区間の型と高次帰納型を持つ
RedPRL、redtt、cooltt	Cartesian キュービカル型理論	試作品としての処理系を重ねてきた系列
Arend	区間の型を持つ HoTT の変種	JetBrains が開発
Narya	高次観測的型理論など	開発中。基本の理論に区間を持たない

キュービカル型理論の処理系は、区間を言語に組み込み、型に依存した計算を行う点で、Rzk と技術的な難しさを共有しています。

ただし、扱うのは向きのない ∞-亜群です。

本稿で扱う RSTT を実装した処理系としては、著者らの知る限り、Rzk が唯一です。

（∞-圏論の形式化には、10-9 で紹介する Lean 4 上の ∞-cosmos のような、別の路線もあります。）

10-6　論文が挙げる今後の課題

論文は、結論の節で、未解決の問題と、具体的な改良の予定を挙げています。

未解決の問題

1. **保存性の議論の機械化**
   - 強制変換の整合性、項レベルの部分型の潰れ、solver の健全性は、述べたが機械化していない
   - RSTT の機械化そのものも、著者らの知る限りまだない

- 候補として、Fiore と Szamozvancev による Agda の 2 階抽象構文の枠組みが挙げられている（Rzk の項の表現とも相性がよい）

2. **tope の断片の決定可能性**
   - 決定可能だと予想しているが、証明と完全な判定手続きは今後の課題
3. **分けた構成の直接の意味論**
   - Weinberger（2026 年）は、融合した extension type について置き換えに対する厳密な安定性を証明した（これで shape-Π は扱える）
   - しかし、自由な位置の型制限は扱えず、直接の意味論は未解決
4. **最も根本的な問題：RSTT の正規化**
   - 停止するかどうかが分かっていない
   - したがって、型検査器の完全なメタ理論も、理論そのものの標準性や正規化も、未解決のまま

具体的な改良の予定

1. **tope solver を完全にする**
   - よく研究された直観主義論理の判定器（Itauto など）を、有向区間の理論で拡張する
   - Voronkov による「等式つき直観主義論理の証明探索」が、最も近い断片
   - → 完全になり、7-6 の重い外れ値も、より一様に扱えるようになるはず
2. **normalization-by-evaluation（NbE）のバックエンド**
   - 現在の「構文の上での直接の簡約」より、定数倍で大きく速くなるはず
   - ただし RSTT に特有の難しさが 2 つある
     - 型に依存した境界の書き換え（8-6）
     - 同じ項が tope の仮定ごとに違う値に計算されること
     - → 値を tope の文脈ごとに分けて持つ必要がある
3. **tope family の定義域を、型で追跡する**
   - 単一の tope 宇宙 TOPE を、「φ のもとで考える tope の部分宇宙 Tope[φ]」に細分する
   - → 第 8 章で見た「定義域の条件を自動で補う」仕組みを、型の側で表せる

論文の自己評価は、慎重です。

Rzk は「使える道具」だが、その理論的な土台には、まだ埋めるべき穴がある。

その穴の場所を、論文自身が地図にして示しています。

10-7　「学部生に教えられるか」への回答

第 1 章で、この問いを立てました。

∞-圏論を、学部生に教えられるか。

本稿を通して見てきたことを、まとめます。

∞-圏論が大学院で何年もかかる理由の一部は、内容ではなく **前提** にありました。

- 射を定義するための組合せ論的な構成
- 境界の一致を管理する証拠
- その証拠どうしの整合性。

数学の中身に入る前に、それだけの前提を学ぶ必要がありました。

Rzk が言語の核に埋め込んだのは、まさにその前提 です。

- **形（線分、三角形、四面体）**：cube と tope が引き受ける
- **境界での一致**：extension type が引き受ける
- **形の包含の確認**：tope solver が引き受ける

利用者は、境界の一致や形の包含を証拠として管理するための技術的な前提を先に学ばなくても、Rzk の記法で射を書き始められます。

`hom` は 5 行で、可換三角形は 1 行でした。

誤った射は、型検査器が拒否してくれます。

もちろん、これで ∞-圏論のすべてが易しくなるわけではありません。

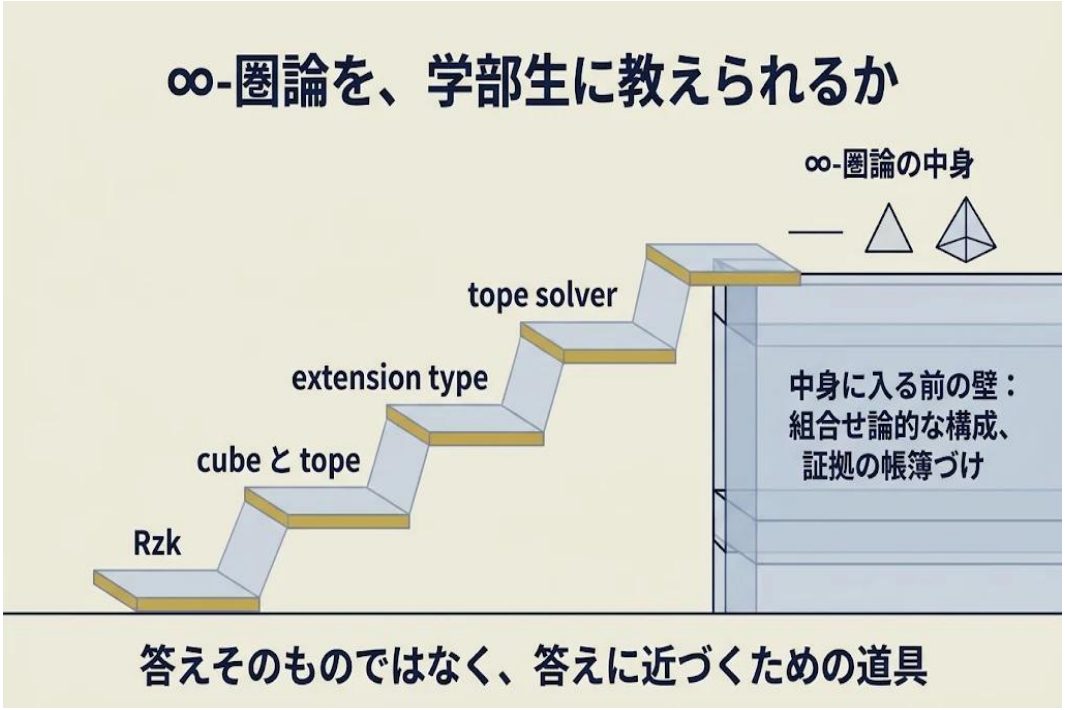
米田の補題の中身を理解するには、やはり数学の学習が要ります。

しかし、「**中身に入る前の壁」の一部を、言語が取り除いた**ことは確かです。

2026 年の ICERM のワークショップが「コンピュータで高次圏論を教える」を掲げ、その実習を Rzk に置いていることは、この方向が教育の現場で試され始めていることを示しています。

これが、「学部生に教えられるか」に対する、現時点での 1 つの技術的な回答です。

答えそのものではなく、**答えに近づくための道具**です。



10-8　本稿の限界

筆者が確かめたことと、確かめていないことを分けておきます。

確かめたこと（Rzk 0.11.4 で実機検証）	確かめていないこと
4 つの基本的な形（Δ^1、Δ^2、Λ^2_1、Δ^3）の定義が通る	sHoTT 全体の型検査
hom、hom2、id-hom の定義が通る	論文の性能測定（7-6）の再現
恒等射の可換三角形 id-triangle が通る	米田の補題の形式化の中身
cofibration composition の前向き・逆向きの写像が、それぞれ通る	2 つのバグが v0.7.7 で実際に再現すること
境界条件に反する定義 bad が、型エラーで拒否される	論文の定理（忠実性・保存性）の証明の細部

注意点も記しておきます。

- **定義の順序**：後方参照はできない。使う前に定義する必要がある
- **組み込み**：prod、iff などは組み込みではない。Equiv を使う例には sHoTT の定義が要る
- **バージョン**：論文が報告しているのは v0.7.8。筆者が検証したのは 0.11.4

10-9　同じ研究者が 2 つの道を試している

最後に、1 つの事実を紹介して、本稿を閉じます。

Emily Riehl は、Rzk / sHoTT とは別に、もう 1 つの形式化プロジェクトを率いています。

Lean 4 の上で ∞-圏論を形式化 する **∞-cosmos プロジェクト**です。

Riehl の 2025 年の発表スライド（ASL 2025 北米年次大会）によると、**∞-圏論を形式化可能にする戦略は 3 つあります**（筆者による要約）。

戦略	内容
(i) 解析的（analytic）	∞-圏を、組合せ論的な構成として定義する
(ii) 公理的（axiomatic）	∞-圏の理論を、公理的な枠組み（∞-cosmos）で展開する
(iii) 合成的（synthetic）	∞-圏が、はじめから内にある型理論で考える

合成的な道は、専用の定理証明支援系（たとえば Rzk）を必要とする

そして **∞-cosmos プロジェクト** は、Mario Carneiro、Dominic Verity、Riehl が共同で率い、**Lean** における **∞-圏の公理的な理論** を目指している、とされています。

> 【出典の性格】この内容は、学会の発表スライド（https://math.nmsu.edu/asl-2025/slides/prospects-asl.pdf）にもとづきます。
>
> 査読を経た論文ではありません。

プロジェクト	道筋	使う言語
Rzk / sHoTT	合成的	Rzk（専用言語）
∞-cosmos	公理的	Lean 4（汎用言語）

分野専用の言語を作るべきか、汎用言語の上でやるべきか。

この問いに、まだ答えは出ていません。

だからこそ、同じ研究者が、両方を並行して試しています。

- **専用言語** は、その分野の記法をそのまま書ける代わりに、利用者と資産が少ない。
- **汎用言語** は、巨大なライブラリと共同体を持つ代わりに、その分野の概念を押し込む手間がかかる。

どちらの代償が小さいかは、分野によっても、時期によっても変わるでしょう。

本稿が示せるのは、片方の道が、ここまで来たということだけです。

∞-圏論の米田の補題を機械で検査し、25,000 行のライブラリを築き、教室で使われ始めるところまで。



10-10　次に読むもの

Rzk に触れてみる

- Rzk Playground
- Yoneda game

ライブラリを読む

- sHoTT

言語そのものを知る

- Kudasov, Sim, Ahrens「Rzk: a Proof Assistant for Synthetic ∞-Categories」
- Rzk のリポジトリ

理論の原典

- Riehl, Shulman「A type theory for synthetic ∞-categories」（2017）　arXiv:1705.07442

動機を知る

- Riehl「Could ∞-category theory be taught to undergraduates?」（2023）

extension type そのものについては、別の記事で扱います。

付録 A　本稿で扱った用語（22 語）

本文で初めて出てきたときに説明した用語を、英語の原語と記号つきで一覧にします。

読み返すときの索引として使ってください。

正式な用語	英語の原語	記号・Rzk での書き方	ひとことで	初出
圏	*category*	—	対象と、対象のあいだの射からなる構造	2-1
射	*morphism*	`hom A x y`	対象から対象への矢印	2-1、5-1
∞-圏	*∞-category*、*(∞,1)-category*	—	射が何段にも重なる構造。1 次元の射は可逆とは限らない	2-2、2-5
∞-亜群	*∞-groupoid*、*(∞,0)-category*	—	すべての射が可逆な ∞-圏	2-5
合成的	*synthetic*	—	外から組み立てず、はじめから内側にあるものとして扱う	1-4
単体的型理論	*simplicial type theory*、RSTT	—	Riehl と Shulman の型理論（2017 年）	2-6
有向型理論	*directed type theory*	—	射に向きがある型理論の総称	2-6
cube	*cube*	`2`、`2 × 2`	点の抽象的な空間	4-2
tope	*tope*	`TOP`、`BOT`、`≡`、`≤`、`∧`、`∨`	cube の点を制約する論理式	4-3
shape	*shape*	`Δ¹`、`Δ²`	cube と tope を合わせて切り出した形	4-4
有向区間	*directed interval*	`2`（端点 0_2、1_2）	向きのある区間	3-1、4-2
単体	*simplex*	Δ^n	点、線分、三角形、四面体、…	3-5、4-5
inner horn	*inner horn*	Λ^2_1	三角形のうち、内側の頂点で交わる 2 辺だけ	4-5
extension type	*extension type*	`(t : ψ) → A [ ϕ ↦ a ]`	境界で値を判定的に固定する型	3-6、4-6

正式な用語	英語の原語	記号・Rzk での書き方	ひとことで	初出
cofibration	*cofibration*	`──`	形の包含	8-3
Segal 型	*Segal type*	`is-segal A`	合成が本質的に一意に定まる型	6-2
Rezk 型	*Rezk type*	`is-rezk A`	Segal 型 + 完備性。合成的な (∞,1)-圏	6-3
忠実性	*faithfulness*	`τ`	翻訳元のすべての証明が、翻訳先でも証明になる	9-1
保存性	*conservativity*	`σ`	翻訳先で、翻訳元について新しいことが証明できない	9-1
双方向型検査	*bidirectional typing*	`⇐`、`⇒`	検査と合成の 2 つのモードで型を確かめるアルゴリズム	8-6
部分型	*subtyping*	`A <: B`	ある型の要素が、そのまま別の型の要素として通る関係	8-3
判定的等しさ	*judgmental / definitional equality*	`≡`	型検査器が計算によって自動で認める等しさ	3-2

本文で説明した、そのほかの用語

用語	英語の原語	ひとことで	初出
定理証明言語	*proof assistant*	証明を書き、機械に検査させる言語	1-1
型検査	*type checking*	値の型が合っているかを機械が確かめること	1-1
形式化	*formalization*	数学を、機械が検査できる形に書き直すこと	1-1
可換図式	*commutative diagram*	どの道をたどっても結果が同じであることを表す絵	2-1
恒等射	*identity morphism*	何もしない射	2-1、5-2
依存型	*dependent type*	値に依存する型	2-6
等式型	*identity type*	「等しいことの証拠」の型	2-6
命題的等しさ	*propositional equality*	証拠として与える等しさ	3-2
可縮	*contractible*	本質的に 1 つしか要素がないこと	6-2
同型	*isomorphism*	逆向きの射を持つ射	6-3

用語	英語の原語	ひとことで	初出
同値	*equivalence*	HoTT における「同型」	6-4
型制限	*type restriction*	任意の型に境界の制約を付ける Rzk の構成要素	8-2
tope family	*tope family*	定義域を持つ tope の引数。包含の前提を型で表す	8-4
tope solver	*tope solver*	tope の含意を自動で確かめる仕組み	8-5
健全性バグ	*soundness bug*	正しくない証明を受け入れてしまうバグ	9-6
正規化	*normalization*	項をこれ以上計算できない形まで計算すること	9-7
暗黙の引数	*implicit arguments*	型から推論できる引数を書かずに済ませる仕組み	10-4

付録 B　参考文献

本稿の主な情報源

- N. Kudasov, V. Sim, B. Ahrens. *Rzk: a Proof Assistant for Synthetic ∞-Categories.* Proceedings of the ACM on Programming Languages, Vol. 11, POPL, 2027. arXiv:2607.12207（https://arxiv.org/abs/2607.12207）

RSTT と、その上の数学

- E. Riehl, M. Shulman. *A type theory for synthetic ∞-categories.* Higher Structures 1, 2017. arXiv:1705.07442
- N. Kudasov, E. Riehl, J. Weinberger. *Formalizing the ∞-Categorical Yoneda Lemma.* CPP 2024.
- B. Lossin. *Fibrations in Directed Type Theory.* 修士論文、ミュンヘン工科大学、2025 年. arXiv:2604.18668
- J. Weinberger. *A Synthetic Perspective on (∞,1)-Category Theory: Fibrational and Semantic Aspects.* 2022. arXiv:2202.13132
- U. Buchholtz, J. Weinberger. *Synthetic fibered (∞,1)-category theory.* Higher Structures 7, 2023.
- J. Weinberger. *Two-sided cartesian fibrations of synthetic (∞,1)-categories.* Journal of Homotopy and Related Structures, 2024.
- C. Bardomiano Martínez. *Limits and colimits of synthetic ∞-categories.* Mathematical Structures in Computer Science, 2025.
- J. Weinberger. *Strict stability of extension types.* Theory and Applications of Categories 45, 2026.
- I. Talipov, N. Kudasov. *Towards Formalization of Directed Univalence in Rzk Proof Assistant.* HoTT/UF 2026 アブストラクト.

型理論の背景

- M. Hofmann, T. Streicher. *The Groupoid Model Refutes Uniqueness of Identity Proofs.* LICS 1994.
- M. Z. Weaver, D. R. Licata. *A Constructive Model of Directed Univalence in Bicubical Sets.* LICS 2020.
- J. Weinberger, B. Ahrens, U. Buchholtz, P. North. *Towards Normalization of Simplicial Type Theory via Synthetic Tait Computability.* TYPES 2022.

実装の技術

- N. Kudasov. *Free Monads, Intrinsic Scoping, and Higher-Order Preunification.* TFP 2024.
- J. Dunfield, N. Krishnaswami. *Bidirectional Typing.* ACM Computing Surveys 54, 2021.
- Z. Luo. *Coercive subtyping.* Journal of Logic and Computation 9, 1999.
- A. Abel, J. Öhman, A. Vezzosi. *Decidability of conversion for type theory in type theory.* POPL 2018.
- A. Voronkov. *Proof Search in Intuitionistic Logic with Equality, or Back to Simultaneous Rigid E-Unification.* Journal of Automated Reasoning 21, 1998.
- F. Besson. *Itauto: An Extensible Intuitionistic SAT Solver.* ITP 2021.
- F. Mazzoli, A. Abel. *Type checking through unification.* 2016. arXiv:1609.09709
- M. Fiore, D. Szamozvancev. *Formal Metatheory of Second-Order Abstract Syntax.* POPL 2022.
- A. Abounegm, N. Kudasov. *Implementing a Language Server for the Rzk Proof Assistant.* System Informatics 22, 2023.

リポジトリとツール

- Rzk Playground：https://rzk-lang.github.io/rzk/playground/
- sHoTT ライブラリ：https://github.com/rzk-lang/sHoTT
- Rzk Playground：https://rzk-lang.github.io/rzk/playground/
- Yoneda game：https://rzk-lang.github.io/yoneda-game/

発表資料と論考（査読なし）

- E. Riehl. *Could ∞-category theory be taught to undergraduates?* 2023 年.
- N. Kudasov. HoTTEST セミナー発表スライド、2023 年 10 月.
- E. Riehl. ASL 2025 北米年次大会 発表スライド. https://math.nmsu.edu/asl-2025/slides/prospects-asl.pdf